

TTK4551 Technical Cybernetics - Specialization Project:
Graph based SLAM for Autonomous Navigation using Side Scan
Sonar on the microAmpere ASV

Martynas Smilingis

September 2025

Contents

1	Introduction	8
1.1	Background	8
1.2	Motivation	8
1.3	UN Sustainability Goals	9
1.4	Goal	10
1.5	Contributions	10
1.6	Overview and Context	10
2	Side Scan Sonar SLAM Architecture	11
2.1	Overview	11
2.2	Detailed Architecture	12
2.2.1	Side Scan Sonar Pipeline	12
2.2.2	Hardware	13
2.2.3	Data Processing	13
2.2.4	SLAM	13
2.2.5	Global Map and Trajectory	15
3	Sonar Theory	16
3.1	Introduction	16
3.2	Acoustic Waves	17
3.3	Sonar Fundamentals	18
3.4	Exteroceptive Sensors and Side Scan Sonar	19
4	Hardware	20
4.1	Introduction	20
4.2	microAmpere ASV	21
4.3	Specifications	23
4.4	Layout	24
4.5	Sensors	28
4.5.1	Introduction	28
4.5.2	Inertial Measurement Unit (IMU) for Navigation and Pose Estimation	28
4.5.3	GNSS for Positioning and Heading Estimation	29
4.5.4	Side Scan Sonar for Environmental Perception and SLAM	30
4.6	Software Architecture	31
5	System Modeling	33
5.1	Introduction	33
5.2	Orientation Representations	34
5.2.1	Euler Angles	34
5.2.2	Quaternions	35
5.2.3	Lie Groups and Manifolds	37
5.2.4	Handy Conversions	39
5.3	Reference Frames and Transformations	40
5.3.1	Basic Translation and Rotation Transformations	40
5.3.2	Global Reference Frames	40
5.3.3	Local Reference Frames	43
5.4	Rigid Body Kinematics	45
5.4.1	Relevance for Navigation and Modeling	45
5.4.2	Position and Orientation	45
5.4.3	Angular Velocity and Euler Angle Relationship	45
5.4.4	Angular Velocity and Quaternion Relationship	46
5.4.5	Linear Velocity Relationship	47
5.4.6	Angular Acceleration in Euler Representation	47
5.4.7	Angular Acceleration in Quaternion Representation	48
5.4.8	Linear Acceleration	48
5.5	ASV Motion and Measurement Models	50
5.5.1	Overview	50

5.5.2	Kinetic Motion Model	50
5.5.3	Kinematic Motion Model	51
5.5.4	Aiding Measurement Model	56
5.5.5	Landmark Measurement Model	58
5.5.6	Model Comparison and Selection	60
5.6	Numerical Solvers	61
5.6.1	Implicit vs Explicit Solvers	61
5.6.2	Newton-Euler Method	62
5.6.3	Runge-Kutta Methods (RK4)	62
5.7	Factor Graphs	64
6	State Estimation	65
6.1	Introduction	65
6.2	Bayes Filter	66
6.3	Kalman Filter	67
6.4	Extended Kalman Filter	69
6.5	Error State Kalman Filter	72
6.5.1	Concept and Motivation	72
6.5.2	Error State Formulation	72
6.5.3	Process Model and Jacobians	72
6.5.4	Discretization of Error Dynamics	73
6.5.5	Measurement Model and Jacobian	73
6.5.6	Filter Algorithm	74
6.5.7	EKF vs ESKF	75
6.6	Unscented Kalman Filter on Manifolds	76
6.6.1	Concept and Motivation	76
6.6.2	Unscented Kalman Filter	76
6.6.3	Manifold Operators	78
6.6.4	Manifold Operators for the Process Model	79
6.6.5	Manifold Operators for the Measurement Model	81
6.6.6	<code>set_weights(dim, α)</code> : Sigma Point Weights Parameter Function	82
6.6.7	Modified Process Model for the UKF-M	84
6.6.8	Filter Algorithm: Prediction Step	84
6.6.9	Filter Algorithm: Correction Step	86
6.6.10	Advantages and Practical Aspects	88
6.7	SLAM based State Update	89
6.8	Tuning and Filter Verification	90
6.8.1	Debugging	90
6.8.2	Model Verification and Consistency Checks	90
6.8.3	Tuning of Process and Measurement Covariances	90
6.8.4	Empirical Tuning Procedure	91
6.8.5	Innovation Monitoring and Outlier Rejection	91
6.8.6	Final State Definition and Conclusion	92
7	Preintegration	93
7.1	Introduction	93
7.2	Preintegration on Manifolds	94
7.2.1	INS Propagation Between IMU Samples	94
7.2.2	Preintegration Initialization	95
7.2.3	Preintegration Algorithm (Recursive Update)	95
7.2.4	Bias Jacobian Propagation	96
7.2.5	Bias Re-Linearization Using Accumulated Jacobians	97
7.2.6	Predicted Motion Reconstruction	97
7.2.7	Predicted Bias Reconstruction	97
7.2.8	Preintegration Covariance Propagation	98
7.2.9	Bias Covariance Propagation	100
7.2.10	Algorithm Summary	100
7.3	Factor Construction	101
7.3.1	Factor Graph Formulation	101

7.3.2	IMU Preintegration Factor	101
7.3.3	Bias Evolution Factor	101
7.3.4	Preintegration Trigger and Factor Connectivity	102
8	Direct Factors	103
8.1	Introduction	103
8.2	GNSS Factor	104
9	Local Map Generation	105
9.1	Introduction	105
9.2	Swath Processing	106
9.2.1	Raw Data and Waterfall Image Formation	106
9.2.2	Pitch and Roll Correction	106
9.2.3	Blind Zone Removal	108
9.2.4	Slant Range Correction	110
9.2.5	Intensity Normalization	111
9.2.6	Stacking and Reconstruction	114
9.3	Probabilistic Mapping	115
9.3.1	Mapping	115
9.3.2	Projection from Polar to Cartesian	115
9.3.3	kNN Interpolation and Grid Reconstruction	116
9.3.4	Probabilistic Interpolation and Grid Reconstruction	117
9.3.5	Optimized Probabilistic Interpolation and Grid Reconstruction	125
9.3.6	Sliding Map Window and Overlap	128
9.4	Feature Extraction	129
9.4.1	Landmarks	129
9.4.2	Landmark Detection	130
9.4.3	Landmark Measurement	138
9.4.4	Landmark Measurement Uncertainty	138
9.4.5	Descriptor Assignment	139
9.4.6	Feature Extraction Output	142
10	Data Association	143
10.1	Introduction	143
10.2	Gating	144
10.3	Nearest Neighbor	146
10.4	Probabilistic Methods	147
10.5	Mixture Models	148
10.6	RANSAC	149
10.7	JCBB	150
10.7.1	Overview	150
10.7.2	Mathematical Preparation for JCBB	150
10.7.3	Covariance Structure and Practical Extraction	151
10.7.4	Local vs Global JCBB Usage	152
10.7.5	JCBB Algorithm	152
10.7.6	FastJCBB	153
10.7.7	Practical Use and Limitations	153
10.8	Loop Closure and Matching	154
10.8.1	The Problem of Loop Closure	154
10.8.2	Descriptor Based Loop Closure Matching	154
10.8.3	Local vs Global Loop Closure Decisions	154
10.8.4	Landmark Revalidation	154
10.9	Integrated Data Association Framework for SLAM	155
10.9.1	Processing Each New Scan	155
10.9.2	Local vs Global DA Branch	155
10.9.3	Landmark Aging	155
10.9.4	Landmark Revalidation	155
10.9.5	Summary of the Full Pipeline	156
10.10	Factor Construction	157

11 Optimizers	158
11.1 Introduction	158
11.2 iSAM	160
11.2.1 Getting to SLAM update step	160
11.2.2 Incremental QR for fast updates (iSAM)	162
11.2.3 What is R? The square root information matrix	162
11.2.4 Matrix Factorization for building QR (Givens rotations)	163
11.2.5 Incremental Updating	164
11.2.6 Loop Closure	165
11.2.7 Re-Linearization	166
11.2.8 Data Association from R	166
11.2.9 Algorithm	168
11.2.10 Limitations	169
11.3 iSAM2	170
11.3.1 Introduction and Motivation	170
11.3.2 Factor Graphs	170
11.3.3 From Factor Graphs to the SLAM Optimization Problem	171
11.3.4 From Factor Graphs to Bayes Networks	173
11.3.5 R as a Bayes tree data structure	174
11.3.6 Incremental Updates directly on Bayes tree	175
11.3.7 Loop Closure and Incremental Reordering	177
11.3.8 Fluid Re-Linearization	178
11.3.9 Sparse Factor Graphs	179
11.3.10 Beyond Gaussian Assumptions (Robust Estimators)	179
11.3.11 Data Association from the Bayes Tree	179
11.3.12 Algorithm	179
11.3.13 Limitations	180
12 Global Map and Trajectory	181
12.1 Introduction	181
12.2 Global Trajectory and Global Map Generation	182
13 Implementation Frameworks And Tools	183
13.1 Introduction	183
13.2 ROS2 Middleware Framework	184
13.3 CAOR-MINES ParisTech UKF-M Library	185
13.4 OpenCV Computer Vision Library	186
13.5 MRPT Data Association and Perception Tools	187
13.6 GTSAM Factor Graph Framework	188

Acronyms

ASV	Autonomous Surface Vessel
AUR Lab	Applied Underwater Robotics Lab
AUV	Autonomous Underwater Vehicle
CDF	Cumulative Distribution Function
COG	Center Of Gravity
COLAMD	Column Approximate Minimum Degree
DA	Data Association
DDS	Data Distribution Service
DOF	Degrees Of Freedom
DVL	Doppler Velocity Log
ECEF	Earth-Centered, Earth-Fixed
EKF	Extended Kalman Filter
ERK	Explicit Runge-Kutta
ESC	Electronic Speed Controller
ESKF	Error State Kalman Filter
FBR	First Bottom Return
FLANN	Fast Library for Approximate Nearest Neighbors
GNSS	Global Navigation Satellite System
GTSAM	Georgia Tech Smoothing And Mapping
HDPE	High Density Polyethylene
IMU	Inertial Measurement Unit
INS	Inertial Navigation System
IPDA	Integrated Probabilistic Data Association
iSAM	Iterative Smoothing And Mapping
JCBB	Joint Compatibility Branch and Bound
JIPDA	Joint Integrated Probabilistic Data Association
JPDA	Joint Probabilistic Data Association
KF	Kalman Filter
kNN	k-Nearest Neighbor
LAUV	Light Autonomous Underwater Vehicle
LiDAR	Light Detection And Ranging
LTI	Linear Time Invariant
LTV	Linear Time Varying
MAP	Maximum A Posteriori
MHT	Multi Hypothesis Tracking
MMSAM	Multi Modal Smoothing and Mapping
MPC	Model Predictive Control
MRPT	Mobile Robot Programming Toolkit
NED	North East Down
NIS	Normalized Innovation Squared
NN	Nearest Neighbor
PDA	Probabilistic Data Association
PPS	Pulse Per Second
PSM	Power Sense Module
PWM	Pulse Width Modulation
RANSAC	Random Sample Consensus

RL	Reinforcement Learning
ROS2	Robot Operating System 2
SBC	Single Board Computer
SE(3)	Special Euclidean Group in three dimensions
SLAM	Simultaneous Localization And Mapping
SLERP	Spherical Linear Interpolation
SO(3)	Special Orthogonal Group in three dimensions
SSS SLAM	Side Scan Sonar Simultaneous Localization And Mapping
UKF	Unscented Kalman Filter
UKF-M	Unscented Kalman Filter on Manifolds
UT	Unscented Transform
WGS84	World Geodetic System 1984
ZOH	Zero Order Hold

1 Introduction

1.1 Background

Coastal navigation depends on accurate environmental perception and robust mapping, especially in shallow or dynamic areas where shifting seabed conditions and variable coastlines make grounding avoidance a critical task. While GNSS normally provides global positioning, its performance degrades near coastal infrastructure due to shadowing and multipath. Autonomous surface vessels therefore require onboard estimation and mapping that remain reliable even when external positioning is weak or unavailable. Side scan sonar is well suited for this role, as acoustic sensing penetrates turbid water and provides high resolution seafloor information independent of visibility. In recent years, Side Scan Sonar based SLAM (SSS SLAM) has become an attractive approach for estimating the vessel trajectory while reconstructing local seabed structure, supporting both safe navigation and situational awareness in coastal environments.

Prior research, including paper on side scan sonar landmark detection [1] and Haraldstads 2023 master thesis on Side Scan Sonar SLAM (SSS SLAM) [2], which are heavily inspired by the earlier work of Hogstad [3], demonstrated the feasibility of using side scan sonar for feature extraction, data association, and landmark based mapping. However, most existing pipelines operate offline, struggle with real-time performance. Modern factor graph SLAM systems, such as those developed by Leonard, Kaess, and the GTSAM community, provide a strong theoretical foundation for building efficient, incremental SLAM architectures suitable for embedded robotic platforms. This project builds upon those foundations and focuses on adapting, simplifying, and optimizing SSS SLAM for deployment on NTNU’s MicroAmpere ASV (Autonomous Surface Vessel).

1.2 Motivation

Operating marine robots in coastal waters demands navigation systems that are resilient to environmental uncertainty and sensor degradation. Certain sensors such as GNSS and its performance degrades in coastal areas with tall structures, narrow passages, bridges, piers, or other large infrastructure, where signal blockage and multipath reflections reduce accuracy or cause intermittent loss. Dead reckoning drifts quickly, and static seafloor charts become outdated due to sediment movement, new constructions, and dynamic marine activity. For autonomous surface vessels such as microAmpere ASV, onboard mapping is essential for avoiding grounding, maintaining situational awareness, and enabling long duration autonomous operation without human supervision.

Side scan sonar offers a practical solution because its data reveals local seabed structure with high spatial consistency, but side scan sonar data is noisy, viewpoint dependent, and difficult to align without accurate motion estimates. Current real-time implementations of SSS SLAM show mixed results, while offline methods achieve good mapping accuracy but are not suited for real-time operation on autonomous vessels. The goals of this project are twofold. First, to study and outline a practical, lightweight SSS SLAM architecture, that can operate in real time on constrained embedded hardware while remaining robust on real datasets. Second, to implement this architecture on the microAmpere ASV, integrate it into the onboard perception stack, and evaluate its performance on real data and in real coastal environments. This shifts the work from theoretical and offline analysis toward a deployable, field ready SLAM capability for autonomous surface vessels.

1.3 UN Sustainability Goals

This project directly supports several UN Sustainability Goals related to ocean safety, resilient infrastructure, and responsible use of maritime environments. Autonomous coastal vessels capable of accurate self localization and seabed mapping contribute to:

- **SDG 9 – Industry, Innovation, and Infrastructure:** Developing advanced robotic systems that improve maritime safety, reduce operational risk, and support next generation autonomous transport.
- **SDG 13 – Climate Action:** Enabling efficient autonomous monitoring of coastal environments, reducing emissions from manned patrol vessels, and supporting long duration data collection with minimal energy consumption.
- **SDG 14 – Life Below Water:** Providing high quality seabed data that can support environmental monitoring, habitat protection, and sustainable marine operations, especially in sensitive or shallow coastal areas.

By enabling autonomous vessels to operate safely and efficiently in coastal waters, the system developed in this thesis contributes to safer marine navigation, reduced environmental impact, and increased accessibility of marine robotics for ecological monitoring and research.



Figure 1: UN Sustainability Goals supported by this project: SDG 9 (Industry, Innovation and Infrastructure), SDG 13 (Climate Action), and SDG 14 (Life Below Water).^[4]

1.4 Goal

The goal of this specialization project is to study, design, and outline a practical side scan sonar SLAM (SSS SLAM) architecture that can operate in real time on constrained embedded hardware. The work focuses on creating a clear and lightweight SSS SLAM pipeline that achieves consistent real-time performance on recorded coastal datasets while remaining suitable for deployment on autonomous surface vessels.

A second goal is to implement this pipeline on the microAmpere ASV and evaluate its performance both on recorded data and in real coastal environments. This includes porting the system to the vessels onboard hardware, integrating it into the perception stack, and validating that the SLAM framework can maintain real-time operation during field trials. Together, these goals shift the work from offline analysis toward a deployable, field ready SSS SLAM module for autonomous surface vessel navigation.

1.5 Contributions

The main contributions of this work are:

- **A streamlined, real-time capable SSS SLAM pipeline:** A complete implementation of core components from previous work, optimized for speed, modularity, and embedded deployment.
- **Improved feature extraction and landmark generation:** New processing techniques using adaptive thresholding, morphological filtering, and robust descriptor assignment for stable, repeatable landmark detection.
- **A 2 layer data association system:** Combining local JCBB matching with global descriptor based loop closure proposals to maintain robustness while remaining computationally efficient.
- **Integration of IMU preintegration and factor graph optimization:** Leveraging GTSAM's iSAM2 for incremental smoothing, enabling consistent trajectory and map estimation over long missions.
- **Embedded integration on microAmpere ASV:** Porting, packaging, and preparing the pipeline for real-time operation on distributed ROS2 based onboard compute systems.

Together, these contributions support practical deployment of side scan sonar SLAM (SSS SLAM) on autonomous surface vessels by providing a clear architecture and a real-time embedded implementation

1.6 Overview and Context

This thesis presents the design and implementation of a complete side scan sonar SLAM (SSS SLAM) system tailored for autonomous surface vessels (ASV's). The core structure of the work follows the flow of a full SLAM pipeline, beginning with a high level architecture and gradually moving toward detailed algorithmic and implementation specific components. The most important part of the thesis is the Side Scan Sonar SLAM Architecture chapter, which provides a unified overview of how all subsystems interact, from raw sonar acquisition to global trajectory estimation and map construction.

The chapters that follow expand each subsystem in depth. First, the Sonar Theory, Hardware, and System Modeling chapters establish the physical and mathematical foundations required for navigation, mapping, and estimation. Since SLAM performance depends directly on correct sensor models, motion models, and coordinate representations, this section defines the specific formulations used for the microAmpere ASV platform.

After establishing the required models and assumptions, the thesis transitions into detailed components of the SLAM system, state estimation, IMU preintegration, direct factors, local map generation, data association, optimizers, and global map construction. Each chapter describes the algorithms, mathematical tools, and design decisions used to form the complete SLAM pipeline.

Finally, a dedicated chapter outlines the frameworks and tools that will be used throughout the implementation. Since modern SLAM relies on well established libraries for factor graphs, optimization, vision processing, and middleware, this chapter summarizes the external software components that make the final system practical to build, deploy, and test on the microAmpere ASV.

2 Side Scan Sonar SLAM Architecture

2.1 Overview

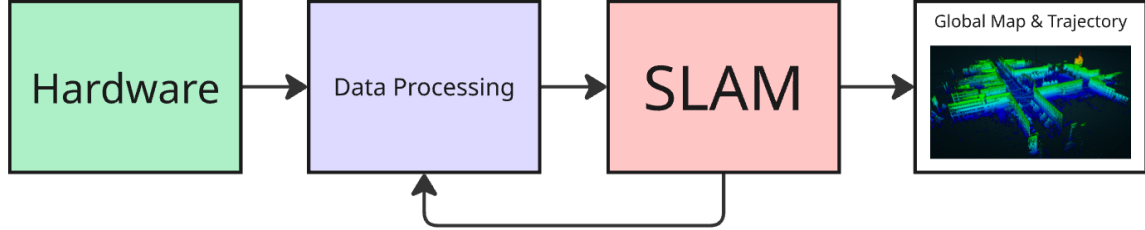


Figure 2: A simplified high level overview of the Side Scan Sonar Simultaneous Localization And Mapping (SSS SLAM) pipeline, from raw hardware inputs to the final global map and trajectory estimate.

The simplified architecture in Figure 2 shows the core flow of the system, sensor hardware produces raw data, this data is processed, the Simultaneous Localization And Mapping (SLAM) module estimates the vehicle motion and environment, and the final output becomes a global map with a trajectory. This overview hides all lower level details and is meant only to illustrate how the major components fit together at a high level.

The SLAM formulation used in this work is landmark based. Many SLAM variants exist, such as EKF-SLAM, dense mapping, or direct scan matching methods, but landmark based SLAM fits naturally with the idea of building a map composed of repeatable features. Since the end goal is to create a global map that is consistent over long distances, using distinct landmark like structures as constraints provides strong geometric information and reduces drift. This strategy also aligns well with modern graph based optimization, where each landmark observation becomes a factor that ties the trajectory together.

The general plan is therefore straightforward, extract useful information from the sonar data, convert it into a set of spatial observations, and feed these into a SLAM back end that optimizes both the trajectory and the map. This approach is chosen because it balances robustness, computational efficiency, and the ability to handle loop closures, which are essential for long duration mapping missions.

This subsection provides the highest level overview of the entire thesis. The following chapters expand each of these blocks in detail, describing the design choices, algorithms, and reasoning behind the final SLAM architecture. Understanding this simple structure is helpful before diving into the more technical parts, as it shows how all components connect from raw input to the final map and trajectory. The next subsection offers a slightly more detailed overview of the full system, which is useful to keep in mind when exploring the individual modules and how they interact throughout the pipeline.

2.2 Detailed Architecture

2.2.1 Side Scan Sonar Pipeline

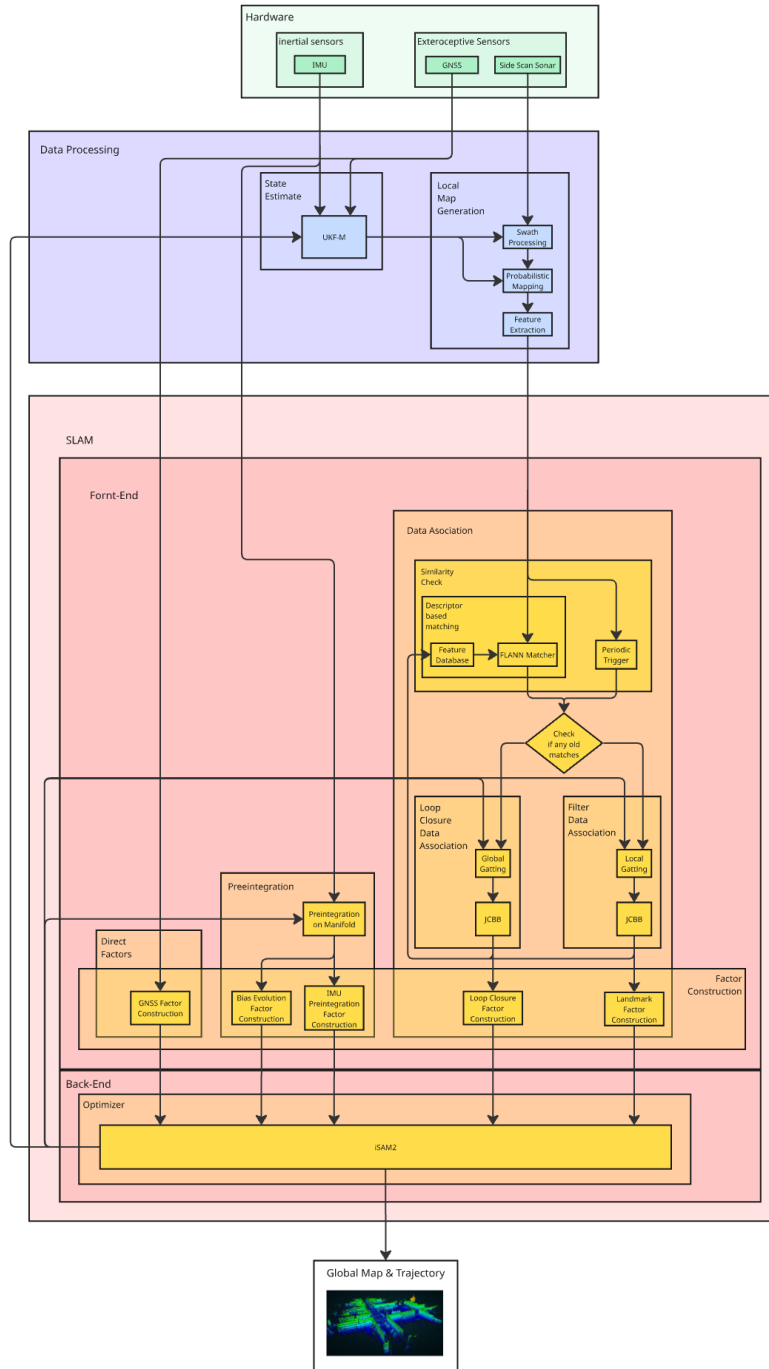


Figure 3: Picture shows the full Side Scan Sonar Simultaneous Localization And Mapping (SSS SLAM) pipeline from raw sensor data to the final global map and estimated trajectory. This diagram expands on the simplified overview by illustrating how hardware, preprocessing, state estimation, local mapping, feature extraction, data association, and the graph based SLAM back end are connected into one continuous system. Each block represents a major subsystem of the SLAM pipeline, and the arrows indicate how information flows from the sensors through processing layers toward the optimizer and the final global reconstruction. This figure acts as a visual “roadmap” for the rest of the thesis, helping the reader keep track of where they are as each subsection is later described in more detail.

2.2.2 Hardware

The hardware layer consists of three key sensors, the IMU (Inertial Measurement Unit), the GNSS (Global Navigation Satellite System), and the side scan sonar. The GNSS and sonar are exteroceptive sensors, meaning they provide information about the external environment. GNSS is used whenever available to provide absolute positioning and heading corrections, but it cannot be fully relied upon due to multipath, occlusion, or complete signal loss in coastal or obstructed areas. Side scan sonar, on the other hand, forms the primary sensing modality for environmental perception and is therefore central to the SLAM system. The IMU provides continuous high rate inertial measurements, allowing dead reckoning between external sensor updates and ensuring smooth, uninterrupted motion estimation even when GNSS is degraded.

Data from these sensors is typically streamed over Ethernet using vendor specific communication protocols. In the actual system deployment, all raw data is distributed throughout the processing pipeline using ROS2 middleware, ensuring synchronized timestamps, modular development, and reliable communication between nodes running on separate embedded computers. This hardware foundation enables the later stages of the pipeline to fuse complementary information sources into a consistent state estimate and surrounding environment understanding.

2.2.3 Data Processing

The data processing stage includes the state estimator and the full local map generation pipeline. The local mapping framework follows the proven structure from prior work by Haraldstad [2], which demonstrated reliable performance on real AUV data. The focus of this thesis is to adapt and improve those methods for real-time use. The swath processing stage projects sonar returns into the ground plane, applies pitch and roll corrections, removes blind zones, performs slant range correction, and normalizes intensities. Afterward, probabilistic interpolation and grid reconstruction convert the processed swaths into coherent local Cartesian maps with missing areas filled using interpolation reconstruction techniques. Normalization and smoothing ensure that each local map is consistent and visually stable before landmark extraction.

Feature extraction in this thesis improves upon the earlier work by adopting more robust computer vision techniques. Edge preserving filtering enhances distinct structures while suppressing noise, adaptive thresholding identifies candidate features under varying intensity conditions, and morphological filtering removes weak or noisy artifacts. A connected components (labelling) procedure groups detected features into coherent landmarks. Each landmark is then assigned descriptors that capture its local appearance and spatial context. These descriptors are used later during data association to match landmarks across time.

To generate each local map, the system must know the vehicles pose at each sonar swath. State estimation is therefore tightly integrated with map generation. Among many possible filtering methods, the UKF-M (Unscented Kalman Filter on Manifolds) is selected due to its ease of implementation and its ability to operate without maintaining Jacobians. While an ESKF (Error State Kalman Filter) could also be used, UKF-M provides comparable performance with simpler maintenance and faster convergence in theory. Although UKF-M requires propagating sigma points, modern embedded hardware easily handles this computational load. The result is a stable, real-time, high rate state estimate that supports accurate local map construction.

2.2.4 SLAM

The SLAM subsystem is divided into a front-end and a back-end. The front-end is responsible for preparing all measurements that will eventually be inserted into the global factor graph, a global data structure of sort, while the back-end optimizes this global data structure called graph to produce the best possible estimate of both the trajectory and the landmark map. At a high level, the SLAM system takes all incoming information from the sensors, compresses and structures it into meaningful constraints, and feeds these constraints into a graph based optimiser that continuously refines the state estimate over time.

The first major component of the front-end is IMU preintegration. The IMU runs at a very high frequency, often hundreds of measurements per second, which makes it impractical to insert every acceleration and gyro sample directly into the graph. Doing so would create thousands of redundant nodes per minute, overwhelming the optimizer and providing no practical benefit. Instead of relying on

a traditional Kalman Filter inside the SLAM loop and propagating a state at every IMU timestamp, preintegration accumulates all IMU data between two keyframes (eks between 2 local sonar 2D maps or 2 GNSS updates) into a single compact factor. This factor encodes the relative motion and uncertainty between the two poses, capturing all the information from hundreds of IMU readings without burdening the optimizer. This design is ideal for real-time SLAM because it preserves the information content of the IMU while keeping the graph compact and manageable.

Direct factors, such as GNSS measurements, are added to the graph whenever available. These factors provide absolute position constraints, which help bound long term drift. When a GNSS update arrives, the preintegration is temporarily paused, the GNSS factor is inserted, and the graph is optimized. Preintegration then resumes from the updated pose. This ensures that the system benefits from GNSS whenever it is reliable, while still functioning robustly in GNSS denied or degraded environments. Sonar landmarks offer a similar anchoring effect. When available, they provide strong positional constraints based on environmental structure, meaning the system does not become dependent on GNSS and can operate independently using only the sonar and IMU.

Data association forms the most challenging and delicate part of the front-end. The goal is to determine whether a newly detected landmark corresponds to one already in the map or whether it represents a new feature. Incorrect data association is extremely harmful, a false positive match can warp the map, distort the trajectory, and corrupt long term consistency. Conversely, overly conservative association can cause the system to miss loop closures, leading to drift. To balance reliability and real-time performance, the system splits data association into two layers, local association and global association.

Before performing any hypothesis testing, the system runs a fast descriptor based matching step using FLANN. This step is not required to be perfect, its purpose is simply to filter a large database of landmarks down to a manageable candidate set. If the descriptors suggest that the new landmarks resemble those observed recently, the system performs local data association. Local data tends to be geometrically consistent, and drift over short time periods is small. For this reason, the system uses JCBB (Joint Compatibility Branch and Bound) with tight gating for local association. JCBB is selected because it provides a strong compromise between accuracy and computational cost. Simpler methods, such as NN (Nearest Neighbour) matching, are fast but too unreliable and prone to incorrect matches. More sophisticated methods, such as MHT (Multiple Hypothesis Tracking) or algorithms like MMSAM (Multi Modal Smoothing and Mapping), are extremely robust but far too slow for real-time operation. JCBB occupies the middle ground, it systematically evaluates compatibility between sets of observations and map landmarks while pruning impossible hypotheses early. This allows it to reject false positives while still maintaining real-time performance on small sets of landmarks.

When the descriptors indicate that a landmark may correspond to something observed long ago, possibly implying a loop closure, the system switches to global JCBB. Global association requires extracting landmark to pose and landmark to landmark covariance information from the back-end, which is significantly slower, but since loop closures are rare compared to local updates, the computational cost remains acceptable. Additionally, a periodic loop closure trigger ensures that the system occasionally attempts a global association even if the FLANN matcher has not suggested a promising candidate. This acts as a safety mechanism to avoid missing opportunities to correct accumulated drift.

The SLAM back-end is responsible for optimizing all constraints collected by the front-end. Many different approaches exist for this, such as EKF-SLAM, FastSLAM, batch Gauss-Newton methods, or sliding-window filters. However, the most successful family of algorithms for modern SLAM systems is smoothing based methods. The key idea behind smoothing is that the optimizer uses all available measurements simultaneously, rather than only maintaining a filtered estimate. This produces far more stable and globally consistent results, especially over long trajectories where drift must be corrected reliably. The challenge, however, is that smoothing requires storing and accessing a very large amount of historical information, which demands an extremely efficient data structure to remain computationally feasible.

In SLAM, this information is most naturally and efficiently represented using a factor graph. A factor graph expresses the probabilistic structure of the problem, variable nodes represent robot poses and landmarks, while factors encode sensor measurements and constraints. Importantly, SLAM factor graphs are highly sparse. Each measurement only depends on a small subset of variables, typically a

single pose or a pose to landmark pair. This sparsity reflects the underlying geometry of the problem and is the critical property that makes large scale SLAM solvable in real-time.

To exploit this sparsity, modern smoothing algorithms rely on optimized graph structures that allow fast updates when new data arrives. Among the many available algorithms, iSAM2 has emerged as the most prominent method, especially for landmark based SLAM. iSAM2 exploits the natural sparsity of SLAM factor graphs by reorganizing the graph into an efficient tree structured representation called a Bayes tree. This structure can be solved and updated easily, allowing the optimizer to modify only the affected parts of the solution instead of re-solving the entire graph. As a result, the system remains fast and scalable even as the map grows. iSAM2 is well tested, handles loop closures reliably, and has a mature implementation in GTSAM, making it a practical and robust choice for real-time marine SLAM.

Together, the front-end and back-end form a complete SLAM architecture tailored for real-time marine operations. The front-end carefully selects and structures information so that only high-quality, compact constraints are sent to the optimizer, while the back-end performs continuous refinement to produce a globally consistent trajectory and map. This design ensures robustness, scalability, and real-time capability, which are central goals of this thesis.

2.2.5 Global Map and Trajectory

The final stage of the pipeline constructs the global map and estimated trajectory. Each optimized pose is used to transform the corresponding local map into a shared global frame. As the map grows, overlapping areas from different local tiles are blended, typically by simple averaging of overlapping pixels. This produces a coherent global seafloor mosaic that incorporates all loop closures and optimization corrections. The trajectory drawn over this map provides a clear visualization of the vehicles path and supports downstream tasks such as collision avoidance, mission planning, and situational awareness. Because the global map reflects the entire history of the mission, it can be used not only for real-time navigation but also for offline analysis and long term decision making.

This detailed architecture ties together all major concepts of the thesis. It provides a complete picture of how sensor data flows through the system, how each subsystem transforms that data, and how everything converges into the final optimized map and trajectory. The following chapters examine each of these components in depth, but this overview serves as the structural backbone for understanding the full SLAM pipeline.

3 Sonar Theory

3.1 Introduction

Sonar is one of the primary sensing modalities available in underwater and near surface environments, where visual sensors suffer from turbidity, light attenuation, and poor visibility. To make effective use of sonar measurements in a SLAM pipeline, it is necessary to understand the basic physics behind acoustic wave propagation and how sonar sensors generate, transmit, and receive these signals. This section provides a brief and practical overview of the underlying principles, serving only as a foundation for the later chapters where sonar data is processed. The goal is not to dive deeply into acoustics or advanced modeling, but to establish a clear understanding of how sonar interacts with the environment and why its characteristics matter for mapping.

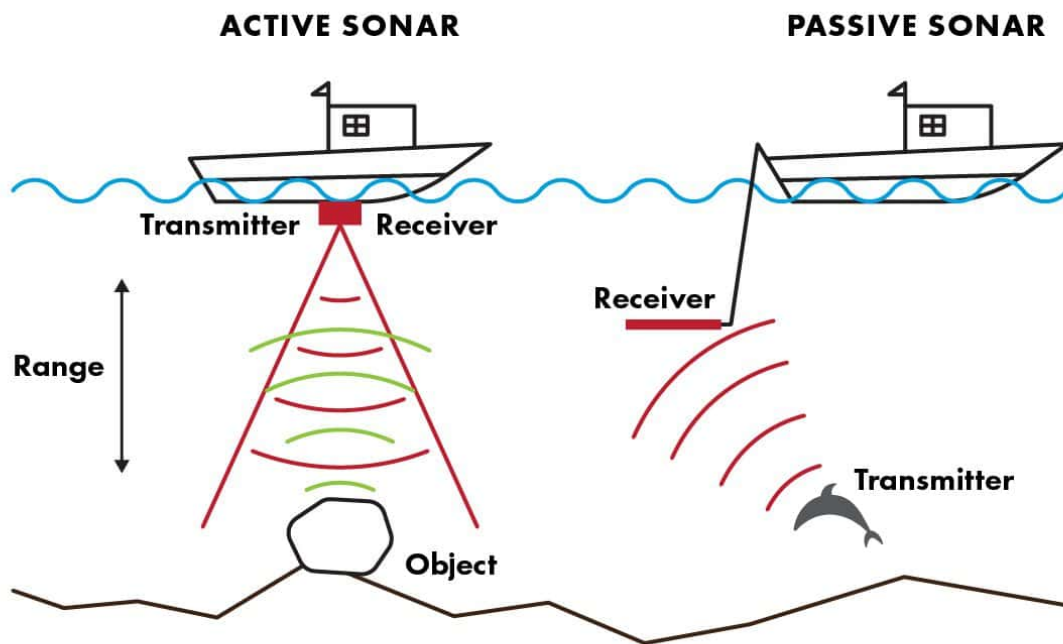


Figure 4: Picture of difference between active and passive sonar, both types can be used for marine navigation, however active is the most used and flexible method for marine navigation with sonar.^[5]

3.2 Acoustic Waves

Sound in water travels as longitudinal pressure waves, generated by small compressions and rarefactions of the surrounding medium. Unlike electromagnetic waves, acoustic waves propagate efficiently through water due to its density and elasticity, resulting in long range transmission and relatively low attenuation. The speed of sound in seawater is approximately 1500 m/s, depending on temperature, salinity, and depth. These parameters slightly modify the propagation path and timing, but for most practical applications in coastal robotics, the speed is assumed constant.

Acoustic waves follow simple geometric spreading, as they travel outward, energy is distributed over a larger area, causing the received amplitude to decrease with range. Additional attenuation arises from absorption in the water and scattering from suspended particles or biological matter. When a sound pulse reaches the seafloor or an object, part of the energy reflects back toward the sensor. The strength and timing of this reflection depend on material properties, surface roughness, angle of incidence, and the distance traveled. These principles form the basis of sonar imaging, where the measured return intensity encodes information about the environment.

Because sonar relies on timing and amplitude of returning echoes, understanding propagation and reflection is essential for interpreting range measurements, correcting distortions, and aligning sonar data with vehicle motion. The raw acoustic behavior directly influences the quality and structure of the maps produced later.

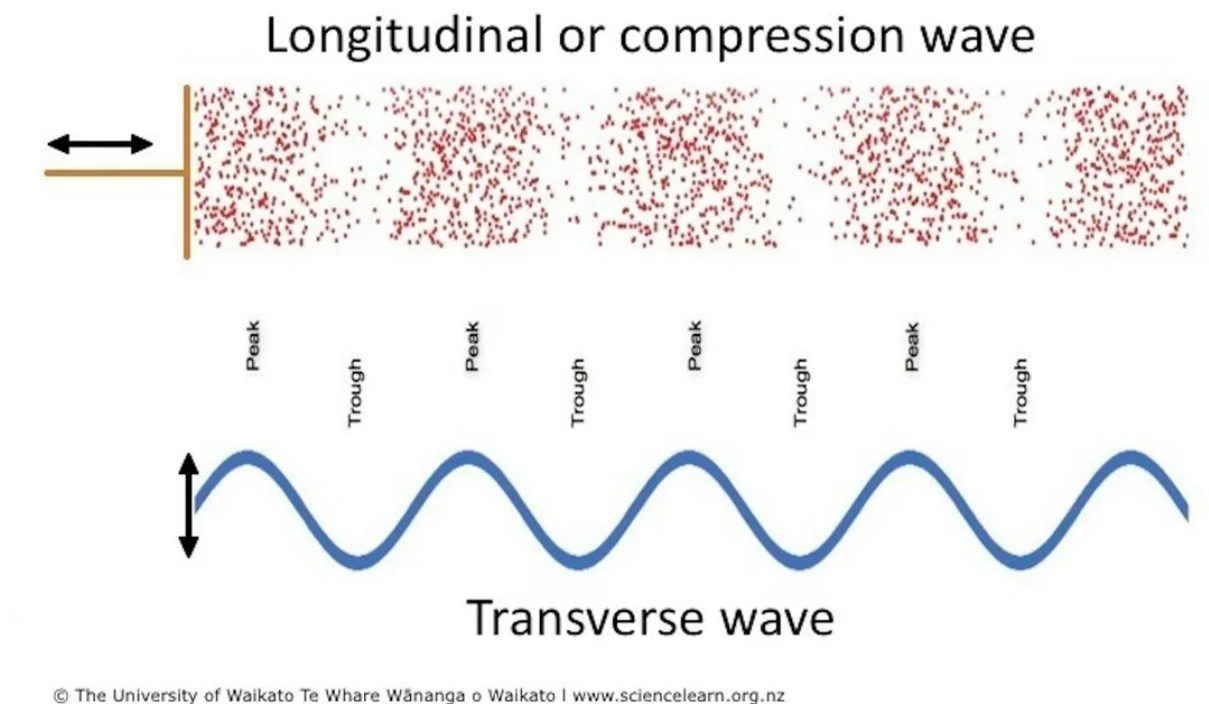


Figure 5: Illustration of acoustic wave propagation, showing how sound travels through a medium as alternating regions of compression and rarefaction. These pressure variations move outward as a longitudinal wave and can be detected by underwater piezoelectric ceramic transducers.^[6]

3.3 Sonar Fundamentals

A sonar system consists of one or more transducers made from piezoelectric ceramic materials. When an electrical signal is applied, the ceramic deforms and produces a mechanical vibration that emits an acoustic wave into the water. Conversely, when an incoming acoustic wave reaches the transducer, the ceramic deforms and generates a small electrical signal. This signal is then amplified and digitized by the receiver electronics. The process is simple at its core, electrical energy becomes sound, sound interacts with the environment, and the returning sound becomes an electrical signal again.

Modern sonar systems emit short pulses (chirps) whose frequency content is optimized for resolution and range. High frequencies provide fine detail but attenuate faster. Lower frequencies travel farther but offer lower resolution. Side scan sonar typically uses high frequency chirps (hundreds of kHz) to produce detailed seafloor imagery. The transmitter and receiver assemblies are arranged so that each ping illuminates a narrow swath of the seabed, and the received intensity profile forms a single scan line. By stacking these scan lines over time as the vessel moves, the system produces a 2D representation of the seafloor.

Sonar measurements are inherently sensitive to orientation, altitude, and motion of the platform. Pitch and roll change the insonified region, while altitude affects the projection from slant range to true ground range. These effects create distortions that must be corrected before any mapping or SLAM step, which is why the early parts of the SLAM data processing pipeline apply normalization and geometric corrections to create stable and accurate seafloor maps.

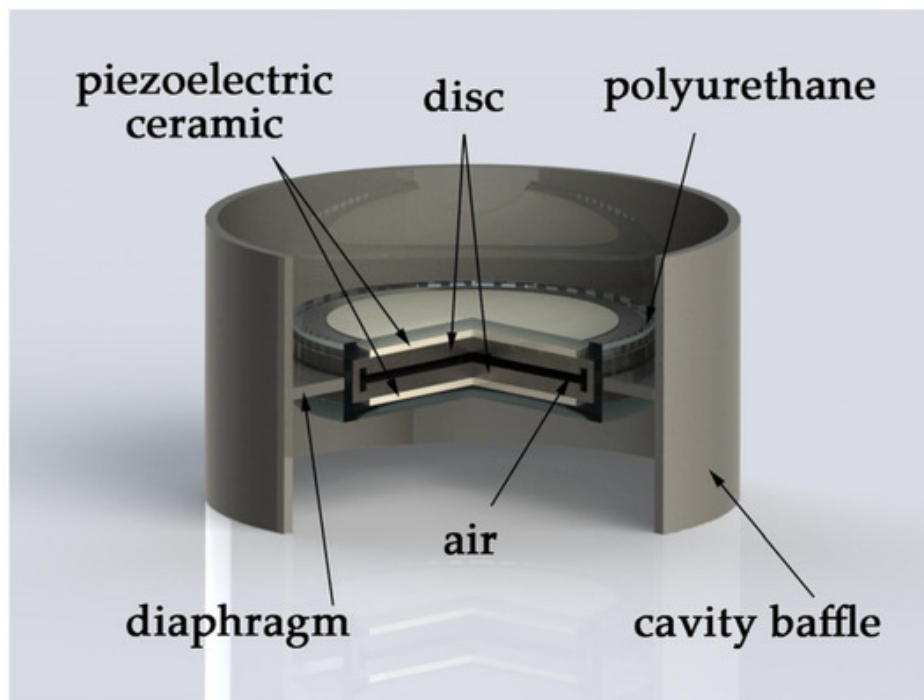


Figure 6: Picture shows construction of a piezoelectric ceramic transducer. Applying an electrical voltage causes the material to deform and generate acoustic vibrations, while incoming pressure variations induce mechanical deformation that produces a measurable electrical signal. These low voltage signals are later amplified and processed by the sonar electronics.^[7]

3.4 Exteroceptive Sensors and Side Scan Sonar

Exteroceptive sensors measure the external world, and in robotics this typically includes cameras, LiDAR, radar, and sonar. Cameras and LiDAR rely on light and are extremely effective in clear air environments, but their performance degrades sharply underwater. Sound, however, propagates efficiently in water and allows perception even in dark, turbid, or visually obstructed conditions. Side scan sonar is a specialized form of acoustic imaging designed to capture high resolution seafloor information across wide swaths, making it particularly useful for marine mapping and navigation tasks.

A side scan system emits two fan shaped beams to port and starboard, measuring the intensity of acoustic returns across the seabed. These intensity patterns reveal structure, texture, and objects, which makes the sensor ideal for environmental perception and feature extraction. In a SLAM context, the side scan data forms a sequence of local 2D images that are later transformed into Cartesian grid maps. These local maps are used for landmark detection, data association, loop closure reasoning, and finally global map construction.

Using side scan sonar for SLAM also enables downstream applications such as obstacle avoidance, seabed classification, and autonomous mission planning. Once a consistent global map and trajectory are produced, the system can identify hazardous areas, detect objects of interest, and support high level decision making for the autonomous vessel. Thus, understanding the fundamental sensing mechanism of side scan sonar is essential for leveraging its data effectively throughout the mapping pipeline.

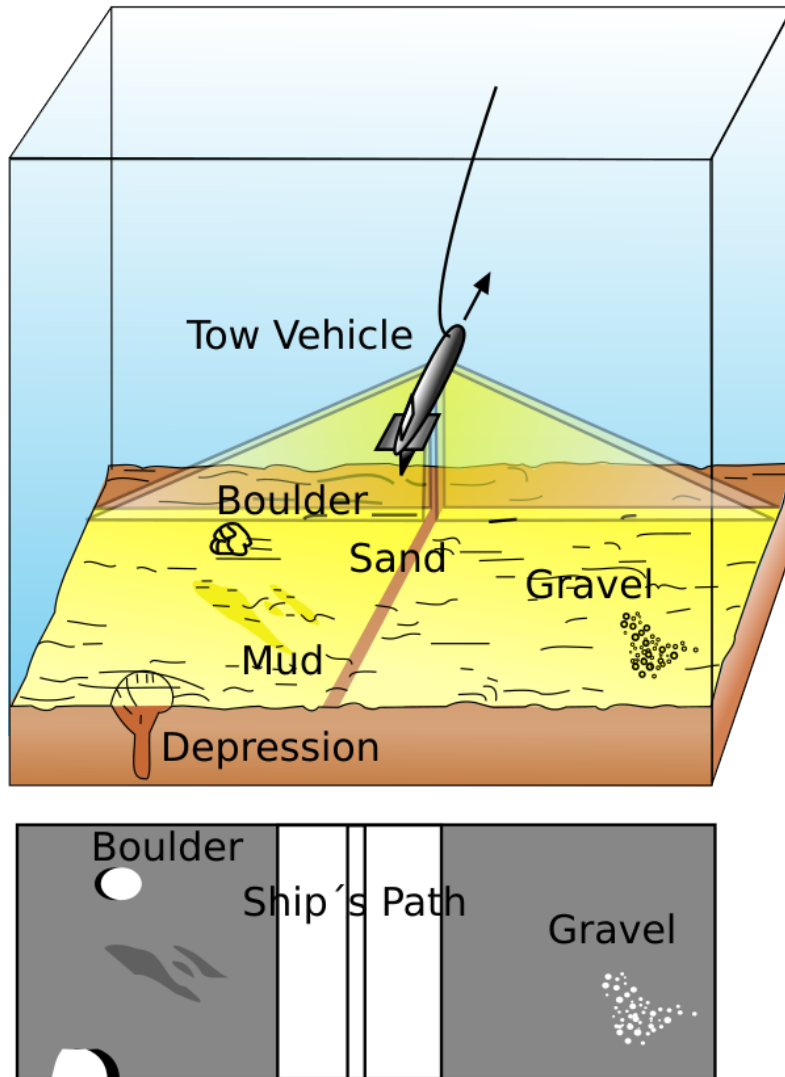


Figure 7: Diagram of side scan sonar.^[8]

4 Hardware

4.1 Introduction

The hardware used in this thesis forms the physical foundation for deploying and evaluating the side-scan sonar SLAM system on the microAmpere ASV. Since all SLAM components, from state estimation to mapping and optimization, depend directly on sensor performance, timing accuracy, and onboard compute capability, understanding the underlying hardware is essential for interpreting the systems overall constraints and behavior.

This chapter introduces the microAmpere ASV platform and highlights the elements most relevant for Side Scan Sonar SLAM (SSS SLAM) integration. The vessels distributed computing architecture, built around synchronized embedded systems, enables real-time processing of inertial, GNSS, and sonar data. Accurate timing, reliable networking, and deterministic data pathways are critical for fusing measurements into a coherent SLAM pipeline, and these characteristics are outlined here to set the operational context.

The chapter also presents the sensor suite used for navigation and mapping. The IMU, GNSS, and side scan sonar define the primary data sources for the SLAM algorithm, each contributing complementary information about motion or the external environment. Their placement, interfaces, and data rates determine how the system must model, synchronize, and process measurements later in the pipeline.

Finally, the hardware introduction positions the microAmpere platform within the broader flow of the thesis. The design choices, limitations, and capabilities described here provide the foundation for the following chapters on system modeling, state estimation, sonar processing, and full SLAM architecture. By establishing the practical constraints early, the thesis ensures that the proposed SLAM system is aligned with the realities of embedded marine robotics.

4.2 microAmpere ASV

This section focuses on the “*microAmpere Autonomous Surface Vessel (ASV)*” platform, which serves as the primary research and development base for this work. The information presented here is summarized from two detailed master theses, Luan Cao Vo Trans thesis on microAmpere hardware and integration [9], and Henrik Reimers thesis on control and system design [10]. More detailed technical descriptions can be found in the referenced theses.

In this section, only the most relevant aspects of the platform are presented, specifically those that are essential for integrating a side scan sonar payload and for enabling SSS SLAM. The focus is therefore on the hardware architecture, computing layout, and synchronization systems that directly affect sonar data acquisition and processing.

The microAmpere is an advanced Autonomous Surface Vessel (ASV) developed at NTNU as a modular platform for autonomous docking and maritime robotics research. It is part of the Autodocking25 project and represents the next evolution of the Blue Robotics BlueBoat platform. The vessel was redesigned to support high performance perception, navigation, and control tasks under realistic marine conditions. Its main purpose is to provide a flexible and open research platform for experimenting with autonomous operations such as docking, waypoint navigation, and sensor fusion at sea.

Physically, the vessel is based on the BlueBoat twin hull design, offering high stability and sufficient deck space for custom payloads. The hulls are constructed from High Density Polyethylene (HDPE), ensuring durability and resistance to saltwater corrosion. The superstructure houses one watertight enclosures containing the compute units, sensor interfaces, and power management systems. This enclosure is designed for quick maintenance and allows for modular upgrades for new hardware.

The system integrates a distributed computing architecture consisting of a “*Jetson Orin NX*” for GPU intensive perception tasks such as LiDAR and camera processing, and a “*LattePanda 3 Delta*” Single Board Computer (SBC) for real-time navigation, control, and communication handling. Both systems run ROS2 with synchronized clocks via GNSS and PPS hardware timing, achieving sub microsecond precision across devices. This allows accurate sensor timestamping and time alignment between the navigation, control, and perception subsystems.

The sensor suite includes dual GNSS antennas for accurate heading estimation, a high grade IMU for attitude and acceleration measurements, an “*Ouster OS1*” LiDAR for 3D environmental mapping, and “*StereoLabs ZED-X*” stereo cameras for visual perception and teleoperation. A dedicated power sensing board monitors voltage and current to ensure system safety, and an industrial “*Teltonika RUTX50*” router provides 4G/5G and GNSS connectivity, extending operational range far beyond line of sight limits. All components are interconnected through a structured internal wiring system with proper isolation, grounding, and surge protection to handle harsh marine environments.

The MicroAmperes software stack is fully containerized, combining real-time control loops with modular high level autonomy nodes. Each module communicates through standardized ROS2 topics and services, enabling independent algorithm development without system reconfiguration. This architecture also facilitates rapid iteration and field testing of advanced autonomy features such as Model Predictive Control (MPC), Reinforcement Learning (RL), and Simultaneous Localization and Mapping (SLAM).

In addition to autonomy research, the microAmpere platform is used for multi sensor data collection and algorithm benchmarking under controlled sea trials. The synchronized data from LiDAR, stereo vision, IMU, and GNSS enables high quality datasets for perception, sensor fusion, and state estimation studies. The vessels modular interfaces also make it suitable for integration with experimental payloads such as side scan sonar, underwater acoustic systems, or alternative control computers for specific research tasks.

From a systems engineering perspective, microAmpere demonstrates the effectiveness of distributed embedded computing in marine robotics. By separating perception and control workloads, developers can independently optimize performance for GPU heavy machine learning pipelines and deterministic control processes. The modular ROS2 setup also supports simulation-in-the-loop and hardware-in-the-loop testing, ensuring smooth transition from virtual models to field deployment.

According to Luan Cao Vo Tran master thesis on microAmpere ASV [9] and Henrik Reimers master thesis on microAmpere ASV [10], the platform has already undergone multiple field tests in Trondheim Fjord. These tests validated microAmpere ASVs hardware robustness, networking reliability, and autonomy stack. Its success has positioned it as a reference platform for NTNUs future autonomous surface vessel research, bridging efforts between the Department of Cybernetics and the AUR Lab.

Overall, the microAmpere ASV serves as a compact but powerful research platform, bridging the gap between simulation and real world autonomous marine systems. Its modular design, distributed computation, and precise timing infrastructure make it ideal for experimentation with advanced autonomy, perception, and control algorithms in challenging maritime environments.



Figure 8: A picture of microAmpere Autonomous Surface Vessel (ASV).^[9]

4.3 Specifications

The microAmpere ASV was designed as a flexible and modular research platform, combining reliable marine hardware with powerful embedded computing and sensing capabilities. Its specification focuses on providing stable, high precision navigation and perception for real-time autonomy experiments in nearshore and harbor environments. The systems modularity allows researchers to swap components and integrate additional sensors without structural modification. The vessels configuration and components are summarized in Table 2 down below.

Table 2: Technical specifications of the microAmpere ASV [9]

Category	Specification
Platform Base	Blue Robotics BlueBoat twin hull (HDPE construction)
Dimensions	Length: 1.8 m Width: 1.1 m Draft: 0.25 m
Weight	Approx. 45 kg (fully equipped)
Propulsion	Dual Blue Robotics T200 thrusters with ESC control, differential thrust for steering
Power Supply	Dual 6S Li-Ion batteries (22.2 V nominal, 20 Ah each), hot-swappable configuration
Compute Units	NVIDIA Jetson Orin NX (perception and LiDAR processing) LattePanda 3 Delta (navigation, control, and communication)
Operating System	Ubuntu 24.04 LTS with ROS2 Jazzy Jalisco
Networking	Teltonika RUTX50 router (4G/5G, Wi-Fi, and GNSS) with Tailscale VPN
Localization Sensors	Dual u-blox ZED-F9P GNSS modules with RTK correction Xsens MTi-680G high-precision IMU
Perception Sensors	Ouster OS1-64 LiDAR StereoLabs ZED-X stereo camera pair
Timing and Synchronization	PPS signal distribution and PTP-based synchronization via custom timing board
Power Monitoring	Custom power sensing PCB (voltage, current, and temperature feedback)
Communication Interfaces	CAN, Ethernet, USB 3.0, Serial (RS232/RS485), GPIO
Software Capabilities	Real-time navigation, control, diagnostics, and data logging Modular autonomy framework for MPC, RL, and SLAM integration
Field Operation	Endurance: ~ 3 hours under typical load Max speed: ~ 2.5 m/s Remote or fully autonomous operation modes

The specifications highlight the microAmpere balance between portability and computational power. The distributed computing setup provides sufficient resources for advanced autonomy tasks while maintaining low latency in the control loop. The integrated sensors enable precise positioning and rich perception, making it an ideal testbed for advanced control, SLAM, and sensor fusion algorithms in dynamic maritime environments.

4.4 Layout

The microAmperes internal and external layout is designed to achieve an optimal balance between functionality, modularity, and serviceability. The vessel is divided into three main physical sections, consisting of the two hulls and the central watertight enclosure. Each section has a specific role in the system: propulsion and power are located in the hulls, computation and communication in the watertight enclosure, and perception sensors are distributed around the vessel. This structure simplifies cabling, improves center-of-gravity stability, and provides a clean and organized platform for integrating research equipment.

The port hull houses the port side battery pack and Electronic Speed Controller (ESC) that drive the left thruster, while the starboard hull contains the mirrored setup together with a Power Sense Module responsible for voltage and current monitoring. Both hulls are interconnected through a reinforced cross tube that carries power, Pulse Width Modulation (PWM), and Ethernet lines to the watertight enclosure. This cross connection ensures a robust and low latency interface between propulsion, control, and sensor systems, while maintaining mechanical symmetry and balance across the hulls.

On the top deck, the vessel supports a LiDAR platform for an unobstructed 360° field of view, in addition a set of stereo cameras, mounted at each hull to give better situational awareness. These sensors enable simultaneous environmental mapping and perception. Additional top mounted features include power switch, diagnostic status LEDs, and the antennas for GNSS, 5G, and radio communication. The overall sensor placement was chosen to ensure optimal visibility, minimal interference between radio and optical systems, and ease of access during maintenance or calibration.

Externally, all antennas and sensors are symmetrically arranged to guarantee signal balance and redundancy. The vessel is equipped with dual GNSS antennas mounted at the rear of each hull (stern) for precise position and heading estimation, a 5G antenna for long range communication, and a radio antenna for local telemetry.

The internal layout focuses on structured modularity, with all high power and propulsion related systems isolated within the hulls, and all sensitive electronics centralized in the watertight enclosure. This separation improves electromagnetic compatibility and enhances maintainability during field operations. The vessels internal wiring is routed through marine grade cable glands, organized in cable harnesses with labeled connectors for quick service and component replacement.

In addition, active cooling fans are installed at the rear of the hulls and inside the watertight enclosure to maintain safe operating temperatures for both computation and power electronics. Proper thermal design and airflow direction have been validated through practical testing to ensure stability under extended mission durations and high computational loads.

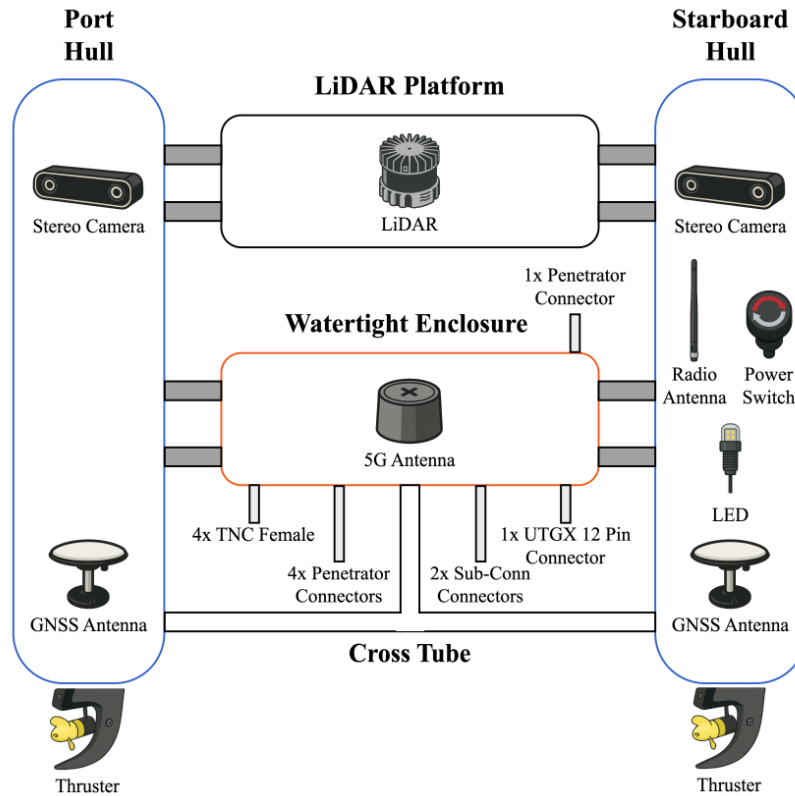


Figure 9: Top level overview of the microAmpere vessel showing the placement of cameras, antennas, thrusters, power switches, status LEDs, and LiDAR platform.^[9]

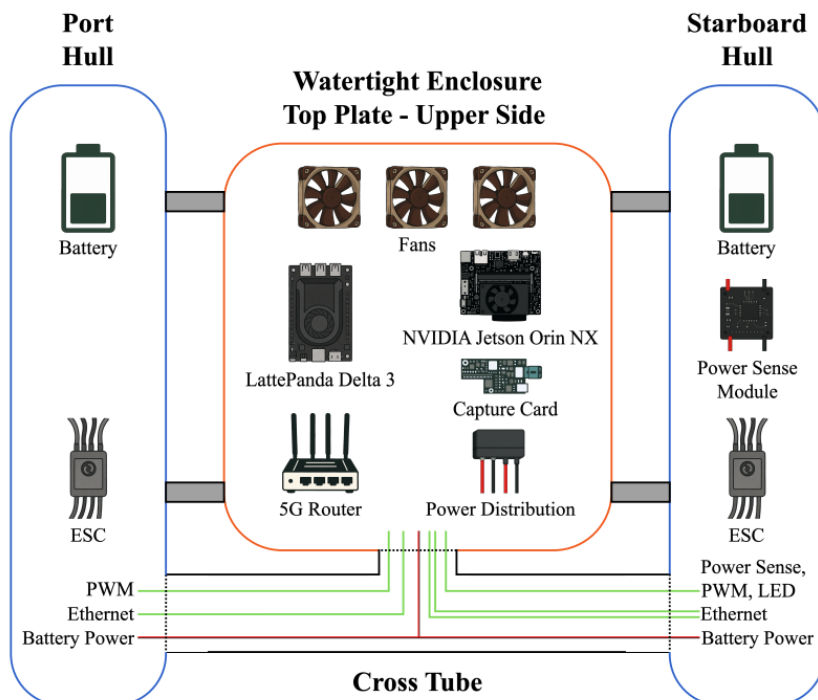


Figure 10: Bottom view of microAmpere showing the internal layout of batteries, ESCs, cooling fans, SBCs, capture card, and power distribution units for optimized performance and accessibility.^[9]

Focusing on the watertight enclosure, this section contains all critical computational, power conversion, and synchronization electronics of the microAmpere system. The internal structure follows a two layer plexiglass mounting design with a top and bottom plate. This setup maximizes thermal efficiency, ensures proper airflow, and allows easy access for maintenance or hardware upgrades.

The top plate upper side holds the main compute stack, including the “*NVIDIA Jetson Orin NX*”, “*LattePanda 3 Delta*”, and the “*Teltonika RUTX50 5G router*”. These components are mounted alongside a power distribution module and a set of “*Noctua 60 mm cooling fans*” for optimal airflow and thermal stability during intensive computational loads. The placement of these units also provides clear cable routing paths and short communication links between processing and networking hardware.

The underside of the top plate contains the “*DC/DC converters*” rated at 30 W and 150 W, a “*fuse board*”, and the fan bracket assembly. These converters supply stable 12 V and 5 V lines to different subsystems including the compute boards, router, Sentiboard, and auxiliary electronics. The components are arranged for efficient heat dissipation and quick replacement in case of hardware maintenance.

The bottom plate top side integrates the primary navigation and synchronization modules. It includes the “*Xsens MTi-680G IMU*”, dual “*u-blox ZED-F9P GNSS receivers*”, and the custom “*Sentiboard*” used for precise hardware timestamping and signal synchronization. The stacked layout minimizes wiring complexity, reduces signal interference, and maintains clear access for diagnostics and firmware updates.

Overall, the watertight enclosure layout emphasizes modularity, cooling efficiency, and reliability. By separating compute, power, and navigation layers, the design reduces thermal coupling between components and improves system robustness during long duration field operations. This configuration allows the microAmpere to operate safely under variable weather and computational conditions while maintaining high data integrity across all subsystems.

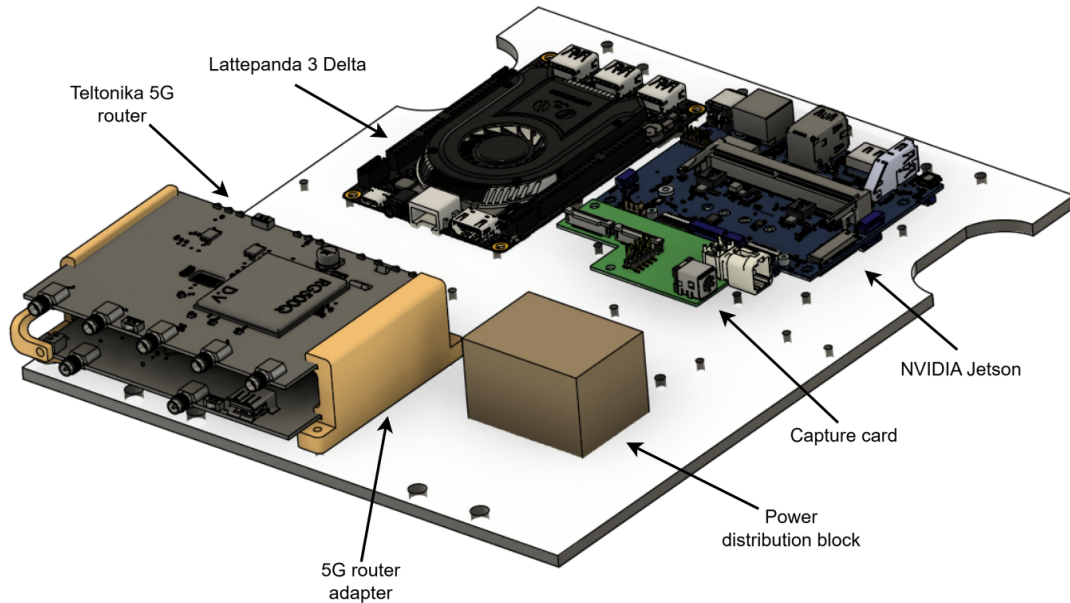


Figure 11: Watertight enclosure top plate (upper side) showing the main compute stack with Jetson Orin NX, LattePanda 3 Delta, and Teltonika RUTX50 router.^[9]

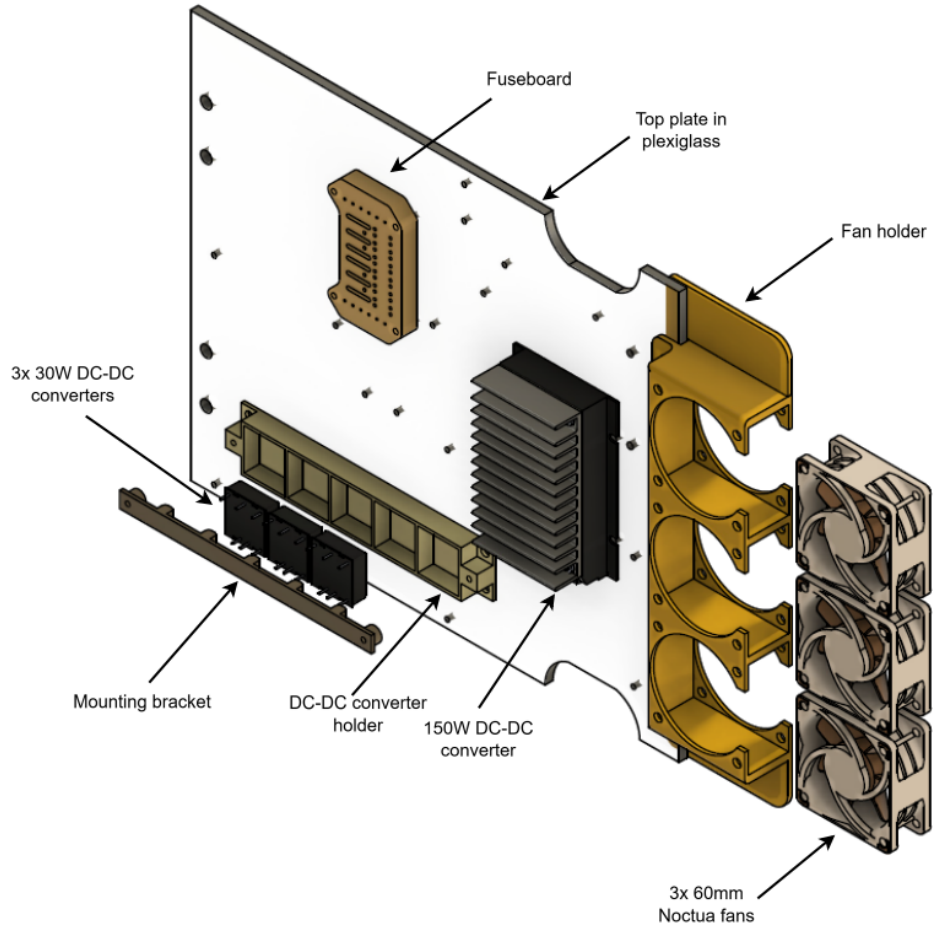


Figure 12: Underside of the top plate showing the DC/DC converters, fuse board, and cooling fans for power regulation and heat management.^[9]

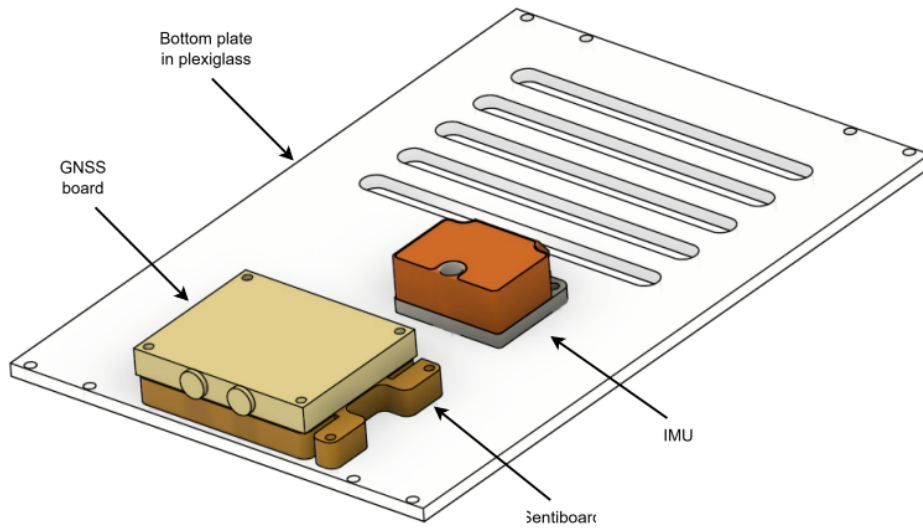


Figure 13: Bottom plate layout containing the IMU, Sentiboard, and GNSS receivers used for precise navigation and time synchronization.^[9]

4.5 Sensors

4.5.1 Introduction

The microAmpere platform is equipped with a wide range of sensors supporting navigation, perception, and control tasks. For this work, the focus lies on the integration of a Side Scan Sonar and the development of SLAM. Two of the existing sensors play a key role in enabling this, the Inertial Measurement Unit (IMU) and the Global Navigation Satellite System (GNSS). These sensors together provide accurate state estimation and position tracking, forming the foundation required for precise sonar data alignment, georeferencing, and mapping.

4.5.2 Inertial Measurement Unit (IMU) for Navigation and Pose Estimation

The “*Sensoror STIM300*” is the primary Inertial Measurement Unit (IMU) onboard microAmpere. It is a tactical grade sensor providing precise angular rate and linear acceleration measurements for dead reckoning, attitude estimation, and control stabilization. Its compact and rugged design makes it well suited for marine robotics, where vibration, temperature variation, and magnetic disturbances are common. The unit integrates three high precision gyroscopes and three accelerometers, enabling full 3-axis motion sensing with excellent mechanical and thermal stability.

The STIM300 offers a ± 400 $^{\circ}/s$ gyroscope range and ± 10 g accelerometer range with 24-bit resolution, ensuring low noise and stable long term performance. It communicates via an RS-422 interface, operates from a 5 V supply, and supports sampling rates up to 2 kHz with deterministic timing. Its low bias instability and minimal drift allow reliable short term dead reckoning when GNSS signals are unavailable, making it a key component for autonomous control and future side scan sonar-based SLAM integration.

An overview of the IMU is shown in Figure 14, and its key specifications are summarized in Table 3.



Figure 14: Sensoror STIM300 high-performance IMU used for internal navigation and pose estimation.^[9]

Table 3: Sensoror STIM300 IMU specifications summary.^[11]

Parameter	Value
Gyroscope range	± 400 $^{\circ}/s$
Accelerometer range	± 10 g
Resolution	24-bit
Max data rate	2 kHz
Interface	RS-422
Input voltage	5 V
Weight	55 g

4.5.3 GNSS for Positioning and Heading Estimation

For accurate positioning and heading estimation, microAmpere employs two “*u-blox ZED-F9P*” GNSS modules integrated on a Dual GNSS card from SentiSystems. These modules support Real-Time Kinematic (RTK) corrections for centimeter level positioning accuracy and operate in a moving base configuration to estimate heading from the relative position of two GNSS antennas.

The external “*SignalPlus*” GNSS antennas provide reliable satellite reception and minimize multipath effects common in harbor environments. With a baseline distance of approximately 0.7m between antennas, the system achieves heading accuracy of around 0.5° , while maintaining position accuracy down to 1.5m, or 0.01m with RTK corrections. This configuration ensures precise navigation and robust performance even under challenging marine conditions.

Together with the onboard IMU, the dual GNSS system forms the foundation for the vessels navigation, control, and future SSS SLAM integration. The Dual GNSS card and antennas are shown in Figure 15, and the system specifications are summarized in Table 4.

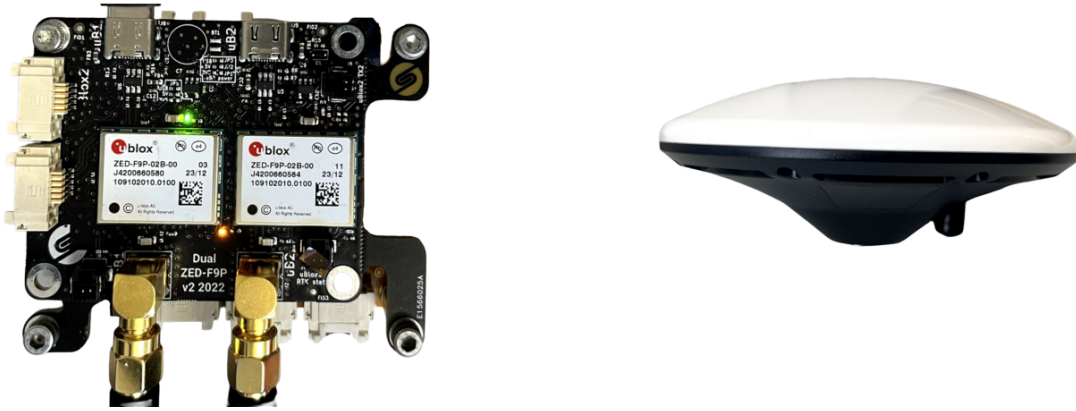


Figure 15: (Left) SentiSystems Dual GNSS card with two u-blox ZED-F9P modules. (Right) SignalPlus GNSS antennas used for position and heading estimation.^[9]

Table 4: GNSS module specifications summary.^[12]

Parameter	ZED-F9P
Position accuracy (CEP)	1.5 m (0.01 m with RTK)
Heading accuracy	0.4° (baseline 1.8 m)
RTK capability	Base + Moving Base
Satellite bands	L1, L2, L5
Max navigation rate	25 Hz (5 Hz moving base)

4.5.4 Side Scan Sonar for Environmental Perception and SLAM

The “*DeepVision OSM Ethernet Sonar System*” is the side scan sonar used onboard the microAmpere platform for seabed imaging, mapping, and SLAM development. Developed by DeepVision AB, this system operates with dual 680 kHz transducers utilizing chirp modulation to produce high resolution acoustic imagery. It supports a maximum operating depth of 100 m and can achieve image resolutions down to 1 cm depending on configuration settings.

The OSM sonar was previously evaluated by Hogstad, Bjørnar Reitan in his masters thesis at NTNU, where it was integrated on a BlueROV2 platform with an Error State Kalman Filter (ESKF) for autonomous navigation and acoustic mapping [3]. In that setup, the transducers were mounted symmetrically and angled downward at $\theta = 45^\circ$, providing a vertical beam width of $\alpha = 60^\circ$ and a horizontal beam width of $\phi = 0.7^\circ$. This configuration allowed both wide area coverage and high spatial resolution across the seafloor.

The sonar connects via Ethernet to the onboard computer and is automatically recognized by the “*DeepView LT*” software for real-time imaging, control, and data logging. Its high operating frequency and narrow horizontal beam enable precise seafloor mapping, while the wide vertical beam ensures efficient coverage for environmental perception. This makes it highly suitable for integration into the microAmpere ASVs perception stack, supporting the development of SSS SLAM and environment aware autonomy.

An overview of the sonar system is shown in Figure 16, and its main specifications are summarized in Table 5.



Figure 16: DeepVision OSM Ethernet Side-Scan Sonar system with dual 680 kHz transducers used for seabed imaging and SLAM development.^[3]

Table 5: DeepVision OSM Ethernet Sonar System specifications summary.^[3]

Parameter	Value
Operating frequency	680 kHz (Chirp signal)
Resolution	Down to 1 cm
Maximum operating depth	100 m
Vertical beam width (α)	60°
Horizontal beam width (ϕ)	0.7°
Mounting angle (θ)	45°
Swath range	Up to 50 m per side (100 m total)
Interface	Ethernet
Software	DeepView LT
Manufacturer	DeepVision AB

4.6 Software Architecture

Although not strictly part of the hardware system, the software architecture is what connects, manages, and coordinates all onboard components. It defines how the sensors, actuators, and computational units communicate to form a complete autonomous system. The design follows a modular and distributed approach, separating computational loads across multiple processors to improve reliability, maintainability, and scalability. This structure also allows new hardware, such as the side scan sonar, to be integrated seamlessly without major reconfiguration of existing modules.

Each subsystem is assigned a specific computational domain with a well-defined purpose. The *“Navigation Unit”* on the *“LattePanda 3 Delta”* is responsible for low-level control, sensor fusion, and actuator management, interfacing directly with the IMU, GNSS, and thruster controllers. The *“Perception Unit”* on the *“NVIDIA Jetson Orin NX”* handles high bandwidth sensors such as LiDAR and stereo cameras, performing real-time perception, object detection, and mapping. The *“Ground Control Station”* provides remote supervision, telemetry visualization, and manual override when required. Together, these systems operate in a synchronized local Ethernet network, exchanging data through a standardized message layer.

The software stack runs on *“Ubuntu 24.04 LTS”* with the *“ROS2 Jazzy Jalisco”* distribution. ROS2 acts as the communication backbone, providing a real-time publish/subscribe framework through the Data Distribution Service (DDS) middleware. Each functional block in the system is implemented as an independent ROS2 node that communicates via topics, services, and actions. This decoupled design simplifies debugging, system upgrades, and integration of experimental modules for research.

A key part of the architecture is precise time synchronization between all onboard computers. The system uses a hardware signal called Pulse Per Second (PPS) from the GNSS modules to align the internal clocks of each computer. This signal marks the exact start of every second and is combined with the GNSS time data to keep all systems synchronized. A background process running in Linux automatically adjusts the system clocks based on this information, ensuring that all sensor data are timestamped with the same reference time. This sub-microsecond synchronization is essential for accurate sensor fusion, navigation, and SLAM.

An overview of the complete software architecture and node interconnections is shown in Figure 17 down below on the next page.

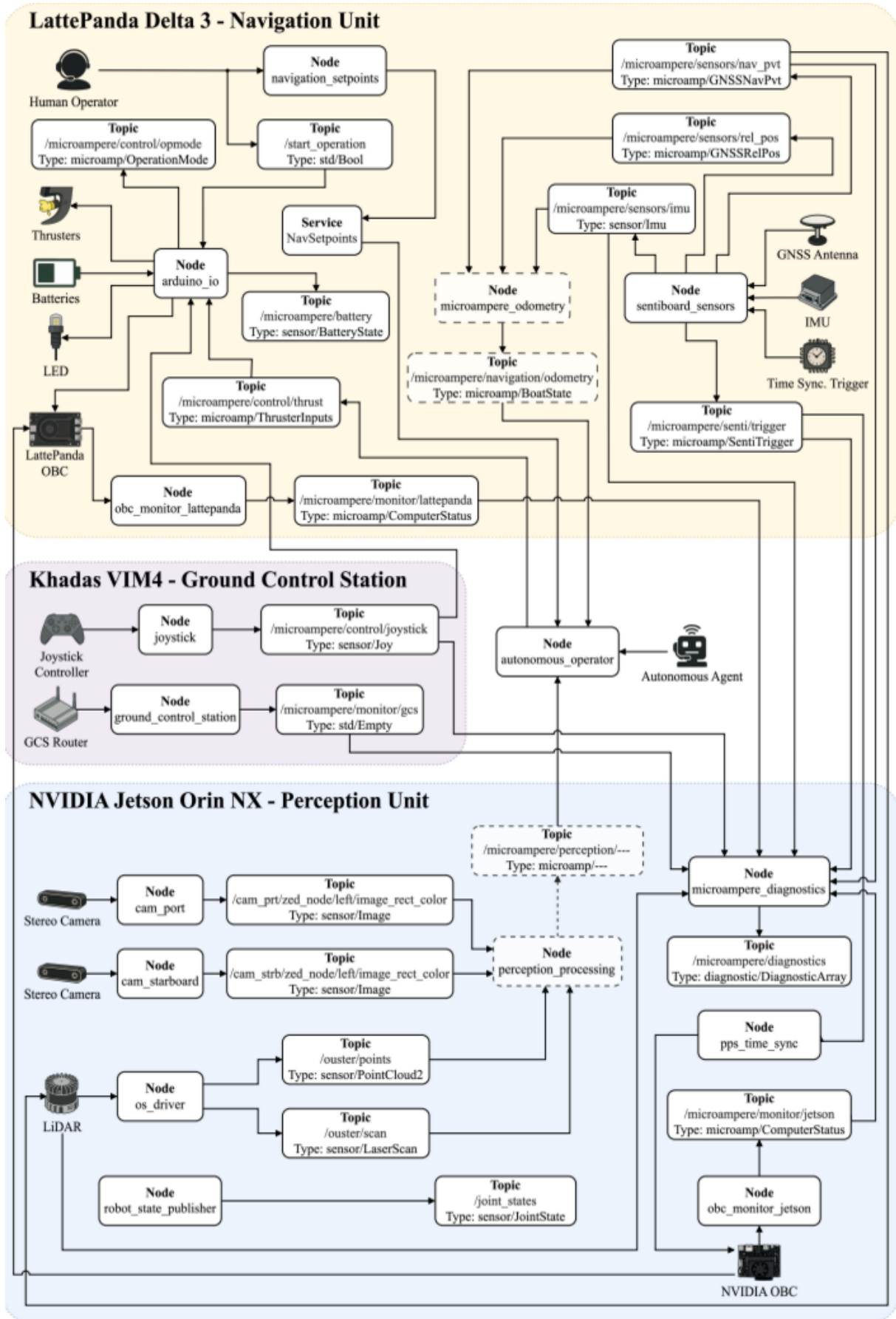


Figure 17: Overview of the software architecture for the microAmpere ASV, showing the distributed communication between perception, navigation, and control units.^[9]

5 System Modeling

5.1 Introduction

Reliable navigation and mapping for an autonomous vessel depend on accurate mathematical models that describe how the vessel moves and how its sensors perceive the surrounding environment. These models form the foundation for state estimation, control, and SLAM algorithms, ensuring that all physical quantities such as position, velocity, and orientation are consistently defined and propagated over time.

The modeling framework is divided into two complementary parts, kinematics and dynamics. Kinematics focuses on the geometric description of motion without considering the forces that cause it. It defines how position, velocity, and orientation are represented and transformed between coordinate frames. Global reference frames such as the World Geodetic System 1984 (WGS84) and the Earth Centered Earth Fixed (ECEF) frame describe the Earth geometry and provide absolute positioning. Local frames such as the North East Down (NED) and Body frames define the vessel motion relative to its own orientation and surroundings. The transformation chain between these frames ensures that all measured and estimated quantities are consistently expressed and can be converted between global and local coordinates.

Dynamics extend the kinematic framework by describing how forces and moments act on the vessel to generate motion. This relationship forms the basis of the motion models used in navigation and estimation. Two main formulations are considered. The Marine Craft Model by Thor Inge Fossen [13] provides a comprehensive six degree of freedom (6 DOF) description that captures hydrodynamic effects, external disturbances, and control inputs, making it ideal for simulation and model based control. The Inertial Navigation System (INS) model used here follows the simplified sensor driven formulation presented in Fundamentals of Sensor Fusion by Edmund Brekke [14], relying directly on IMU measurements. This approach is better suited for real-time operation and SLAM applications, where computational efficiency and robustness to environmental uncertainty are prioritized over detailed hydrodynamic accuracy.

GNSS is modeled as a direct noisy position measurement in the NED frame. The landmark model uses a 2D range-bearing formulation, with both a simple deterministic version and an extended variant that accounts for sonar specific effects, providing geometric constraints that help correct drift during SLAM.

The complete mathematical framework connects these kinematic and dynamic representations into a unified structure. It defines how vessel states are represented, transformed, and propagated in time using rotation formulations such as Euler angles, quaternions, and Lie group representations. It establishes consistent conventions between global and local reference frames and expresses the relationships between linear and angular motion necessary for deriving velocities, accelerations, and time derivatives of position and orientation.

To achieve stable and accurate temporal evolution of the state, numerical solvers are used to integrate the underlying differential equations. Classical integration schemes such as Newton-Euler and Runge-Kutta methods are employed to propagate the vessel dynamics in discrete time while minimizing numerical drift and maintaining physical consistency. The choice of solver has a significant impact on model accuracy, especially in real-time applications where integration stability and computational efficiency determine overall system performance.

5.2 Orientation Representations

5.2.1 Euler Angles

Euler angles provide a simple and intuitive method for representing three dimensional orientation through a sequence of rotations about the principal axes. The orientation of a rigid body is described by three angles corresponding to yaw, pitch, and roll, which define the body's rotation relative to a fixed reference frame. These angles are widely used in navigation and control applications due to their clear geometric interpretation and direct relationship to measurable quantities such as heading and inclination.

In the North East Down (NED) convention commonly used for marine and aerial vehicles, the rotation sequence is typically defined as a rotation about the z -axis (yaw, ψ), followed by the y -axis (pitch, θ), and finally the x -axis (roll, ϕ). This corresponds to the rotation matrix:

$$R = R_x(\phi)R_y(\theta)R_z(\psi)$$

$$R_x(\phi) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \phi & -\sin \phi \\ 0 & \sin \phi & \cos \phi \end{bmatrix}, \quad R_y(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ -\sin \theta & 0 & \cos \theta \end{bmatrix}, \quad R_z(\psi) = \begin{bmatrix} \cos \psi & -\sin \psi & 0 \\ \sin \psi & \cos \psi & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Combining these gives the full NED transformation from the body frame to the navigation frame:

$$R_b^n = \begin{bmatrix} c_\theta c_\psi & s_\phi s_\theta c_\psi - c_\phi s_\psi & c_\phi s_\theta c_\psi + s_\phi s_\psi \\ c_\theta s_\psi & s_\phi s_\theta s_\psi + c_\phi c_\psi & c_\phi s_\theta s_\psi - s_\phi c_\psi \\ -s_\theta & s_\phi c_\theta & c_\phi c_\theta \end{bmatrix}$$

where $s_\bullet = \sin(\bullet)$ and $c_\bullet = \cos(\bullet)$ for brevity. The NED frame transformation can be visualized in Figure 18, which illustrates the intrinsic yaw-pitch-roll rotation sequence following the NED convention. This sequence shows how the body frame is successively rotated about the global z -axes, y -axes, and x -axes to achieve the desired orientation.

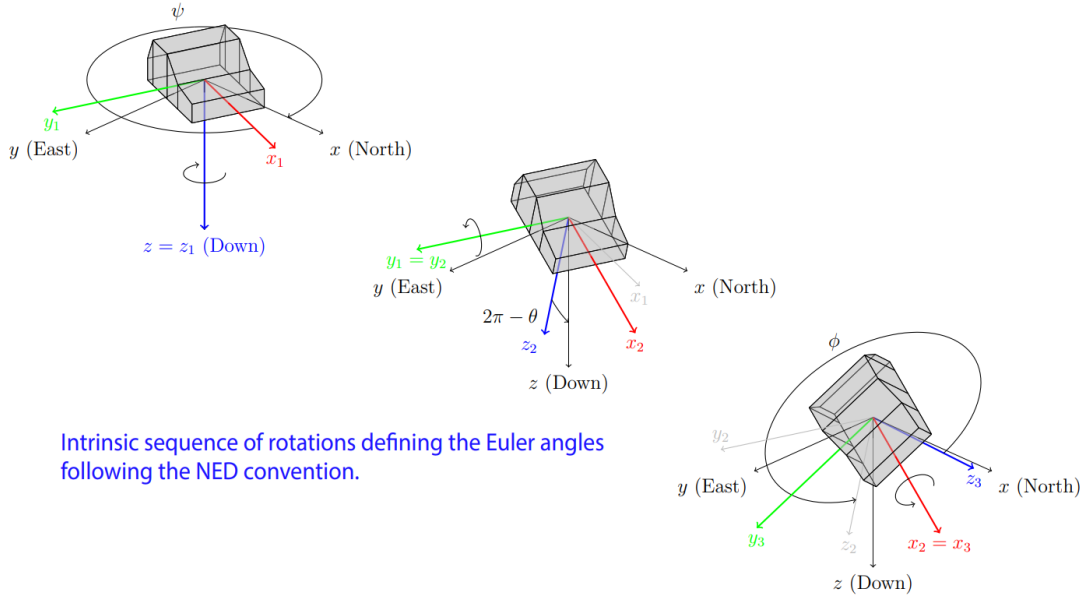


Figure 18: Intrinsic sequence of rotations defining the Euler angles following the NED convention.^[14]

The main advantages of Euler angles are their simplicity, minimal parameter count, and intuitive physical meaning. Each angle directly corresponds to an observable rotational motion, which simplifies both system design and debugging. They also integrate naturally with classical control systems, where angular rates are easily expressed as time derivatives of Euler angles.

However, the Euler representation suffers from a singularity problem known as “*gimbal lock*”, which in the NED convention occurs when the pitch angle approaches $\theta = \pm 90^\circ$. At this configuration, the yaw and roll axes align, resulting in the loss of one rotational degree of freedom. In this state,

small changes in pitch can cause disproportionately large or undefined changes in yaw and roll rates. Mathematically, the transformation matrix that relates Euler angle rates to body angular velocities becomes ill-conditioned, leading to singularities where the angular rate terms tend toward infinity. This introduces numerical instability and unreliable attitude estimates, which are particularly problematic for navigation and control systems relying on smooth and continuous angular motion.

The relationship between the body angular velocity vector $\boldsymbol{\omega}_b = [p \ q \ r]^T$ and the Euler angle rate vector $\dot{\boldsymbol{\Theta}} = [\dot{\phi} \ \dot{\theta} \ \dot{\psi}]^T$ for the NED yaw-pitch-roll convention is expressed as:

$$\begin{bmatrix} p \\ q \\ r \end{bmatrix} = \begin{bmatrix} 1 & 0 & -\sin \theta \\ 0 & \cos \phi & \sin \phi \cos \theta \\ 0 & -\sin \phi & \cos \phi \cos \theta \end{bmatrix} \begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix}$$

When the pitch angle approaches $\theta = \pm 90^\circ$, the cosine term $\cos \theta$ tends to zero, causing the matrix to lose rank and become singular. As a result, any attempt to compute Euler rate derivatives or integrate attitude dynamics at this configuration leads to undefined or infinite angular velocities.

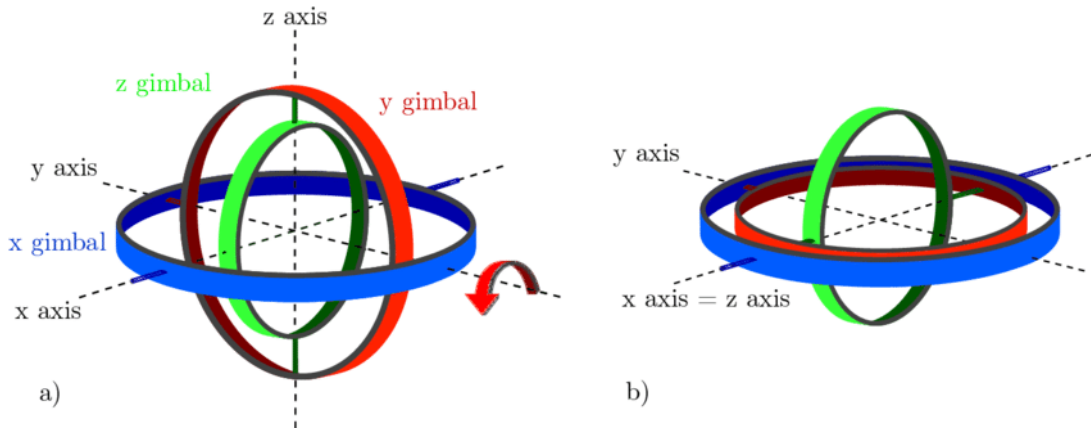


Figure 19: Illustration of gimbal lock where the yaw and roll axes coincide, causing loss of one rotational degree of freedom.^[15]

Despite this limitation, Euler angles remain a practical choice for surface and marine robotics, where pitch and roll angles typically remain small. Their intuitive interpretation and computational simplicity make them well suited for real-time estimation and control near level operating conditions. In the case of an ASV, large attitude excursions are highly unlikely, as excessive roll or pitch would indicate a loss of stability or complete capsizing. Under such operational constraints, the singularity at $\theta = \pm 90^\circ$ is never encountered, making the Euler representation both sufficient and efficient for navigation and SLAM applications.

5.2.2 Quaternions

Quaternions provide a compact and singularity free alternative to Euler angles for representing three dimensional orientations. They extend complex numbers into four dimensions and are defined as a scalar and a three component vector

$$\mathbf{q} = \begin{bmatrix} q_w \\ q_x \\ q_y \\ q_z \end{bmatrix} = \begin{bmatrix} \cos \frac{\theta}{2} \\ \hat{u}_x \sin \frac{\theta}{2} \\ \hat{u}_y \sin \frac{\theta}{2} \\ \hat{u}_z \sin \frac{\theta}{2} \end{bmatrix}$$

where θ is the rotation angle and $\hat{u} = [\hat{u}_x \ \hat{u}_y \ \hat{u}_z]^T$ is the unit vector defining the rotation axis. The quaternion $\mathbf{q}$ therefore represents a rotation of θ radians about $\hat{u}$. To ensure that it encodes a valid rotation, it must satisfy the unit norm constraint

$$\|\mathbf{q}\| = \sqrt{q_w^2 + q_x^2 + q_y^2 + q_z^2} = 1$$

Normalization is typically enforced after each update step in numerical integration or filtering to avoid drift due to floating point errors. This is done by dividing the quaternion by its magnitude:

$$\mathbf{q}_{\text{normalized}} = \frac{\mathbf{q}}{\|\mathbf{q}\|} = \frac{1}{\sqrt{q_w^2 + q_x^2 + q_y^2 + q_z^2}} \begin{bmatrix} q_w \\ q_x \\ q_y \\ q_z \end{bmatrix}$$

A geometric interpretation of quaternions can be seen in Figure 20. Quaternions can be viewed as points on the unit four dimensional hypersphere S^3 , composed of a real component and a three dimensional imaginary subspace spanned by (i, j, k) . The real component represents the cosine of half the rotation angle, while the imaginary vector component encodes the rotation axis scaled by the sine of half the angle. Together, they define a rotation in three dimensional space through the relation $\mathbf{q} = \cos \frac{\theta}{2} + \hat{u} \sin \frac{\theta}{2}$. Each point on this hypersphere corresponds to a unique orientation of a rigid body, and continuous motion along its surface represents a smooth change in attitude without encountering any discontinuities or singularities.

When a vector $\mathbf{v}$ is rotated using $\mathbf{q}\mathbf{v}\mathbf{q}^{-1}$, the operation effectively applies a double rotation, first through $\mathbf{q}$ and then through its conjugate $\mathbf{q}^{-1}$, producing a single pure 3D rotation about the axis $\hat{u}$ by an angle θ . The scalar part $\cos(\theta/2)$ defines the rotation magnitude, while the vector part $\sin(\theta/2)\hat{u}$ specifies the rotation axis in the imaginary (i, j, k) space. This representation forms the basis for quaternion based attitude kinematics used in estimation and control.

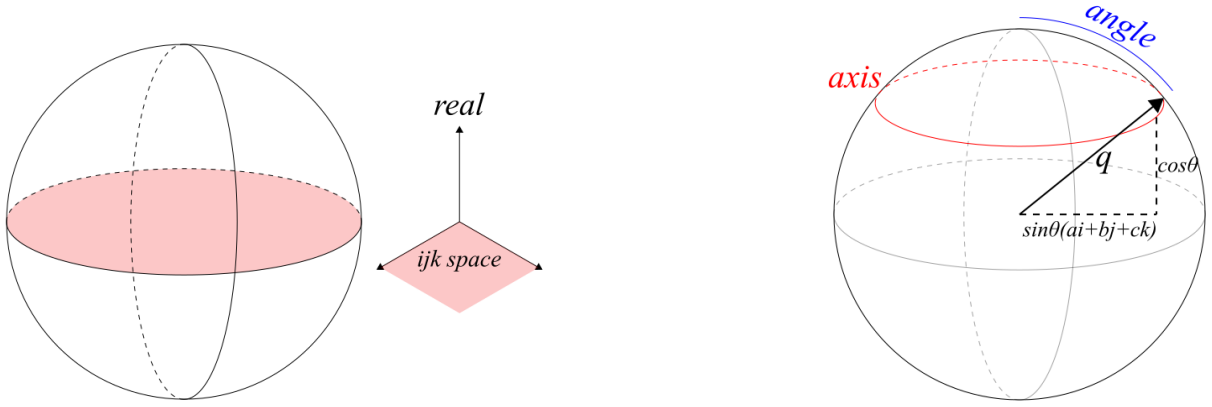


Figure 20: Geometric interpretation of quaternions on the unit hypersphere S^3 . (Left) Decomposition into the real axis and imaginary (i, j, k) subspace. (Right) Axis-angle representation showing how a quaternion encodes rotation by θ about the unit axis $\hat{u}$.^[16]

Rotations using quaternions are performed through quaternion multiplication, which is associative but not commutative. Given two orientations $\mathbf{q}_1$ and $\mathbf{q}_2$, their combined rotation is expressed as

$$\mathbf{q}_{\text{combined}} = \mathbf{q}_2 \otimes \mathbf{q}_1$$

where $\otimes$ denotes the quaternion product. The rotation of a vector $\mathbf{v}$ in three dimensional space can then be performed as

$$\mathbf{v}' = \mathbf{q} \otimes \mathbf{v}_q \otimes \mathbf{q}^{-1}$$

where $\mathbf{v}_q = [0 \ v_x \ v_y \ v_z]^T$ is the quaternion form of the vector $\mathbf{v}$. This operation applies the rotation represented by $\mathbf{q}$ to $\mathbf{v}$ in a smooth and continuous manner without encountering singularities.

In practical applications, quaternions are widely used in estimators and sensors where singularity free orientation handling is critical. The IMU on the microAmpere ASV platform outputs attitude in quaternion form, which integrates naturally with sensor fusion algorithms such as the Error State Kalman Filter (ESKF) discussed later in the report. These algorithms rely on quaternion algebra to represent small attitude perturbations efficiently and avoid numerical instability associated with Euler angle singularities.

The main advantages of quaternions include their smooth and continuous representation of orientation, absence of gimbal lock, and computational efficiency in rotation composition and interpolation. They maintain numerical stability under integration and are well suited for optimization and estimation algorithms. However, quaternions require one additional parameter compared to Euler angles and lack intuitive physical meaning, as their four components do not directly correspond to measurable angular quantities. Furthermore, they are less straightforward to interpret and manipulate mathematically, often requiring specialized operations such as normalization, conjugation, and noncommutative multiplication. Despite these challenges, their robustness and compatibility with modern sensor fusion and control frameworks make quaternions a preferred internal representation for real-time navigation and estimation systems.

5.2.3 Lie Groups and Manifolds

Lie groups provide a mathematical framework for representing continuous and smooth transformations such as rotation and translation in three dimensional space. They combine the properties of algebraic groups and differentiable manifolds, allowing both analytical manipulation and geometric interpretation of motion. In navigation, control, and robotics, Lie groups enable consistent handling of orientation and pose without the discontinuities or ambiguities present in minimal representations such as Euler angles.

The rotation group $SO(3)$, called the Special Orthogonal Group, represents all possible 3D rotations. Each rotation is described by a 3×3 matrix R that satisfies

$$R^T R = I, \quad \det(R) = 1$$

This group is smooth and continuous, meaning that small changes in orientation correspond to small movements on the manifold. To describe small or incremental rotations, $SO(3)$ is associated with its tangent space, the Lie algebra $\mathfrak{so}(3)$.

Elements of $\mathfrak{so}(3)$ are skew-symmetric matrices that encode infinitesimal rotations. A rotation vector $\boldsymbol{\omega} = [\omega_x \ \omega_y \ \omega_z]^T$ can be written as

$$\boldsymbol{\omega}^\times = \begin{bmatrix} 0 & -\omega_z & \omega_y \\ \omega_z & 0 & -\omega_x \\ -\omega_y & \omega_x & 0 \end{bmatrix}$$

This matrix form is simply another way of expressing the cross product $\boldsymbol{\omega} \times \mathbf{v}$, which rotates a vector $\mathbf{v}$ by a small angle.

The exponential map connects this local representation to an actual finite rotation on $SO(3)$:

$$R = \exp(\boldsymbol{\omega}^\times) \quad (Tangent \rightarrow Manifold) \quad (1)$$

And the inverse operation (the logarithmic map) retrieves the corresponding small rotation from R :

$$\boldsymbol{\omega} = \log(R) \quad (Tangent \leftarrow Manifold) \quad (2)$$

These two functions allow smooth transitions between local angular velocity representations and full 3D orientations, which is highly useful for filters and optimizers for SLAM.

To include translation as well as rotation, the concept extends to $SE(3)$, the so called Special Euclidean Group, which describes full 3D rigid body motion:

$$T = \begin{bmatrix} R & \mathbf{p} \\ 0 & 1 \end{bmatrix}, \quad T \in SE(3) \quad (3)$$

Here R is orientation and $\mathbf{p}$ is position. This group provides a consistent mathematical way to represent a vehicles full pose and combine both rotational and translational motion.

In robotics and navigation, $SO(3)$ and $SE(3)$ are used to describe smooth and continuous motion in three dimensional space. $SO(3)$ represents pure rotations, while $SE(3)$ extends this to include both rotation and translation, forming the full rigid body pose. These groups define motion directly on the

manifold, ensuring mathematically consistent and globally valid transformations without singularities.

The curved surface of the $SE(3)$ manifold represents all possible poses of a rigid body in space. The tangent plane at the identity, denoted $\mathfrak{se}(3)$, represents small local motions that can be combined and integrated into full poses using the exponential map $\exp(\cdot)$. Conversely, the logarithmic map $\log(\cdot)$ converts a pose difference back into this local space. Together, these mappings allow smooth transitions between small local displacements and full 3D transformations, which is essential for analyzing and composing motion in robotics and control.

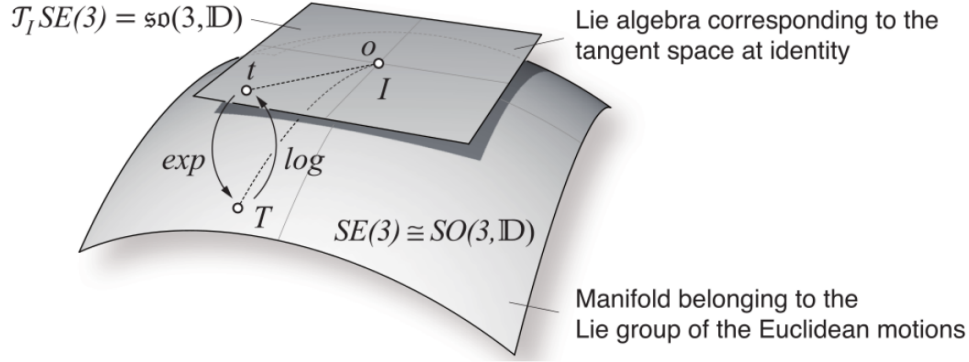


Figure 21: Visualization of the $SE(3)$ manifold and its tangent space $\mathfrak{se}(3)$. Small motions are represented in the tangent space and mapped to the manifold through the exponential map $\exp(\cdot)$, while the logarithmic map $\log(\cdot)$ projects manifold elements back to the local linear space.^[17]

Lie group formulations are fundamental in modern SLAM systems because they provide a mathematically consistent way to represent and manipulate poses in three dimensions. When building and updating a map, each robot or camera pose is an element of $SE(3)$, combining both rotation and translation into a single compact structure. This allows pose composition, inversion, and differentiation to be performed directly on the manifold without approximations or singularities. In practice, this means that motion updates, sensor transformations, and loop closure corrections can all be handled through clean matrix operations that are both efficient and numerically stable.

Using Lie groups also enables fast and scalable computation, a necessity when optimizing over thousands of poses and landmarks in large scale SLAM problems. The same mathematical principles are applied in computer graphics and 3D rendering, where $SE(3)$ transformations are used to efficiently manipulate hundreds or thousands of objects and vertices in real time. By working on the manifold, transformations remain consistent regardless of scale or complexity, ensuring that rotations and translations compose correctly without distortion.

Overall, Lie group representations allow SLAM and mapping algorithms to combine geometry, motion, and optimization within a single unified framework. They ensure that the estimated trajectory and map remain globally consistent, even after many iterations of motion and correction, making them the foundation of accurate and robust 3D perception in robotics.

5.2.4 Handy Conversions

The following conversion formulas are taken directly from Edmund Brekke's book on "*Fundamentals of Sensor Fusion*" [14]. They are commonly used to convert between different orientation representations for navigation, control, and estimation.

Euler to Quaternion

$$\mathbf{q} = \begin{bmatrix} q_w \\ q_x \\ q_y \\ q_z \end{bmatrix} = \begin{bmatrix} \cos \frac{\phi}{2} \cos \frac{\theta}{2} \cos \frac{\psi}{2} + \sin \frac{\phi}{2} \sin \frac{\theta}{2} \sin \frac{\psi}{2} \\ \sin \frac{\phi}{2} \cos \frac{\theta}{2} \cos \frac{\psi}{2} - \cos \frac{\phi}{2} \sin \frac{\theta}{2} \sin \frac{\psi}{2} \\ \cos \frac{\phi}{2} \sin \frac{\theta}{2} \cos \frac{\psi}{2} + \sin \frac{\phi}{2} \cos \frac{\theta}{2} \sin \frac{\psi}{2} \\ \cos \frac{\phi}{2} \cos \frac{\theta}{2} \sin \frac{\psi}{2} - \sin \frac{\phi}{2} \sin \frac{\theta}{2} \cos \frac{\psi}{2} \end{bmatrix}$$

Quaternion to Euler

$$\begin{bmatrix} \phi \\ \theta \\ \psi \end{bmatrix} = \begin{bmatrix} \text{atan2}(2(q_w q_x + q_y q_z), 1 - 2(q_x^2 + q_y^2)) \\ \text{asin}(2(q_w q_y - q_z q_x)) \\ \text{atan2}(2(q_w q_z + q_x q_y), 1 - 2(q_y^2 + q_z^2)) \end{bmatrix}$$

Euler to SO(3)

$$R_{from}^{to} = R_b^n = R_x(\phi)R_y(\theta)R_z(\psi)$$

$$R_x(\phi) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \phi & -\sin \phi \\ 0 & \sin \phi & \cos \phi \end{bmatrix}, \quad R_y(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ -\sin \theta & 0 & \cos \theta \end{bmatrix}, \quad R_z(\psi) = \begin{bmatrix} \cos \psi & -\sin \psi & 0 \\ \sin \psi & \cos \psi & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Quaternion to SO(3)

$$R_{from}^{to} = R_b^n = \begin{bmatrix} 1 - 2(q_y^2 + q_z^2) & 2(q_x q_y - q_z q_w) & 2(q_x q_z + q_y q_w) \\ 2(q_x q_y + q_z q_w) & 1 - 2(q_x^2 + q_z^2) & 2(q_y q_z - q_x q_w) \\ 2(q_x q_z - q_y q_w) & 2(q_y q_z + q_x q_w) & 1 - 2(q_x^2 + q_y^2) \end{bmatrix}$$

These relationships provide a convenient reference for converting between orientation representations depending on the application, whether for computation, visualization, or integration with sensor data.

5.3 Reference Frames and Transformations

5.3.1 Basic Translation and Rotation Transformations

Rigid body motion in three dimensions can be expressed compactly using the homogeneous transformation matrix $T \in \text{SE}(3)$ defined in Equation 3. These matrices combine rotation and translation into a single representation and are used to map coordinates between frames such as the navigation (NED), body, and sensor frames.

The pose of the body frame relative to the navigation frame is written as

$$T_b^n = \begin{bmatrix} R_b^n & \mathbf{p}_b^n \\ 0 & 1 \end{bmatrix} \quad (4)$$

where R_b^n is the rotation matrix from the body to the NED frame, and $\mathbf{p}_b^n$ is the position of the body origin expressed in NED coordinates.

A point $\mathbf{x}_b^b$ expressed in the body frame can be transformed to the NED frame as:

$$\mathbf{x}_b^n = T_b^n \begin{bmatrix} \mathbf{x}_b^b \\ 1 \end{bmatrix} = R_b^n \mathbf{x}_b^b + \mathbf{p}_b^n \quad (5)$$

Composing multiple transformations is performed through matrix multiplication. For instance, if T_s^b defines the transformation from a sensor frame to the body frame, then the sensor pose in the NED frame becomes

$$T_s^n = T_b^n T_s^b$$

The inverse transformation, which converts coordinates from NED back to the body frame, is given by

$$T_n^b = (T_b^n)^{-1} = \begin{bmatrix} (R_b^n)^T & -(R_b^n)^T \mathbf{p}_b^n \\ 0 & 1 \end{bmatrix}$$

This matrix formulation provides a clean and consistent way to represent spatial relationships between the navigation, body, and sensor frames. It is fundamental in robotics, navigation, and control, where accurate frame alignment and transformation chaining are essential for pose estimation and sensor fusion.

These homogeneous transformations form the foundation for defining global and local reference frames such as WGS84, ECEF, NED, and Body, which are described in the following sections

5.3.2 Global Reference Frames

Global reference frames provide the foundation for representing absolute positions on Earth and are essential for all GNSS based navigation and mapping systems. Since GNSS receivers provide position estimates in a global Earth fixed frame, while navigation and control systems typically operate in local frames such as NED or body coordinates, it becomes necessary to define consistent global reference models to enable accurate transformations between these coordinate systems.

The two main reference systems used for this purpose are the World Geodetic System 1984 (WGS84) geodetic model, which defines the Earths ellipsoidal shape and geodetic coordinates (latitude, longitude, altitude), and the Earth-Centered Earth-Fixed (ECEF) Cartesian frame, which expresses these same positions as 3D Cartesian coordinates centered at the Earths center of mass. These systems provide the common link between GNSS measurements and local navigation frames used in estimation and control.

WGS84

The World Geodetic System 1984 (WGS84) is the global geodetic reference used by all major GNSS systems. It defines an Earth-fixed ellipsoidal model that closely approximates the planet's mean shape, accounting for the equatorial bulge caused by rotation. Although regional realizations such as “*ETRS89*” (European Terrestrial Reference System 1989) are used in Europe to reduce tectonic drift, the WGS84 reference remains the standard for GNSS based navigation in Norway and most marine applications, with differences between the two systems being only a few decimeters.

The WGS84 model defines the Earth as an oblate ellipsoid, flattened at the poles and expanded at the equator. It serves as the global geodetic reference for GPS and most modern satellite navigation systems. The ellipsoid parameters are defined as $a = 6378137.0$ m and $f = 1/298.257223563$, where a is the semi major axis and f is the flattening factor. The semi minor axis can then be computed as $b = a(1 - f)$.

A position on Earth is defined by three geodetic coordinates. First the latitude φ , which is the angle between the equatorial plane and the ellipsoid normal. Then the longitude λ , which is the angle between the Greenwich meridian and the projection of the point onto the equatorial plane. Finally the altitude h , which is the height above the ellipsoidal surface. These coordinates describe a point's global position and are the standard output format from all GNSS receivers.

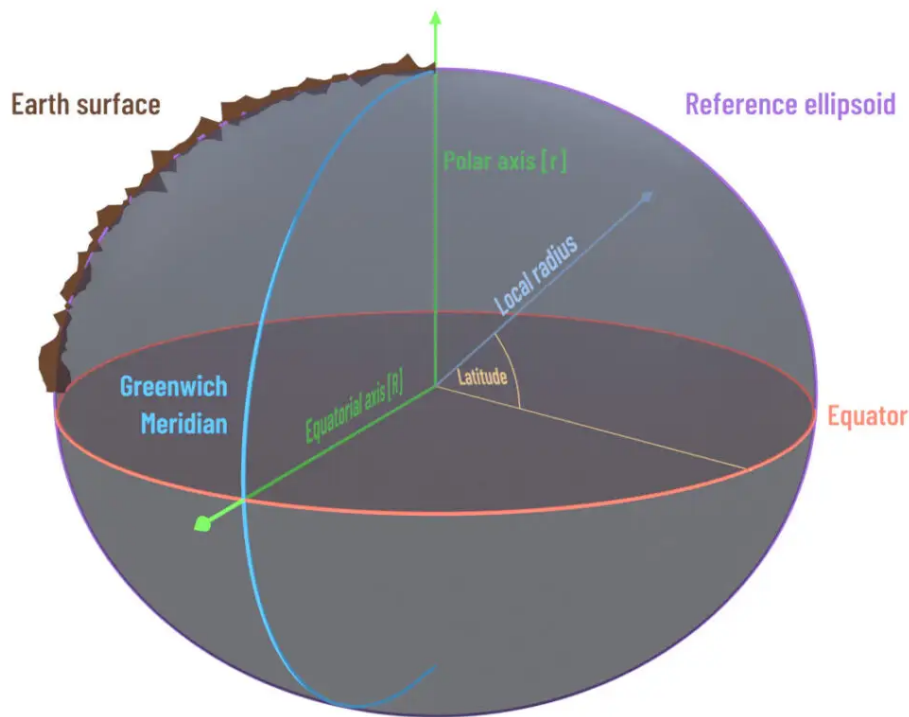


Figure 22: WGS84 ellipsoidal Earth model showing the relationship between latitude, longitude, and altitude.^[18]

ECEF

The Earth-Centered Earth-Fixed (ECEF) frame is a three dimensional Cartesian coordinate system with its origin at the Earth's center of mass. The x -axis passes through the intersection of the equator and the Greenwich meridian, the y -axis lies along the equator 90° east of the x -axis, and the z -axis points toward the North Pole. The ECEF frame rotates with the Earth and is commonly used for expressing GNSS positions in Cartesian form.

The transformation from geodetic coordinates WGS84 (φ, λ, h) to ECEF coordinates (x, y, z) is given by:

$$\begin{aligned} e^2 &= 2f - f^2, \\ N(\varphi) &= \frac{a}{\sqrt{1 - e^2 \sin^2 \varphi}}, \\ x &= (N(\varphi) + h) \cos \varphi \cos \lambda, \\ y &= (N(\varphi) + h) \cos \varphi \sin \lambda, \\ z &= [(1 - e^2)N(\varphi) + h] \sin \varphi \end{aligned}$$

where $N(\varphi)$ is the prime vertical radius of curvature and e is the first eccentricity of the ellipsoid. The inverse conversion from ECEF to geodetic coordinates is typically solved iteratively due to the nonlinear nature of the equations.

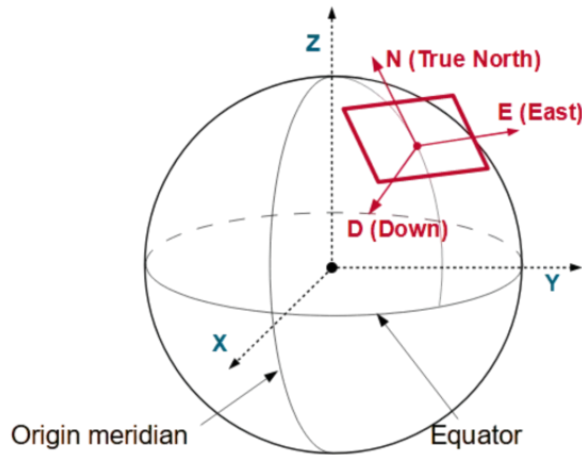


Figure 23: Earth-Centered Earth-Fixed (ECEF) coordinate system with origin at Earth's center and axes aligned with the equator and rotational axis.^[19]

The WGS84 and ECEF frames together form the foundation of all global navigation and positioning systems. While WGS84 defines the geometric reference ellipsoid used by GNSS measurements, ECEF provides a Cartesian coordinate representation that is better suited for numerical computation, sensor fusion, and integration with local navigation frames such as NED or body fixed coordinates.

5.3.3 Local Reference Frames

Local reference frames provide a convenient way to describe vehicle motion and orientation for navigation, estimation, and control. Marine and aerial systems commonly use the North East Down (NED) frame and the Body frame to represent position, velocity, and attitude consistently. The NED frame is a local tangent frame with axes pointing north, east, and down, aligned with compass directions and suited for interpreting GNSS and IMU data. Its origin is typically placed near the vehicles initial position.

The Body frame is fixed to the vehicle, with x forward, y to the right, and z downward, usually defined at the center of gravity. Sensor measurements such as accelerations, angular rates, and forces are naturally expressed in this frame, making it essential for modeling and control.

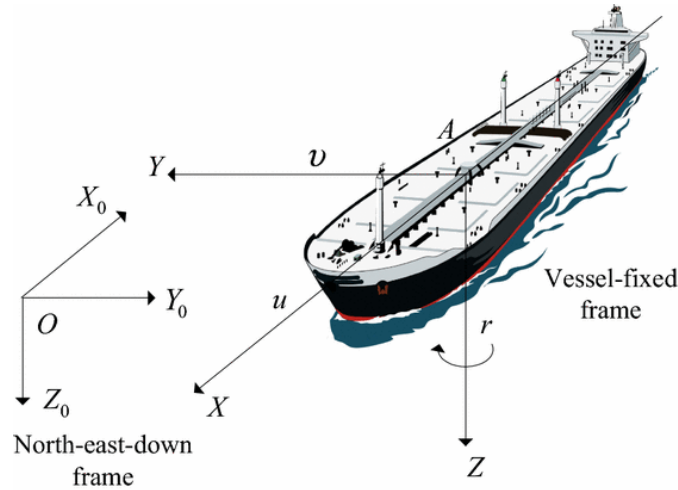


Figure 24: NED coordinate system attached to a marine vehicle. The illustration shows the local navigation axes and their relation to the body frame.^[20]

The relationship between the global and local reference frames follows the transformation chain:

$$\text{WGS84 (geodetic)} \rightarrow \text{ECEF (Cartesian)} \rightarrow \text{NED (local tangent)} \rightarrow \text{Body (vehicle-fixed)}$$

GNSS receivers typically provide positions in the WGS84 frame, expressed as geodetic coordinates (φ, λ, h) , where φ is latitude, λ is longitude, and h is altitude above the reference ellipsoid. For computational purposes, these coordinates are first converted to the ECEF frame, which represents positions in Cartesian coordinates (x, y, z) relative to the Earth's center of mass.

To obtain a local navigation frame, the ECEF position is then rotated and translated into the NED frame, which defines a tangent plane fixed at a chosen reference point (φ_0, λ_0) . The rotation from ECEF to NED is given by

$$R_e^n = \begin{bmatrix} -\sin \varphi_0 \cos \lambda_0 & -\sin \varphi_0 \sin \lambda_0 & \cos \varphi_0 \\ -\sin \lambda_0 & \cos \lambda_0 & 0 \\ -\cos \varphi_0 \cos \lambda_0 & -\cos \varphi_0 \sin \lambda_0 & -\sin \varphi_0 \end{bmatrix}$$

where φ_0 and λ_0 are the latitude and longitude of the local origin. The superscript n and subscript e indicate that this matrix transforms a vector expressed in the ECEF frame $\{e\}$ into the NED frame $\{n\}$.

Given the ECEF position of the vehicle $\mathbf{p}_{b/e}^e$ and the ECEF position of the NED origin $\mathbf{p}_{O/e}^e$, the vehicles position in NED coordinates is computed as

$$\mathbf{p}_{b/O}^n = \mathbf{p}_{b/e}^n + \mathbf{p}_{e/O}^n = R_e^n (\mathbf{p}_{b/e}^e - \mathbf{p}_{O/e}^e)$$

For onboard sensors, the position of a sensor $\{s\}$ relative to the vehicle body origin, expressed in the NED frame, is given by

$$\mathbf{p}_{s/b}^n = R_b^n \mathbf{p}_{s/b}^b$$

The absolute position of the sensor relative to the NED origin can then be found as

$$\mathbf{p}_{s/O}^n = \mathbf{p}_{s/b}^n + \mathbf{p}_{b/O}^n$$

where $\mathbf{p}_{s/b}^b$ is the known position of the sensor in the body frame, and R_b^n is the rotation matrix representing the vehicles attitude from the Body to the NED frame.

This formulation relates the sensor position to the global reference frame by first rotating the sensor offset from the body frame into the navigation frame, then translating it using the vehicles global position. The rotation matrix R_b^n is obtained from the vehicles orientation representation, typically parameterized using Euler angles or a unit quaternion, ensuring consistent alignment between the sensor, body, and navigation coordinate frames.

Maintaining a consistent chain of transformations between these frames is essential for reliable navigation and estimation. In practice, GNSS delivers global positions in WGS84 or ECEF coordinates, while onboard IMU sensors provide measurements in the Body frame. Sensor fusion algorithms such as the Extended Kalman Filter (EKF) and Error State Kalman Filter (ESKF) depend on these transformations to express all quantities consistently within the NED frame used for state estimation and control.

5.4 Rigid Body Kinematics

5.4.1 Relevance for Navigation and Modeling

Rigid body kinematics provides the mathematical foundation for expressing motion, orientation, and acceleration consistently across coordinate frames. In navigation systems, these relationships allow the integration of measurements from inertial sensors, GNSS, and other sources within a unified dynamic model.

The transformation equations for position, velocity, and acceleration ensure that sensor data expressed in the body frame can be accurately related to the navigation frame, enabling correct estimation of the vehicles state. This consistency is essential for inertial navigation, attitude estimation, and sensor fusion, where body fixed measurements such as angular velocity and specific force must be mapped into global coordinates for integration and correction.

Moreover, the rigid body framework is fundamental for simulation and control system design. It allows modeling of vehicle dynamics, actuator response, and sensor placement with precise spatial relationships. The kinematic expressions derived in this section thus serve as the basis for dynamic modeling, state estimation, and motion prediction used in autonomous and navigation systems.

5.4.2 Position and Orientation

The position of a rigid body is represented by the vector $\mathbf{p}_{b/O}^n$, which denotes the location of the body frame origin expressed in the navigation (NED) frame. The orientation of the body is represented by the rotation matrix $R_b^n \in \text{SO}(3)$, which maps a vector from the body frame $\{b\}$ to the navigation frame $\{n\}$.

The combined rigid body pose is described by the homogeneous transformation matrix $T_{b/O}^n \in \text{SE}(3)$:

$$T_{b/O}^n = \begin{bmatrix} R_b^n & \mathbf{p}_{b/O}^n \\ 0 & 1 \end{bmatrix}$$

The kinematic relationship between the time derivative of position and the linear velocity is then given by

$$\dot{\mathbf{p}}_{b/O}^n = \mathbf{v}_{b/O}^n$$

where $\mathbf{v}_{b/O}^n$ is the velocity of the body origin expressed in the navigation frame.

5.4.3 Angular Velocity and Euler Angle Relationship

The angular velocity vector $\boldsymbol{\omega}_{b/O}^b = [p, q, r]^T$ represents the instantaneous rotation rate of the body about its own axes, roll rate p about the x_b -axis (North), pitch rate q about the y_b -axis (East), and yaw rate r about the z_b -axis (Down).

When the body orientation is represented by ZYX Euler angles (yaw ψ , pitch θ , roll ϕ), the time derivative of the Euler angles relates to the body angular velocity through

$$\begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix} = T(\phi, \theta) \boldsymbol{\omega}_{b/O}^b$$

where the transformation matrix $T(\phi, \theta)$ for the NED convention is defined as

$$T(\phi, \theta) = \begin{bmatrix} 1 & \sin \phi \tan \theta & \cos \phi \tan \theta \\ 0 & \cos \phi & -\sin \phi \\ 0 & \sin \phi / \cos \theta & \cos \phi / \cos \theta \end{bmatrix}$$

This maps the body angular velocity to the Euler angle rates according to the NED convention.

The inverse relationship, expressing body angular velocity from Euler angle derivatives, is given by

$$\boldsymbol{\omega}_{b/O}^b = \begin{bmatrix} p \\ q \\ r \end{bmatrix} = T^{-1}(\phi, \theta) \begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix}$$

where

$$T^{-1}(\phi, \theta) = \begin{bmatrix} 1 & 0 & -\sin \theta \\ 0 & \cos \phi & \sin \phi \cos \theta \\ 0 & -\sin \phi & \cos \phi \cos \theta \end{bmatrix}$$

This formulation aligns with the NED coordinate convention and ensures correct mapping between angular velocities and Euler angle derivatives. The transformation matrix becomes singular at $\theta = \pm 90^\circ$, corresponding to gimbal lock, where $\tan \theta$ diverges.

5.4.4 Angular Velocity and Quaternion Relationship

Quaternions provide a compact and singularity free representation of rotation. A unit quaternion $q = [q_0, q_1, q_2, q_3]^\top$ consists of one scalar and three vector components, often written as

$$q = \begin{bmatrix} q_0 \\ \mathbf{q}_v \end{bmatrix} = \begin{bmatrix} q_0 \\ q_1 \\ q_2 \\ q_3 \end{bmatrix}$$

where q_0 is the scalar part and $\mathbf{q}_v = [q_1, q_2, q_3]^\top$ is the vector part. The quaternion represents a rotation of angle θ around the unit axis $\hat{\mathbf{u}}$:

$$q = \begin{bmatrix} \cos \frac{\theta}{2} \\ \hat{\mathbf{u}} \sin \frac{\theta}{2} \end{bmatrix}$$

The time evolution of the quaternion is governed by the angular velocity vector $\boldsymbol{\omega}_{b/O}^b = [p, q, r]^\top$ measured in the body frame. The quaternion kinematics are given by

$$\dot{q} = \frac{1}{2} \Omega(\boldsymbol{\omega}_{b/O}^b) q$$

where $\Omega(\boldsymbol{\omega}_{b/O}^b)$ is the quaternion rate matrix:

$$\Omega(\boldsymbol{\omega}_{b/O}^b) = \begin{bmatrix} 0 & -p & -q & -r \\ p & 0 & r & -q \\ q & -r & 0 & p \\ r & q & -p & 0 \end{bmatrix}$$

Given the quaternion $q = [q_0, \mathbf{q}_v]^\top$ and its time derivative $\dot{q} = [\dot{q}_0, \dot{\mathbf{q}}_v]^\top$, the body angular velocity can be reconstructed directly as

$$\boldsymbol{\omega}_{b/O}^b = 2(q_0 \dot{\mathbf{q}}_v - \dot{q}_0 \mathbf{q}_v - \mathbf{q}_v \times \dot{\mathbf{q}}_v).$$

Here, q_0 is the scalar part of the quaternion and $\mathbf{q} * v$ its vector part. This relation provides a direct and numerically stable way to compute the instantaneous angular velocity from quaternion derivatives while maintaining full consistency with rigid body rotational kinematics in the body frame. This formulation complements the quaternion rate equation which integrates angular velocity to update attitude over time. The quaternion must remain normalized, i.e. $|q| = 1$, to represent a valid rotation, and is therefore periodically renormalized during numerical integration as

$$q = \frac{q}{|q|}.$$

The corresponding rotation matrix from body to navigation frame is obtained as

$$R_b^n(q) = \begin{bmatrix} 1 - 2(q_2^2 + q_3^2) & 2(q_1 q_2 - q_0 q_3) & 2(q_1 q_3 + q_0 q_2) \\ 2(q_1 q_2 + q_0 q_3) & 1 - 2(q_1^2 + q_3^2) & 2(q_2 q_3 - q_0 q_1) \\ 2(q_1 q_3 - q_0 q_2) & 2(q_2 q_3 + q_0 q_1) & 1 - 2(q_1^2 + q_2^2) \end{bmatrix}$$

This rotation matrix provides the mapping between the body and navigation frames, analogous to R_b^n obtained from Euler angles, but without the gimbal lock problem.

In practice, quaternion based orientation integration using $\dot{q} = \frac{1}{2} \Omega(\boldsymbol{\omega}_{b/O}^b) q$ is preferred for real-time attitude propagation, while the equivalent rotation matrix or Euler angles are extracted only when needed for visualization or control.

5.4.5 Linear Velocity Relationship

The linear velocity of any point P fixed on a rigid body can be related to the velocity of a reference point O on the same body as

$$\mathbf{v}_{P/O}^b = \mathbf{v}_{b/O}^b + \mathbf{v}_{P/b}^b + \boldsymbol{\omega}_{b/O}^b \times \mathbf{p}_{P/b}^b$$

Here, $\mathbf{v}_{b/O}^b$ is the translational velocity of the body origin expressed in the body frame, $\mathbf{v}_{P/b}^b$ is the velocity of point P relative to the body (zero for fixed sensors), $\boldsymbol{\omega}_{b/O}^b$ is the body angular velocity, and $\mathbf{p}_{P/b}^b$ is the position of P relative to the body origin, both expressed in the body frame.

The cross product term $\boldsymbol{\omega}_{b/O}^b \times \mathbf{p}_{P/b}^b$ represents the additional linear velocity experienced by point P due to the rotational motion of the body. This term becomes more significant for points located farther away from the rotation axis, such as antennas or sensors mounted away from the vessels center of gravity.

In many applications, it is necessary to express all velocities in a common reference frame. The velocity of the body origin expressed in the navigation (NED) frame is obtained by rotating the body frame velocity using the rotation matrix R_b^n :

$$\mathbf{v}_{b/O}^n = R_b^n \mathbf{v}_{b/O}^b.$$

Conversely, the body frame velocity can be obtained from the navigation frame velocity by

$$\mathbf{v}_{b/O}^b = (R_b^n)^\top \mathbf{v}_{b/O}^n,$$

since $(R_b^n)^\top = R_b^n$. The rotation matrix R_b^n is derived from the vehicles orientation, represented either by Euler angles or a unit quaternion.

Similarly, for a sensor or point P rigidly fixed to the vehicle (i.e., $\mathbf{v}_{P/b}^b = 0$), the velocity expressed in the navigation frame simplifies to

$$\mathbf{v}_{P/O}^n = R_b^n \left(\mathbf{v}_{b/O}^b + \boldsymbol{\omega}_{b/O}^b \times \mathbf{p}_{P/b}^b \right),$$

where $\mathbf{p}_{P/b}^b$ defines the sensors position in the body frame. This formulation provides a consistent way to compute the motion of any fixed point on the vehicle in both body and navigation frames, ensuring proper alignment between inertial and navigation states.

5.4.6 Angular Acceleration in Euler Representation

The angular acceleration $\boldsymbol{\alpha}_{b/O}^b$ describes the rate of change of the body's angular velocity vector $\boldsymbol{\omega}_{b/O}^b = [p, q, r]^\top$, which represents the instantaneous rotation of the body about its own x , y , and z axes. Physically, $\boldsymbol{\alpha}_{b/O}^b$ captures how fast the rotational motion itself is changing, and it directly relates to the torques acting on the body through the rotational dynamics equations.

When the body orientation is represented using Euler angles (ϕ, θ, ψ) , the angular acceleration can be obtained by differentiating the Euler angle to angular velocity relationship:

$$\boldsymbol{\alpha}_{b/O}^b = T^{-1}(\phi, \theta, \psi) \ddot{\boldsymbol{\Theta}} + \dot{T}^{-1}(\phi, \theta, \psi) \dot{\boldsymbol{\Theta}}$$

where $\dot{\boldsymbol{\Theta}} = [\dot{\phi}, \dot{\theta}, \dot{\psi}]^\top$ and $\ddot{\boldsymbol{\Theta}} = [\ddot{\phi}, \ddot{\theta}, \ddot{\psi}]^\top$ are the first and second derivatives of the Euler angles. The first term represents the direct contribution from angular rate changes, while the second term captures coupling effects caused by the nonlinearity of rotational kinematics, for instance, when pitch or roll rates influence yaw acceleration.

In differential form, the time derivative of angular velocity is expressed as

$$\dot{\boldsymbol{\omega}}_{b/O}^b = \boldsymbol{\alpha}_{b/O}^b$$

which defines angular acceleration in the same frame as $\boldsymbol{\omega}_{b/O}^b$. To express angular acceleration in another reference frame, such as the navigation frame, it can be transformed using the rotation matrix:

$$\boldsymbol{\alpha}_{b/O}^n = R_b^n \boldsymbol{\alpha}_{b/O}^b$$

where R_b^n is the rotation from the body frame to the navigation (NED) frame.

In practical navigation and control systems, the angular acceleration $\alpha_{b/O}^b$ is rarely measured directly. It is usually approximated by differentiating the body angular velocity $\omega_{b/O}^b$ obtained from gyroscopes or inferred from rigid body dynamics. Maintaining consistent frame alignment between body and navigation frames is essential for accurate attitude propagation and torque computation.

5.4.7 Angular Acceleration in Quaternion Representation

The quaternion representation provides a singularity free alternative to Euler angles for describing rotational motion. The unit quaternion $q = [q_0, q_1, q_2, q_3]^\top$ defines the orientation of the body frame relative to the navigation frame, where q_0 is the scalar part and $[q_1, q_2, q_3]^\top$ is the vector part.

The time evolution of the quaternion is governed by the rotational kinematics equation:

$$\dot{q} = \frac{1}{2}\Omega(\omega_{b/O}^b)q$$

where $\omega_{b/O}^b = [p, q, r]^\top$ is the angular velocity in the body frame and $\Omega(\omega_{b/O}^b)$ is the quaternion rate matrix:

$$\Omega(\omega_{b/O}^b) = \begin{bmatrix} 0 & -p & -q & -r \\ p & 0 & r & -q \\ q & -r & 0 & p \\ r & q & -p & 0 \end{bmatrix}$$

Differentiating this expression gives the quaternion based angular acceleration:

$$\ddot{q} = \frac{1}{2}\Omega(\alpha_{b/O}^b)q + \frac{1}{2}\Omega(\omega_{b/O}^b)\dot{q}$$

which captures both the direct contribution of angular acceleration $\alpha_{b/O}^b$ and the coupling effect from the current angular velocity.

In vector form, the time derivative of angular velocity is expressed as

$$\dot{\omega}_{b/O}^b = \alpha_{b/O}^b$$

which defines angular acceleration in the same frame as ω^b . To express angular acceleration in another reference frame, such as the navigation frame, it can be transformed using the rotation matrix:

$$\alpha_{b/O}^n = R_b^n \alpha_{b/O}^b$$

where R_b^n is the rotation matrix from body to navigation frame.

To ensure q remains a valid rotation, it must satisfy the unit norm constraint:

$$\|q\| = 1$$

which is typically enforced by periodic normalization during numerical integration like so:

$$q = \frac{q}{\|q\|}$$

This quaternion based formulation is computationally efficient and avoids the gimbal lock issue inherent in Euler angle representations. For this reason, quaternion propagation is widely preferred for representing orientation and modeling angular dynamics in navigation and control systems.

5.4.8 Linear Acceleration

The linear acceleration of a point P on a rigid body consists of translational, tangential, centripetal, and Coriolis components. It is expressed in the body frame as

$$\mathbf{a}_{P/O}^b = \mathbf{a}_{b/O}^b + \mathbf{a}_{P/b}^b + \alpha_{b/O}^b \times \mathbf{p}_{P/b}^b + \omega_{b/O}^b \times (\omega_{b/O}^b \times \mathbf{p}_{P/b}^b) + 2\omega_{b/O}^b \times \mathbf{v}_{P/b}^b$$

Here, $\mathbf{a}_{b/O}^b$ is the translational acceleration of the body origin, $\mathbf{a}_{P/b}^b$ is acceleration of point P on sensor relative to body, $\boldsymbol{\alpha}_{b/O}^b$ is the angular acceleration, $\boldsymbol{\omega}_{b/O}^b$ is the angular velocity, $\mathbf{p}_{P/b}^b$ is the position of point P relative to the body origin, and $\mathbf{v}_{P/b}^b$ is the velocity of P relative to the body frame.

Each term represents a specific physical contribution to the total acceleration. The first term $\mathbf{a}_{b/O}^b$ describes the translational acceleration of the body origin. Second term $\mathbf{a}_{P/b}^b$ denotes the local acceleration of P relative to the body frame, which becomes zero for fixed sensors, while $\boldsymbol{\alpha}_{b/O}^b \times \mathbf{p}_{P/b}^b$ represents the tangential acceleration caused by changes in rotational rate. The term $\boldsymbol{\omega}_{b/O}^b \times (\boldsymbol{\omega}_{b/O}^b \times \mathbf{p}_{P/b}^b)$ accounts for the centripetal acceleration directed toward the axis of rotation, and $2\boldsymbol{\omega}_{b/O}^b \times \mathbf{v}_{P/b}^b$ captures the Coriolis acceleration arising from relative motion within the rotating body.

For a sensor rigidly mounted on the body, where $\mathbf{v}_{P/b}^b = 0$ and $\mathbf{a}_{P/b}^b = 0$, the equation simplifies to

$$\mathbf{a}_{P/O}^b = \mathbf{a}_{b/O}^b + \boldsymbol{\alpha}_{b/O}^b \times \mathbf{p}_{P/b}^b + \boldsymbol{\omega}_{b/O}^b \times (\boldsymbol{\omega}_{b/O}^b \times \mathbf{p}_{P/b}^b)$$

This formulation is widely used to compute the acceleration at sensor locations such as IMUs or GNSS antennas mounted away from the vehicles center of mass.

To express the acceleration in the navigation (NED) frame, the transformation is performed using the rotation matrix R_b^n :

$$\mathbf{a}_{P/O}^n = R_b^n \mathbf{a}_{P/O}^b$$

and conversely,

$$\mathbf{a}_{P/O}^b = (R_b^n)^\top \mathbf{a}_{P/O}^n$$

In inertial navigation systems, accelerometers measure the specific force, which is the total acceleration excluding gravity:

$$\mathbf{f}^b = \mathbf{a}_{b/O}^b - R_n^b \mathbf{g}^n$$

where $\mathbf{g}^n = [0, 0, g]^\top$ is the gravity vector in the NED frame. The specific force $\mathbf{f}^b$ is the quantity directly measured by IMUs and serves as the basis for estimating velocity and position through numerical integration.

5.5 ASV Motion and Measurement Models

5.5.1 Overview

This chapter presents the mathematical framework used to model the motion dynamics and sensor measurements of the ASV. Two main modeling approaches are discussed, the nonlinear 6 DOF marine craft model developed by Fossen [13], and the Inertial Navigation System (INS) based kinematic model described in Edmund Brekke book on Sensor Fusion [14]. Both formulations provide complementary perspectives on vehicle motion, Fossen’s model emphasizes a physically consistent hydrodynamic representation, while the INS model focuses on inertial propagation and sensor fusion suitable for real-time navigation.

The chapter begins by introducing Fossen’s marine craft model, which captures the rigid body and hydrodynamic behavior of marine vehicles through a compact matrix vector formulation that includes added mass, Coriolis, damping, and restoring effects. The INS based kinematic model is then presented as the practical implementation used in this work, describing the evolution of position, velocity, and attitude from inertial data while modeling sensor biases and stochastic errors. Aiding measurements from a dual antenna GNSS system are incorporated to correct drift and estimate heading. Although Fossen’s model provides higher physical fidelity, it requires extensive parameter identification, whereas the INS model achieves sufficient accuracy for SLAM and local navigation when aided by GNSS. Thus, the INS based approach is chosen as the optimal balance between simplicity, computational efficiency, and practical performance.

5.5.2 Kinetic Motion Model

The kinetic motion model employed in this work is based on Fossens nonlinear 6 DOF marine craft formulation, which provides a unified mathematical framework for describing the dynamic behavior of marine vehicles [13]. This model captures both rigid body and hydrodynamic effects using a compact matrix vector representation, making it suitable for analysis, control, and simulation of ships, underwater vehicles, and surface vessels. By bridging classical rigid body mechanics with hydrodynamic theory, the formulation offers a complete description of marine craft dynamics while preserving key physical properties such as symmetry, passivity, and energy consistency.

The general 6 DOF equations of motion can be written as

$$\mathbf{M}\dot{\boldsymbol{\nu}} + \mathbf{C}(\boldsymbol{\nu})\boldsymbol{\nu} + \mathbf{D}(\boldsymbol{\nu})\boldsymbol{\nu} + \mathbf{g}(\boldsymbol{\eta}) = \boldsymbol{\tau}$$

where $\boldsymbol{\eta} = [x, y, z, \phi, \theta, \psi]^\top$ represents position and orientation, and $\boldsymbol{\nu} = [u, v, w, p, q, r]^\top$ denotes linear and angular velocities in the body frame. The vector $\boldsymbol{\tau}$ contains external forces and moments acting on the vessel from propulsion, wind, waves, and current. This formulation expresses the total balance between inertia, Coriolis, damping, and restoring forces on the left hand side, and all external excitations on the right hand side, forming the foundation for the dynamics of any marine craft.

The inertia matrix $\mathbf{M}$ consists of both rigid body and added mass contributions,

$$\mathbf{M} = \mathbf{M}_{RB} + \mathbf{M}_A$$

where $\mathbf{M}_{RB}$ represents the rigid body mass and inertia, while $\mathbf{M}_A$ captures the hydrodynamic added mass caused by the acceleration of surrounding water as the hull moves. The added mass terms are particularly important for underwater and high speed vehicles, as they significantly influence acceleration response and stability. This total inertia matrix determines how the craft resists changes in motion, acting as a coupling term between linear and angular accelerations.

The Coriolis and centripetal effects are similarly expressed as

$$\mathbf{C}(\boldsymbol{\nu}) = \mathbf{C}_{RB}(\boldsymbol{\nu}) + \mathbf{C}_A(\boldsymbol{\nu})$$

with $\mathbf{C}_{RB}$ and $\mathbf{C}_A$ representing the rigid body and added mass contributions, respectively. These matrices account for dynamic coupling between translational and rotational motion, such as how a turn induces sway or roll. The Coriolis matrix is skew-symmetric, ensuring that it does not contribute to net energy gain or loss, only redistributing kinetic energy among the motion axes.

The matrix $\mathbf{D}(\boldsymbol{\nu})$ models hydrodynamic damping from viscous drag, wave radiation, and flow separation effects. It typically includes linear damping terms valid for small velocities and nonlinear or quadratic terms that dominate at higher speeds. The damping effects are always dissipative, converting kinetic energy into heat or wave energy, thus stabilizing the system over time.

The restoring forces and moments $\mathbf{g}(\boldsymbol{\eta})$ describe gravitational and buoyancy effects, depending on the vehicles geometry and displacement. These terms act to return the vessel to equilibrium when displaced, defining roll, pitch, and heave stability through hydrostatic stiffness. For surface vessels, the restoring forces are strongly dependent on metacentric heights and the waterplane area, while for submerged vehicles, they depend mainly on the distance between the centers of gravity and buoyancy.

The kinematic relationships between the body and NED frames are given by

$$\dot{\boldsymbol{\eta}} = \mathbf{J}(\boldsymbol{\eta})\boldsymbol{\nu}$$

where $\mathbf{J}(\boldsymbol{\eta})$ is a block diagonal transformation matrix containing the rotation matrix R_b^n for translational motion and $T(\phi, \theta)$ for angular motion. This mapping relates the velocity expressed in the body frame to the rate of change of position and attitude in the navigation frame, and can be implemented using either Euler angles or quaternions to avoid singularities.

Environmental effects such as ocean currents, wind, and wave disturbances can be included by introducing the relative velocity $\boldsymbol{\nu}_r = \boldsymbol{\nu} - \boldsymbol{\nu}_c$, where $\boldsymbol{\nu}_c$ is the current velocity expressed in the body frame. This modifies the hydrodynamic forces and damping terms to account for the vehicles motion relative to the surrounding water, which is essential for accurate modeling of drift and current induced forces.

For surface vessels, the equations can be simplified to the planar 3 DOF surge, sway, and yaw model:

$$\mathbf{M}\dot{\boldsymbol{\nu}} + \mathbf{C}(\boldsymbol{\nu})\boldsymbol{\nu} + \mathbf{D}(\boldsymbol{\nu})\boldsymbol{\nu} = \boldsymbol{\tau}$$

where $\boldsymbol{\nu} = [u, v, r]^\top$. This reduced model captures the dominant motion in the horizontal plane and is widely used for uncrewed surface vehicles and control applications where roll, pitch, and heave motions are negligible.

The full nonlinear state space representation of Fossen's marine craft model can be written as

$$\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u}) = \begin{bmatrix} \dot{\boldsymbol{\eta}} \\ \dot{\boldsymbol{\nu}} \end{bmatrix} = \begin{bmatrix} \mathbf{J}(\boldsymbol{\eta})\boldsymbol{\nu} \\ \mathbf{M}^{-1}(\boldsymbol{\tau} - \mathbf{C}(\boldsymbol{\nu})\boldsymbol{\nu} - \mathbf{D}(\boldsymbol{\nu})\boldsymbol{\nu} - \mathbf{g}(\boldsymbol{\eta})) \end{bmatrix}$$

where

$$\mathbf{x} = [\boldsymbol{\eta} \quad \boldsymbol{\nu}]^\top = [x, y, z, \phi, \theta, \psi, u, v, w, p, q, r]^\top, \quad \mathbf{u} = \boldsymbol{\tau}$$

Here, the first part of the function $f(\mathbf{x}, \mathbf{u})$ describes the kinematic mapping from body frame velocities to the time derivative of position and attitude through the transformation matrix $\mathbf{J}(\boldsymbol{\eta})$, while the second part represents the vehicle kinetics defined by Newton-Euler dynamics. The term $\mathbf{M}^{-1}$ acts as an inverse inertia operator, mapping the net generalized forces and moments (after accounting for Coriolis, damping, and restoring effects) to the acceleration in the body frame.

This formulation provides a complete and compact representation of the vessels motion, directly linking control inputs and environmental disturbances to the time evolution of the vehicles state.

Fossen's formulation exhibits several key properties, the mass matrix $\mathbf{M}$ is symmetric and positive definite, the Coriolis matrix $\mathbf{C}(\boldsymbol{\nu})$ is skew-symmetric, and the damping matrix $\mathbf{D}(\boldsymbol{\nu})$ is positive definite. These properties guarantee passivity and energy conservation, ensuring that the system dissipates energy over time and remains physically consistent. This structure also provides a strong foundation for control system design and stability analysis, as it aligns with fundamental principles of energy based modeling and Lyapunov stability.

5.5.3 Kinematic Motion Model

The kinematic motion model adopted in this work is based on the Inertial Navigation System (INS) formulation, which describes the time evolution of a vehicles position, velocity, and attitude as functions

of inertial sensor measurements [14]. Unlike hydrodynamic models that depend on physical parameters such as mass, damping, or buoyancy, the INS model is purely kinematic and relies solely on accelerometer and gyroscope data. This independence from the vehicles geometry, operating environment, and fluid interaction makes it well suited for generic navigation and SLAM applications, where a simplified yet robust representation of motion is sufficient for accurate state propagation between aiding measurements.

IMUs typically represent attitude using quaternions rather than Euler angles. This avoids singularities and ensures smooth rotational updates even under large orientation changes, making quaternion based propagation the standard approach for INS implementations.

The continuous time state vector is defined as

$$\mathbf{x} = \begin{bmatrix} \mathbf{p}_{b/O}^n & \mathbf{v}_{b/O}^n & \mathbf{q} & \mathbf{b}_a & \mathbf{b}_g \end{bmatrix}^\top$$

where $\mathbf{p}_{b/O}^n = [x, y, z]^\top$ is the position of the body origin relative to the navigation frame, $\mathbf{v}_{b/O}^n = [v_x, v_y, v_z]^\top$ is the velocity in the same frame, $\mathbf{q}$ is the quaternion representing the rotation from body to navigation frame, and $\mathbf{b}_a$ and $\mathbf{b}_g$ are the accelerometer and gyroscope biases, respectively.

The position and velocity states describe translational motion in the navigation frame, while the quaternion captures attitude evolution through rotational kinematics. The bias states account for the slowly varying sensor offsets that develop over time due to temperature changes, component aging, and other environmental factors. Together, these quantities form a complete minimal representation of the navigation state used in inertial systems, forming the basis for the continuous time kinematic model that governs their time evolution.

The inertial navigation model therefore describes the kinematic evolution of the vehicles position, velocity, and attitude as driven by inertial sensor measurements. The foundation of this formulation is the deterministic, noise free relationship between translational and rotational motion, which forms the core of all INS propagation models.

In the ideal case without any sensor errors, the motion of the sensor can be expressed as

$$\begin{aligned} \dot{\mathbf{p}}_{s/O}^n &= \mathbf{v}_{s/O}^n \\ \dot{\mathbf{v}}_{s/O}^n &= R_b^n(\mathbf{q}) (R_s^b \mathbf{a}_m) + \mathbf{g}^n \\ \dot{\mathbf{q}} &= \frac{1}{2} \mathbf{q} \otimes (R_s^b \boldsymbol{\omega}_m) \end{aligned}$$

where $\mathbf{p}_{s/O}^n$ and $\mathbf{v}_{s/O}^n$ are the position and velocity of the sensor with respect to an inertial reference frame like NED, $R_b^n(\mathbf{q})$ is the rotation matrix transforming vectors from the body frame to the navigation frame, and R_s^b is the rotation matrix from the sensor to body frame. The measured quantities $\mathbf{a}_m$ and $\boldsymbol{\omega}_m$ are the specific force and angular rate readings provided by the IMU, expressed in the sensors own coordinate frame. These measurements are therefore related to the true body frame motion through the sensors fixed mounting orientation, such that $\mathbf{a}_m = \mathbf{a}_{s/b}^s$ and $\boldsymbol{\omega}_m = \boldsymbol{\omega}_{s/b}^s$.

In practice, the inertial sensors are rarely placed exactly at the vehicles center of gravity (COG). When the sensor is offset by a position vector $\mathbf{r}_{s/b}^b$ from the body frame origin usually defined at the COG, the accelerations measured by the IMU no longer correspond directly to the translational accelerations of the vehicle. Instead, the rotational motion of the body introduces additional terms due to centripetal and tangential accelerations. The total acceleration of the sensor can then be expressed as

$$\mathbf{a}_{s/O}^b = \mathbf{a}_{b/O}^b + \boldsymbol{\alpha}_{b/O}^b \times \mathbf{r}_{s/b}^b + \boldsymbol{\omega}_{b/O}^b \times (\boldsymbol{\omega}_{b/O}^b \times \mathbf{p}_{s/b}^b)$$

where $\mathbf{a}_{b/O}^b$ is the linear acceleration of the body origin, $\boldsymbol{\omega}_{b/O}^b$ is the angular velocity of the body, and $\boldsymbol{\alpha}_{b/O}^b$ its angular acceleration. The second term represents the tangential acceleration component caused by rotational acceleration, while the third term corresponds to the centripetal acceleration arising from constant rotation. These effects are collectively known as the “*lever arm effect*”, and they cause the IMU to measure accelerations that differ from those experienced at the COG.

Accurately compensating for the lever arm effect requires knowledge of the exact mounting offset $\mathbf{p}_{s/b}^b$ and precise estimates of both angular velocity and angular acceleration. The latter is particularly difficult to obtain in real time, as most MEMS-based IMU (like microAmpere IMU) do not measure angular acceleration directly. Numerical differentiation of gyroscope signals amplifies measurement noise and introduces additional uncertainty, making real-time compensation computationally challenging and often unreliable.

Therefore, in this master thesis work, it is assumed that the inertial sensors are mounted close to the vehicles center of gravity. For the microAmpere ASV, this assumption is valid since the IMU is physically located near the COG within the central electronics housing. As a result, the lever arm terms become negligible, and the simplified kinematic model given above provides an accurate and practical representation of the vehicle motion. This assumption greatly simplifies the mathematical model while retaining sufficient fidelity for navigation and SLAM applications.

Under this assumption, the IMU can be considered approximately coincident with the body frame origin, effectively treating the sensor as rigidly aligned with the vehicles center of gravity. The motion model can then be expressed directly in terms of the body frame as

$$\begin{aligned}\dot{\mathbf{p}}_{b/O}^n &= \mathbf{v}_{b/O}^n \\ \dot{\mathbf{v}}_{b/O}^n &= R_b^n(\mathbf{q}) \mathbf{a}_m + \mathbf{g}^n \\ \dot{\mathbf{q}} &= \frac{1}{2} \mathbf{q} \otimes \boldsymbol{\omega}_m\end{aligned}$$

which defines the nominal, noise free kinematic model used in this work. It captures the translational and rotational motion of the ASV based solely on the IMUs accelerometer and gyroscope measurements, with attitude expressed through quaternion integration for smooth and singularity free orientation propagation.

In practice, the accelerometer and gyroscope measurements are not perfect and are affected by sensor biases and measurement noise. These imperfections cause the estimated position and attitude to drift over time, as small integration errors accumulate during motion. The measured sensor outputs can therefore be modeled as

$$\begin{aligned}\mathbf{a}_m &= R^T(q_t) (\mathbf{a}_t - \mathbf{g}) + \mathbf{a}_{bt} + \mathbf{a}_n \\ \boldsymbol{\omega}_m &= \boldsymbol{\omega}_t + \boldsymbol{\omega}_{bt} + \boldsymbol{\omega}_n\end{aligned}$$

where $\mathbf{a}_m$ and $\boldsymbol{\omega}_m$ are the measured accelerations and angular rates, $\mathbf{a}_{bt}$ and $\boldsymbol{\omega}_{bt}$ denote slowly varying sensor biases, and $\mathbf{a}_n$ and $\boldsymbol{\omega}_n$ are zero mean Gaussian noise processes representing measurement uncertainty. The reason for choosing zero mean Gaussian noise processes representation is just to make calculations easier to manage, and usually inertial sensor noise can be approximated to a pure gaussian.

While the previous kinematic model assumed ideal, noise free measurements, real inertial sensors inherently produce signals corrupted by both bias and random noise. These effects introduce long term drift in velocity, position, and orientation estimates if uncorrected. To account for this, the noise and bias terms are explicitly included in the dynamic equations.

By substituting the noisy sensor models into the noise free kinematics, we obtain the full continuous time INS motion model that represents the true system behavior under realistic sensor conditions:

$$\begin{aligned}\dot{\mathbf{p}}_{b/O}^n &= \mathbf{v}_{b/O}^n \\ \dot{\mathbf{v}}_{b/O}^n &= R_b^n(\mathbf{q}) (\mathbf{a}_m - \mathbf{a}_{bt} - \mathbf{a}_n) + \mathbf{g}^n \\ \dot{\mathbf{q}} &= \frac{1}{2} \mathbf{q} \otimes (\boldsymbol{\omega}_m - \boldsymbol{\omega}_{bt} - \boldsymbol{\omega}_n)\end{aligned}$$

where $R_b^n(\mathbf{q})$ transforms body frame accelerations into the navigation frame according to the current orientation quaternion.

This formulation captures the complete inertial motion propagation process, accounting for both deterministic motion and stochastic sensor effects. It forms the basis for all subsequent modeling of bias dynamics and sensor noise processes.

Inertial measurement systems such as accelerometers and gyroscopes inherently suffer from drift over time, as they measure relative motion rather than absolute quantities. Small integration errors

in acceleration and angular rate accumulate, causing increasing uncertainty in position and attitude estimates. To account for this phenomenon, bias terms are introduced to represent slow, time varying offsets in the sensor readings. Accurate bias modeling is essential for achieving reliable and stable navigation performance, particularly for low cost MEMS-based sensors where bias instability is a dominant error source.

Sensor biases are typically represented as stochastic processes that evolve gradually over time. Several modeling approaches exist, each offering a trade off between realism and complexity. The simplest approach is the constant bias model,

$$\dot{\mathbf{b}} = 0$$

which assumes a fixed, time invariant offset in the sensor readings. Although straightforward, this model is generally insufficient for practical applications since real world sensors exhibit continuous drift due to thermal, mechanical, and electronic variations.

A more flexible formulation is the Wiener process, or better known as random walk model,

$$\dot{\mathbf{b}} = \mathbf{b}_n$$

where $\mathbf{b}_n$ is a zero mean white noise process. This allows the bias to evolve stochastically over time, capturing unmodeled variations in sensor output. However, this model tends to be overly conservative, as the bias uncertainty grows without bound, making it unsuitable for long duration navigation where bounded behavior is required.

A more realistic representation for modern inertial sensors is the first-order Gauss-Markov process,

$$\dot{\mathbf{b}} = -\frac{1}{\tau}\mathbf{b} + \mathbf{b}_n$$

where τ is the correlation time constant that defines how quickly the bias decays toward zero. This model captures both short term fluctuations and long term drift, providing a balanced compromise between physical accuracy and numerical stability. It ensures that bias uncertainty remains bounded while reflecting the natural stochastic variation of real IMU behavior.

Accordingly, the accelerometer and gyroscope biases are modeled as first order Gauss-Markov processes:

$$\begin{aligned}\dot{\mathbf{a}}_{bt} &= -p_{ab} I \mathbf{a}_{bt} + \mathbf{a}_w \\ \dot{\boldsymbol{\omega}}_{bt} &= -p_{\omega b} I \boldsymbol{\omega}_{bt} + \boldsymbol{\omega}_w\end{aligned}$$

where $p_{ab} = 1/\tau_a$ and $p_{\omega b} = 1/\tau_g$ denote the inverse correlation times for the accelerometer and gyroscope bias models, respectively. The terms $\mathbf{a}_w$ and $\boldsymbol{\omega}_w$ represent zero mean white noise processes driving the bias evolution.

This bias modeling framework provides a physically consistent and numerically stable representation of inertial sensor drift. By incorporating these bias dynamics, the INS model can accurately describe both the deterministic vehicle motion and the stochastic effects introduced by the inertial sensors.

With the inclusion of bias dynamics, the complete continuous time true state kinematic model of the system is expressed as

$$\begin{aligned}\dot{\mathbf{p}}_{b/O}^n &= \mathbf{v}_{b/O}^n \\ \dot{\mathbf{v}}_{b/O}^n &= R_b^n(\mathbf{q}) (\mathbf{a}_m - \mathbf{a}_{bt} - \mathbf{a}_n) + \mathbf{g}^n \\ \dot{\mathbf{q}} &= \frac{1}{2} \mathbf{q} \otimes (\boldsymbol{\omega}_m - \boldsymbol{\omega}_{bt} - \boldsymbol{\omega}_n) \\ \dot{\mathbf{a}}_{bt} &= -p_{ab} I \mathbf{a}_{bt} + \mathbf{a}_w \\ \dot{\boldsymbol{\omega}}_{bt} &= -p_{\omega b} I \boldsymbol{\omega}_{bt} + \boldsymbol{\omega}_w\end{aligned}$$

This formulation constitutes the complete nonlinear inertial motion model, capturing both the deterministic kinematics of the vehicle and the stochastic processes that characterize real sensor behavior.

The complete INS model previously presented captures the full physical and stochastic behavior of the vehicle, incorporating both deterministic motion and random processes such as measurement noise and bias drift. While this representation accurately reflects real sensor behavior, it is often desirable to

isolate the deterministic part of the dynamics to describe the nominal system behavior. This simplified formulation, known as the nominal state kinematic model, represents how the system would evolve in the absence of stochastic disturbances. It provides a clear and noise-free description of the vehicles motion based purely on inertial measurements, forming the foundation for state propagation in navigation and estimation frameworks.

The nominal model assumes that all noise components are zero, leaving only the essential deterministic relationships between position, velocity, attitude, and sensor biases. In this formulation, the accelerometer and gyroscope measurements are treated as control inputs driving the systems motion. This perspective simplifies the dynamics while preserving the physical structure of the original model, allowing efficient and accurate propagation of the navigation states between external aiding updates.

By removing the stochastic noise terms from the true state equations, the nominal state kinematic model becomes

$$\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u}) = \begin{bmatrix} \dot{\mathbf{p}}_{b/O}^n \\ \dot{\mathbf{v}}_{b/O}^n \\ \dot{\mathbf{q}} \\ \dot{\mathbf{a}}_b \\ \dot{\omega}_b \end{bmatrix} = \begin{bmatrix} \mathbf{v}_{b/O}^n \\ R_b^n(\mathbf{q}) (\mathbf{a}_m - \mathbf{a}_b) + \mathbf{g}^n \\ \frac{1}{2} \mathbf{q} \otimes (\boldsymbol{\omega}_m - \omega_b) \\ -p_{ab} I \mathbf{a}_b \\ -p_{\omega b} I \omega_b \end{bmatrix} \quad (6)$$

where

$$\mathbf{x} = [\mathbf{p}_{b/O}^n \quad \mathbf{v}_{b/O}^n \quad \mathbf{q} \quad \mathbf{a}_b \quad \omega_b]^\top, \quad \mathbf{u} = [\mathbf{a}_m \quad \boldsymbol{\omega}_m]^\top \quad (7)$$

Here, $\mathbf{p}_{b/O}^n$ and $\mathbf{v}_{b/O}^n$ represent the vehicles position and velocity expressed in the navigation frame, $\mathbf{q}$ denotes the attitude quaternion defining the rotation from the body to the navigation frame, and $\mathbf{a}_b$ and ω_b represent the accelerometer and gyroscope biases, respectively. The parameters $p_{ab} = 1/\tau_a$ and $p_{\omega b} = 1/\tau_g$ correspond to the inverse correlation times governing the exponential decay rates of the bias dynamics.

This nominal model represents the idealized, noise free motion of the vehicle as inferred from IMU measurements. It captures the deterministic propagation of position, velocity, attitude, and bias under perfect sensor conditions, serving as the baseline dynamic model for integrated navigation systems.

In the real world, this motion model is never perfectly deterministic. Even though Equation 6 describes an ideal system, the actual process is continuously disturbed by sensor noise and model imperfections. Since this model is derived directly from the INS model, it effectively represents the IMU itself, and the IMU is a sensor with stochastic noise and bias drift. To account for these imperfections, a process noise term is introduced, characterized by the covariance matrix $\mathbf{Q}$.

The true system evolution can therefore be represented as the nominal model driven by process noise

$$\dot{\mathbf{x}}_{\text{real}} = f(\mathbf{x}, \mathbf{u}) + \mathbf{G} \mathbf{n}_{\text{INS}}, \quad \mathbf{n}_{\text{INS}} \sim \mathcal{N}(\mathbf{0}, \mathbf{Q})$$

where $\mathbf{G}$ is the noise input matrix that maps the 12 dimensional process noise into the full 16 dimensional state space, and $\mathbf{Q}$ defines the covariance of the IMU noise sources. This formulation captures how the real system deviates from the ideal deterministic model due to stochastic disturbances originating from the inertial sensors.

The process noise covariance $\mathbf{Q}$ models how random accelerometer and gyroscope disturbances drive uncertainty into the system states over time. In this work, $\mathbf{Q}$ is assumed Gaussian, which makes the mathematics tractable and aligns well with the typical near Gaussian behavior of MEMS IMUs. The covariance is constructed from the standard deviations of the IMU noise parameters as

$$\mathbf{Q} = \text{diag}(\sigma_a^2 \mathbf{I}_3, \sigma_\omega^2 \mathbf{I}_3, \sigma_{b_a}^2 \mathbf{I}_3, \sigma_{b_\omega}^2 \mathbf{I}_3) \quad (8)$$

where σ_a and σ_ω are the accelerometer and gyroscope white noise densities, and σ_{b_a} and σ_{b_ω} describe the random walk components of the accelerometer and gyroscope bias drift. Each parameter is 3D (x, y, z). These values are typically taken from the IMU datasheet or estimated empirically from static measurements using Allan variance analysis.

It is important to note that while the full INS state $\mathbf{x}$ has 16 elements (position, velocity, attitude quaternion, and two bias vectors), $\mathbf{Q}$ only has 12 dimensions. This is because $\mathbf{Q}$ represents the “driving noise sources”, accelerometer and gyro noise plus their bias drifts, rather than direct noise on the state itself. The position, velocity, and attitude uncertainties arise indirectly as this noise propagates through the system dynamics over time.

How this propagation is carried out mathematically is handled through system linearization, transformation, and discretization of the process noise. These operations map the continuous time noise characteristics into discrete time uncertainty that spreads through all 16 states of the filter. The detailed formulation of this process, including the transformation matrices and covariance propagation equations, will be presented later in the “*State Estimation*” chapter.

For now, it is sufficient to understand that $\mathbf{Q}$ defines the statistical properties of the IMU noise processes that drive the uncertainty growth in the system. This provides the essential stochastic foundation for the subsequent estimation framework, where these disturbances are explicitly propagated and corrected using aiding measurements such as GNSS.

By separating deterministic kinematics from stochastic sensor effects, the INS formulation provides a structured foundation for real-time state estimation. This nominal model is particularly suited for use within an Error State Kalman Filter (ESKF), where it defines the expected system evolution between measurement updates and enables correction using external aiding sources such as GNSS or magnetometers.

Overall, the INS based motion model offers a compact and computationally efficient representation of the ASVs dynamics. While it omits detailed hydrodynamic effects, its balance of simplicity, generality, and compatibility with sensor fusion frameworks makes it a robust and practical choice for autonomous surface vehicle navigation and SLAM applications.

5.5.4 Aiding Measurement Model

The aiding measurements are obtained from a dual antenna GNSS system that provides absolute position and heading information in the NED frame. The two antennas are rigidly mounted on the stern, one on the port side and one on the starboard side, at known offsets from the vessels COG. This setup enables direct estimation of both the vessels global position and its yaw orientation, offering an absolute reference that continuously corrects the drift accumulated in the INS.

Accurate aiding of position and attitude is crucial for maintaining reliable long term navigation performance. While the INS can propagate the motion of the vessel accurately over short time spans, it inevitably accumulates errors due to sensor noise and bias drift. The GNSS updates counteract this drift by providing absolute position corrections, while the dual antenna baseline yields a precise heading measurement that stabilizes the yaw estimate, one of the most critical states for surface vessel guidance and control. Reliable yaw information directly improves trajectory tracking, waypoint following, and steering stability.

For autonomous surface vessels like the microAmpere, roll and pitch angles can be neglected due to the platforms inherently stable catamaran style hull. The dual pontoons provide strong passive stability and a nearly constant deck attitude even under moderate disturbances. Consequently, the vessel motion is effectively modeled in two dimensions, considering surge, sway, and yaw, while the vertical position z is retained from GNSS for completeness. This simplification reduces system complexity without sacrificing accuracy and aligns well with the physical behavior and operational characteristics of small, inherently stable surface craft.

Each GNSS receiver provides a position measurement expressed in the navigation frame as

$$\mathbf{z}_{p,i} = \mathbf{p}_{\text{GNSS}i/O}^n = \mathbf{p}_{b/O}^n + R_b^n(\mathbf{q}) \mathbf{p}_{\text{GNSS}i/b}^b + \mathbf{n}_{p,i}, \quad i \in \{1, 2\}$$

where $\mathbf{p}_{b/O}^n$ denotes the body origin (COG) position in the navigation frame, $\mathbf{p}_{\text{GNSS}i/b}^b$ are the known lever arm offsets of each antenna expressed in the body frame, and $\mathbf{n}_{p,i}$ represents zero mean Gaussian measurement noise. Since both antennas are positioned forward of the COG, this transformation is necessary to align the raw GNSS positions with the INS state representation.

From these two absolute position measurements, the relative vector between the antennas can be expressed in the navigation frame as

$$\mathbf{r}_{1/2}^n = \mathbf{r}_{GNSS1/GNSS2}^n = \mathbf{p}_{GNSS2/O}^n - \mathbf{p}_{GNSS1/O}^n$$

This baseline vector defines the spatial relationship between the antennas and allows extraction of the vessels yaw angle ψ through its horizontal components as

$$\psi = \text{atan2}(r_{1/2,y}^n, r_{1/2,x}^n)$$

Here, ψ represents the heading of the line connecting the port and starboard antennas relative to true north. The accuracy of the heading estimate depends on the baseline length, where a longer lateral separation provides greater sensitivity and higher precision. The dual antenna setup therefore yields a robust and drift free heading measurement that complements the attitude propagation performed by the INS.

The two GNSS position measurements and the derived yaw angle are then combined into a single aiding measurement vector that provides both global position and orientation information. The complete GNSS measurement is expressed as

$$\mathbf{z}_{\text{GNSS}} = \begin{bmatrix} \mathbf{p}_{b/O}^n \\ \psi \end{bmatrix} = h(\mathbf{x}) + \mathbf{n}_{\text{GNSS}}$$

where the nonlinear measurement function $h(\mathbf{x})$ relates the navigation state

$$\mathbf{x} = \begin{bmatrix} \mathbf{p}_{b/O}^n & \mathbf{v}_{b/O}^n & \mathbf{q} & \mathbf{b}_a & \mathbf{b}_g \end{bmatrix}^\top$$

to the observed quantities as

$$h(\mathbf{x}) = \begin{bmatrix} \mathbf{p}_{b/O}^n + R_b^n(\mathbf{q}) \mathbf{p}_{GNSS1/b}^b \\ \psi(\mathbf{q}) \end{bmatrix}, \quad \psi(\mathbf{q}) = \text{atan2}(2(q_w q_z + q_x q_y), 1 - 2(q_y^2 + q_z^2)) \quad (9)$$

In this formulation, the first component represents the transformed GNSS position measurement expressed in the navigation frame, while the second component computes the expected yaw angle ψ directly from the attitude quaternion $\mathbf{q}$. This quaternion based expression provides the same horizontal heading angle as that obtained geometrically from the dual antenna baseline, ensuring a consistent definition of yaw across both the INS and aiding measurement models. The term $\mathbf{n}_{\text{GNSS}}$ denotes zero mean Gaussian noise combining both position and heading uncertainties, with covariance $\mathbf{R}_{\text{GNSS}} = \text{diag}(\mathbf{R}_p, \sigma_\psi^2)$.

For small autonomous surface vessels such as the MicroAmpere, the platform demonstrates inherently stable roll and pitch dynamics due to its catamaran style pontoon design. This hydrodynamic stability allows the assumption that roll and pitch angles remain close to zero during nominal operation, effectively reducing the attitude representation to a single dominant rotation about the vertical axis, corresponding to the yaw angle. With this simplification, the GNSS aiding measurements primarily correct horizontal position and heading, while small vertical displacements and tilting motions are neglected, as they have minimal impact on overall navigation accuracy.

The statistical characteristics of the aiding measurements are described by the covariance matrix $\mathbf{R}_{\text{GNSS}}$, which defines the uncertainty associated with both position and yaw observations. The GNSS measurement noise is modeled as a zero mean Gaussian process, as this assumption simplifies the mathematical formulation of the estimation problem and allows the use of linear statistical tools such as the Kalman filter. Furthermore, most sensor noise processes exhibit approximately Gaussian behavior, and therefore all models in this work adopt Gaussian noise representations for analytical tractability and consistency.

$$\mathbf{n}_{\text{GNSS}} \sim \mathcal{N}(\mathbf{0}, \mathbf{R}_{\text{GNSS}})$$

with covariance

$$\mathbf{R}_{\text{GNSS}} = \begin{bmatrix} \sigma_x^2 & 0 & 0 & 0 \\ 0 & \sigma_y^2 & 0 & 0 \\ 0 & 0 & \sigma_z^2 & 0 \\ 0 & 0 & 0 & \sigma_\psi^2 \end{bmatrix} \quad (10)$$

where the diagonal entries represent the individual variances of the GNSS position and heading measurements. The horizontal position variances σ_x^2 and σ_y^2 , together with the vertical variance σ_z^2 , can either be taken from the GNSS receiver specifications or estimated empirically by collecting a sequence of static position samples and computing the sample covariance.

While all GNSS measurement variances are influenced by external factors such as satellite visibility, signal quality, and atmospheric conditions, the yaw variance σ_ψ^2 introduces additional complexity due to its dependence on the geometric configuration of the dual antenna baseline. Unlike position variances, which can be directly related to the GNSS positional dilution of precision and signal quality, the yaw covariance is a nonlinear function of baseline length, relative orientation, and phase measurement accuracy between the two antennas. Analytical methods exist for modeling this relationship using differential carrier phase error propagation and baseline geometry. However, for practical implementation in this work, σ_ψ^2 is determined empirically by recording heading data under static or low dynamic conditions and computing the sample variance. This provides a realistic and representative characterization of the true heading uncertainty while inherently capturing the combined effects of baseline geometry and signal conditions.

In practice, the resulting covariance matrix $\mathbf{R}_{\text{GNSS}}$ provides a quantitative description of the expected accuracy of the aiding measurements and directly influences the filter update weighting. Accurate estimation of $\mathbf{R}_{\text{GNSS}}$ is therefore critical to achieve consistent sensor fusion performance and to maintain the correct balance between inertial propagation and GNSS correction.

In summary, the dual antenna GNSS system provides a reliable and mathematically well defined aiding source that delivers both absolute position and heading information in the navigation frame. By constraining the inertial navigation solution with globally referenced measurements, it effectively limits drift accumulation and ensures consistent long term state estimation. This makes the aiding model well suited for marine applications where stable attitude and accurate horizontal localization are of primary importance.

5.5.5 Landmark Measurement Model

The landmark measurement model establishes the mathematical relationship between the ASV pose and observable landmarks within the map. This formulation follows the classical range-bearing model commonly adopted in SLAM literature, as it provides a compact and geometrically consistent way of expressing 2D landmark observations [14] [2]. The choice of the range-bearing model is motivated by its direct correspondence to the type of information available from the processed side scan sonar 2D map, where detected landmarks can be interpreted in terms of their relative distance and angle with respect to the vessels current position.

The ASV state $\mathbf{x}$ is defined by the INS model presented earlier in Equation (7). From this full state, only the position component $\mathbf{p}_{b/O}^n$ and the attitude quaternion $\mathbf{q}$ are required for the landmark observation. The vehicle pose is expressed in the NED frame, while the landmark measurements are expressed relative to the local body frame of the sonar.

Let the map contain a set of landmarks $\mathbf{m} = \{m^1, m^2, \dots, m^N\}$, where each landmark $m^i = [m_x^i, m_y^i, 0]^\top$ is fixed in the NED frame. Although the ASV state $\mathbf{p}_{b/O}^n = [x, y, z]^\top$ is fully 3D, the side scan sonar and corresponding landmark map are inherently 2D, describing features on a locally planar surface. Consequently, only the horizontal components (x, y) of both the vehicle position and the landmark locations are relevant for the range-bearing measurement, while variations in depth (z) are neglected.

The measurement function $h(\mathbf{x}, m^i)$ predicts the expected observation of the i -th landmark in the local body frame by transforming it from the navigation frame using the full quaternion rotation matrix $R_n^b(\mathbf{q})$. The expected range and bearing observation is expressed as

$$h(\mathbf{x}, m^i) = \begin{bmatrix} \sqrt{(m_{bx}^i)^2 + (m_{by}^i)^2} \\ \text{atan2}(m_{by}^i, m_{bx}^i) \end{bmatrix}, \quad m_b^i = R_n^b(\mathbf{q}) (m^i - \mathbf{p}_{b/O}^n) \quad (11)$$

This transformation converts the landmark position from the NED frame to the sensors local body frame, ensuring that both range and bearing are expressed relative to the vehicles pose and attitude. Although

the full quaternion rotation $R_n^b(\mathbf{q})$ is used for generality, only the planar motion components (surge, sway, yaw) contribute to the range-bearing measurement, while roll and pitch variations are small and can be neglected for the microAmpere ASV.

The range-bearing model provides an effective framework for representing side scan sonar landmark measurements, where image features such as rocks, seabed edges, or submerged structures are interpreted as discrete landmarks. Each detection is expressed as $\mathbf{z}^i = [z_r^i, z_\theta^i]^\top$, corresponding to the range and bearing of a landmark relative to the locally generated 2D sonar map.

In practical conditions, the sonar measurement is influenced by several environmental and geometric factors. The intensity and detectability of a landmark strongly depend on the incidence angle between the sonar beam and the surface normal of the object. At near normal incidence, strong reflections occur, while at grazing angles, reflections weaken due to scattering and shadowing effects. In addition, local sound speed variations caused by temperature and salinity gradients introduce refraction, bending the acoustic path and altering the apparent range of the detected feature.

To account for both environmental and geometric effects, the deterministic range bearing model in Equation (11) is extended with a systematic correction term and a stochastic noise component.

The measurement model relates the observed landmark measurement $\mathbf{z}^i$ to its predicted observation based on the current vehicle state $\mathbf{x}$ and the landmark position m^i . It is expressed as

$$\begin{bmatrix} z_r^i \\ z_\theta^i \end{bmatrix} = \mathbf{z}^i = h_{\text{corr}}(\mathbf{x}, m^i) + \mathbf{n}^i \quad (12)$$

where $\mathbf{z}^i = [z_r^i, z_\theta^i]^\top$ denotes the measured range and bearing of the i -th landmark, and $\mathbf{n}^i$ represents the stochastic measurement noise.

The predicted or expected landmark observation is described by the corrected deterministic measurement model

$$h_{\text{corr}}(\mathbf{x}, m^i) = h(\mathbf{x}, m^i) + \Delta h(\mathbf{x}, m^i, \theta^i) \quad (13)$$

where $h(\mathbf{x}, m^i)$ is the nominal geometric prediction of range and bearing based solely on the vehicle pose and landmark position as defined in Equation (11). The term $\Delta h(\mathbf{x}, m^i, \theta^i)$ introduces a deterministic correction that compensates for angle dependent reflection and refraction effects, while θ^i denotes the local incidence angle between the sonar beam and the line of sight to the landmark, ie bearing. The remaining term $\mathbf{n}^i$ models zero mean Gaussian noise with range dependent covariance $\mathbf{R}_{z^i}$, representing the random uncertainty in the measurement. Together, these components define how the actual landmark observation $\mathbf{z}^i$ deviates from the ideal geometric prediction due to both deterministic environmental effects and stochastic sensor noise.

The bearing θ^i is local incidence angle defined as the angle between the sonar beam direction and the line of sight to the landmark in the body frame:

$$\theta^i = \text{atan2}(m_{by}^i, m_{bx}^i), \quad m_b^i = R_n^b(\mathbf{q}) (m^i - \mathbf{p}_{b/O}^n)$$

where $R_n^b(\mathbf{q})$ transforms the landmark position from the navigation to the body frame. This angle changes with the ASV yaw and influences both reflection strength and measurement bias.

The correction term $\Delta h(\mathbf{x}, m^i, \theta^i)$ models this deterministic bias and is approximated as

$$\Delta h(\mathbf{x}, m^i, \theta^i) \approx \begin{bmatrix} \kappa_r(1 - \cos \theta^i) \\ \kappa_\theta \sin \theta^i \end{bmatrix}$$

where κ_r and κ_θ are empirically tuned parameters that describe how range and bearing measurements vary with incidence angle. The range component reduces for grazing angles due to weaker echoes, while the bearing term introduces a small angular offset caused by asymmetric scattering.

The random noise component is modeled as

$$\mathbf{n}^i \sim \mathcal{N}(\mathbf{0}, \mathbf{R}_{z^i}), \quad \mathbf{R}_{z^i} = \begin{bmatrix} \sigma_r^2(1 + \alpha_r(r^i)^2) & 0 \\ 0 & \sigma_\theta^2(1 + \alpha_\theta(r^i)^2) \end{bmatrix} \quad (14)$$

where $r^i = \sqrt{(m_{bx}^i)^2 + (m_{by}^i)^2}$ is the predicted landmark range. The parameters σ_r and σ_θ represent the nominal measurement noise at short distance, while α_r and α_θ model the range dependent growth in uncertainty due to acoustic attenuation, multipath effects, and beam spreading.

5.5.6 Model Comparison and Selection

The modeling framework combines two complementary representations, the Fossen kinetic model for high fidelity dynamics and the INS based kinematic model for efficient state propagation. The kinetic model provides a complete physical description including rigid body dynamics, hydrodynamic effects, and restoring forces, making it suitable for control design and dynamic analysis. However, its use requires extensive parameter identification that is impractical for small autonomous vessels, where only a limited subset of hydrodynamic coefficients can be reliably estimated.

The INS based kinematic model offers a simplified alternative focused on direct propagation of position, velocity, and attitude from inertial measurements. It removes the dependency on hydrodynamic parameters, enabling lightweight and robust integration with aiding sensors such as GNSS. The assumption that the IMU is located near the center of gravity minimizes lever arm effects, making this approach sufficiently accurate for navigation, SLAM, and control tasks. For these reasons, the INS formulation is adopted as the primary motion model in this thesis, providing an optimal trade off between model fidelity, computational cost, and implementation simplicity.

Similarly, for landmark sensing, the deterministic range-bearing model is sufficient for basic experiments or low noise environments. In contrast, the extended model accounts for incidence angle and range dependent effects, providing a more realistic representation of sonar measurement behavior under varying environmental and geometric conditions. The choice between models therefore depends on the desired accuracy and computational constraints, with the extended formulation preferred for high fidelity SLAM or sonar mapping applications.

5.6 Numerical Solvers

5.6.1 Implicit vs Explicit Solvers

Numerical solvers are used to integrate the system dynamics over time, approximating the continuous evolution of a state $\dot{x} = f(t, x)$ through discrete time steps Δt . Depending on how the derivative is evaluated, integration schemes are categorized as either “*explicit*” or “*implicit*”.

In an “*explicit*” method, the state update is computed directly from known quantities at the current time step as such

$$x_{k+1} = x_k + \Delta t f(t_k, x_k)$$

Making it simple and computationally efficient. However, explicit methods are only “*conditionally stable*”, meaning the chosen time step must be sufficiently small to maintain numerical stability. In contrast, an “*implicit*” method evaluates the derivative at the next time step,

$$x_{k+1} = x_k + \Delta t f(t_{k+1}, x_{k+1})$$

This requires solving a nonlinear equation for x_{k+1} , often using iterative techniques such as the Newton-Raphson method. Implicit solvers are generally “*unconditionally stable*”, allowing larger time steps and better handling of stiff systems, but at the cost of significantly higher computational complexity.

Figure 25 illustrates the stability regions of several explicit Runge-Kutta (ERK) methods of increasing order. As the order increases (eks ERK1 to ERK4), the stability region expands, allowing larger integration time steps before instability occurs. However, this improvement comes with additional intermediate evaluations of $f(t, x)$ per time step, increasing computational cost. Advanced explicit methods such as adaptive Runge-Kutta schemes (eks ARK4(5) or Dormand-Prince) improve stability by adjusting the time step dynamically, whereas implicit methods like Gauss-Legendre Runge-Kutta (GLRK) maintain stability by directly enforcing an error tolerance rather than relying on small time steps. This makes implicit methods far more robust but also significantly more computationally expensive, making implicit methods not practical for real-time.

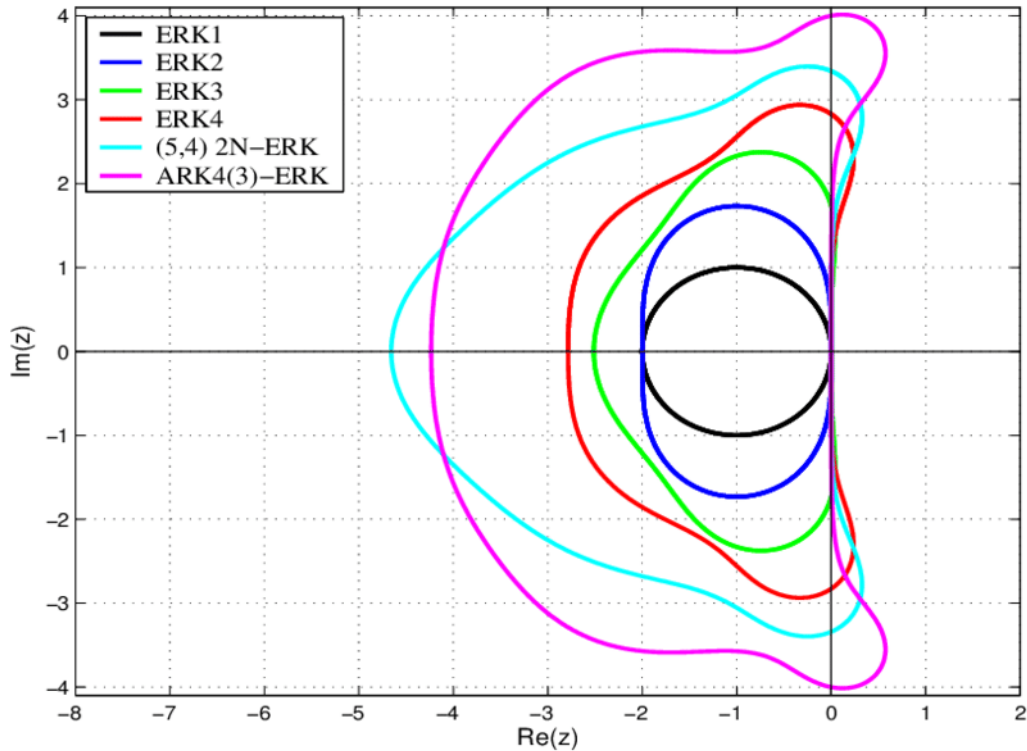


Figure 25: Stability regions of various explicit Runge-Kutta methods. Higher order methods support larger integration time steps before instability occurs, at the cost of increased computational effort.^[21]

In real-time navigation and estimation applications, such as the INS model implemented in this work, explicit solvers provide the optimal balance between accuracy and efficiency. Although implicit solvers are preferred for offline simulation or highly stiff systems, their iterative nature and computational load make them impractical for embedded execution. Moreover, any minor numerical drift introduced by the explicit integration scheme is compensated during sensor fusion, where aiding measurements (eks GNSS) correct accumulated errors at each update step.

Overall, ERK methods, particularly the 4th order ERK4 formulation, offer an efficient and sufficiently accurate approach for forward state propagation in real-time marine navigation systems.

5.6.2 Newton-Euler Method

The Newton-Euler method represents the simplest and most direct approach to numerically integrating a systems dynamics over time. It forms the foundation for explicit state propagation in real-time navigation and control systems, providing a fast and straightforward way to approximate the continuous evolution of the system state.

The general continuous time system is described by

$$\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u})$$

where $\mathbf{x}$ denotes the state vector, $\mathbf{u}$ represents the system inputs, and $f(\mathbf{x}, \mathbf{u})$ defines the continuous time motion model.

Using the explicit or forward Euler discretization, the system is approximated over discrete time intervals Δt as

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \Delta t f(\mathbf{x}_k, \mathbf{u}_k)$$

This update relies only on the current state and input, making the method fully explicit with no need for iteration or matrix inversion. Its direct formulation ensures minimal computational overhead, which is particularly important for real-time embedded execution.

The Newton-Euler method is explicit, first order accurate, and computationally efficient. Its simplicity makes it ideal for systems where high update rates and frequent measurement corrections are available. However, it is conditionally stable, and numerical errors can accumulate for large time steps or rapidly changing dynamics. In aided estimation systems, these effects are typically compensated by sensor fusion corrections such as GNSS or magnetometer updates.

Although higher order explicit methods like Runge-Kutta improve numerical accuracy, they require additional evaluations of $f(\mathbf{x}, \mathbf{u})$ and therefore increase computational cost. For real-time navigation and control, the Newton-Euler method provides an optimal trade-off between computational efficiency and integration accuracy, making it well suited for embedded state propagation in this work. in this work.

5.6.3 Runge-Kutta Methods (RK4)

The Runge-Kutta family of methods generalizes the explicit Newton-Euler integration scheme by combining several intermediate evaluations of the system dynamics to achieve higher accuracy. The general continuous time system is written as

$$\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u})$$

where $\mathbf{x}$ represents the state vector and $\mathbf{u}$ the control inputs.

The Runge-Kutta approach evaluates $f(\mathbf{x}, \mathbf{u})$ at multiple points within each time step Δt , and combines these intermediate slopes using a set of weighting coefficients defined by the Runge-Kutta formulation. This structure can be compactly represented using the Butcher tableau:

c_1				
c_2	a_{21}			
c_3	a_{31}	a_{32}		
c_4	a_{41}	a_{42}	a_{43}	
	b_1	b_2	b_3	b_4

Each constant defines how intermediate stages are calculated and combined.

For the classical 4th order Explicit Runge-Kutta (ERK4) method, the coefficients are given by:

$$\begin{array}{c|ccc}
 0 & & & \\
 \frac{1}{2} & \frac{1}{2} & & \\
 \frac{1}{2} & 0 & \frac{1}{2} & \\
 1 & 0 & 0 & 1 \\
 \hline
 & \frac{1}{6} & \frac{1}{3} & \frac{1}{3} & \frac{1}{6}
 \end{array}$$

Using these coefficients, the RK4 method is computed as

$$\begin{aligned}
 k_1 &= f(\mathbf{x}_k, \mathbf{u}_k) \\
 k_2 &= f(\mathbf{x}_k + \frac{\Delta t}{2} k_1, \mathbf{u}_k) \\
 k_3 &= f(\mathbf{x}_k + \frac{\Delta t}{2} k_2, \mathbf{u}_k) \\
 k_4 &= f(\mathbf{x}_k + \Delta t k_3, \mathbf{u}_k) \\
 \mathbf{x}_{k+1} &= \mathbf{x}_k + \Delta t \sum_{i=1}^4 b_i k_i
 \end{aligned}$$

Each k_i represents an intermediate slope that estimates the systems rate of change at different points inside the time step. The final update is a weighted average of these slopes, providing a more accurate approximation of the state evolution compared to the single slope Newton-Euler approach.

The RK4 method achieves a global truncation error of $\mathcal{O}(\Delta t^4)$, making it more accurate for smooth nonlinear systems. However, this improvement requires four evaluations of $f(\mathbf{x}, \mathbf{u})$ per integration step, which increases computational cost and memory usage.

The Newton-Euler method can be viewed as the 1st order Explicit Runge-Kutta scheme (ERK1), where only a single evaluation of $f(\mathbf{x}, \mathbf{u})$ is used. While ERK4 provides higher numerical precision, its added complexity offers minimal benefit in real-time embedded systems where aiding sensors, such as GNSS or magnetometers, continuously correct small integration errors.

In practical navigation and estimation systems, especially on embedded platforms, the Newton-Euler (ERK1) method remains the preferred choice due to its low computational load and sufficient accuracy. ERK4 is mainly reserved for offline simulation or analysis, where increased precision justifies the additional computational effort.

5.7 Factor Graphs

Factor graphs provide a formal framework for representing probabilistic relationships between system variables and sensor measurements. The system is expressed as a collection of local factors, each describing a specific probabilistic constraint such as a motion model, measurement model, or prior. Instead of working with the complete joint probability distribution, which is often high dimensional and computationally expensive, the estimation problem is decomposed into smaller, locally defined probability functions. Each factor depends only on the subset of variables relevant to that constraint, allowing independent evaluation and efficient combination of information from different parts of the system. This structure is conceptually related to the Hidden Markov Model discussed in the following chapter on State Estimation, where states are conditionally dependent over time. However, the factor graph provides a more general formulation that can represent arbitrary dependencies between variables, making it a suitable mathematical framework for expressing probabilistic relationships in problems such as SLAM and other state estimation tasks.

In this work, the factor graph is used to represent the relationships between the microAmpere ASV state variables and their associated sensor data for SLAM problem. The INS motion model defines the motion factors that connect consecutive time steps, while the processed Side Scan Sonar observations provide measurement factors that relate observed features to the estimated trajectory. This formulation gives a clear and consistent way to integrate different sensing modalities within one unified probabilistic system model.

The factor graph structure also forms the basis for optimization based estimation methods used later in Smoothing and Mapping (SAM), where the sparse connections between variables enable efficient computation of the most probable system trajectory.

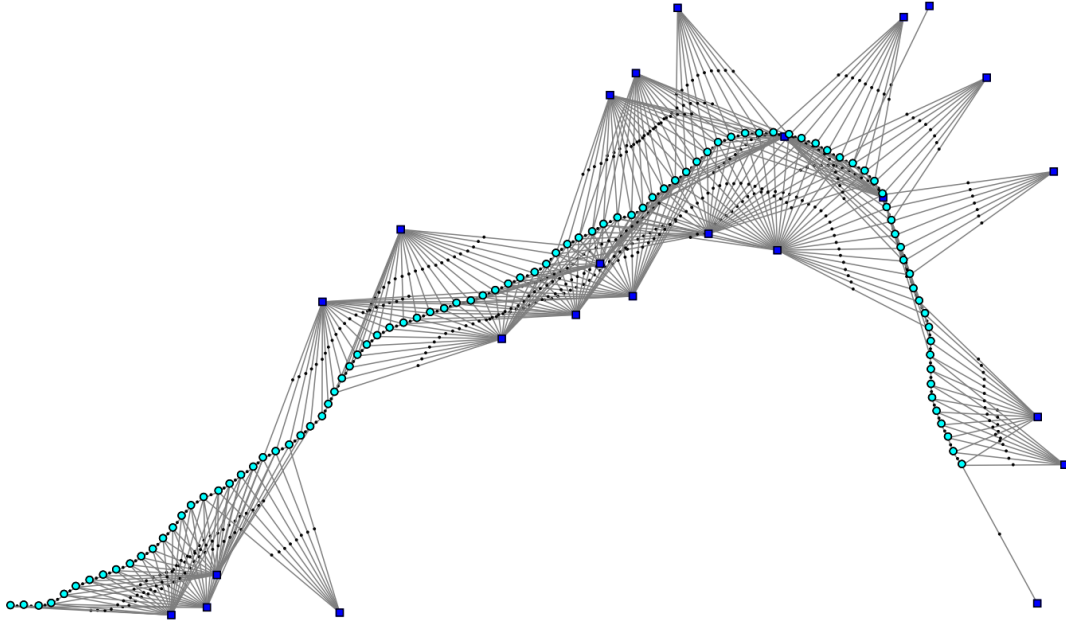


Figure 26: Example of a factor graph in a simulated SLAM problem. Light blue circles represent the estimated robot trajectory, and dark blue squares represent observed landmarks. The connecting black edges/lines are factors, each defining a local probabilistic constraint between variables.^[22]

6 State Estimation

6.1 Introduction

Estimation theory provides the mathematical foundation for inferring unknown quantities from noisy sensor data. In general, two main paradigms are used in estimation and signal processing, the Bayesian and the Frequency domain approaches. Each framework offers different strengths and assumptions depending on the type of system being analyzed.

The frequency domain approach is commonly applied in classical signal processing, where the primary goal is to extract information from stationary or periodic signals. Techniques such as spectral analysis, transfer function estimation, and classical filtering operate efficiently when system dynamics are linear and time invariant. However, these methods struggle to represent the nonlinear, time varying behavior typical of real world dynamic systems such as autonomous vehicles or navigation platforms. Their focus on steady state frequency content limits their ability to perform recursive state estimation in systems subject to stochastic disturbances and nonlinear motion.

The Bayesian approach, in contrast, provides a probabilistic framework for representing and updating uncertainty over time. Instead of working in the frequency domain, Bayesian estimation directly models the probability distribution of the system state given all available measurements. This allows the estimation process to incorporate both process dynamics and measurement models, enabling consistent fusion of heterogeneous sensor data. The Bayesian framework forms the foundation for modern state estimation algorithms such as the Kalman Filter, Extended Kalman Filter, and Unscented Kalman Filter.

The core idea of Bayesian estimation is recursive probabilistic inference. At each time step, the state distribution is first “*predicted*” forward in time using the motion model, and then “*corrected*” using the latest aiding sensor measurements. This recursive structure enables efficient real-time updates of the system state without requiring storage of the entire measurement history.

The recursive form of the Bayes filter can be expressed in two steps [14]. First, the “*prediction step*” propagates the state distribution forward in time:

$$p(\mathbf{x}_k | \mathbf{z}_{1:k-1}) = \int p(\mathbf{x}_k | \mathbf{x}_{k-1}) p(\mathbf{x}_{k-1} | \mathbf{z}_{1:k-1}) d\mathbf{x}_{k-1}$$

Then, the “*update step*” incorporates the new measurement:

$$p(\mathbf{x}_k | \mathbf{z}_{1:k}) = \frac{p(\mathbf{z}_k | \mathbf{x}_k) p(\mathbf{x}_k | \mathbf{z}_{1:k-1})}{\int p(\mathbf{z}_k | \mathbf{x}_k) p(\mathbf{x}_k | \mathbf{z}_{1:k-1}) d\mathbf{x}_k}$$

Together, these equations define the recursive Bayesian estimation framework used throughout this thesis.

In summary, the Bayesian paradigm offers a flexible and mathematically consistent method for estimating nonlinear system states under uncertainty, making it particularly suitable for navigation and sensor fusion applications. The frequency domain approach remains valuable for analyzing stationary signals or identifying system dynamics, but it is less effective for recursive, model-based estimation of dynamic processes.

The theoretical concepts and formulations presented in this section are based on the work by Edmund Brekke in “*Fundamentals of Sensor Fusion*” [14]. The discussion and implementations of the Unscented Kalman Filter and its manifold based extension are supported by the research papers [23] and [24].

6.2 Bayes Filter

The Bayes filter provides the probabilistic foundation for recursive state estimation by modeling the system as a Markov process, where the current state $\mathbf{x}_k$ depends only on the previous state $\mathbf{x}_{k-1}$ and control input $\mathbf{u}_{k-1}$, and each measurement $\mathbf{z}_k$ depends only on the current state. This forms a Hidden Markov Model in which the true state is not directly observable and must be inferred from noisy sensor measurements. Both the process and measurement noise are assumed to be white, zero mean, and statistically independent, allowing the state distribution to be updated consistently within the Bayesian framework.

The Bayes filter operates recursively in two stages: prediction and update. The “*prediction step*” propagates the previous state estimate forward in time using the process model:

$$p(\mathbf{x}_k | \mathbf{z}_{1:k-1}) = \int p(\mathbf{x}_k | \mathbf{x}_{k-1}) p(\mathbf{x}_{k-1} | \mathbf{z}_{1:k-1}) d\mathbf{x}_{k-1}$$

This step accounts for system dynamics and process noise, providing a prior belief of the current state before any new measurement is considered. The “*update step*” then incorporates the new measurement $\mathbf{z}_k$ to refine the estimate:

$$p(\mathbf{x}_k | \mathbf{z}_{1:k}) = \frac{p(\mathbf{z}_k | \mathbf{x}_k) p(\mathbf{x}_k | \mathbf{z}_{1:k-1})}{\int p(\mathbf{z}_k | \mathbf{x}_k) p(\mathbf{x}_k | \mathbf{z}_{1:k-1}) d\mathbf{x}_k}$$

Here, the numerator represents the likelihood of observing the measurement given a state hypothesis, multiplied by the predicted prior, while the denominator ensures normalization of the resulting probability distribution.

This recursive formulation captures the fundamental idea of probabilistic state estimation, where one combines prior knowledge with new information to obtain a posterior estimate. In practice, however, exact computation of these integrals is rarely feasible due to nonlinear process models, non Gaussian noise, and the high dimensionality of the state space. Evaluating the full probability distributions quickly becomes computationally intractable for real-time systems.

For this reason, practical implementations rely on approximate Bayesian filters that make simplifying assumptions to achieve tractable computation. The Kalman Filter assumes linear models with Gaussian noise, while the Extended Kalman Filter and Unscented Kalman Filter extend these ideas to nonlinear systems using local linearization or deterministic sampling. These methods retain the core Bayesian structure but operate with compact parametric representations of uncertainty, typically through the state mean and covariance.

A simplified overview of the process and measurement relationships is shown in Figure 27. The figure illustrates how control inputs $\mathbf{u}_k$ influence the system state $\mathbf{x}_k$, which in turn generates measurements $\mathbf{z}_k$ through a noisy observation model.

The formulation and structure presented here follow the conventions introduced by Edmund Brekke in “*Fundamentals of Sensor Fusion*” [14], which form the theoretical foundation for the estimation framework implemented in this thesis.

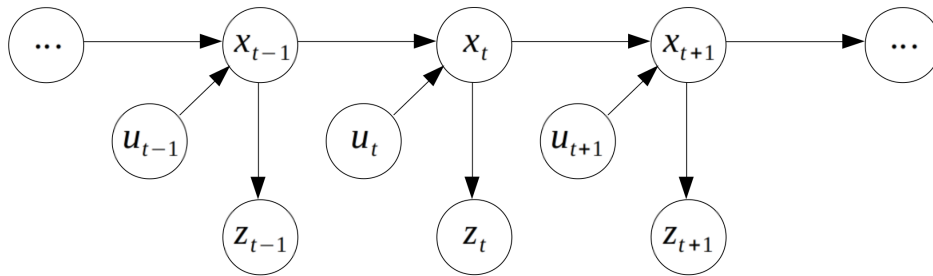


Figure 27: Hidden Markov Model representation of the Bayes filter, showing the relationships between state $\mathbf{x}$, measurement $\mathbf{z}$, and control input $\mathbf{u}$.^[25]

6.3 Kalman Filter

Because the full Bayesian filter formulation is computationally intractable for most practical systems, the Kalman filter provides an efficient analytical solution under the assumption of linear system dynamics and Gaussian noise. It represents a special case of the Bayesian recursion where both the process and measurement models are linear, and all uncertainties are modeled as zero mean white Gaussian noise.

The discrete time linear Gaussian system is defined as

$$\mathbf{x}_{k+1} = F\mathbf{x}_k + G\mathbf{u}_k + \mathbf{w}_k, \quad \mathbf{z}_k = H\mathbf{x}_k + \mathbf{v}_k$$

where $\mathbf{x}_k$ is the state vector, $\mathbf{u}_k$ is the control input, and $\mathbf{z}_k$ is the measurement vector. The matrices F , G , and H define the system dynamics, control influence, and measurement relationship, respectively. The process noise $\mathbf{w}_k \sim \mathcal{N}(0, Q)$ and measurement noise $\mathbf{v}_k \sim \mathcal{N}(0, R)$ are assumed white, uncorrelated, and Gaussian distributed.

In practical systems, the continuous time model $\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u})$ must be converted into a discrete time form suitable for digital implementation. For a linear system of the form

$$\dot{\mathbf{x}} = A\mathbf{x} + B\mathbf{u}$$

the equivalent discrete time model is expressed as

$$\mathbf{x}_{k+1} = F\mathbf{x}_k + G\mathbf{u}_k$$

where F and G represent the discrete state transition and control matrices, respectively.

The most accurate way to compute these matrices is through the exact exponential discretization,

$$F = e^{A\Delta t}, \quad G = \int_0^{\Delta t} e^{A\tau} B d\tau$$

where Δt is the sampling period. This formulation comes directly from the analytical solution of the linear time invariant (LTI) differential equation and preserves the systems true dynamics. It is commonly used in simulation and offline analysis where computational cost is less critical.

For real-time embedded systems, a simpler and computationally efficient approximation is usually applied. When the sampling interval Δt is small, the matrix exponential can be approximated using a first order Taylor expansion

$$F \approx I + A\Delta t, \quad G \approx B\Delta t$$

This approach, known as the forward Euler or Newton-Euler discretization, provides sufficient accuracy for high rate estimation and control applications while requiring minimal computation. The resulting discrete system maintains stability and precision as long as Δt remains small relative to the systems dominant time constants.

Higher order integration methods such as ERK schemes can also be used to improve numerical accuracy. These methods evaluate the process function multiple times within each integration step, producing a more accurate approximation of the continuous dynamics. However, in most practical navigation and estimation systems, such increased precision provides limited benefit. Since the filter state is frequently corrected by external sensor updates, the integration error accumulated between prediction steps remains small. Therefore, the forward Euler discretization is generally preferred for real-time embedded implementations due to its simplicity, deterministic execution time, and sufficient numerical accuracy.

The Kalman filter operates recursively through two main stages that alternate over time. The “*prediction step*” advances the state estimate and its associated uncertainty based on the process model, using the latest available control input. This step provides a prior estimate of the system state before any new measurement is processed.

The predicted mean and covariance are computed as

$$\begin{aligned} \hat{\mathbf{x}}_{k|k-1} &= F\hat{\mathbf{x}}_{k-1|k-1} + G\mathbf{u}_{k-1}, \\ P_{k|k-1} &= FP_{k-1|k-1}F^\top + Q, \end{aligned}$$

where $\hat{\mathbf{x}}_{k|k-1}$ and $P_{k|k-1}$ represent the predicted state and covariance, F and G are the discrete system matrices, $\mathbf{u}_{k-1}$ is the control input, and Q is the process noise covariance capturing model uncertainty.

The “*update step*” incorporates the most recent measurement to refine the predicted estimate. This is achieved by computing the Kalman gain, which determines the optimal weighting between the predicted state and the new measurement:

$$\begin{aligned} K &= P_{k|k-1} H^\top (H P_{k|k-1} H^\top + R)^{-1}, \\ \hat{\mathbf{x}}_{k|k} &= \hat{\mathbf{x}}_{k|k-1} + K(\mathbf{z}_k - H \hat{\mathbf{x}}_{k|k-1}), \\ P_{k|k} &= (I - KH)P_{k|k-1}. \end{aligned}$$

Here, H is the measurement model matrix, R is the measurement noise covariance, and $\mathbf{z}_k$ is the received measurement. The innovation term $(\mathbf{z}_k - H \hat{\mathbf{x}}_{k|k-1})$ represents the discrepancy between the predicted measurement and the actual observation. The Kalman gain K determines how much influence the new measurement has relative to the predicted state, optimally balancing model and sensor uncertainty.

By iterating the prediction and update steps, the filter continuously refines the state estimate to provide an optimal and unbiased estimate of the true system state under linear and Gaussian noise assumptions.

The Kalman filter is optimal only under the assumptions of a linear system, Gaussian noise, and white uncorrelated process and measurement disturbances. For nonlinear systems, such as those encountered in INS motion model, these assumptions no longer hold, and extensions like the Extended Kalman Filter (EKF) or Unscented Kalman Filter (UKF) are required. Nonetheless, the Kalman filter remains the conceptual foundation of all modern recursive estimation algorithms.

Figure 28 illustrates the core idea of the Kalman filter, showing how the predicted and measured probability distributions are combined to yield an optimal state estimate with minimum variance.

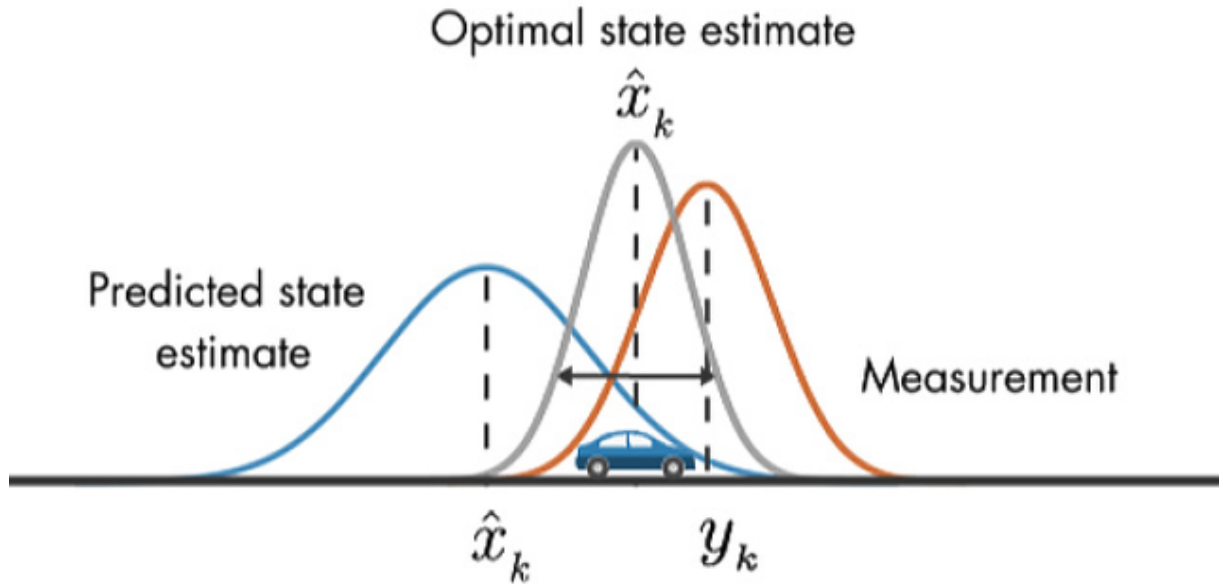


Figure 28: Kalman filter estimation principle combining prediction and measurement distributions to produce the optimal state estimate $\hat{\mathbf{x}}_k$.^[26]

6.4 Extended Kalman Filter

The Extended Kalman Filter (EKF) adapts the Kalman filter to nonlinear systems by linearizing the process and measurement models around the current state estimate. This lets the standard Kalman filter equations operate locally while still handling nonlinear vehicle dynamics, navigation, and orientation systems. The nonlinear models are given by:

$$\mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k) + \mathbf{w}_k, \quad \mathbf{z}_k = h(\mathbf{x}_k) + \mathbf{v}_k$$

where $\mathbf{x}_k$ is the system state, $\mathbf{u}_k$ is the control input, and $\mathbf{z}_k$ is the measurement vector. The process and measurement noises $\mathbf{w}_k$ and $\mathbf{v}_k$ are assumed to be zero mean, white, and Gaussian distributed with covariances Q and R , respectively.

To apply the Kalman filter framework, the nonlinear functions $f(\cdot)$ and $h(\cdot)$ must be approximated by their first order Taylor expansion around the current state estimate $\hat{\mathbf{x}}_{k|k-1}$. This yields the linearized system matrices

$$F_k = \left. \frac{\partial f}{\partial \mathbf{x}} \right|_{\hat{\mathbf{x}}_{k|k-1}, \mathbf{u}_k}, \quad H_k = \left. \frac{\partial h}{\partial \mathbf{x}} \right|_{\hat{\mathbf{x}}_{k|k-1}}$$

The Jacobian matrices F_k and H_k describe how small perturbations in the state affect the predicted dynamics and the expected measurements. This linearization step is local and valid only within the vicinity of the current state estimate, as illustrated in Figure 29.

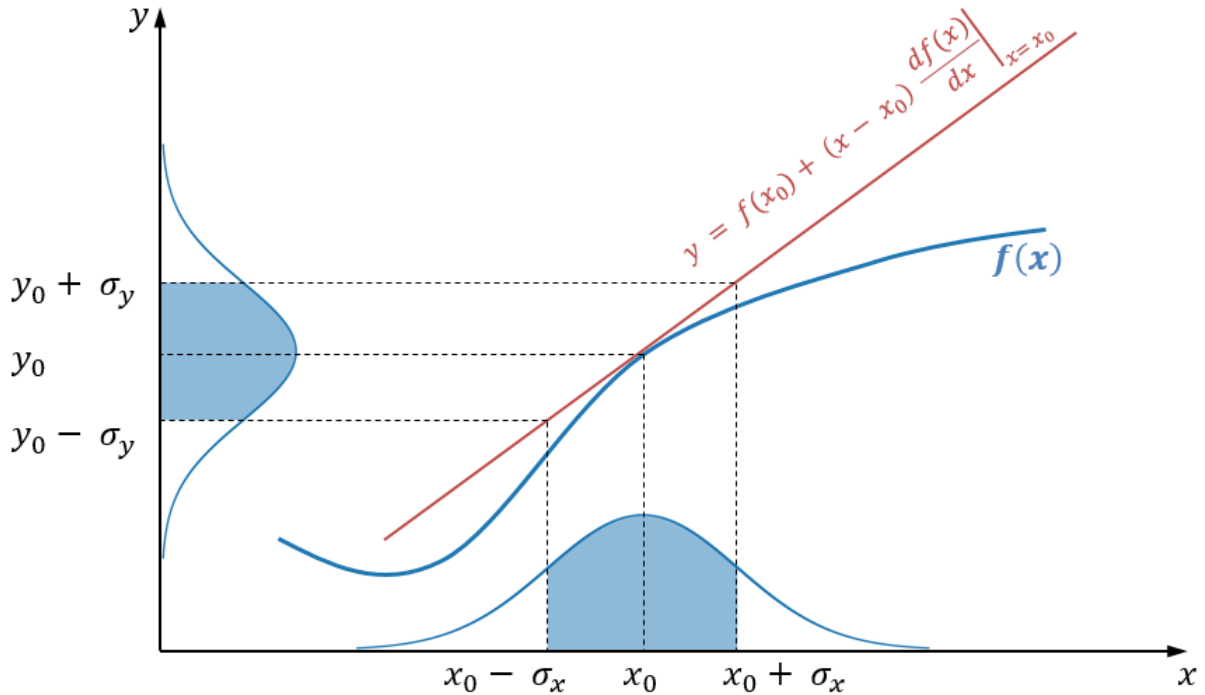


Figure 29: Illustration of local linearization in the Extended Kalman Filter. The nonlinear function $f(x)$ is approximated by its tangent around the current estimate, allowing local linear propagation.^[27]

An accurate discretization of the continuous time dynamics is essential to ensure consistent state propagation. Here, the continuous model $\dot{\mathbf{x}} = A\mathbf{x} + B\mathbf{u}$ is discretized using the “Zero-Order Hold (ZOH)” method, which assumes that the control input remains constant within each sampling interval $[k\Delta t, (k+1)\Delta t)$.

The resulting discrete time model is given by

$$\mathbf{x}_{k+1} = F_k \mathbf{x}_k + G_k \mathbf{u}_k$$

where

$$F_k = e^{A\Delta t}, \quad G_k = \int_0^{\Delta t} e^{A\tau} B d\tau$$

This can be computed compactly using the augmented matrix exponential

$$e^{\begin{bmatrix} A & B \\ 0 & 0 \end{bmatrix} \Delta t} = \begin{bmatrix} F_k & G_k \\ 0 & I \end{bmatrix} \quad (15)$$

The ZOH method preserves both the stability and dynamic behavior of the original continuous time system exactly for LTI models. It provides a numerically stable and accurate discretization, making it particularly suitable for embedded real-time implementations.

In digital systems, the matrix exponential $e^{A\Delta t}$ cannot be computed analytically and must be approximated numerically. A reliable and efficient method is the “*Padé approximation with scaling and squaring*”, which provides a good balance between computational speed and numerical accuracy.

The Padé approximation expresses the exponential as a rational function:

$$e^A \approx \left(I - \frac{A}{2} + \frac{A^2}{12} - \frac{A^3}{120} + \dots \right)^{-1} \left(I + \frac{A}{2} + \frac{A^2}{12} + \frac{A^3}{120} + \dots \right) \quad (16)$$

where higher order terms improve accuracy at the cost of computation. To avoid instability for large $\|A\|$, the matrix is first scaled down by a power of two, computed as

$$A_s = \frac{A}{2^s} \quad (17)$$

where s is chosen such that $\|A_s\|$ is below a defined threshold. The exponential of A_s is then computed using the Padé rational form, and the final result is obtained by repeated squaring:

$$e^A = (e^{A_s})^{2^s} \quad (18)$$

This method ensures stable and efficient computation of matrix exponentials even for high order systems, making it practical for embedded discretization of continuous time models.

When using nonlinear dynamics, the continuous time system is generally expressed as

$$\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u})$$

To implement the continuous time dynamics $\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u})$ on a digital system, the time derivative must be approximated numerically. This is achieved through time discretization, where the continuous evolution of the state is expressed as a difference equation over discrete sampling intervals.

One of the simplest and most widely used discretization schemes is the forward (Newton-Euler) method.

$$\mathbf{x}_{k+1} \approx \mathbf{x}_k + \Delta t f(\mathbf{x}_k, \mathbf{u}_k)$$

In this discrete form, the function $f(\mathbf{x}, \mathbf{u})$ used within the EKF no longer represents the instantaneous derivative $\dot{\mathbf{x}}$, but rather the integrated or “*propagated*” state over one time step. For notational simplicity, many EKF formulations redefine the discrete transition function as

$$f_d(\mathbf{x}_k, \mathbf{u}_k) = \mathbf{x}_k + \Delta t f(\mathbf{x}_k, \mathbf{u}_k)$$

where $f_d(\cdot)$ implicitly includes the time step integration. This convention is used throughout this work to maintain concise notation while ensuring consistency with the underlying continuous time model.

The EKF prediction and update steps then follow directly from the linearized system. During the “*prediction step*”, the nonlinear process model is propagated as

$$\hat{\mathbf{x}}_{k|k-1} = f_d(\hat{\mathbf{x}}_{k-1|k-1}, \mathbf{u}_{k-1})$$

However, for the INS motion model, this propagation method can be significantly improved by replacing the simple Newton-Euler forward integration with a more accurate “*hybrid inertial propagation scheme*” that directly integrates IMU measurements over each time step. This approach captures the coupling between rotation, acceleration, and gravity, providing a physically consistent motion update. The discrete time transition function $f_d(\mathbf{x}_k, \mathbf{u}_k)$ operates on the previous state and control input as defined in Equation 7, and evolves according to

$$f_d(\mathbf{x}_k, \mathbf{u}_k) = \mathbf{x}_{k+1} = \begin{bmatrix} \mathbf{p}_{b/O_{k+1}}^n \\ \mathbf{v}_{b/O_{k+1}}^n \\ \mathbf{q}_{k+1} \\ \mathbf{a}_{b_{k+1}} \\ \omega_{b_{k+1}} \end{bmatrix} = \begin{bmatrix} \mathbf{p}_{b/O_k}^n + \mathbf{v}_{b/O_k}^n \Delta t + \frac{1}{2}(R(\mathbf{q}_k)(\mathbf{a}_m - \mathbf{a}_{b_k}) + \mathbf{g}^n)\Delta t^2 \\ \mathbf{v}_{b/O_k}^n + (R(\mathbf{q}_k)(\mathbf{a}_m - \mathbf{a}_{b_k}) + \mathbf{g}^n)\Delta t \\ \mathbf{q}_k \otimes \text{Exp}([\omega_m - \omega_{b_k}]_{\times} \Delta t) \\ \mathbf{a}_{b_k} - p_{ab}\mathbf{a}_{b_k}\Delta t \\ \omega_{b_k} - p_{\omega b}\omega_{b_k}\Delta t \end{bmatrix} \quad (19)$$

It can be shown that this discrete propagation arises from integrating the continuous time INS motion equations and discretizing them according to Equation 6. This formulation inherently accounts for rotational motion through quaternion based attitude integration and incorporates gravity in the navigation frame, ensuring physically consistent position and velocity updates. Consequently, this hybrid propagation scheme provides a more accurate and numerically stable prediction model, forming the basis for modern ESKF and preintegration frameworks.

Now the covariance prediction is computed using the linearized Jacobian:

$$P_{k|k-1} = F_k P_{k-1|k-1} F_k^\top + Q$$

The “*update step*” corrects the predicted estimate using the measurement model:

$$\begin{aligned} K &= P_{k|k-1} H^\top (H P_{k|k-1} H^\top + R)^{-1} \\ \hat{\mathbf{x}}_{k|k} &= \hat{\mathbf{x}}_{k|k-1} + K[\mathbf{z}_k - h(\hat{\mathbf{x}}_{k|k-1})] \\ P_{k|k} &= (I - K H) P_{k|k-1} \end{aligned}$$

After each update, attitude states represented by quaternions are normalized to maintain a valid unit quaternion. This step is crucial in inertial navigation systems, as numerical drift in the quaternion magnitude can lead to significant orientation errors over time. The normalization ensures that the quaternion remains on the unit hypersphere, preserving a valid rotation representation:

$$\mathbf{q}_{k|k} \leftarrow \frac{\mathbf{q}_{k|k}}{\|\mathbf{q}_{k|k}\|}.$$

Here, $\mathbf{q}_{k|k}$ denotes the updated quaternion after the correction step, and $\|\mathbf{q}_{k|k}\|$ is its Euclidean norm. This operation enforces the constraint $\|\mathbf{q}\| = 1$, guaranteeing that the quaternion continues to represent a proper rotation without scale distortion.

The EKF provides a computationally efficient framework for nonlinear estimation. However, it relies heavily on the validity of local linearization, which may lead to divergence if the system dynamics are strongly nonlinear or the initial state uncertainty is large. In particular, when dealing with orientation states represented by quaternions or with highly coupled nonlinear motion models such as those in INS, the EKF’s 1st order approximation may fail to capture the true system behavior.

Despite these limitations, the EKF remains widely used in embedded navigation systems due to its simplicity, real-time efficiency, and well understood theoretical foundation. An important improvement to the EKF is the Error State Kalman Filter (ESKF), which reformulates the estimation problem by linearizing not around the full nominal states but around the small error states instead. These error variables tend to behave more linearly and remain bounded, resulting in improved numerical stability and consistency, especially for systems with rotational dynamics such as those using quaternions. Essentially, the ESKF provides a smarter way to linearize highly nonlinear systems by focusing on the deviation from the nominal trajectory rather than the absolute state itself. For applications requiring even higher robustness to nonlinearity, alternative methods such as the Unscented Kalman Filter (UKF) or manifold based approaches can provide improved performance at the cost of increased computational complexity.

6.5 Error State Kalman Filter

6.5.1 Concept and Motivation

While the Extended Kalman Filter provides a powerful framework for nonlinear estimation, it still linearizes directly around the full nominal state. For systems such as the INS, where the state includes both position, velocity, and attitude (represented by quaternions), this approach can lead to numerical inconsistencies, unstable orientation updates, and loss of orthogonality in rotation representations.

To address these issues, the Error State Kalman Filter (ESKF) reformulates the estimation problem by separating the system into two parts, a nonlinear “nominal state” $\mathbf{x}$ and a small linear “error state” $\delta\mathbf{x}$. The nominal state follows the nonlinear dynamics of the INS, while the error state captures small deviations from this nominal trajectory. This approach allows the filter to maintain accurate nonlinear propagation of the nominal state while estimating only small, approximately linear error quantities.

6.5.2 Error State Formulation

The nominal INS model is already defined earlier in this work in Equation 6 with nominal states Equation 7.

From the nominal INS model, the corresponding error state INS model is obtained by linearizing the nonlinear equations around the current nominal trajectory. This process isolates the small deviations between the true state and the estimated nominal state, allowing the linearized error dynamics to be described by a 1st order system. Following the formulation by Edmund Brekke in “*Fundamentals of Sensor Fusion*” [14], the continuous time linear time varying (LTV) error state dynamics are expressed as

$$\delta\mathbf{x} = \begin{bmatrix} \delta\mathbf{p}_{b/O}^n \\ \delta\mathbf{v}_{b/O}^n \\ \delta\theta \\ \delta\mathbf{a}_b \\ \delta\omega_b \end{bmatrix} \quad (20)$$

$$\delta\dot{\mathbf{x}} = A(\mathbf{x}) \delta\mathbf{x} + G(\mathbf{x}) \mathbf{n} \quad (21)$$

Here, $\delta\mathbf{x}$ represents the small deviation between the true and nominal state vectors, and $\mathbf{n}$ is the continuous time process noise, typically modeled as zero mean Gaussian white noise. The system matrices $A(\mathbf{x})$ and $G(\mathbf{x})$ depend on the current nominal state and describe how these small deviations evolve in time.

The attitude error is represented using the small angle quaternion approximation, which linearizes the rotation error between the estimated and true attitudes. This approximation holds for small orientation deviations and is expressed as

$$\delta\mathbf{q} \approx \begin{bmatrix} 1 \\ \frac{\delta\theta}{2} \end{bmatrix}$$

This relation ensures that the attitude error $\delta\theta$ can be treated as a 3D vector in the local tangent space of the unit quaternion manifold, simplifying the linearization of rotational dynamics while maintaining geometric consistency.

6.5.3 Process Model and Jacobians

The linearized system matrices $A(\mathbf{x})$ and $G(\mathbf{x})$ are given by

$$A(\mathbf{x}) = \begin{bmatrix} \mathbf{0} & \mathbf{I} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & -R_b^n(\mathbf{q})[\mathbf{a}_m - \mathbf{a}_b]_{\times} & -R_b^n(\mathbf{q}) & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & -[\omega_m - \omega_b]_{\times} & \mathbf{0} & -\mathbf{I} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & -\mathbf{p}_{ab}\mathbf{I} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & -\mathbf{p}_{\omega b}\mathbf{I} \end{bmatrix}, \quad G(\mathbf{x}) = \begin{bmatrix} \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ -\mathbf{R}_b^n(\mathbf{q}) & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & -\mathbf{I} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{I} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{I} \end{bmatrix} \quad (22)$$

In this representation, $R_b^n(\mathbf{q})$ is the rotation matrix from the body to the navigation frame obtained from the current quaternion estimate, and $[\cdot]_{\times}$ denotes the skew symmetric cross product operator. The matrix $A(\mathbf{x})$ describes how small perturbations in position, velocity, attitude, and sensor biases evolve, including the coupling between kinematics and IMU bias dynamics, while $G(\mathbf{x})$ maps continuous time IMU and

bias noise into the error states. This linearized model enables covariance propagation in the ESKF by providing a well behaved approximation of how uncertainty evolves, while the nonlinear nominal INS runs independently.

6.5.4 Discretization of Error Dynamics

Discretization of the continuous error dynamics is critical for digital implementation. In theory, the discrete time transition matrix is given by

$$F_k = e^{A(\mathbf{x})\Delta t}, \quad Q_k = \int_0^{\Delta t} e^{A(\mathbf{x})\tau} G(\mathbf{x}) Q G(\mathbf{x})^\top e^{A(\mathbf{x})^\top \tau} d\tau \quad (23)$$

However, directly computing these expressions is infeasible in real-time embedded systems due to the computational cost of evaluating the matrix exponential and noise covariance integral at every time step. Several discretization strategies exist to approximate this process efficiently while maintaining filter stability and accuracy.

The “*Cayley Hamilton series*” provides an exact polynomial expansion of the exponential using the characteristic polynomial of $A(\mathbf{x})$. However, it requires evaluating high order matrix powers and is therefore impractical for real-time estimation.

A more robust and widely used approach is the Zero Order Hold (ZOH) discretization introduced in (15). By assuming constant inputs and noise within each sampling interval, ZOH preserves the exact dynamics of linear time invariant systems and yields the discrete transition matrix F_k and process noise covariance Q_k through an augmented matrix exponential. This provides an accurate continuous to discrete mapping while avoiding the instabilities typical of low order approximations.

However, evaluating the augmented matrix exponential remains expensive for large or time varying systems. To address this, the “*Padé approximation with scaling and squaring*” (16)-(18) is used in this work to compute the exponential efficiently and with good numerical conditioning. Padé offers a strong balance between speed and accuracy, making it suitable for embedded implementation. In contrast, crude first order approximations such as $F_k \approx I + A(\mathbf{x})\Delta t$ should be avoided, as they degrade covariance consistency and may cause divergence under high dynamics or long sampling intervals.

6.5.5 Measurement Model and Jacobian

Before defining the discrete time ESKF stages, the measurement Jacobian structure must be introduced. The total Jacobian used in the update step is expressed as

$$H = H_{\mathbf{x}} X_{\delta\mathbf{x}}.$$

Here, $H_{\mathbf{x}}$ is the Jacobian of the measurement function with respect to the nominal state, obtained by linearizing the measurement model in Equation 9. Since this measurement function provides position and yaw observations, $H_{\mathbf{x}}$ captures how small variations in the nominal position and attitude affect the predicted GNSS position and yaw outputs.

However, because the ESKF performs estimation in the error state domain, the measurement sensitivity must be expressed with respect to the error state $\delta\mathbf{x}$. The matrix $X_{\delta\mathbf{x}}$ provides this mapping by relating perturbations in the nominal state to those in the error state. It has a fixed block diagonal structure:

$$X_{\delta\mathbf{x}} = \begin{bmatrix} I_6 & 0 & 0 \\ 0 & Q_{\delta\theta} & 0 \\ 0 & 0 & I_6 \end{bmatrix}$$

where $Q_{\delta\theta}$ relates small attitude errors $\delta\theta$ to quaternion perturbations and is derived from the differential of $\mathbf{q} \otimes \delta\mathbf{q}$ evaluated at $\delta\theta = 0$, where $\mathbf{q} = [\eta \ \epsilon_1 \ \epsilon_2 \ \epsilon_3]^T$:

$$Q_{\delta\theta} \approx \frac{1}{2} \begin{bmatrix} -\epsilon_1 & -\epsilon_2 & -\epsilon_3 \\ \eta & -\epsilon_3 & \epsilon_2 \\ \epsilon_3 & \eta & -\epsilon_1 \\ -\epsilon_2 & \epsilon_1 & \eta \end{bmatrix}.$$

Together, H_x and $X_{\delta x}$ form the complete measurement Jacobian $H = H_x X_{\delta x}$, which links measurement residuals to the linearized error state and enables consistent correction of both translational and rotational components during the update phase of the ESKF.

6.5.6 Filter Algorithm

The ESKF estimation process consists of several distinct stages, each responsible for different aspects of the nominal and error state propagation. The filter propagates the nonlinear nominal state using the INS equations, while maintaining a linearized covariance model of the small error dynamics. Unlike the EKF, the ESKF does not directly propagate the error-state mean, as it is always reset to zero after each correction. Instead, only the nominal state and the error covariance are propagated during the prediction stage.

Prediction step:

At the beginning of each iteration, the error state mean is initialized to zero:

$$\delta \hat{\mathbf{x}}_{k|k-1} = \mathbf{0}$$

For the ESKF, the nominal state is propagated using the same nonlinear discrete time INS process model as in the EKF (Equation 19):

$$\hat{\mathbf{x}}_{k|k-1} = f_d(\hat{\mathbf{x}}_{k-1|k-1}, \mathbf{u}_{k-1})$$

Simultaneously, the covariance of the error state is propagated using the linearized system matrices F_k and Q_k , derived from the Jacobians of the continuous time error dynamics:

$$P_{k|k-1} = F_k P_{k-1|k-1} F_k^\top + Q_k$$

This step predicts the evolution of uncertainty around the nominal trajectory, incorporating the effects of process and sensor noise. The mean of the error state remains zero since the nominal state already represents the best estimate of the system.

Update step:

When a new measurement $\mathbf{z}_k$ becomes available, the error state is updated using the linearized measurement model:

$$\begin{aligned} K &= P_{k|k-1} H^\top (H P_{k|k-1} H^\top + R)^{-1} \\ \delta \hat{\mathbf{x}}_{k|k} &= K [\mathbf{z}_k - h(\hat{\mathbf{x}}_{k|k-1})] \\ P_{k|k} &= (I - KH) P_{k|k-1} \end{aligned}$$

Here, H denotes the Jacobian of the measurement function with respect to the error state. The correction term $\delta \hat{\mathbf{x}}_{k|k}$ represents a small estimated deviation between the predicted nominal state and the measurement-derived state.

Injection step:

After computing the correction, it is injected back into the nominal state using the nonlinear composition operator $\oplus$, which updates the nominal quantities according to the estimated error:

$$\hat{\mathbf{x}}_{k|k} \leftarrow \hat{\mathbf{x}}_{k|k-1} \oplus \delta \hat{\mathbf{x}}_{k|k}$$

For translational states, this operation corresponds to vector addition, whereas for attitude states represented by quaternions, it involves quaternion multiplication between the nominal quaternion and the small-angle correction quaternion derived from $\delta \hat{\boldsymbol{\theta}}$.

Reset step:

Following the injection, the error state mean is reset to zero:

$$\delta \hat{\mathbf{x}}_{k|k} \leftarrow \mathbf{0}$$

To maintain a consistent uncertainty representation after the nominal state update, the covariance must be transformed using the reset Jacobian G :

$$P_{k|k} \leftarrow G P_{k|k} G^\top, \quad G = \begin{bmatrix} I_6 & 0 & 0 \\ 0 & I - [\frac{\delta \hat{\boldsymbol{\theta}}}{2}]_\times & 0 \\ 0 & 0 & I_6 \end{bmatrix}$$

Here, $S = [\frac{\delta\hat{\theta}}{2}]_{\times}$ denotes the small angle correction matrix related to the attitude quaternion update. This transformation ensures that the new error state is correctly defined relative to the updated nominal trajectory and that the covariance remains properly aligned, preventing the accumulation of linearization errors over time.

Quaternion normalization:

Finally, quaternion normalization is applied to maintain a valid unit quaternion representation:

$$\mathbf{q}_{k|k} \leftarrow \frac{\mathbf{q}_{k|k}}{\|\mathbf{q}_{k|k}\|}$$

This step guarantees that the attitude estimate remains on the unit hypersphere, preventing numerical drift and preserving a consistent rotation representation.

The combined structure of the prediction, update, injection, and reset stages allows the ESKF to maintain high numerical stability and consistency. By separating the nonlinear nominal state from the small linear error state, the filter achieves superior accuracy and robustness for inertial navigation systems involving rotational dynamics compared to the standard EKF formulation.

6.5.7 EKF vs ESKF

Overall, the ESKF provides significantly improved numerical stability, robustness, and estimation consistency compared to the EKF. By estimating only small, locally linear error quantities while maintaining a nonlinear nominal state propagation, it achieves an effective balance between accuracy and computational efficiency. This separation ensures that orientation updates remain well conditioned and free from singularities, even during highly dynamic motion. As a result, the ESKF has become the preferred framework for modern inertial navigation and sensor fusion systems, particularly in applications involving rotational dynamics and quaternion based attitude estimation.

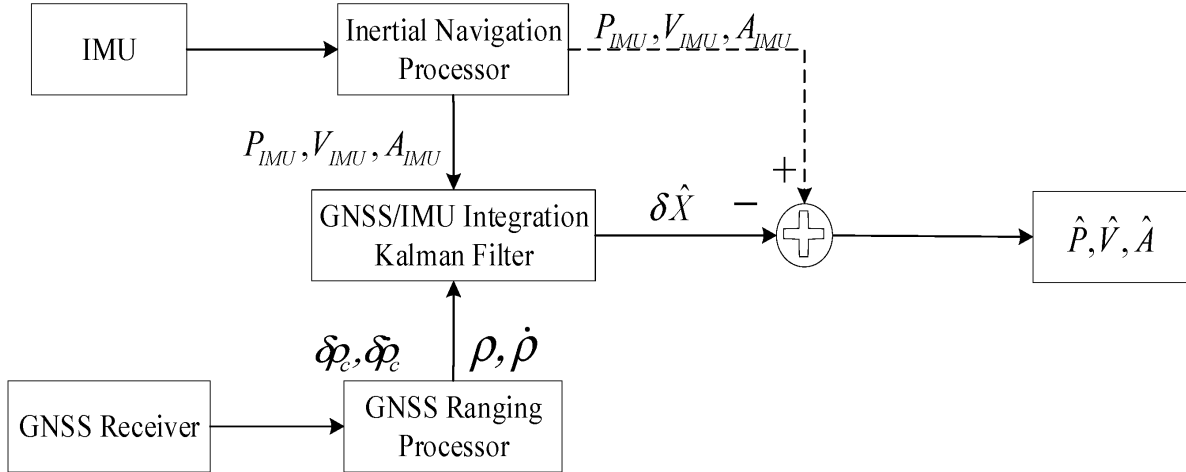


Figure 30: Functional overview of the ESKF architecture. The filter propagates and estimates the small error state based on IMU data, updates it using GNSS measurements, and then injects the correction into the nominal state. After injection, the error state is reset to zero. This loop ensures stable and consistent estimation by combining fast IMU propagation with slower GNSS updates.^[28]

6.6 Unscented Kalman Filter on Manifolds

6.6.1 Concept and Motivation

Up to this point, all Kalman Filter variants, including the EKF and ESKF, approximate nonlinear systems locally through 1st order linearization. Although effective for mildly nonlinear dynamics, these methods rely on Jacobians, which introduce errors when the system exhibits strong nonlinearity or discontinuous dynamics. An alternative approach to local linearization is sampling based nonlinear approximation. A well known technique of this class is the “*Unscented Transform (UT)*”, which represents a Gaussian distribution using a small, deterministically chosen set of sample points (called sigma points) and then propagates them through the nonlinear function to capture the transformed mean and covariance up to the third order for Gaussian inputs [23].

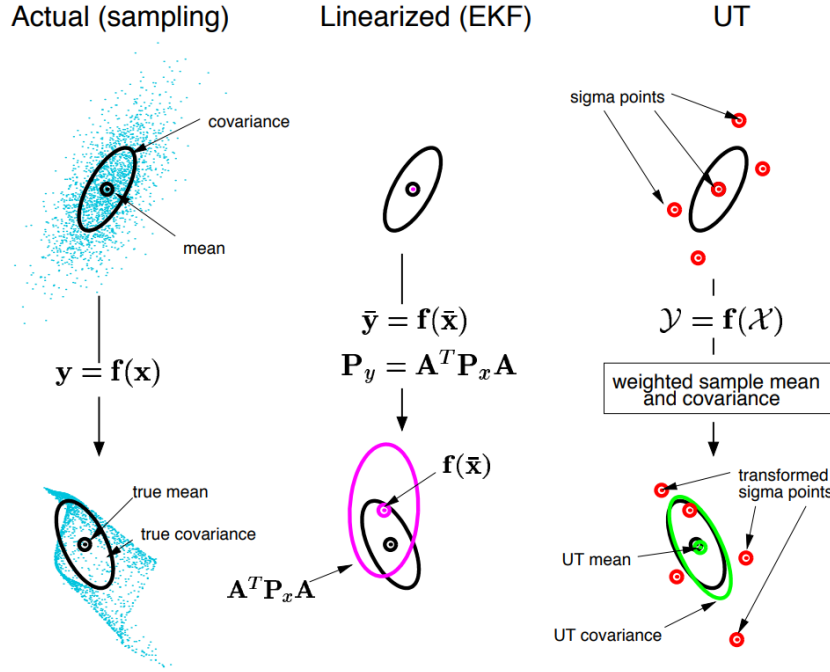


Figure 31: Comparison of different methods for nonlinear mean and covariance propagation. The figure contrasts (a) direct sampling of the true nonlinear distribution, (b) linearization using the Extended Kalman Filter (EKF), and (c) the Unscented Transform (UT), which captures the mean and covariance more accurately without linearization.^[23]

6.6.2 Unscented Kalman Filter

The Unscented Transform is defined by generating a set of $2L + 1$ sigma points from the prior mean $\mathbf{x}$ and covariance P , where L denotes the dimensionality of the state vector. Each sigma point represents a deterministic sample capturing the local mean and covariance structure of the state distribution, ensuring accurate nonlinear propagation up to the second order for any nonlinearity.

$$\begin{aligned}
 \chi_0 &= \mathbf{x} \\
 \chi_i &= \mathbf{x} + (\sqrt{(L + \lambda)P_i}) \quad i = 1, \dots, L \\
 \chi_i &= \mathbf{x} - (\sqrt{(L + \lambda)P_{i-L}}) \quad i = L + 1, \dots, 2L \\
 \lambda &= \alpha^2(L + \kappa) - L
 \end{aligned}$$

where λ is a scaling parameter controlling the spread of the sigma points. Each sigma point is assigned associated weights W_i^m and W_i^c for mean and covariance reconstruction. These weights ensure that both the central and surrounding sigma points contribute correctly to the nonlinear mean and covariance

propagation according to their statistical significance. The weights are defined as

$$\begin{aligned} W_0^m &= \frac{\lambda}{L + \lambda} \\ W_0^c &= \frac{\lambda}{L + \lambda} + (1 - \alpha^2 + \beta) \\ W_i^m &= W_i^c = \frac{1}{2(L + \lambda)} \quad i = 1, \dots, 2L \end{aligned}$$

The parameters α , β , and κ govern the spread, scaling, and higher order accuracy of the sigma point distribution. According to the original Unscented Kalman Filter formulation by Julier and Uhlmann [23], these parameters should be tuned to balance numerical stability and approximation accuracy. The parameter α determines the overall spread of the sigma points around the mean, it is typically chosen as a small positive value ($10^{-3} \leq \alpha \leq 1$), with smaller values resulting in sigma points closer to the mean and larger values increasing the nonlinear coverage at the cost of potential numerical instability. The parameter κ acts as a secondary scaling term that adjusts the effective spread of the sigma points; it is often set to 0 for simplicity or $3 - L$ to guarantee positive semi definiteness of the covariance. The parameter β encodes prior knowledge of the underlying distribution, for Gaussian distributions, $\beta = 2$ is recommended, as it ensures optimal 2nd order accuracy in the covariance reconstruction.

Together, these parameters define how the sigma points are positioned and weighted to best approximate the true nonlinear mean and covariance transformation while maintaining numerical stability across a wide range of system non-linearities.

Using these sigma points and weights, the propagated mean and covariance are computed as

$$\begin{aligned} \hat{\mathbf{x}}^- &= \sum_{i=0}^{2L} W_i^m f_d(\chi_i, \mathbf{u}) \\ P^- &= \sum_{i=0}^{2L} W_i^c [f_d(\chi_i, \mathbf{u}) - \hat{\mathbf{x}}^-][f_d(\chi_i, \mathbf{u}) - \hat{\mathbf{x}}^-]^\top \end{aligned}$$

This process effectively replaces linearization and analytical Jacobian computation with a deterministic sampling of the nonlinear function. A similar procedure is applied during the measurement update step, where the sigma points are propagated through the nonlinear measurement model $h(\mathbf{x})$ to compute the predicted observation mean and covariance.

During the measurement update step, each predicted sigma point χ_i^- is passed through the nonlinear measurement model $h(\mathbf{x})$ (for example, the GNSS measurement model in Equation 9) to get predicted measurement samples:

$$\mathcal{Z}_i = h(\chi_i^-)$$

The predicted measurement sample mean is computed as

$$\hat{\mathbf{z}} = \sum_{i=0}^{2L} W_i^m \mathcal{Z}_i$$

The corresponding innovation covariance and cross covariance are then obtained as

$$\begin{aligned} S &= \sum_{i=0}^{2L} W_i^c (\mathcal{Z}_i - \hat{\mathbf{z}})(\mathcal{Z}_i - \hat{\mathbf{z}})^\top + R \\ P_{xz} &= \sum_{i=0}^{2L} W_i^c (\chi_i^- - \hat{\mathbf{x}}^-)(\mathcal{Z}_i - \hat{\mathbf{z}})^\top, \end{aligned}$$

where R is the measurement noise covariance matrix.

The Kalman gain is then computed as

$$K = P_{xz} S^{-1}.$$

The state and covariance are updated according to

$$\begin{aligned}\hat{\mathbf{x}} &= \hat{\mathbf{x}}^- + K(\mathbf{z} - \hat{\mathbf{z}}), \\ P &= P^- - KSK^\top.\end{aligned}$$

Finally, the quaternion component of the state is normalized to maintain unit length and ensure a valid rotation representation:

$$\mathbf{q} \leftarrow \frac{\mathbf{q}}{\|\mathbf{q}\|}.$$

This completes the classical UKF update stage, where non-linearities are handled through sigma point sampling rather than analytic Jacobian linearization.

6.6.3 Manifold Operators

The classical UKF assumes that all system states evolve in Euclidean space $\mathbb{R}^n$. However, many real world systems and motion models, such as INS motion model 6, include quantities that lie on nonlinear manifolds. A common example is the attitude represented by unit quaternions $\mathbf{q} \in \mathbb{S}^3$, which form a curved space where addition and averaging are not globally valid operations. Applying the standard UKF directly to such states can lead to inconsistencies, since linear updates may move the estimate off the manifold's surface.

The “*A Code for Unscented Kalman Filtering on Manifolds (UKF-M)*” [24] extends the standard UKF by performing all statistical operations within the tangent space of the manifold. The Unscented Transform is carried out locally in this Euclidean tangent space, and the resulting sigma points are mapped back to the manifold using the exponential and logarithmic maps introduced in Equations 1 and 2. This approach ensures that all propagated and updated states remain geometrically consistent.

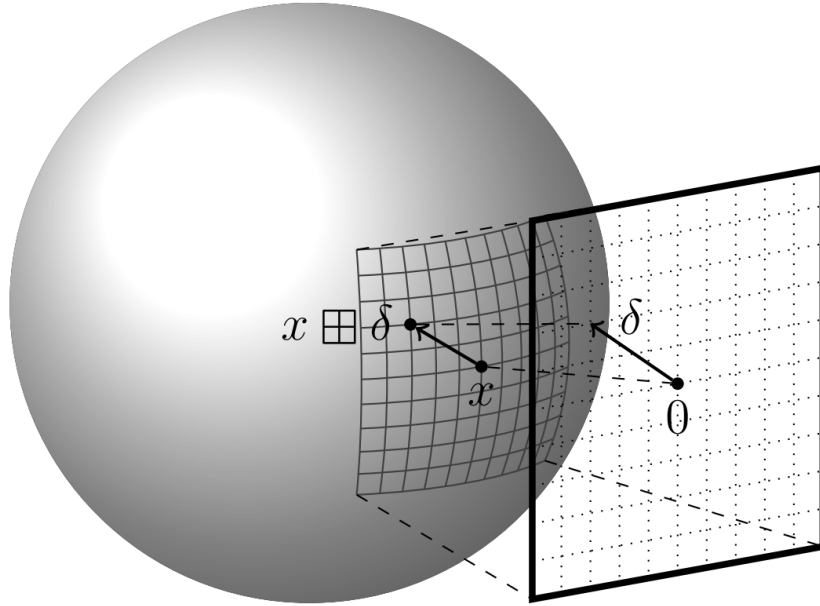


Figure 32: Mapping a local neighborhood in the state space (here: on the unit sphere $\mathbb{S}^2$) into $\mathbb{R}^n$ (here: the plane) allows for the use of standard sensor fusion algorithms without explicitly encoding the global topological structure.^[29]

To enable consistent state updates on a nonlinear manifold, the UKF-M introduces two generalized operators, $\varphi(\cdot, \cdot)$ and $\varphi^{-1}(\cdot)$, which replace standard addition and subtraction. They relate the manifold to its tangent space:

$$\begin{aligned}\mathbf{x}' &= \varphi(\mathbf{x}, \boldsymbol{\delta}) \\ \boldsymbol{\delta} &= \varphi_{\mathbf{x}}^{-1}(\mathbf{y})\end{aligned}$$

Note that δ is NOT the error state as used in the ESKF formulation. Instead, δ represents a small perturbation defined in the local tangent space at $\mathbf{x}$, and can take any arbitrary value within that space.

Here, $\mathbf{x}$ and $\mathbf{y}$ are elements on the manifold $\mathcal{M}$, while $\delta \in T_{\mathbf{x}}\mathcal{M}$ is the corresponding vector in the tangent space. The φ operator applies a perturbation from the tangent space to move the state along the manifold, producing an updated manifold element $\mathbf{x}'$. Conversely, the φ^{-1} operator computes the minimal difference between two manifold states by mapping that displacement back into the tangent space.

Together, these two operators define a consistent way to move back and forth between the manifold $\mathcal{M}$ and its tangent space $T_{\mathbf{x}}\mathcal{M}$, enabling the UKF-M to perform all statistical operations (eks, mean and covariance propagation) in a locally Euclidean space while keeping the final state representation on the true manifold.

In Euclidean space, these operators reduce to standard vector addition and subtraction, ie $\mathbf{x}' = \varphi(\mathbf{x}, \delta) = \mathbf{x} + \delta$ and $\delta = \varphi_{\mathbf{x}}^{-1}(\mathbf{y}) = \mathbf{y} - \mathbf{x}$. However, on curved manifolds like $\mathbb{S}^3$, directly adding vectors can move the state off the manifold, violating its geometric constraints. To address this, the φ and φ^{-1} operators rely on the exponential and logarithmic maps that move between the manifold and its tangent space:

$$\begin{aligned}\varphi(\mathbf{x}, \delta) &= \exp(\delta) \circ \mathbf{x} && (Tangent \rightarrow Manifold) \\ \varphi_{\mathbf{x}}^{-1}(\mathbf{y}) &= \log(\mathbf{y} \circ \mathbf{x}^{-1}) && (Tangent \leftarrow Manifold)\end{aligned}$$

where $\circ$ denotes the group composition operator. The exponential map projects a tangent space perturbation onto the manifold, while the logarithmic map computes the smallest displacement between two manifold elements within the tangent space.

These mappings are directly analogous to the Lie group relations (See Equations 1-2 and Figure 21)

$$\begin{aligned}R &= \exp(\boldsymbol{\omega}^\times) && (Tangent \rightarrow Manifold) \\ \boldsymbol{\omega} &= \log(R) && (Tangent \leftarrow Manifold)\end{aligned}$$

which connect a rotation matrix $R \in SO(3)$ and its corresponding rotation vector $\boldsymbol{\omega} \in \mathbb{R}^3$. In the UKF-M framework, the same concept generalizes to full state vectors composed of multiple manifold and Euclidean components.

6.6.4 Manifold Operators for the Process Model

In the UKF-M framework, the motion model defined in Equation 6 evolves the system state over time using inertial measurements. Since the state vector

$$\mathbf{x} = \begin{bmatrix} \mathbf{p}_{b/O}^n & \mathbf{v}_{b/O}^n & \mathbf{q} & \mathbf{a}_b & \omega_b \end{bmatrix}^\top$$

contains both Euclidean components $(\mathbf{p}, \mathbf{v}, \mathbf{a}_b, \omega_b)$ and a manifold component $(\mathbf{q} \in \mathbb{S}^3)$, the standard addition and subtraction operations used in classical UKF cannot be directly applied. The attitude quaternion lies on the unit sphere $\mathbb{S}^3$, which is a nonlinear manifold, meaning that the linear operations used for Euclidean states are not globally valid.

To handle this properly, the UKF-M defines two special manifold consistent operators, $\varphi(\cdot, \cdot)$ and $\varphi^{-1}(\cdot)$, that map between the nonlinear manifold and its local tangent space. These operators are used throughout the prediction and update stages whenever states or covariances are combined or compared.

φ : Mapping a tangent space perturbation to the manifold

The φ operator defines how a state on the manifold is updated using a perturbation vector $\boldsymbol{\xi}$ that lives in the tangent space. In this formulation, $\mathbf{x}$ represents the nominal state on the manifold, while $\boldsymbol{\xi}$ is a small local offset expressed in the linear tangent space around $\mathbf{x}$. The tangent space acts as a flat, Euclidean approximation of the manifold near the current mean, allowing linear algebra operations such as addition

and covariance computation to be performed safely.

Taking an example of sigma point generation from the UKF-M algorithm discussed later down below takes form

$$\mathcal{X} = \varphi(\mathbf{x}, \boldsymbol{\xi})$$

where $\mathbf{x}$ is the current estimated state and $\boldsymbol{\xi}$ represents a small perturbation drawn from the covariance in tangent space. The vector $\boldsymbol{\xi}$ has the same structure as the state but replaces the quaternion with a 3D rotation vector. Explicitly,

$$\mathbf{x} = \begin{bmatrix} \mathbf{p}_{b/O}^n & \mathbf{v}_{b/O}^n & \mathbf{q} & \mathbf{a}_b & \omega_b \end{bmatrix}^\top \quad \boldsymbol{\xi} = \begin{bmatrix} \boldsymbol{\xi}_p & \boldsymbol{\xi}_v & \boldsymbol{\xi}_\theta & \boldsymbol{\xi}_{a_b} & \boldsymbol{\xi}_{\omega_b} \end{bmatrix}^\top$$

Here, the quaternion $\mathbf{q}$ is an element of the unit hypersphere $\mathbb{S}^3$, while the vector $\boldsymbol{\xi}_\theta \in \mathbb{R}^3$ is a tangent space representation of a small rotation around the mean orientation.

In tangent space, quaternions do not exist, only 3D rotation vectors do. The vector $\boldsymbol{\xi}_\theta$ simply points in the direction and magnitude of the local rotational perturbation, describing how the attitude should change infinitesimally around the current estimate.

When converting this local perturbation back to the manifold, the rotation vector must be mapped to a unit quaternion on $\mathbb{S}^3$. This is done through the exponential map, which transforms a small 3D vector into a valid quaternion rotation. The mapping is

$$\mathbf{q}_{\boldsymbol{\xi}_\theta} \triangleq \text{Exp} \left(\frac{1}{2} \boldsymbol{\xi}_\theta \right) = \begin{bmatrix} \cos \left(\frac{|\boldsymbol{\xi}_\theta|}{2} \right) \\ \frac{\boldsymbol{\xi}_\theta}{|\boldsymbol{\xi}_\theta|} \sin \left(\frac{|\boldsymbol{\xi}_\theta|}{2} \right) \end{bmatrix}$$

The division by two appears because quaternion rotations encode angles that are twice the corresponding rotation vector magnitude. Using this, the complete state perturbation becomes

$$\varphi(\mathbf{x}, \boldsymbol{\xi}) = \begin{bmatrix} \mathbf{p}_{b/O}^n + \boldsymbol{\xi}_p \\ \mathbf{v}_{b/O}^n + \boldsymbol{\xi}_v \\ \mathbf{q} \otimes \text{Exp} \left(\frac{1}{2} \boldsymbol{\xi}_\theta \right) \\ \mathbf{a}_b + \boldsymbol{\xi}_{a_b} \\ \omega_b + \boldsymbol{\xi}_{\omega_b} \end{bmatrix}$$

The linear components like position, velocity, and biases are updated using simple vector addition, since they exist in Euclidean space. The quaternion, however, must be updated multiplicatively using the exponential map to remain on the manifold $\mathbb{S}^3$. This ensures that every sigma point maintains unit norm and represents a valid orientation.

In essence, $\boldsymbol{\xi}$ is never a physical state but a local linear displacement defined in the tangent space. It expresses how much the manifold state should move in each direction, including rotation, to represent local uncertainty. When mapped back through φ , the translational components shift linearly, while the rotational component “*bends*” along the hypersphere surface via the exponential map. This motion allows the UKF-M to handle quaternions and other manifold states consistently, performing uncertainty propagation and updates in linear tangent space while always keeping the resulting states valid on their nonlinear manifold.

φ^{-1} : Mapping a manifold difference to the tangent space

The φ^{-1} operator defines how the difference between two states on the manifold, $\mathbf{x}_1$ and $\mathbf{x}_2$, is expressed as a perturbation vector in the tangent space. While φ projects a local perturbation from the tangent space onto the manifold, φ^{-1} performs the inverse operation, it measures how far one manifold element is from another and maps that difference back into the linear tangent domain. This operation is essential for computing residuals, innovations, and covariance updates within the UKF-M, since all statistical quantities must reside in Euclidean tangent space.

For two states

$$\mathbf{x}_1 = \begin{bmatrix} \mathbf{p}_1 & \mathbf{v}_1 & \mathbf{q}_1 & \mathbf{a}_{b_1} & \omega_{b_1} \end{bmatrix}^\top, \quad \mathbf{x}_2 = \begin{bmatrix} \mathbf{p}_2 & \mathbf{v}_2 & \mathbf{q}_2 & \mathbf{a}_{b_2} & \omega_{b_2} \end{bmatrix}^\top$$

the manifold consistent subtraction is defined as

$$\varphi_{\mathbf{x}_2}^{-1}(\mathbf{x}_1) = \begin{bmatrix} \mathbf{p}_1 - \mathbf{p}_2 \\ \mathbf{v}_1 - \mathbf{v}_2 \\ 2 \text{Log}(\mathbf{q}_2^{-1} \otimes \mathbf{q}_1) \\ \mathbf{a}_{b_1} - \mathbf{a}_{b_2} \\ \omega_{b_1} - \omega_{b_2} \end{bmatrix}$$

Here, the linear quantities (position, velocity, and sensor biases) can be subtracted directly, as they live in Euclidean space. However, the quaternion part requires special handling because it resides on the unit hypersphere $\mathbb{S}^3$. To obtain a meaningful rotational difference between $\mathbf{q}_1$ and $\mathbf{q}_2$, the relative rotation

$$\mathbf{q}_{\text{relative}} = \mathbf{q}_2^{-1} \otimes \mathbf{q}_1$$

is computed, representing the rotation that brings $\mathbf{q}_2$ to $\mathbf{q}_1$.

This relative quaternion is then converted into its corresponding tangent space vector using the logarithmic map, which projects the quaternion from $\mathbb{S}^3$ to $\mathbb{R}^3$. For a quaternion $\mathbf{q}_{\text{relative}} = [q_w, \mathbf{q}_v]^\top$ where $\mathbf{q}_v = [q_x, q_y, q_z]^\top$ composed of scalar q_w and vector $\mathbf{q}_v$ parts, the mapping is defined as

$$\boldsymbol{\theta}_{\text{relative}} \triangleq \text{Log}(\mathbf{q}_{\text{relative}}) = \begin{cases} \frac{2 \arctan 2(\|\mathbf{q}_v\|, q_w)}{\|\mathbf{q}_v\|} \mathbf{q}_v, & \text{if } \|\mathbf{q}_v\| \neq 0 \\ \mathbf{0}, & \text{otherwise} \end{cases}$$

The result of this operation is a 3D rotation vector $\boldsymbol{\theta}_{\text{relative}} \in \mathbb{R}^3$ that lies in the tangent space of the unit quaternion manifold. This vector points along the rotation axis and has a magnitude equal to the rotation angle between the two orientations. It therefore provides a minimal, linear representation of the orientation error. The factor of two ensures consistency with the quaternion exponential map used in the φ operator.

Intuitively, φ^{-1} answers the question: “*What small rotation and translation in tangent space would transform $\mathbf{x}_2$ into $\mathbf{x}_1$?*” For Euclidean components, the answer is simply the difference between vectors. For rotational components, it is the smallest rotation vector on the manifold surface that aligns $\mathbf{q}_2$ with $\mathbf{q}_1$. By performing this mapping, the UKF-M can compute orientation residuals, innovations, and covariance deviations in a consistent, linearized space without ever violating the unit quaternion constraint.

Interpretation

The φ and φ^{-1} operators together define the mathematical bridge between the nonlinear manifold and its locally linear tangent space, ensuring that attitude, position, and bias states are updated consistently. The φ operator applies a perturbation from tangent space to the manifold, producing a new valid state on $\mathbb{S}^3$, while the φ^{-1} operator performs the reverse, projecting the difference between two manifold states back into tangent space where linear operations such as mean and covariance can be computed.

Within the UKF-M motion framework, φ is used when generating sigma points or updating the state estimate after correction, effectively moving the state along the manifold surface in response to perturbations. Conversely, φ^{-1} is used when comparing predicted and updated states or computing innovations, allowing differences in orientation and position to be expressed in a linearized space suitable for statistical computation.

Together, these mappings preserve the unit quaternion constraint and maintain full geometric consistency of the nonlinear attitude dynamics described by the motion model 6, preventing normalization errors or distortion of uncertainty during repeated prediction and update cycles.

6.6.5 Manifold Operators for the Measurement Model

The measurement model in Equation 9 maps the nonlinear navigation state $\mathbf{x}$ into a measurable quantity $\mathbf{z}$ composed of GNSS position and yaw. The measurement manifold therefore combines both Euclidean and circular components:

$$\mathbf{z} = \begin{bmatrix} \mathbf{p}_{b/O}^n \\ \psi \end{bmatrix}, \quad \mathbf{z} \in \mathbb{R}^3 \times \mathbb{S}^1$$

where ψ is the yaw angle, living on the circle manifold $\mathbb{S}^1$.

The process $f(\cdot)$ and measurement $h(\cdot)$ manifolds thus differ in structure: while the state includes quaternion orientation $\mathbf{q} \in \mathbb{S}^3$, the measurement only includes a single angular degree of freedom (yaw) along with the position.

φ : Mapping a tangent space perturbation to the manifold

The φ operator defines how a measurement correction or innovation is consistently applied on the measurement manifold. This ensures that angular quantities, such as yaw, remain wrapped within the valid interval $[-\pi, \pi]$ when updated.

For a predicted measurement $\hat{\mathbf{z}} = h(\hat{\mathbf{x}})$ and a correction $\delta\mathbf{z}$ obtained from the filter update, the operation is

$$\varphi(\hat{\mathbf{z}}, \delta\mathbf{z}) = \begin{bmatrix} \hat{\mathbf{z}}_p + \delta\mathbf{z}_p \\ \text{wrap_to_pi}(\hat{\mathbf{z}}_\psi + \delta\mathbf{z}_\psi) \end{bmatrix}$$

where $\text{wrap_to_pi}(\cdot)$ ensures angular consistency on the $\mathbb{S}^1$ manifold. Without this wrapping, the filter could incorrectly interpret heading discontinuities near $\pm\pi$ as large jumps, destabilizing the update step.

φ^{-1} : Mapping a manifold difference to the tangent space

For completeness, the inverse operator φ^{-1} defines how a difference between two measurement manifold elements can be represented in tangent space. It is conceptually expressed as

$$\boldsymbol{\nu} = \varphi_{\hat{\mathbf{z}}}^{-1}(\mathbf{z}) = \begin{bmatrix} \mathbf{z}_p - \hat{\mathbf{z}}_p \\ \text{wrap_to_pi}(\mathbf{z}_\psi - \hat{\mathbf{z}}_\psi) \end{bmatrix}$$

Although mathematically defined, this operation is seldom used in practice, as measurement innovations are usually handled directly through φ in the update step.

Interpretation

In the measurement model, the φ operator is the one predominantly used, as it applies measurement space corrections in a geometrically consistent manner, maintaining continuity of the angular components across the $\pm\pi$ boundary. The φ^{-1} operator, on the other hand, is primarily included for formal completeness, providing a way to express manifold differences in tangent space when required by specific formulations such as smoothing or optimization frameworks.

Together, these operators maintain consistency between the nonlinear measurement manifold $\mathbb{R}^3 \times \mathbb{S}^1$ and its tangent space. The φ operator ensures that corrections are applied while respecting the circular geometry of $\mathbb{S}^1$, while φ^{-1} provides the theoretical foundation for inverse mappings when needed. This guarantees stable, geometrically consistent measurement updates and accurate handling of both position and yaw measurements within the UKF-M framework.

6.6.6 `set_weights(dim, $\alpha$ )`: Sigma Point Weights Parameter Function

The UKF-M defines sigma point weighting through a generalized function:

$$\{\lambda, w_0, w_i^{(m)}, w_i^{(c)}\} = \text{set_weights}(\text{dim}, \alpha) \quad (24)$$

This function specifies how sigma points are placed and weighted in the tangent space of the manifold during both prediction and update steps.

The sigma point weights determine how uncertainty is distributed around the mean state. Unlike the classical UKF formulation, which uses the parameter triplet (α, β, κ) , the UKF-M retains only a single spread parameter α . The manifold retraction $\varphi(\cdot)$ and its inverse $\varphi^{-1}(\cdot)$ already guarantee second order accuracy and preserve geometric consistency, rendering β and κ unnecessary. This simplification improves numerical stability and reduces tuning complexity, as detailed in “*Unscented Kalman Filtering on Lie Groups*” [30] and “*A Code for Unscented Kalman Filtering on Manifolds (UKF-M)*” [24].

For a given tangent space dimension dim and spread parameter α , the scaling parameter λ is computed as

$$\lambda = (\alpha^2 - 1) dim$$

Smaller α values lead to tightly clustered sigma points suitable for near linear dynamics, while larger α values spread them further to better capture nonlinear behavior. Typical values lie within $\alpha \in [10^{-3}, 1]$

Each sigma point set is defined by symmetric weights around the mean:

$$\begin{aligned} w_i^{(m)} &= \frac{\lambda}{dim + \lambda} & i = 1, \dots, 2 dim \\ w_i^{(c)} &= \frac{1}{2(dim + \lambda)} & i = 1, \dots, 2 dim \\ w_0 &= \frac{\lambda}{dim + \lambda} + 3 - \alpha^2 \end{aligned}$$

Here, $w_i^{(m)}$ is the mean weight of the central sigma point, $w_i^{(c)}$ are the symmetric weights for all other sigma points, and w_0 adds a correction term that stabilizes the covariance evolution on manifolds. The term $(3 - \alpha^2)$ is unique to the manifold formulation and does not appear in the classical UKF. It compensates for higher order effects introduced by nonlinear retractions $\varphi(\cdot)$ and its inverse $\varphi^{-1}(\cdot)$, improving numerical stability and covariance consistency [24].

The weight computation used in UKF-M can be summarized as:

$$\text{set_weights}(dim, \alpha) \Rightarrow \begin{cases} \lambda = (\alpha^2 - 1) dim \\ w_i^{(m)} = \frac{\lambda}{dim + \lambda} \\ w_i^{(c)} = \frac{1}{2(dim + \lambda)} \\ w_0 = \frac{\lambda}{dim + \lambda} + 3 - \alpha^2 \end{cases}$$

This unified expression is used in all stages of the UKF-M algorithm with the appropriate tangent space dimension.

The argument dim corresponds to the dimension of the tangent space associated with the uncertainty being represented:

$$dim \in \{d, q, u\},$$

where:

- $d = \dim(P)$ is used for the **state uncertainty** during prediction and for the **measurement update**
- $q = \dim(Q)$ is used for the **process noise** during prediction
- $u = \dim(P_{up})$ is defined in algorithm but never actually used for state estimation.

In the UKF-M code, each step uses its own scaling parameter $\alpha_d, \alpha_q, \alpha_u$ respectively:

$$\alpha = [\alpha_d, \alpha_q, \alpha_u]$$

allowing independent control over sigma point spread in each stage.

For a tangent space of dimension dim , the number of sigma points generated is $2 dim + 1$. For example, for INS model 7 with 16 dimensions the total number of sigma points generated is ($d = 16$), 33 sigma points are generated. Meanwhile, for the process noise model, which includes accelerometer and gyroscope measurement noise as well as their biases ($q = 12$), the filter generates 25 sigma points. Note that in the current UKF-M implementation, the parameter u is defined but never actually used. All sigma point generation in both propagation and update steps relies on the state dimension d and q . The reason u exists in the formulation is purely for generality or future extensions, but it remains unused in the standard UKF-M algorithm.

6.6.7 Modified Process Model for the UKF-M

The deterministic discrete time motion model introduced in Equation 19 defines the state evolution as

$$\mathbf{x}_{k+1} = f_d(\mathbf{x}_k, \mathbf{u}_k)$$

where $\mathbf{u}_k$ represents the measured IMU input (accelerometer and gyroscope), and $f_d(\cdot)$ is fully deterministic. However, in the UKF-M, the process model must explicitly accept an additive noise term $\mathbf{w}_k$:

$$\mathbf{x}_{k+1} = f_d(\mathbf{x}_k, \mathbf{u}_k, \mathbf{w}_k)$$

This stochastic extension is necessary because the UKF-M propagates a set of sigma points that represent samples of both state and process noise. Each sigma point includes a perturbation $\mathbf{w}_k$ drawn from the process noise distribution $\mathcal{N}(\mathbf{0}, Q_k)$, where Q_k is the discretized process noise covariance. By propagating these noise sigma points through the dynamics, the filter captures how uncertainty in the process model affects the state evolution, without requiring any linearization.

For the discretized INS model in Equation 19, process noise is naturally introduced by perturbing the measured IMU inputs. Thus, the model becomes:

$$f_d(\mathbf{x}_k, \mathbf{u}_k, \mathbf{w}_k) = \mathbf{x}_{k+1} = \begin{bmatrix} \mathbf{p}_{b/O_{k+1}}^n \\ \mathbf{v}_{b/O_{k+1}}^n \\ \mathbf{q}_{k+1} \\ \mathbf{a}_{b_{k+1}} \\ \boldsymbol{\omega}_{b_{k+1}} \end{bmatrix} = \begin{bmatrix} \mathbf{p}_{b/O_k}^n + \mathbf{v}_{b/O_k}^n \Delta t + \frac{1}{2} (R(\mathbf{q}_k)[(\mathbf{a}_m + \mathbf{w}_k^a) - \mathbf{a}_{b_k}] + \mathbf{g}^n) \Delta t^2 \\ \mathbf{v}_{b/O_k}^n + (R(\mathbf{q}_k)[(\mathbf{a}_m + \mathbf{w}_k^a) - \mathbf{a}_{b_k}] + \mathbf{g}^n) \Delta t \\ \mathbf{q}_k \otimes \text{Exp}((\boldsymbol{\omega}_m + \mathbf{w}_k^\omega) - \boldsymbol{\omega}_{b_k}) \times \Delta t \\ (\mathbf{a}_{b_k} + \mathbf{w}_k^{a_b}) - p_{ab} \mathbf{a}_{b_k} \Delta t \\ (\boldsymbol{\omega}_{b_k} + \mathbf{w}_k^{\omega_b}) - p_{\omega b} \boldsymbol{\omega}_{b_k} \Delta t \end{bmatrix}$$

where

$$\mathbf{w}_k = [\mathbf{w}_k^\omega \quad \mathbf{w}_k^a \quad \mathbf{w}_k^{\omega_b} \quad \mathbf{w}_k^{a_b}]^\top$$

collects all process noise components:

- $\mathbf{w}_k^\omega$: gyroscope measurement noise
- $\mathbf{w}_k^a$: accelerometer measurement noise
- $\mathbf{w}_k^{\omega_b}$: gyro bias random walk
- $\mathbf{w}_k^{a_b}$: accelerometer bias random walk

The noise $\mathbf{w}_k$ is not drawn randomly during runtime. Instead, the UKF-M generates a deterministic set of noise sigma points $\{\mathbf{w}_i\}_{i=1}^{2q}$ from the covariance Q_k :

$$\mathbf{w}_i = \begin{cases} \text{col} \left(\sqrt{(q + \lambda) Q_k} \right)_i & i = 1, \dots, q, \\ -\text{col} \left(\sqrt{(q + \lambda) Q_k} \right)_{i-q} & i = q + 1, \dots, 2q \end{cases}$$

Here, q denotes the dimension of the process noise covariance matrix Q_k , λ is the scaling factor from the sigma point generation, and $\text{col}(\cdot)_i$ extracts the i -th column of the matrix square root.

Each noise sigma point is passed into the process model $f_d(\mathbf{x}_k, \mathbf{u}_k, \mathbf{w}_j)$ during prediction. This allows the filter to account for the effect of process uncertainty without Monte Carlo sampling or Jacobian linearization.

This modification preserves the structure of the original INS model while enabling the UKF-M to propagate uncertainty on both the state and process noise manifolds. It ensures that both the deterministic mean $\hat{\mathbf{x}}_k^-$ and its covariance P_k^- evolve consistently according to the stochastic dynamics of the system.

6.6.8 Filter Algorithm: Prediction Step

The prediction step in the UKF-M propagates both the mean state and its covariance through the nonlinear process model while ensuring full geometric consistency on the underlying manifold [24]. All statistical operations are performed in the tangent space of the manifold, while the mean state itself always remains on the manifold.

Input:

$$\hat{\mathbf{x}}_{k-1}, P_{k-1}, \mathbf{u}_k, Q_k, \alpha$$

where $\hat{\mathbf{x}}_{k-1}$ is the previous mean state on the manifold, P_{k-1} the covariance in its tangent space, $\mathbf{u}_k$ the input (eks, control or IMU increment), Q_k the process noise covariance, and α the sigma point scaling parameter.

Step 1 - Propagate the mean state:

The deterministic state prediction is first computed using the process model $f_d(\mathbf{x}_k, \mathbf{u}_k)$ (see Equation 19):

$$\hat{\mathbf{x}}_k^- = f_d(\hat{\mathbf{x}}_{k-1}, \mathbf{u}_k, \mathbf{0})$$

This represents the nominal propagation of the mean without process noise. The result $\hat{\mathbf{x}}_k^-$ remains on the manifold due to the definition of $f_d(\mathbf{x}_k, \mathbf{u}_k)$.

Step 2 - Compute sigma-point weights:

Using the scaling parameter α and the tangent space dimension d , the sigma point scaling and weights are obtained from:

$$\{\lambda, w_0, w_i^{(m)}, w_i^{(c)}\}_{i=0,\dots,2d} = \text{set_weights}(d, \alpha)$$

as defined in Equation 24. These parameters determine how far and how strongly each sigma point contributes to the propagated covariance. For the INS motion model in Equation 19, the tangent space dimension is $d = 15$, corresponding to position, velocity, attitude, and accelerometer and gyroscope bias states.

Step 3 - Generate state sigma points in the tangent space:

From the Cholesky decomposition of the covariance P_{k-1} , the tangent sigma point perturbations are constructed as:

$$\boldsymbol{\xi}_i = \begin{cases} \text{col}\left(\sqrt{(d+\lambda)P_{k-1}}\right)_i & d = 1, \dots, d, \\ -\text{col}\left(\sqrt{(d+\lambda)P_{k-1}}\right)_{i-d} & i = d+1, \dots, 2d \end{cases}$$

where $\text{col}(\cdot)_i$ extracts the i -th column of the matrix square root. Each $\boldsymbol{\xi}_j \in \mathbb{R}^d$ represents a local perturbation in the tangent space of the current state.

Step 4 - Retract sigma points onto the manifold and propagate through the model:

Each perturbation $\boldsymbol{\xi}_i$ from the tangent space is mapped onto the manifold using the retraction operator $\varphi(\cdot)$ and then propagated through the process model $f_d(\cdot)$. For each sigma point:

$$\chi_i^k = f_d(\varphi(\hat{\mathbf{x}}_{k-1}, \boldsymbol{\xi}_i), \mathbf{u}_k, \mathbf{0}), \quad i = 1, \dots, 2d$$

This produces $2d$ propagated manifold states χ_i^k that represent how local perturbations evolve under the nonlinear dynamics. Each χ_i^k remains a valid manifold element (eks, normalized quaternion or valid SE(3) pose).

Step 5 - Compute the propagated state covariance:

After propagation, each sigma point is mapped back into the tangent space of the predicted mean $\hat{\chi}_k$ using the inverse retraction:

$$\boldsymbol{\eta}_i = \varphi_{\hat{\chi}_k}^{-1}(\chi_i^k)$$

These deviations express how far each propagated sigma point lies from the predicted mean in local linear coordinates. The state induced covariance contribution is then:

$$\Sigma_k = \sum_{i=1}^{2d} w_i^{(c)} \boldsymbol{\eta}_i \boldsymbol{\eta}_i^\top$$

which captures the spread due to state uncertainty.

Step 6 - Generate process noise sigma points:

Using the scaling parameter α and the process noise dimension q , the sigma point scaling and weights are obtained from:

$$\{\lambda, w_0, w_i^{(m)}, w_i^{(c)}\}_{i=0,\dots,2q} = \text{set_weights}(q, \alpha)$$

as defined in Equation 24. These parameters determine the spread and relative weighting of the noise sigma points generated from the process noise covariance Q_k :

$$\mathbf{w}_i = \begin{cases} \text{col}\left(\sqrt{(q+\lambda)Q_k}\right)_i & i = 1, \dots, q, \\ -\text{col}\left(\sqrt{(q+\lambda)Q_k}\right)_{i-q} & i = q+1, \dots, 2q \end{cases}$$

Here, q corresponds to the dimension of the process noise vector, which for the INS model is typically $q = 12$ (covering gyroscope, accelerometer, and bias random walk noise terms). $\text{col}(\cdot)_i$ extracts the i -th column of the matrix square root.

Step 7 - Propagate noise sigma points through the model:

Each noise sigma point $\mathbf{w}_i$ is injected into the process model to determine its effect on the predicted state:

$$\tilde{\chi}_i^k = f_d(\hat{\mathbf{x}}_{k-1}, \mathbf{u}_k, \mathbf{w}_i), \quad i = 1, \dots, 2q$$

and is then mapped back into the tangent space of the predicted mean:

$$\tilde{\boldsymbol{\eta}}_i = \varphi_{\hat{\mathbf{x}}_k^-}^{-1}(\tilde{\chi}_i^k)$$

Here, each $\tilde{\chi}_i^k$ represents the propagated manifold state obtained by applying a specific noise realization $\mathbf{w}_i$ to the nominal prediction. The corresponding $\tilde{\boldsymbol{\eta}}_i$ is its local deviation in the tangent space around the predicted mean $\hat{\mathbf{x}}_k^-$, expressing how that noise perturbation alters the state in linear coordinates suitable for covariance computation.

Step 8 - Compute the full predicted covariance:

The total predicted covariance combines the contributions from both the propagated state and the injected process noise:

$$P_k^- = \Sigma_k + \sum_{i=1}^{2q} w_i^{(c)} \tilde{\boldsymbol{\eta}}_i \tilde{\boldsymbol{\eta}}_i^\top$$

Here, P_k^- remains in the tangent space centered at $\hat{\mathbf{x}}_k^-$, ensuring that covariance operations stay Euclidean while the mean remains on the manifold.

Summary:

The prediction step produces the a priori state estimate $\hat{\mathbf{x}}_k^-$ and its covariance P_k^- . The mean state is propagated deterministically through the process model, while the covariance is updated using the spread of both state and process noise sigma points mapped through the manifold retraction. This ensures that the predicted mean remains on the manifold and the covariance evolves consistently in its tangent space, providing a second order accurate and geometrically valid prediction.

6.6.9 Filter Algorithm: Correction Step

The correction step refines the predicted state $\hat{\mathbf{x}}_k^-$ and covariance P_k^- using the incoming measurement $\mathbf{z}_k$. In the UKF-M framework, all computations occur in the local tangent space to maintain geometric consistency on the manifold [24][30].

Input:

$$\hat{\mathbf{x}}_k^-, P_k^-, \mathbf{z}_k, R_k, \alpha$$

where $\hat{\mathbf{x}}_k^-$ is the predicted mean on the manifold, P_k^- its covariance, $\mathbf{z}_k$ the measurement, R_k the measurement noise covariance, and α the sigma point scaling parameter.

Step 1 - Compute sigma-point weights:

The sigma point scaling and weights are computed in the tangent space of dimension $d = \dim(P_k^-)$:

$$\{\lambda, w_0, w_i^{(m)}, w_i^{(c)}\}_{i=0,\dots,2d} = \text{set_weights}(d, \alpha)$$

as defined in Equation 24. These weights determine the spread and influence of each sigma point during the update.

Step 2 - Generate state sigma points:

The Cholesky decomposition of P_k^- provides the tangent-space perturbations:

$$\xi_i = \begin{cases} \text{col}\left(\sqrt{(d+\lambda)P_k^-}\right)_i & i = 1, \dots, d, \\ -\text{col}\left(\sqrt{(d+\lambda)P_k^-}\right)_{i-d} & i = d+1, \dots, 2d \end{cases}$$

Each ξ_i represents a local perturbation of $\hat{\mathbf{x}}_k^-$ in the tangent space. $\text{col}(\cdot)_i$ extracts the i -th column of the matrix square root.

Step 3 - Propagate sigma points through the measurement model:

Each sigma point is retracted onto the manifold using $\varphi(\cdot)$ and propagated through the nonlinear measurement model $h(\cdot)$ to produce a set of predicted measurement sigma points:

$$\begin{aligned} \mathcal{Z}_0 &= h(\hat{\mathbf{x}}_k^-) \\ \mathcal{Z}_i &= h(\varphi(\hat{\mathbf{x}}_k^-, \xi_i)), \quad i = 1, \dots, 2d \end{aligned}$$

The predicted mean measurement is then obtained by weighted averaging:

$$\hat{\mathbf{z}}_k = w_0^{(m)} \mathcal{Z}_0 + \sum_{i=1}^{2d} w_i^{(m)} \mathcal{Z}_i$$

Step 4 - Compute innovation and cross-covariance matrices:

After centering the measurement sigma points around $\hat{\mathbf{z}}_k$, the innovation covariance S and the state measurement cross covariance $P_{\xi\mathcal{Z}}$ are computed as:

$$\begin{aligned} S &= w_0^{(c)} (\mathcal{Z}_0 - \hat{\mathbf{z}}_k)(\mathcal{Z}_0 - \hat{\mathbf{z}}_k)^\top + \sum_{i=1}^{2d} w_i^{(c)} (\mathcal{Z}_i - \hat{\mathbf{z}}_k)(\mathcal{Z}_i - \hat{\mathbf{z}}_k)^\top + R_k \\ P_{\xi\mathcal{Z}} &= \sum_{i=1}^{2d} w_i^{(c)} \xi_i (\mathcal{Z}_i - \hat{\mathbf{z}}_k)^\top \end{aligned}$$

Here, S represents the innovation covariance in measurement space, accounting for both predicted measurement spread and sensor noise, while $P_{\xi\mathcal{Z}}$ quantifies the correlation between state perturbations and measurement deviations.

Step 5 - Apply Kalman update:

The Kalman gain is computed using the cross covariance and innovation covariance as

$$K = P_{\xi\mathcal{Z}} S^{-1}$$

which determines how much the new measurement influences the state correction. The state innovation in tangent space is then computed as

$$\xi_k^+ = K(\mathbf{z}_k - \hat{\mathbf{z}}_k)$$

Next, the corrected state on the manifold is obtained by retracting the tangent update onto the manifold:

$$\hat{\mathbf{x}}_k^+ = \varphi(\hat{\mathbf{x}}_k^-, \xi_k^+)$$

which ensures that the update remains consistent with the nonlinear geometry of the state space.

Finally, the state covariance is updated to reflect the reduced uncertainty after incorporating the measurement:

$$P_k^+ = P_k^- - K S K^\top$$

To preserve numerical symmetry and positive semi definiteness, the covariance is symmetrized as

$$P_k^+ = \frac{1}{2}(P_k^+ + P_k^{+\top})$$

Summary:

At the end of the correction step, the filter outputs the updated mean state $\hat{\mathbf{x}}_k^+$ and covariance P_k^+ . The state $\hat{\mathbf{x}}_k^+$ represents the best estimate of the system after incorporating the latest measurement, while the covariance P_k^+ reflects the remaining uncertainty. Together, they form the posterior estimate that serves as the prior for the next prediction step.

6.6.10 Advantages and Practical Aspects

The UKF-M offers several advantages over Jacobian based filters like the EKF and ESKF. It removes the need for analytic Jacobians, eliminating discretization and modeling errors while simplifying implementation. The Unscented Transform provides full 2nd order accuracy in mean and covariance propagation, improving stability and consistency under strong nonlinearities.

Operating directly on manifolds allows the UKF-M to handle non Euclidean states such as rotations ($\text{SO}(3)$), quaternions ($\mathbb{S}^3$), and rigid body poses ($\text{SE}(3)$) in a unified framework. This ensures consistent attitude updates without small-angle approximations and removes the need for tuning parameters like β and κ , while retaining α to control the sigma point spread in the tangent space. The result is a robust and geometrically correct treatment of uncertainty across mixed Euclidean manifold states.

Compared to the standard UKF, the manifold formulation is more general and directly applicable to Lie groups and hybrid models used in robotics and navigation. Its main drawbacks are higher computational cost, since each step evaluates multiple nonlinear models, and the need to tune additional scaling parameters such as α . However, with modern embedded hardware and careful parameter selection, these costs are minor compared to the gain in robustness, geometric consistency, and implementation simplicity.

In practice, the UKF-M achieves accurate and stable estimation of position and orientation without Jacobians or linearization. Its balance of simplicity, geometric correctness, and numerical stability makes it well suited for nonlinear systems such as SLAM and inertial navigation on the microAmpere ASV, where computational resources are sufficient.

6.7 SLAM based State Update

The SLAM based state update provides globally consistent pose corrections to the state estimation filter, counteracting drift accumulated during inertial propagation. While the IMU supplies high rate relative motion updates, it lacks an absolute reference, causing position and attitude errors to increase over time. The SLAM output introduces an external correction derived from landmark observations, delivering drift free pose estimates in the navigation frame.

In cases where GNSS coverage is limited or temporarily unavailable, the SLAM update serves as an alternative absolute reference, maintaining consistent localization by leveraging environmental features. This allows the vessel to preserve accurate navigation performance even in GNSS denied conditions.

The SLAM system provides the estimated states $\mathbf{X}$, $\mathbf{V}$, $\mathbf{R}$, $\mathbf{B}$, and $\mathbf{P}$, corresponding to position, velocity, rotation, IMU bias, and the associated pose covariance. The position and velocity are already expressed in the NED frame and can be directly used in the update step. The bias terms are also compatible, as they represent the same accelerometer and gyroscope bias components used by the state estimation filter. The covariance $\mathbf{P}$ describes the SLAM pose uncertainty and is used as the measurement noise covariance during the update.

The only required adjustment concerns the attitude representation. The state estimation filter expresses orientation as a unit quaternion $\mathbf{q}$, while the SLAM system provides a rotation matrix R_b^n on $SO(3)$. To maintain consistency, the quaternion is converted to its equivalent rotation matrix form $R(\mathbf{q})$ before the update. This ensures both systems operate in the same rotational space.

The SLAM update step is then straightforward. The measurement model is defined as

$$h(\mathbf{x}) = \mathbf{x}$$

with the measurement noise covariance taken directly from the SLAM pose uncertainty,

$$\mathbf{R} = \mathbf{P}_{SLAM} = \mathbf{P}_{xx}$$

If the Jacobian of the measurement model is required, it simplifies to $H = I$. This makes the correction step efficient and direct, allowing the global SLAM pose to realign the estimated navigation state and compensate for accumulated drift with minimal computational cost.

6.8 Tuning and Filter Verification

6.8.1 Debugging

When developing and implementing nonlinear state estimators such as the UKF-M, numerous factors can lead to degraded performance or instability. Common sources of error include incorrect model formulations, poor parameter initialization, and improper tuning of process and measurement noise covariances. These issues can cause the filter to become overconfident, overly conservative, or even diverge completely. Therefore, systematic verification and tuning procedures are essential to ensure reliable operation under both simulation and real world conditions.

6.8.2 Model Verification and Consistency Checks

Before tuning the filter, the correctness of the process model $f(\mathbf{x}, \mathbf{u})$ and measurement model $h(\mathbf{x})$ must be verified. This step ensures that the system dynamics and observation mappings used by the estimator accurately represent the real physical behavior of the vehicle and sensors. Verification is typically performed through simulation or offline replay of recorded sensor data, where predicted quantities are compared with measured ones under both static and dynamic conditions. Discrepancies often indicate modeling errors such as incorrect frame transformations, sign inversions, or unmodeled delays, which must be corrected before filter tuning can be meaningfully performed.

A powerful statistical tool for verifying model consistency is the “*Normalized Innovation Squared*” (NIS) test [14]. The NIS evaluates whether the statistical properties of the filter’s measurement innovations agree with the assumed noise characteristics. For each measurement update, the innovation is computed as

$$\boldsymbol{\nu}_k = \mathbf{z}_k - h(\hat{\mathbf{x}}_k^-)$$

with associated innovation covariance

$$S_k = H_k P_k^- H_k^\top + R_k,$$

where H_k is the measurement Jacobian or its sigma point equivalent in the UKF-M framework. The NIS value is then defined as

$$\epsilon_k = \boldsymbol{\nu}_k^\top S_k^{-1} \boldsymbol{\nu}_k.$$

If the filter is statistically consistent, ϵ_k should follow a χ^2 distribution with N degrees of freedom, where N is the dimensionality of the measurement vector.

Over a long sequence of measurements, the mean value of the NIS should approximate N , and its time history should remain within the confidence bounds of the χ_N^2 distribution. If ϵ_k frequently exceeds the upper confidence limit, the filter is overconfident, indicating that Q or R are underestimated or the models are inaccurate. Conversely, if ϵ_k is consistently below the lower limit, the filter is too conservative, suggesting that noise covariances are overestimated.

The NIS test is particularly useful because it provides a quantitative and objective way to verify both model fidelity and covariance tuning without relying on ground truth data. It helps identify whether inconsistencies stem from modeling errors or incorrect noise assumptions and is therefore an essential diagnostic tool during the design and validation of nonlinear estimators such as the UKF-M.

6.8.3 Tuning of Process and Measurement Covariances

The process noise covariance Q and the measurement noise covariance R are two of the most influential parameters governing the performance and stability of any Kalman filter. The matrix Q represents uncertainty in the process model, effectively describing how much trust the filter places in its predicted dynamics. In contrast, R defines the expected noise characteristics of the sensors and determines the degree of confidence placed in new measurements. Together, these matrices control the balance between model prediction and measurement correction.

Selecting Q too small makes the filter overly confident in its process model, leading to poor adaptation when the system encounters unmodeled disturbances or parameter variations. In such cases, the filter may ignore valid sensor information and exhibit lagging or diverging behavior. Conversely, choosing an excessively large Q injects too much uncertainty into the state propagation, resulting in noisy and

unstable estimates. A similar trade off exists for the measurement covariance R , where smaller values cause the filter to overreact to sensor noise, while larger values reduce update responsiveness and delay corrections.

A practical tuning strategy is to begin with nominal noise levels provided by the sensor manufacturer and gradually adjust them using logged test data. Innovation monitoring can then be used to assess whether the residuals ν_k are statistically consistent with their predicted covariance S_k . If innovations are systematically larger than expected, Q or R should be increased. If they are consistently smaller, these values can be reduced. This empirical process is often repeated iteratively until the NIS statistic converges toward its theoretical expectation.

In practice, final tuning often reflects a compromise between theoretical accuracy and robustness. Environmental effects, unmodeled dynamics, and sensor degradation can all influence the optimal noise parameters. For this reason, the selected Q and R matrices are sometimes treated as design parameters rather than fixed physical quantities, optimized for overall stability, smoothness, and performance under representative operating conditions.

6.8.4 Empirical Tuning Procedure

A practical way to tune and verify the filter performance is to use recorded test data from the target platform. The process and measurement noise covariances, Q and R , can be iteratively adjusted until the estimator produces stable and realistic results. Since the microAmpere ASV platform runs on a ROS2 based system, the available tooling such as “*ROS bags*” provides a convenient way to record, replay, and analyze sensor data offline. This allows testing the estimator repeatedly under identical conditions and evaluating its behavior during different maneuvers such as straight motion, turns, and accelerations.

During this tuning phase, the consistency of the filter can be evaluated by observing the innovation covariance S_k and NIS. This data help determine whether the chosen Q and R values properly represent the actual system and sensor noise. If the NIS frequently exceeds its expected bounds, the process or measurement noise is likely underestimated, while values that are consistently low indicate an overly conservative setup. By iteratively adjusting the noise parameters based on these observations, a statistically consistent and well-behaved estimator configuration can be achieved.

6.8.5 Innovation Monitoring and Outlier Rejection

Some extra aspects to consider when designing and testing the filter include how it handles measurement outliers and how its performance is monitored over time. Sensor data can occasionally contain large errors caused by multipath effects, dropouts, or temporary disturbances. Residual gating helps detect and reject such outliers by computing the Mahalanobis distance of the innovation and comparing it against a threshold from the χ^2 distribution. This ensures that only statistically consistent measurements are accepted during the update step, improving overall filter stability and fault tolerance.

The Mahalanobis distance is computed as

$$d^2 = \nu^\top S^{-1} \nu$$

where ν is the innovation and S its covariance. Note that the Mahalanobis distance is mathematically identical to the NIS test, the difference being in context. While NIS is typically used for offline filter consistency analysis, the Mahalanobis distance is applied online for real-time outlier detection. If the computed distance d^2 becomes larger than a predefined threshold $\chi_{\text{threshold}}^2$, the measurement is considered statistically inconsistent and is therefore rejected as an outlier (ie if : $d^2 > \chi_{\text{threshold}}^2 \rightarrow \text{Outlier!}$). Conversely, if $d^2 \leq \chi_{\text{threshold}}^2$, the measurement is accepted and used in the update step. This simple gating procedure is commonly used in practical filtering systems, as it provides a statistically grounded and computationally lightweight method for maintaining estimator robustness in the presence of faulty or inconsistent sensor data.

Innovation monitoring is also a useful tool for evaluating estimator behavior during operation. By observing the innovation sequence ν_k and its covariance S_k , it becomes possible to identify bias, drift, or incorrect noise assumptions in the models. In more advanced cases, adaptive tuning methods can be used to adjust Q and R dynamically in response to changing conditions, such as varying motion regimes or sensor degradation, helping maintain consistent and reliable filter performance without manual retuning.

6.8.6 Final State Definition and Conclusion

The final estimator state used in this work is defined as

$$\mathbf{x} = \begin{bmatrix} \mathbf{p}_{b/O}^n & \mathbf{v}_{b/O}^n & \mathbf{q} & \mathbf{a}_b & \omega_b \end{bmatrix}^\top$$

where position, velocity, and attitude are expressed in the NED frame, and accelerometer and gyroscope biases evolve in the Body frame. This structure provides a compact yet complete representation of the vehicle motion and sensor dynamics.

Overall, the Manifold based Unscented Kalman Filter (UKF-M) appears to be the most suitable framework for this project, offering a good compromise between numerical stability, modeling accuracy, and implementation simplicity. It eliminates the need for Jacobians while preserving geometric consistency for quaternion based attitude estimation.

7 Preintegration

7.1 Introduction

In graph based SLAM, the preintegration method addresses the computational inefficiency caused by high frequency inertial measurements relative to low frequency exteroceptive sensors such as sonar. In a typical Side Scan Sonar SLAM (SSS SLAM) setup, a new sonar 2D map image is produced every few seconds at a rate of less than 0.1 Hz, while the onboard IMU operates at several hundred hertz. If all IMU measurements were to be directly added to the factor graph as individual odometry factors by just simply using ESKF or UKF-M estimate, this would result in hundreds of redundant nodes between consecutive 2D sonar map frames. Such dense factor creation would significantly increase the computational load during optimization, while contributing limited additional information to the overall estimation process.

Preintegration resolves this problem by summarizing all IMU measurements between two 2D sonar map frames into a single compound odometry factor. This preintegrated factor represents the total relative motion, orientation change, and accumulated uncertainty over the integration interval, without introducing intermediate IMU states into the graph. The resulting factor provides the same essential motion constraints as full IMU integration using ESKF or UKF-M as Dead Reckoning estimate, but in a compact and computationally efficient form. This approach is particularly beneficial for SSS SLAM, where the sensor rate mismatch between the IMU and sonar is substantial.

The preintegration method uses the same INS motion model $f(x, u)$ as presented in Equation 6, with minor modifications in how biases are handled and how the state propagation is performed. In contrast to the discrete propagation used in the State Estimation chapter described in Equation 19, preintegration assumes the accelerometer and gyroscope biases remain constant over the short integration window. This assumption simplifies the computation while retaining sufficient accuracy for most robotic applications.

The outcome of preintegration is a single, bias correctable odometry factor connecting two consecutive keyframes at the sonar update rate. This allows the SLAM system to maintain precise and consistent motion constraints while keeping the factor graph sparse. The approach achieves comparable accuracy to full state propagation methods such as ESKF or UKF-M but avoids the overhead associated with large numbers of intermediate factors.

Preintegration has been widely adopted in modern SLAM frameworks, including the Georgia Tech Smoothing and Mapping library (GTSAM), which provides dedicated classes for implementing preintegrated IMU factors discussed in later chapters of the thesis. The method was originally introduced for visual inertial odometry paper [31] and has been used in many modern SLAM problems such as for LiDAR and radar inertial fusion [32]. The same principles directly apply to sonar inertial systems, where the slow image acquisition rate makes preintegration a critical component for efficient factor graph optimization.

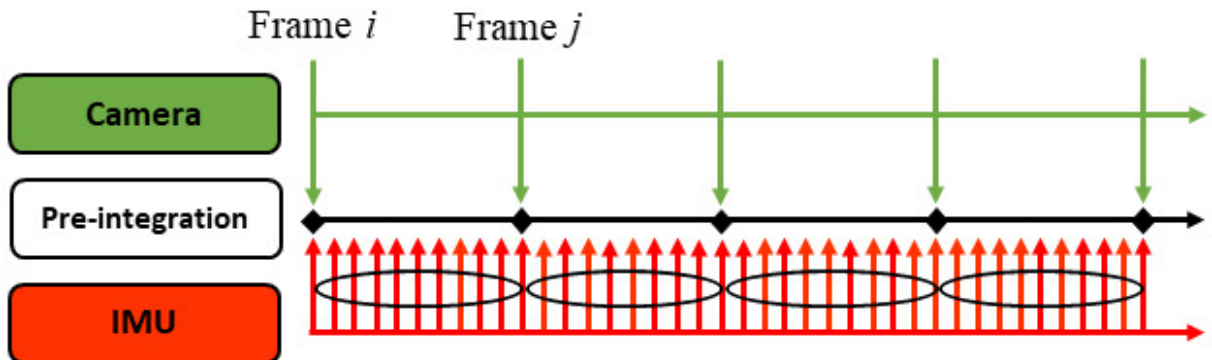


Figure 33: Illustration of the sampling rate mismatch between an IMU and an exteroceptive sensor such as a camera or sonar. The IMU operates at hundreds of hertz, producing dense inertial measurements, while the sonar or camera generates new observations at a much lower rate. Preintegration combines all intermediate IMU readings into a single relative motion constraint that aligns with the moment a camera or sonar image is acquired.^[33]

7.2 Preintegration on Manifolds

7.2.1 INS Propagation Between IMU Samples

The preintegration algorithm builds directly on the deterministic continuous time INS motion model defined in Equation 6, where the system state is given by $\mathbf{x} = [\mathbf{p}_{b/O}^n, \mathbf{v}_{b/O}^n, \mathbf{q}, \mathbf{a}_b, \boldsymbol{\omega}_b]^\top$ and the input vector by $\mathbf{u} = [\mathbf{a}_m, \boldsymbol{\omega}_m]^\top$, as presented in Equation 7. The same state representation is used here, consisting of position $\mathbf{p}$, velocity $\mathbf{v}$, and attitude $R(\mathbf{q})$, along with the accelerometer and gyroscope bias states $\mathbf{a}_b$ and $\boldsymbol{\omega}_b$. The motion model follows the same nominal kinematic equations as defined in the system modeling chapter, where the rigid body dynamics evolve according to the IMU measurements $(\mathbf{a}_m, \boldsymbol{\omega}_m)$ corrected by their respective biases. Hence, the same continuous time INS equations described in Equation 6 and the corresponding discrete form in Equation 19 apply here.

In contrast to the full state estimator formulation used previously, which employed a 1st order Gauss Markov process for modeling bias drift, preintegration adopts a simpler Brownian motion bias model. This simplification is made because the preintegration window is short, and any bias evolution within this interval is small compared to the measurement noise and will be corrected later by the backend optimizer. The Brownian model is defined as

$$\begin{aligned}\dot{\mathbf{a}}_b &= \boldsymbol{\eta}_{a_b} & \boldsymbol{\eta}_{a_b} &\sim \mathcal{N}(0, \sigma_{a_b}^2 I_3) \\ \dot{\boldsymbol{\omega}}_b &= \boldsymbol{\eta}_{\omega_b} & \boldsymbol{\eta}_{\omega_b} &\sim \mathcal{N}(0, \sigma_{\omega_b}^2 I_3)\end{aligned}\tag{25}$$

where $\boldsymbol{\eta}_{a_b}$ and $\boldsymbol{\eta}_{\omega_b}$ are zero mean Gaussian noise processes representing the bias random walk. The discrete time equivalent form becomes

$$\begin{aligned}\mathbf{a}_{b,k+1} &= \mathbf{a}_{b,k} + \boldsymbol{\eta}_{a_b,d} \\ \boldsymbol{\omega}_{b,k+1} &= \boldsymbol{\omega}_{b,k} + \boldsymbol{\eta}_{\omega_b,d}\end{aligned}\tag{26}$$

with $\text{Cov}(\boldsymbol{\eta}_{a_b,d}) = \sigma_{a_b}^2 \Delta t I_3$ and $\text{Cov}(\boldsymbol{\eta}_{\omega_b,d}) = \sigma_{\omega_b}^2 \Delta t I_3$. Over short preintegration intervals, these biases are assumed constant, meaning their change between IMU samples is negligible. The integration therefore proceeds using frozen bias estimates $\mathbf{a}_b$ and $\boldsymbol{\omega}_b$ from the start of the interval.

The nominal discrete time propagation of position, velocity, and rotation between IMU samples follows the same structure as the state estimation chapter in Equation 19, but is here expressed explicitly for clarity as

$$\begin{aligned}R_{k+1} &= R_k \exp([\boldsymbol{\omega}_{m,k} - \boldsymbol{\omega}_{b,k}]_\times \Delta t) \\ v_{k+1} &= v_k + R_k(\mathbf{a}_{m,k} - \mathbf{a}_{b,k})\Delta t + \mathbf{g}\Delta t \\ p_{k+1} &= p_k + v_k\Delta t + \frac{1}{2}R_k(\mathbf{a}_{m,k} - \mathbf{a}_{b,k})\Delta t^2 + \frac{1}{2}\mathbf{g}\Delta t^2\end{aligned}$$

In this formulation, the index k denotes consecutive IMU samples integrated at a high rate (typically hundreds of hertz). These samples are accumulated over the time interval between two keyframes i and j , where each keyframe corresponds to a slower exteroceptive measurement such as a sonar 2D image frame. The goal of preintegration is to compress all intermediate IMU updates ($k = i, \dots, j-1$) into a single compound relative motion estimate connecting the two keyframes i and j .

The raw IMU outputs $\mathbf{a}_{m,k}$ and $\boldsymbol{\omega}_{m,k}$ represent the measured specific force and angular velocity in the body frame. The bias terms $\mathbf{a}_{b,k}$ and $\boldsymbol{\omega}_{b,k}$ are the current estimates of the accelerometer and gyroscope biases and are subtracted once within the propagation equations to obtain the corrected physical quantities. The resulting propagation is therefore already bias compensated and forms the deterministic foundation for the preintegration and subsequent error propagation steps described in the following sections.

At the beginning of each new preintegration interval $(t_j, t_{j+1}]$, the initial state (R_i, v_i, p_i) is set equal to the optimized estimates from the previous keyframe, ie:

$$R_i = R_j^{\text{opt}}, \quad v_i = v_j^{\text{opt}}, \quad p_i = p_j^{\text{opt}}, \quad B_i = B_j^{\text{opt}}$$

This ensures that the preintegration always starts from the most accurate state and bias estimates provided by the backend optimizer, maintaining temporal consistency across all keyframe intervals.

7.2.2 Preintegration Initialization

Preintegration is initialized at the time of keyframe i , corresponding to the most recent exteroceptive measurement such as a sonar frame. All IMU samples collected between keyframes i and j are subsequently integrated in the local coordinate frame of keyframe i . This ensures that the resulting preintegrated quantities describe motion relative to keyframe i rather than the global frame, maintaining numerical stability and simplifying later optimization.

At the start of preintegration, the relative motion increments are initialized as

$$\Delta R_{ii} = I_3, \quad \Delta v_{ii} = \mathbf{0}_3, \quad \Delta p_{ii} = \mathbf{0}_3$$

which represent, respectively, the initial relative rotation, velocity, and position between the same keyframe. These quantities form the starting point for integrating subsequent IMU samples.

The IMU bias used during preintegration is frozen at its current estimate from keyframe i , denoted by

$$B_i^{\text{preint}} = [\mathbf{a}_{b,i}, \omega_{b,i}]$$

and is held constant throughout the preintegration interval $(t_i, t_j]$. The assumption of frozen bias is reasonable since the bias drift is slow compared to the short duration between keyframes, and any accumulated error is later corrected by the backend optimizer.

For uncertainty propagation, the Jacobians of the preintegrated quantities with respect to bias are initialized as

$$J_R^{\omega_b} = J_v^{\mathbf{a}_b} = J_v^{\omega_b} = J_p^{\mathbf{a}_b} = J_p^{\omega_b} = 0$$

These matrices are propagated forward with each IMU sample to capture how small bias perturbations affect the resulting preintegrated deltas, allowing efficient bias correction without re-integrating all IMU data.

All IMU readings are integrated relative to the orientation R_i of the starting keyframe, ensuring that the computed preintegrated deltas $(\Delta R_{ij}, \Delta v_{ij}, \Delta p_{ij})$ remain expressed consistently in the local frame of keyframe i . The index k denotes consecutive IMU samples integrated at a high rate (typically hundreds of hertz), while the index j represents the current end of the ongoing preintegration interval. As new IMU data arrive, j moves forward in time, meaning that $(\Delta R_{ij}, \Delta v_{ij}, \Delta p_{ij})$ are continuously updated until the next exteroceptive keyframe (eks sonar frame) is reached. Once keyframe j is established, the accumulated deltas at that moment represent the complete preintegrated motion between frames i and j .

This initialization therefore defines the starting state, bias configuration, and Jacobian setup for the recursive preintegration algorithm described in the following section.

7.2.3 Preintegration Algorithm (Recursive Update)

Once initialized, the preintegration proceeds recursively by integrating each incoming IMU measurement between the current keyframe i and the evolving endpoint j . The integration runs at IMU rate (typically hundreds of hertz), using the frozen bias estimate $B_i^{\text{preint}} = [\mathbf{a}_{b,i}, \omega_{b,i}]$. The preintegrated quantities ΔR_{ij} , Δv_{ij} , and Δp_{ij} are updated incrementally with every IMU sample k according to

$$\begin{aligned} \Delta R_{i,k+1} &= \Delta R_{i,k} \exp([\omega_{m,k} - \omega_{b,i}]_{\times} \Delta t) \\ \Delta v_{i,k+1} &= \Delta v_{i,k} + \Delta R_{i,k}(\mathbf{a}_{m,k} - \mathbf{a}_{b,i})\Delta t \\ \Delta p_{i,k+1} &= \Delta p_{i,k} + \Delta v_{i,k}\Delta t + \frac{1}{2}\Delta R_{i,k}(\mathbf{a}_{m,k} - \mathbf{a}_{b,i})\Delta t^2 \end{aligned} \tag{27}$$

The exponential map $\exp([\cdot]_{\times})$ ensures that the orientation update remains consistent on the manifold $SO(3)$, maintaining a valid rotation representation after each integration step. These updates are applied sequentially for all IMU samples within the preintegration window $(t_i, t_j]$.

The integration is performed in the local frame of keyframe i , meaning that all quantities $(\Delta R_{ij}, \Delta v_{ij}, \Delta p_{ij})$ are expressed relative to the orientation R_i . The index k refers to the current IMU sample, while j marks the progressively advancing endpoint of the preintegration interval as new IMU data arrive. When the next exteroceptive keyframe j (eks a 2D sonar image) is received, the accumulated deltas represent the complete preintegrated motion between the two complete keyframes i and j .

This recursive integration scheme effectively compresses all high frequency IMU updates into a single relative motion constraint while preserving the nonlinear structure of the underlying kinematics on $SE(3)$. It forms the foundation for the subsequent Jacobian propagation and covariance update described in the following sections.

7.2.4 Bias Jacobian Propagation

The bias Jacobian propagation captures how small changes in accelerometer and gyroscope biases affect the preintegrated quantities $(\Delta R_{ij}, \Delta v_{ij}, \Delta p_{ij})$. Although the biases are assumed constant within each preintegration interval, they are later refined by the optimizer. By maintaining their partial derivatives, the preintegration can be corrected efficiently without re-integrating all IMU data. The Jacobians represent the first-order sensitivity of the preintegrated deltas with respect to the bias states:

$$J_R^{\omega_b} = \frac{\partial \text{Log}(\Delta R_{ij})}{\partial \omega_b}, \quad J_v^{\mathbf{a}_b} = \frac{\partial \Delta v_{ij}}{\partial \mathbf{a}_b}, \quad J_v^{\omega_b} = \frac{\partial \Delta v_{ij}}{\partial \omega_b}, \quad J_p^{\mathbf{a}_b} = \frac{\partial \Delta p_{ij}}{\partial \mathbf{a}_b}, \quad J_p^{\omega_b} = \frac{\partial \Delta p_{ij}}{\partial \omega_b}$$

The $\text{Log}(\cdot)$ operator in $J_R^{\omega_b}$ maps the incremental rotation ΔR_{ij} from the manifold $SO(3)$ to its tangent space $\mathfrak{so}(3)$, allowing the small rotation errors to be represented as 3D vectors in Euclidean space where derivatives can be taken linearly (See Equations 1 and 2). All Jacobians are initialized to zero at the start of preintegration and propagated at every IMU timestep using 1st order linearization:

$$J(t + \Delta t) = J(t) + \frac{\partial(\Delta R, \Delta v, \Delta p)}{\partial b} \Delta t$$

No higher order bias dynamics are modeled since bias drift is slow and follows a Brownian process. Given the nominal preintegration updates with Equation 27 the bias Jacobians evolve as

$$\begin{aligned} J_R^{\omega_b}(k+1) &\approx J_R^{\omega_b}(k) - \Delta R_{i,k} \Gamma_1 \Delta t \\ J_v^{\mathbf{a}_b}(k+1) &\approx J_v^{\mathbf{a}_b}(k) - \Delta R_{i,k} \Delta t \\ J_v^{\omega_b}(k+1) &\approx J_v^{\omega_b}(k) - \Delta R_{i,k} [\mathbf{a}_{m,k} - \mathbf{a}_{b,i}]_{\times} J_R^{\omega_b}(k) \Delta t \\ J_p^{\mathbf{a}_b}(k+1) &\approx J_p^{\mathbf{a}_b}(k) + J_v^{\mathbf{a}_b}(k) \Delta t - \frac{1}{2} \Delta R_{i,k} \Delta t^2 \\ J_p^{\omega_b}(k+1) &\approx J_p^{\omega_b}(k) + J_v^{\omega_b}(k) \Delta t - \frac{1}{2} \Delta R_{i,k} [\mathbf{a}_{m,k} - \mathbf{a}_{b,i}]_{\times} J_R^{\omega_b}(k) \Delta t^2 \end{aligned}$$

where Γ_1 is the 1st order right Jacobian of $SO(3)$, evaluated at the small rotation vector $\phi_k = (\omega_{m,k} - \omega_{b,i}) \Delta t$. It maps perturbations in the Lie algebra (tangent space) to perturbations on the manifold, ensuring correct rotation updates for small angular increments (See Equations 1 and 2). In closed form

$$\Gamma_1(\phi_k) = I_3 - \frac{1 - \cos \|\phi_k\|}{\|\phi_k\|^2} [\phi_k]_{\times} + \frac{\|\phi_k\| - \sin \|\phi_k\|}{\|\phi_k\|^3} [\phi_k]_{\times}^2$$

and for small rotations, it can be approximated as

$$\Gamma_1(\phi_k) \approx I_3 - \frac{1}{2} [\phi_k]_{\times}$$

This Jacobian maintains manifold consistency by correctly relating incremental angular changes to their corresponding local linearizations on $SO(3)$. The $\text{Log}(\cdot)$ operator in $J_R^{\omega_b}$ projects the rotational error from the manifold $SO(3)$ to the tangent space $\mathfrak{so}(3)$, providing a linear representation of small rotation errors that enables differentiation with respect to the gyroscope bias.

Finally, the Jacobians computed at each IMU timestep are accumulated over the full preintegration interval $(t_i, t_j]$ to form the total bias sensitivity between keyframes i and j . This accumulation corresponds to summing the incremental contributions from each IMU update:

$$J_R^{\omega_b} = \sum_{k=i}^{j-1} J_R^{\omega_b}(k), \quad J_v^{\mathbf{a}_b} = \sum_{k=i}^{j-1} J_v^{\mathbf{a}_b}(k), \quad J_v^{\omega_b} = \sum_{k=i}^{j-1} J_v^{\omega_b}(k), \quad J_p^{\mathbf{a}_b} = \sum_{k=i}^{j-1} J_p^{\mathbf{a}_b}(k), \quad J_p^{\omega_b} = \sum_{k=i}^{j-1} J_p^{\omega_b}(k)$$

Thus, each Jacobian term represents the cumulative 1st order effect of bias perturbations over all IMU samples between the two keyframes. These accumulated Jacobians form the final bias correction matrices used in the subsequent preintegration update step.

7.2.5 Bias Re-Linearization Using Accumulated Jacobians

When the optimizer send update and corrects the bias estimates, the stored preintegrated quantities must be re-linearized to stay consistent with the new bias values. Instead of re-integrating all IMU samples, this correction is efficiently performed using the accumulated Jacobians computed during preintegration.

The bias correction increment is defined as

$$\delta B_i = [\delta \mathbf{a}_b, \delta \boldsymbol{\omega}_b] = B_i^{\text{opt}} - B_i^{\text{preint}}$$

where B_i^{preint} is the frozen bias used during preintegration and B_i^{opt} is the updated bias from the optimizer. The corrected preintegrated quantities are obtained by applying a 1st order correction using the propagated Jacobians:

$$\begin{aligned}\Delta R_{ij}^* &\approx \Delta R_{ij} \exp(J_R^{\omega_b} \delta \boldsymbol{\omega}_b) \\ \Delta v_{ij}^* &\approx \Delta v_{ij} + J_v^{\mathbf{a}_b} \delta \mathbf{a}_b + J_v^{\omega_b} \delta \boldsymbol{\omega}_b \\ \Delta p_{ij}^* &\approx \Delta p_{ij} + J_p^{\mathbf{a}_b} \delta \mathbf{a}_b + J_p^{\omega_b} \delta \boldsymbol{\omega}_b\end{aligned}$$

where $(\cdot)^*$ denotes the bias corrected preintegrated quantities. The $\exp(\cdot)$ operator maps the small correction $J_R^{\omega_b} \delta \boldsymbol{\omega}_b$ from the tangent space back onto the rotation manifold $SO(3)$, ensuring consistent attitude updates (See Equations 1 and 2).

The accumulated Jacobians $(J_R^{\omega_b}, J_v^{\mathbf{a}_b}, J_v^{\omega_b}, J_p^{\mathbf{a}_b}, J_p^{\omega_b})$ compactly represent the total 1st order sensitivity of the preintegrated deltas to bias changes across the entire preintegration interval $(t_i, t_j]$. Using these, the optimizer can instantly re-evaluate the preintegrated measurements without reprocessing any IMU data, maintaining full geometric consistency while minimizing computational cost.

7.2.6 Predicted Motion Reconstruction

Once the preintegrated measurements have been bias corrected, they can be used to reconstruct the nominal motion between the two keyframes i and j . This reconstruction provides the predicted navigation state $(\hat{R}_j, \hat{v}_j, \hat{p}_j)$ at keyframe j , given the known state (R_i, v_i, p_i) at keyframe i and the corrected preintegrated deltas $(\Delta R_{ij}^*, \Delta v_{ij}^*, \Delta p_{ij}^*)$.

The nominal motion prediction is obtained as

$$\begin{aligned}\hat{R}_j &= R_i \Delta R_{ij}^* \\ \hat{v}_j &= v_i + \mathbf{g} \Delta t_{ij} + R_i \Delta v_{ij}^* \\ \hat{p}_j &= p_i + v_i \Delta t_{ij} + \frac{1}{2} \mathbf{g} \Delta t_{ij}^2 + R_i \Delta p_{ij}^*\end{aligned}$$

where $\mathbf{g}$ is the gravity vector expressed in the navigation frame, and Δt_{ij} is the total elapsed time between keyframes i and j . The predicted quantities $(\hat{R}_j, \hat{v}_j, \hat{p}_j)$ represent the best estimate of the motion over the interval $(t_i, t_j]$ using only IMU data.

The resulting predicted state for keyframe j is compactly expressed as

$$\hat{X}_j = \begin{bmatrix} \hat{p}_j \\ \hat{v}_j \\ \hat{R}_j \end{bmatrix}$$

which serves as the nominal prior for the next optimization step and ensures temporal consistency across keyframes.

7.2.7 Predicted Bias Reconstruction

In addition to predicting the nominal motion, the IMU bias state must also be propagated between keyframes. The bias follows the Brownian random walk model introduced earlier in Equation 25, which assumes slow, uncorrelated drift over short time intervals.

The discrete time bias propagation between consecutive IMU samples is given by Equation 26,

where the bias mean remains constant, but its uncertainty grows due to the additive white noise processes $\boldsymbol{\eta}_{a_b,d}$ and $\boldsymbol{\eta}_{\omega_b,d}$. Over the preintegration interval $(t_i, t_j]$, this simplifies to

$$\begin{aligned}\hat{\mathbf{a}}_{b,j} &= \mathbf{a}_{b,i} \\ \hat{\boldsymbol{\omega}}_{b,j} &= \boldsymbol{\omega}_{b,i}\end{aligned}$$

indicating that the bias mean is held constant throughout the preintegration. The corresponding uncertainty growth due to the Brownian process is captured later through the bias covariance matrix $\mathbf{Q}_b$.

The predicted bias state at keyframe j is therefore

$$\hat{B}_j = \begin{bmatrix} \hat{\mathbf{a}}_{b,j} \\ \hat{\boldsymbol{\omega}}_{b,j} \end{bmatrix} = \begin{bmatrix} \mathbf{a}_{b,i} \\ \boldsymbol{\omega}_{b,i} \end{bmatrix}$$

and is passed together with the predicted motion state $\hat{X}_j$ to the backend optimizer for joint correction and relinearization.

7.2.8 Preintegration Covariance Propagation

In addition to the predicted navigation states, the associated uncertainty must also be propagated to quantify the confidence of the preintegrated motion estimate. This covariance describes how IMU process noise accumulates over the preintegration interval and directly influences the weighting of the IMU constraint during optimization.

The propagation of covariance during IMU preintegration follows the same mathematical principles as the ESKF formulation presented in the State Estimation chapter. In particular, it corresponds to a reduced version of the full error state INS model in Equation 21, using only the first three state components, position, velocity, and attitude. The bias states are omitted here since their uncertainty is treated separately under the Brownian motion bias model described previously. The resulting reduced error state vector is therefore defined as

$$\delta \mathbf{x} = \begin{bmatrix} \delta \mathbf{p} \\ \delta \mathbf{v} \\ \delta \mathbf{q} \end{bmatrix}$$

where $\delta \mathbf{q}$ represents the small quaternion attitude error, approximated as in Equation 20 using the small angle representation

$$\delta \mathbf{q} \approx \begin{bmatrix} 1 \\ \frac{1}{2} \delta \boldsymbol{\theta} \end{bmatrix}$$

This small angle approximation allows attitude uncertainty to be represented in a minimal 3D tangent space, preserving the quaternion manifold structure while maintaining linear error propagation.

Continuous Time Error Dynamics

The continuous time error dynamics describe how small perturbations in position, velocity, and attitude evolve under the influence of IMU noise. These equations are not used to compute the nominal motion itself, only to propagate covariance and track how uncertainty grows over time.

The reduced error state is defined as

$$\delta \mathbf{x}(t) = \begin{bmatrix} \delta \mathbf{p}(t) \\ \delta \mathbf{v}(t) \\ \delta \boldsymbol{\phi}(t) \end{bmatrix}$$

where $\delta \mathbf{p}$, $\delta \mathbf{v}$, and $\delta \boldsymbol{\phi}$ are small perturbations in position, velocity, and attitude. The attitude error $\delta \boldsymbol{\phi}$ lies in the tangent space of $SO(3)$ and locally perturbs the nominal rotation as

$$R(\mathbf{q}) = \hat{R}(\mathbf{q}) \exp([\delta \boldsymbol{\phi}]_{\times})$$

This represents a minimal 3D error model that allows linear propagation of orientation uncertainty while preserving the manifold constraint.

The linearized error dynamics follow the first three block rows of the ESKF system matrix in Equation 22, excluding bias terms:

$$\delta\dot{\mathbf{x}}(t) = A(\mathbf{x})\delta\mathbf{x}(t) + G(\mathbf{x})\mathbf{n}(t)$$

where $\mathbf{n}(t) = [\mathbf{n}_a, \mathbf{n}_\omega]^\top$ represents zero-mean white Gaussian accelerometer and gyroscope noise.

The system matrices are

$$A(\mathbf{x}) = \begin{bmatrix} 0 & I_3 & 0 \\ 0 & 0 & -R_i^\top R(\mathbf{q})[\mathbf{a}_m - \mathbf{a}_b]_\times \\ 0 & 0 & -[\boldsymbol{\omega}_m - \boldsymbol{\omega}_b]_\times \end{bmatrix}, \quad G(\mathbf{x}) = \begin{bmatrix} 0 & 0 \\ -R_i^\top R(\mathbf{q}) & 0 \\ 0 & -I_3 \end{bmatrix} \quad (28)$$

Here $R_i^\top R(\mathbf{q})$ gives the relative rotation between the start keyframe i and the current IMU orientation, ensuring that all quantities remain expressed in the local frame of keyframe i .

Although the true error states $\delta\mathbf{x}$ are never explicitly computed, the corresponding linearized dynamics through $A(\mathbf{x})$ and $G(\mathbf{x})$ are used to propagate the covariance over time. This provides an efficient and consistent way to capture the growth of IMU induced uncertainty between keyframes.

Discretization and Covariance Propagation

As discussed in previous chapters under State Estimation in ESKF discretization section (See Equation 23), discretization of the continuous error dynamics is necessary for digital implementation. Similar to the ESKF, the Zero Order Hold (ZOH) method can be applied here, or similar methods, assuming the IMU inputs and noise remain constant within each sampling interval Δt . The discrete time covariance propagation is therefore given by

$$P_{IMU,k+1} = A_k P_{IMU,k} A_k^\top + B_k Q_\eta B_k^\top$$

where

$$Q_\eta = \text{diag}(\sigma_a^2 I_3, \sigma_\omega^2 I_3)$$

is the continuous time IMU noise covariance, scaled by the timestep Δt within the discretization. This formulation is mathematically equivalent to the discretized process noise computation in Equation 23, except that only the preintegrated motion subspace $(\mathbf{p}, \mathbf{v}, \mathbf{q})$ is considered. The initial covariance at the start of preintegration is set to

$$P_{IMU,ii} = 0_{9 \times 9},$$

indicating no accumulated uncertainty at keyframe i . As IMU samples are integrated, $P_{IMU,k}$ evolves recursively with each step, incorporating both linearized system dynamics and sensor noise effects.

Resulting Preintegration Covariance

After integrating all IMU measurements between keyframes i and j , the final propagated covariance is obtained as

$$P_{IMU,ij} = P_{IMU,k_{\text{end}}}$$

This 9×9 covariance matrix represents the total uncertainty of the preintegrated relative motion $(\Delta p_{ij}, \Delta v_{ij}, \Delta R_{ij})$ in the local frame of keyframe i . The inverse covariance, referred to as the information matrix,

$$\Lambda_{IMU,ij} = P_{IMU,ij}^{-1}$$

is used to weight the IMU factor in the nonlinear optimizers such as iSAM2 discussed in later chapters. A higher $\Lambda_{IMU,ij}$ corresponds to greater trust in the IMU preintegration constraint, while a lower weight allows more correction from exteroceptive sensor updates such as sonar or vision.

In summary, this preintegration covariance propagation step is mathematically identical to the error state covariance propagation in the ESKF (Equations 21-23), except that it operates on the reduced state $(\mathbf{p}, \mathbf{v}, \mathbf{q})$ and uses the local frame of keyframe i . This reduction significantly improves computational efficiency while maintaining consistency with the underlying inertial error dynamics.

7.2.9 Bias Covariance Propagation

In parallel with the preintegrated motion covariance, the uncertainty of the IMU bias states must also be propagated between keyframes. As introduced in Equation 25, both accelerometer and gyroscope biases follow a Brownian random walk process, representing slow stochastic drift with zero mean. Over the short duration of a preintegration interval, the bias means remain effectively constant, while their uncertainty increases linearly with time.

Applying the same discretization method used for the preintegration covariance like ZOH or similar methods (see Equation 23), the discrete time bias covariance propagation is given by

$$P_{b,ij} = \begin{bmatrix} \sigma_{a_b}^2 I_3 & 0 \\ 0 & \sigma_{\omega_b}^2 I_3 \end{bmatrix} \Delta t_{ij}$$

This represents the accumulated uncertainty in the bias states between keyframes i and j , arising from the continuous time noise processes $\boldsymbol{\eta}_{a_b}$ and $\boldsymbol{\eta}_{\omega_b}$. The bias covariance evolves independently from the preintegrated motion covariance since the bias random walk is assumed uncorrelated with the instantaneous IMU measurement noise.

The corresponding bias information matrix is therefore defined as

$$\Lambda_{b,ij} = P_{b,ij}^{-1}$$

and represents the statistical weighting of the bias factor in the optimizer. This ensures that the bias evolution constraint contributes independently from the motion factor, allowing consistent and decoupled correction of both motion and bias estimates during optimization.

7.2.10 Algorithm Summary

The complete preintegration procedure can be summarized as follows:

- **Initialization:** Set $\Delta R_{ii} = I_3$, $\Delta v_{ii} = 0$, $\Delta p_{ii} = 0$, and initialize bias Jacobians $J = 0$. Freeze bias estimate $B_i^{\text{preint}} = [\mathbf{a}_{b,i}, \boldsymbol{\omega}_{b,i}]$
- **Recursive IMU Integration:** Integrate all IMU samples $(\mathbf{a}_{m,k}, \boldsymbol{\omega}_{m,k})$ between keyframes i and j to obtain $\Delta R_{ij}, \Delta v_{ij}, \Delta p_{ij}$
- **Bias Jacobian Propagation:** Propagate and accumulate $J_R^{\omega_b}, J_v^{\mathbf{a}_b}, J_v^{\omega_b}, J_p^{\mathbf{a}_b}, J_p^{\omega_b}$
- **Bias Correction:** Apply bias correction using accumulated Jacobians to get $\Delta R_{ij}^*, \Delta v_{ij}^*, \Delta p_{ij}^*$
- **Predicted State Reconstruction:** Compute $\hat{R}_j, \hat{v}_j, \hat{p}_j$ from (R_i, v_i, p_i) and $(\Delta R_{ij}^*, \Delta v_{ij}^*, \Delta p_{ij}^*)$
- **Predicted Bias Reconstruction:** Propagate $\hat{B}_j = [\hat{\mathbf{a}}_{b,j}, \hat{\boldsymbol{\omega}}_{b,j}] = [\mathbf{a}_{b,i}, \boldsymbol{\omega}_{b,i}]$
- **State Covariance Propagation:** Compute $P_{IMU,ij}$ and corresponding information matrix $\Lambda_{IMU,ij} = P_{IMU,ij}^{-1}$
- **Bias Covariance Propagation:** Compute $P_{b,ij}$ and corresponding information matrix $\Lambda_{b,ij} = P_{b,ij}^{-1}$
- **Output:** Return $(\Delta R_{ij}^*, \Delta v_{ij}^*, \Delta p_{ij}^*, \hat{B}_j, P_{IMU,ij}, P_{b,ij}, \Lambda_{IMU,ij}, \Lambda_{b,ij})$

7.3 Factor Construction

7.3.1 Factor Graph Formulation

The optimizer operates on a factor graph representation of the estimation problem, where each node corresponds to a keyframe state and each edge, or factor, represents a probabilistic constraint derived from sensor measurements. For the preintegration framework to function within this optimization structure, the outputs from the preintegration algorithm must be converted into factors that connect consecutive keyframes. These factors define how motion and bias estimates evolve between time steps and provide the necessary mathematical relationships that the optimizer uses to minimize overall error.

The factor graph formulation is crucial because it transforms the continuous time IMU dynamics into discrete, optimization ready constraints. Each factor contributes a residual that measures the discrepancy between the predicted and observed relationships of connected keyframes. During optimization, all residuals are jointly minimized to produce the most consistent estimate of both motion and bias states over time.

In the context of IMU preintegration, two distinct but related factors are constructed. The first one is the IMU preintegration factor, which captures the relative motion between keyframes using the preintegrated measurements. The second one is the bias evolution factor, which models the slow stochastic drift of accelerometer and gyroscope biases. Together, these form the complete inertial constraint within the factor graph, allowing the optimizer to refine all state and bias variables simultaneously and propagate the optimal corrected state back into the next preintegration cycle.

Once the optimization converges, the updated states $(R_j^{\text{opt}}, v_j^{\text{opt}}, p_j^{\text{opt}})$ and biases B_j^{opt} are extracted from the graph and used to initialize the next preintegration interval $(t_j, t_{j+1}]$ and then constructing the factors again. This recursive update maintains temporal consistency between keyframes and ensures that each new preintegration step starts from the best available estimate.

7.3.2 IMU Preintegration Factor

This factor represents the relative motion constraint between keyframes i and j based on all IMU data integrated over the interval $(t_i, t_j]$. It enforces consistency between the estimated states (R_i, v_i, p_i) and (R_j, v_j, p_j) and the preintegrated measurements $(\Delta R_{ij}^*, \Delta v_{ij}^*, \Delta p_{ij}^*)$ computed previously.

Here, R_i , v_i , and p_i correspond to the optimized state estimates from the previous keyframe, denoted as R_i^{opt} , v_i^{opt} , and p_i^{opt} , provided by the backend optimizer. Similarly, R_j , v_j , and p_j represent the current state estimates under optimization, denoted as $\hat{R}_j$, $\hat{v}_j$, and $\hat{p}_j$. The preintegrated quantities $(\Delta R_{ij}^*, \Delta v_{ij}^*, \Delta p_{ij}^*)$ are fixed measurements obtained from the preintegration process and serve as the relative motion observation linking these two states.

The residual formulation is given as

$$r_{\text{IMU}}^{ij} = \begin{bmatrix} \log((\Delta R_{ij}^*)^\top R_i^\top R_j) \\ R_i^\top (v_j - v_i - g\Delta t_{ij}) - \Delta v_{ij}^* \\ R_i^\top (p_j - p_i - v_i\Delta t_{ij} - \frac{1}{2}g\Delta t_{ij}^2) - \Delta p_{ij}^* \end{bmatrix}$$

Each residual term penalizes deviation from the preintegrated motion predicted by the IMU. The factor is weighted by the corresponding information matrix $\Lambda_{\text{IMU},ij} = P_{\text{IMU},ij}^{-1}$, defining its confidence based on accumulated IMU noise.

This single factor efficiently summarizes hundreds of raw IMU readings, maintaining accuracy while drastically reducing computational cost.

7.3.3 Bias Evolution Factor

To maintain bias consistency between keyframes, a separate factor enforces smooth bias evolution according to the Brownian motion model.

Here, B_i represents the optimized bias estimate from the previous keyframe, denoted as

$B_i^{\text{opt}} = [\mathbf{a}_{b,i}^{\text{opt}}, \boldsymbol{\omega}_{b,i}^{\text{opt}}]^\top$, while B_j corresponds to the current bias state being optimized, denoted as $\hat{B}_j = [\hat{\mathbf{a}}_{b,j}, \hat{\boldsymbol{\omega}}_{b,j}]^\top$.

The residual formulation is given as

$$r_b^{ij} = B_j - B_i$$

weighted by the bias information matrix $\Lambda_{b,ij} = P_{b,ij}^{-1}$.

This factor captures the slow, random drift of gyroscope and accelerometer biases over time and ensures stable long-term estimation.

7.3.4 Preintegration Trigger and Factor Connectivity

IMU preintegration operates continuously between discrete keyframe events, such as GNSS updates or landmark detections. A new odometry pose is created whenever such an event occurs. During the interval between two events $(t_i, t_j]$, all incoming IMU measurements are accumulated and preintegrated to form a compact relative motion constraint. When the event is triggered, preintegration stops, a new pose node j is created, and a preintegrated IMU factor is added between the previous and current pose nodes (i, j) .

All other sensor measurements that arrived within this interval such as GNSS positions, landmark detections, or visual observations are attached to the new pose node as unary factors, each providing an independent residual and noise model. This approach ensures that all sensor data are temporally aligned to the event that created the keyframe while keeping the number of poses in the graph minimal.

The bias state is connected through its own dedicated bias evolution factor, which links the bias variable B_i of the previous keyframe to the new bias variable B_j at the current keyframe. This maintains smooth bias propagation across time and ensures that both the motion (IMU preintegration) and bias drift are jointly optimized. The preintegrated IMU factor depends directly on both pose and bias states, meaning that changes in B_i and B_j influence the predicted motion residual. Together, the IMU preintegration factor, bias evolution factor, and sensor measurement factors form a fully coupled constraint system that updates pose, velocity, and bias consistently at each event driven keyframe.

8 Direct Factors

8.1 Introduction

In contrast to complex factors such as odometry or landmark constraints that require state propagation, data association, or nonlinear optimization, some sensor measurements can be directly converted into factor graph constraints with minimal processing.

These are referred to as “*direct factors*”. They provide immediate information about the system state, typically through absolute measurements such as position or orientation. For the microAmpere ASV, one of the main direct factors is the GNSS factor, which directly relates the estimated vehicle pose to globally referenced position and heading measurements obtained from the dual antenna GNSS system.

A direct factor acts as an event trigger in the estimation process. When a new measurement event occurs, such as a GNSS update or a landmark detection, the IMU preintegration is paused, a new odometry pose node is created, and the preintegrated IMU factor is added between the previous and current poses. The new measurement is then attached as a unary direct factor to the latest pose before the optimizer updates the state estimate, the preintegration is reset, and continuous integration resumes. This mechanism ensures that all direct measurements remain synchronized with the factor graph and that each event driven update refines the system state in real time without introducing unnecessary intermediate nodes that have nothing attached to it.

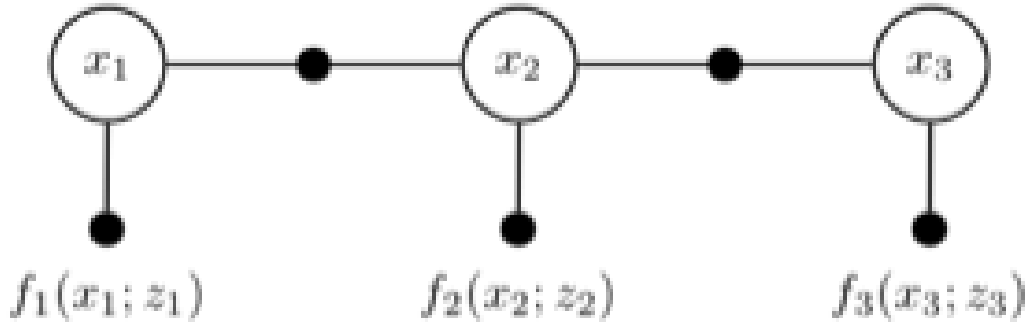


Figure 34: Example of a direct factor graph structure with unary measurement factors such as GNSS, each adding an absolute constraint on position and orientation directly to the associated pose/odometry node.^[34]

8.2 GNSS Factor

The GNSS factor represents a direct measurement constraint that links the estimated vehicle pose to the absolute position and heading obtained from the dual-antenna GNSS receiver. Unlike odometry or landmark based factors, which require preintegration or feature association, the GNSS factor can be constructed directly from the raw measurements with minimal preprocessing, classifying it as a *direct factor*.

Following the formulation of unary factors in GTSAM tutorial on Factor Graphs [34], the GNSS factor can be expressed as a probabilistic constraint between the estimated navigation state and the measured quantities in the NED frame:

$$\mathbf{r}_{GNSS} = \mathbf{z}_{GNSS} - h(\mathbf{x})$$

where $\mathbf{z}_{GNSS}$ is the measurement vector containing the absolute position and yaw angle, and $h(\mathbf{x})$ is the nonlinear measurement model defined in Equation 9. The corresponding measurement noise is modeled as a zero mean Gaussian:

$$\mathbf{r}_{GNSS} \sim \mathcal{N}(0, \mathbf{R}_{GNSS})$$

The corresponding measurement noise is modeled as a zero mean Gaussian with covariance $\mathbf{R}_{GNSS}$, as defined in Equation 10. This covariance represents the uncertainty associated with the GNSS aiding measurements, where the first three terms correspond to positional uncertainty in the NED frame, and the fourth term describes the yaw variance derived from the dual antenna baseline configuration.

In the factor graph framework, the GNSS observation is implemented as a unary factor connected to the current pose node, contributing a residual term of the form

$$\|\mathbf{z}_{GNSS} - h(\mathbf{x})\|_{\mathbf{R}_{GNSS}^{-1}}^2$$

This factor is added incrementally at each GNSS update epoch using solvers such as iSAM2, which re-linearize only the affected local variables within the corresponding clique [35]. These optimization methods, including iSAM2 and batch solvers, are discussed in more detail later in the “*Optimizers*” chapter, where their operation and impact on factor graph efficiency are explained.

By incorporating both position and yaw, the GNSS factor provides a global reference that anchors the estimated trajectory in the navigation frame and stabilizes heading drift over time. For the microAmpere ASV, this dual constraint serves as a reliable absolute reference that complements the relative motion information derived from IMU preintegration, ensuring globally consistent and drift free navigation.

9 Local Map Generation

9.1 Introduction

This chapter presents the methodology used for generating local side scan sonar maps, which serve as input to the data association stage in the SLAM frontend. The main objective is to transform raw 1D sonar swaths into geometrically and radiometrically corrected 2D intensity maps that represent the local seafloor reflectivity in Cartesian coordinates. These maps provide spatially consistent seafloor representations that can be used for feature extraction and localization in autonomous navigation systems.

The mapping process follows a structured pipeline consisting of three main stages, swath processing, probabilistic map construction, and feature extraction. Swath processing compensates for vehicle motion, corrects for beam geometry, and normalizes sonar intensity. The probabilistic construction stage then fuses consecutive swaths into a coherent 2D local reflectivity map using a beam based uncertainty model that accounts for angular spread and measurement overlap. Finally, distinctive acoustic landmarks are extracted from the map and passed to the SLAM frontend for data association and state estimation.

Local maps are generated incrementally using a sliding window approach, where each new map partially overlaps with the previous one. Typically, 1/3rd of the earlier data is retained while 2/3rds are replaced with new sonar information. This overlap ensures temporal and spatial continuity between consecutive maps, preventing landmark loss and maintaining stable feature tracking.

The methodology presented here is primarily based on and extends previous work by Haralstad [2] and Hoff [1], incorporating several adaptations for real-time implementation on the MicroAmpere ASV platform. These include a refined probabilistic mapping model for uncertainty aware construction and optimized data flow for embedded processing.

9.2 Swath Processing

9.2.1 Raw Data and Waterfall Image Formation

Side scan sonar data acquisition begins with the transmission of acoustic pulses and the recording of their backscattered echoes along each sonar beam. Each transmitted ping produces a 1D return signal that represents the intensity of acoustic reflections as a function of slant range. This signal corresponds to the strength of the backscattered energy received from the seafloor at varying distances from the transducer.

As the vessel advances, consecutive pings are recorded sequentially and combined to form a 2D intensity representation known as a “*waterfall image*” or raw swath. In this format, the horizontal axis corresponds to the ping index or along track distance, while the vertical axis represents the across track slant range. The pixel intensity encodes the relative strength of the seabed backscatter, providing an initial qualitative visualization of seafloor texture and reflectivity.

This raw waterfall image forms the foundational representation of side scan sonar data. However, it remains expressed in slant range space rather than true horizontal ground coordinates, introducing geometric distortions, particularly near the nadir region where the sonar beam intersects the seafloor at steep angles. Consequently, the apparent scale and spatial arrangement of features vary with range, and the image cannot yet be interpreted as a direct top down map of the seafloor.

The subsequent stages of swath processing correct for these geometric and radiometric effects to recover an accurate, physically consistent representation of seabed reflectivity. These stages include attitude compensation for pitch and roll, blind zone removal, slant range correction, and intensity normalization. Together, they transform the raw sonar measurements into a geometrically corrected and radiometrically balanced swath suitable for Cartesian projection and feature extraction in later mapping and SLAM stages.

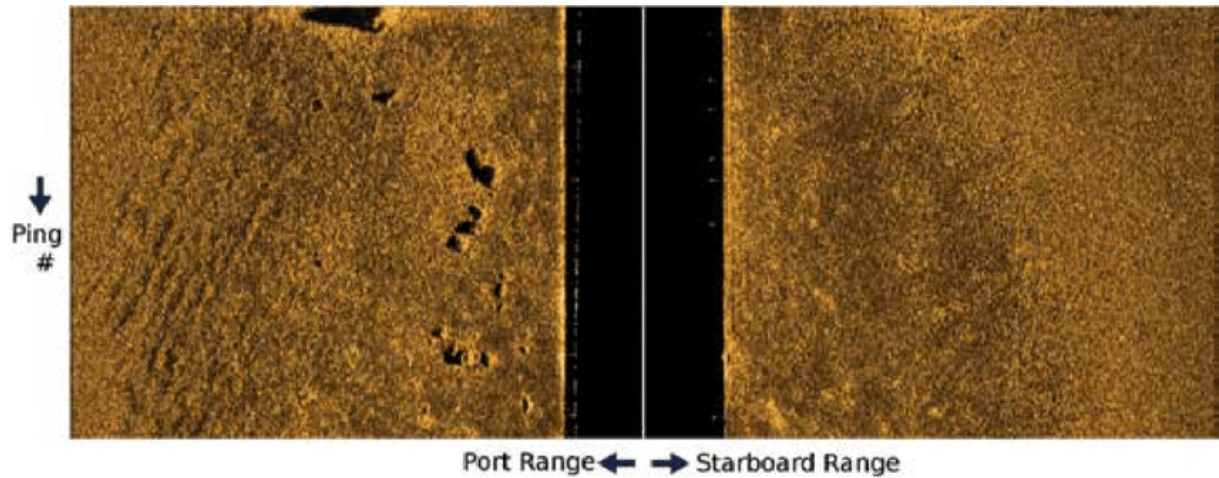


Figure 35: Example of a raw side scan sonar waterfall image showing backscatter intensity versus slant range for consecutive pings. The dark central band corresponds to the sonar blind zone.^[36]

9.2.2 Pitch and Roll Correction

Variations in vessel attitude introduce geometric distortions in side scan sonar data. When the vessel experiences pitch or roll motion, the sonar beams deviate from their nominal orientation, causing the seafloor to appear tilted, curved, or stretched in the recorded swath. Without compensation, these effects lead to errors in the perceived seabed slope and displacement of features when the data is projected into Cartesian coordinates.

Pitch primarily influences the along track geometry by changing the effective depression angle of the sonar beam in the vertical plane. A positive pitch tilts the sonar forward, causing features to appear shifted ahead of their true positions. Roll, on the other hand, tilts the transducer laterally, altering the ground projection between the port and starboard channels. This asymmetry results in apparent lateral displacement or scaling differences of identical seabed features.

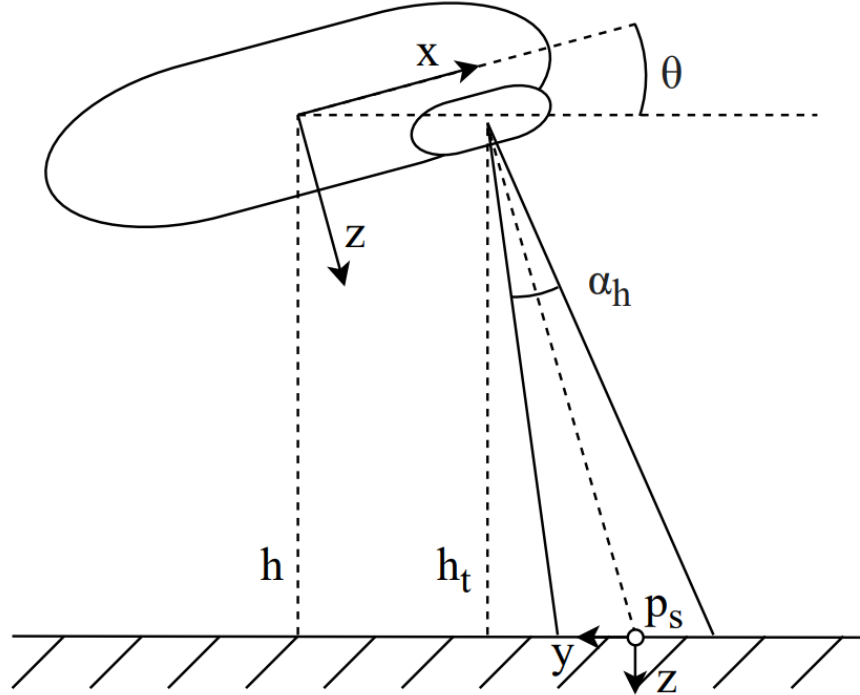


Figure 36: Starboard transducer and vessel geometry with non zero pitch. The figure shows how a positive pitch angle θ tilts the sonar beam forward relative to the horizontal plane, altering the beam incidence angle α_h and changing the apparent intersection point p_s on the seafloor. h_t denotes the vertical transducer height, while h is the true vertical altitude of the vessels reference frame.^[2]

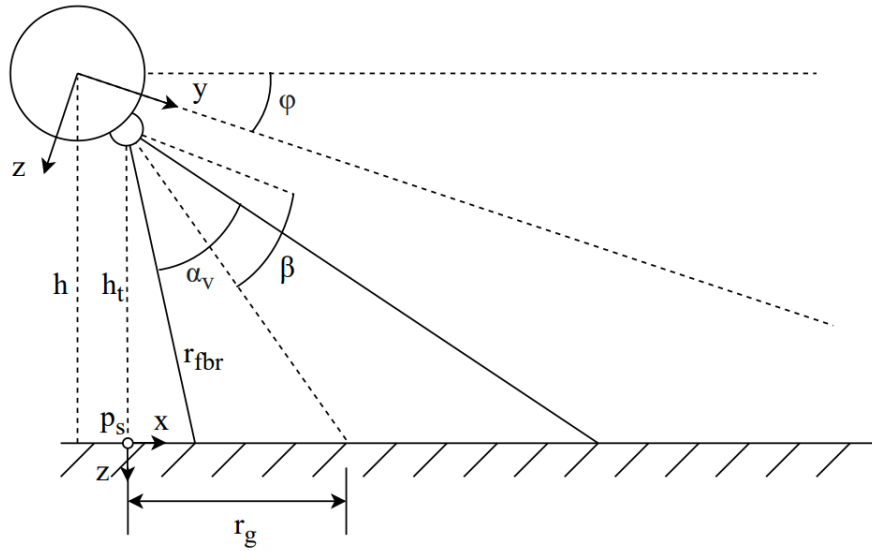


Figure 37: Starboard transducer and vessel geometry with non zero roll. The roll angle ϕ tilts the transducer laterally, changing the apparent ground range r_g and slant range r_{fbr} for each channel. The mounting angle β and the sonars vertical beamwidth α_v define the nominal beam orientation. A positive roll reduces the effective depression angle on the port side and increases it on the starboard side, causing geometric asymmetry between the channels.^[2]

To correct these distortions, each ping is compensated using the vessels estimated orientation obtained from the navigation state estimator fusing GNSS and IMU data. The estimated roll ϕ and pitch θ angles define the instantaneous orientation of the sonar beams relative to the horizontal plane and are used to transform the sonar measurements back into a consistent reference frame.

Explicit geometric correction

The explicit geometric correction method estimates the true vertical altitude of each sonar transducer relative to the seabed using the first bottom return (FBR) in every ping. The FBR represents the first strong acoustic reflection received from the seafloor. Given the two way travel time t_{fbr} to this reflection and the sound speed c in water, the corresponding slant range is

$$r_{fbr} = \frac{c t_{fbr}}{2} \quad (29)$$

which is the distance along the acoustic beam between the transducer and the point of reflection on the seafloor. This distance forms the hypotenuse of a right triangle, where the vertical component corresponds to the true transducer altitude above the seabed.

The vertical transducer height is obtained by projecting this slant range onto the vertical axis using the effective beam depression angle α :

$$h_t = r_{fbr} \sin(\alpha)$$

The total depression angle α depends on the fixed sonar mounting angle β and the instantaneous pitch θ and roll ϕ of the vessel:

$$\alpha = \beta + \phi + \theta$$

Since the vessel rolls about its longitudinal axis, the starboard and port transducers experience opposite roll directions relative to the horizontal plane. A positive roll angle ϕ tilts the starboard side downward (toward the seafloor) while lifting the port side upward (away from the seafloor). Therefore, the effective roll correction must be applied with opposite signs for each channel. Substituting these relationships yields:

$$h_{t, stb} = \frac{c t_{fbr, stb}}{2} \sin(\beta + \phi + \theta), \quad h_{t, port} = \frac{c t_{fbr, port}}{2} \sin(\beta - \phi + \theta) \quad (30)$$

These heights represent the true vertical distance from each transducer to the seabed, accounting for both pitch and roll effects. Even for small roll angles, the difference between $h_{t, stb}$ and $h_{t, port}$ can be significant, especially in shallow water where small angular deviations cause noticeable geometric shifts.

Once the individual transducer heights are known, the corrected slant ranges to the first bottom return can be calculated as:

$$r_{fbr, stb} = \frac{h_{t, stb}}{\sin(\beta + \phi + \frac{\alpha_v}{2})}, \quad r_{fbr, port} = \frac{h_{t, port}}{\sin(\beta - \phi + \frac{\alpha_v}{2})} \quad (31)$$

where α_v is the sonars vertical beamwidth. These relations account for roll induced asymmetry between the two channels, ensuring that each beam is projected correctly onto the seafloor. The resulting values are later used to identify the blind zone boundary and valid seafloor return bins.

Although the system used in this work does not include a Doppler Velocity Log (DVL) or altimeter, the transducer altitudes $h_{t, stb}$ and $h_{t, port}$ can still be estimated directly from the side scan sonar data using the FBR. The absence of external altitude sensors introduces additional uncertainty in these estimates, as the FBR detection depends on seabed reflectivity and acoustic noise. Nevertheless, the method provides sufficiently accurate geometric correction for surface vessels like MicroAmpere when combined with reliable pitch and roll estimates from the navigation state estimator.

By compensating each ping using the measured attitude angles and the estimated per channel transducer altitudes, the side scan sonar data are geometrically corrected for vessel motion, establishing a consistent base for blind zone removal and slant range correction.

9.2.3 Blind Zone Removal

After compensating for pitch and roll, the next processing stage identifies and removes the sonar blind zone. The blind zone corresponds to the near nadir region directly beneath the vessel where the sonar beam does not produce valid seabed returns. In this region, the acoustic pulse travels only through the water column before striking the seafloor, resulting in weak or noisy backscatter dominated by multipath reflections and surface interference.

The blind zone starts at the transducer position and extends to the first bottom return (FBR) for each ping. From the two-way travel time t_{fbr} , the raw slant range to the first seabed echo is given by Equation 29. However, this distance alone does not specify the true seabed intersection point, since vessel pitch and roll modify the effective beam depression angle. To correct for this, the per-channel transducer altitudes $h_{t, stb}$ and $h_{t, port}$ are estimated using Equation 30, which projects the measured slant range onto the vertical axis according to the current attitude.

Using these altitudes, the corrected slant ranges to the first valid seabed returns are computed from Equation 31. The resulting values, $r_{fbr, stb}$ and $r_{fbr, port}$, represent the roll compensated limits beyond which the acoustic signal first reaches the seafloor. Each sonar beam contains a set of slant range samples denoted by r_s , and the first valid bottom return is located at r_{fbr} . When $r_s < r_{fbr}$, the signal originates from the water column and is discarded. When $r_s \geq r_{fbr}$, the return corresponds to the seafloor and is retained for mapping. This separation ensures that only physically meaningful data are passed to later correction stages.

In the raw waterfall image, the blind zone appears as a dark vertical band between the port and starboard channels. Its width scales with sonar altitude, higher transducer placement (larger h_t) produces a wider blind zone, while shallow operation results in a narrower central gap. The horizontal width can be derived geometrically from the sonar altitude and beam depression angle.

Consider one side of the sonar where the transducer height above the seabed is h_t and the beam depression angle relative to the horizontal plane is β . The horizontal distance from the nadir (the point directly below the sonar) to the seabed intersection is denoted x_b . From the right triangle formed by the sonar, nadir, and seabed intersection,

$$\tan(\beta) = \frac{h_t}{x_b}$$

Solving for x_b gives

$$x_b = \frac{h_t}{\tan(\beta)}$$

Since the blind zone covers both sides of the nadir, the total horizontal width becomes

$$w_b = 2x_b = 2\frac{h_t}{\tan(\beta)} = 2h_t \cot(\beta)$$

where h_t represents the mean of $h_{t, stb}$ and $h_{t, port}$, and β is the fixed mounting depression angle. This simple geometric relation shows that increasing sonar altitude or reducing beam angle widens the blind zone proportionally.

Accurate detection and removal of the blind zone are essential for maintaining data quality. Unfiltered water column samples can introduce false intensities and distortions in later steps such as slant range correction and intensity normalization. After blind zone removal, each ping contains only valid seabed returns, providing a clean and geometrically consistent input for subsequent processing.

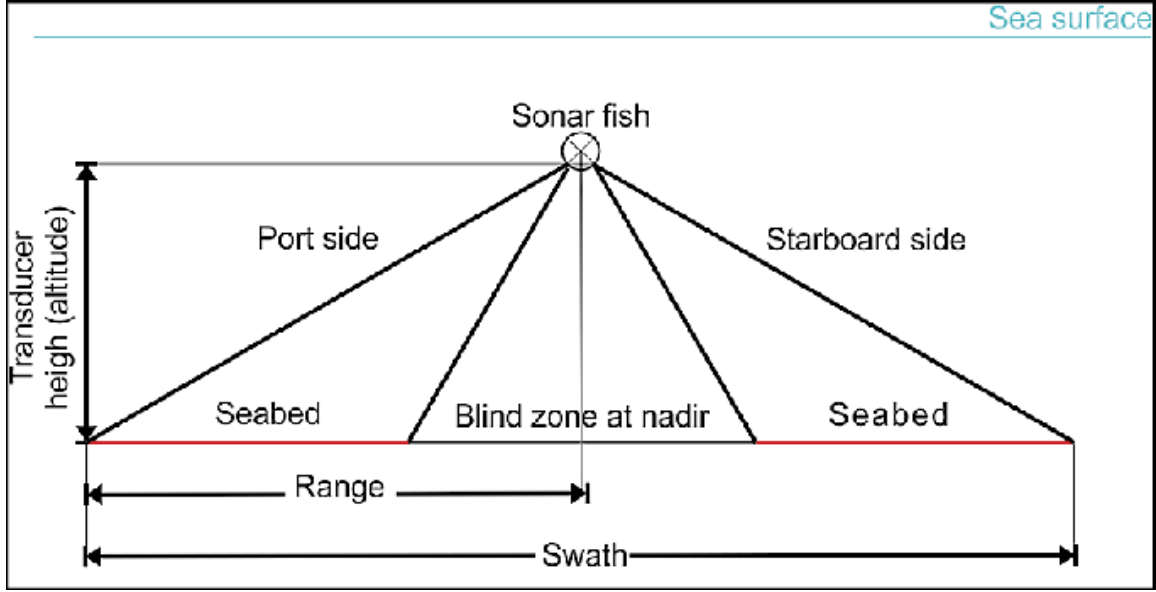


Figure 38: Transverse view of a side scan sonar geometry showing the blind zone at nadir. The blind zone is the near-vertical region beneath the sonar where no valid seabed echoes are received, separating the port and starboard swaths.^[37]

9.2.4 Slant Range Correction

After blind zone removal, the remaining valid backscatter samples still represent distances along the sonar beam, known as “*slant range*”. To correctly project these samples into a map of the seafloor, each measurement must be converted from slant range space to horizontal ground range coordinates. This step geometrically flattens the sonar data, forming the basis for accurate seabed mapping.

Although the corrected slant ranges $r_{fbr, stb}$ and $r_{fbr, port}$ obtained from Equation 31 account for vessel roll and pitch, they still describe distances along the acoustic path, not the horizontal ground distance. To produce a geometrically correct swath, each slant range sample is projected onto the seafloor plane assuming a locally flat seabed, which is a reasonable approximation for short range, high frequency side scan sonars.

Considering a right triangle formed by the sonar transducer, the seabed intersection point, and the vertical projection of the transducer onto the seabed, the geometric relation between the slant range r_s , sonar altitude h_t , and horizontal ground range r_g is given by

$$r_s^2 = r_g^2 + h_t^2$$

Here, r_s represents the total slant range to a specific backscatter sample along the sonar beam, while r_{fbr} denotes the first bottom return, the shortest valid slant range where the sonar beam first intersects the seabed. All samples with $r_s < r_{fbr}$ correspond to water column echoes and are discarded during blind zone removal, whereas samples with $r_s \geq r_{fbr}$ are considered valid seabed returns and are geometrically projected to ground range using the relation above. This conversion ensures that each valid sonar bin is positioned according to its true horizontal distance on the seafloor.

Rearranging for r_g gives the horizontal projection formula:

$$r_g = \sqrt{r_s^2 - h_t^2}$$

This transformation converts each intensity sample from sonar coordinates to its corresponding position on the seafloor. It effectively “*flattens*” the swath geometry, ensuring that equal pixel distances in the final image correspond to equal ground distances on the seabed.

Without this correction, near nadir regions would appear compressed and far range regions stretched, leading to geometric distortion in the mapped imagery. Using the previously estimated per channel

transducer heights $h_{t, stb}$ and $h_{t, port}$ from Equation 30, the transformation is applied separately to the port and starboard channels:

$$r_{g, stb} = \sqrt{r_{s, stb}^2 - h_{t, stb}^2}, \quad r_{g, port} = \sqrt{r_{s, port}^2 - h_{t, port}^2} \quad (32)$$

This step converts the 1D slant based scan line into a horizontal projection, enabling accurate along track stacking and Cartesian map generation in later stages.

The slant range correction relies on the assumption of a locally flat seafloor. While this assumption introduces small errors over complex terrain, it remains valid for the relatively short and narrow swaths typical of shallow water side scan systems such as MicroAmperes sonar. More advanced approaches, such as bathymetry aided correction or shadow based terrain estimation, can further refine this transformation by accounting for local seabed elevation variations. However, these methods are computationally intensive and typically require high resolution bathymetric data or complex image analysis. As such, they are not suitable for real-time operation on computationally constrained embedded systems or during online SLAM processing. For this work, the flat seabed assumption provides an optimal balance between accuracy and efficiency for onboard sonar mapping.

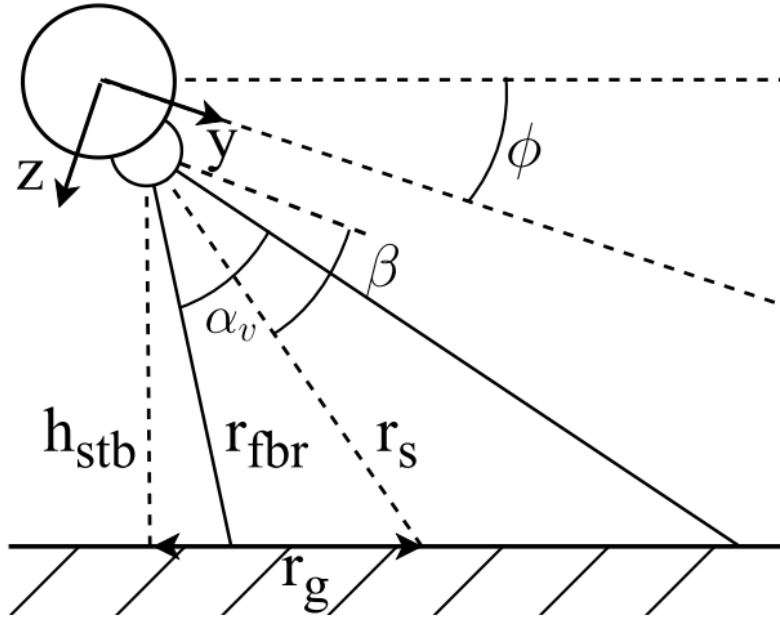


Figure 39: Geometric relation between slant range, transducer altitude, and horizontal ground range. The roll angle ϕ influences both the transducer height h_t and the slant range r_s , affecting the final ground projection r_g .^[1]

9.2.5 Intensity Normalization

The received sonar intensity naturally decays with range due to spherical spreading, absorption, and other propagation losses. These effects introduce unwanted attenuation in the measured signal, causing distant regions of the seabed to appear artificially darker. To counteract this, intensity normalization is applied to compensate for range dependent loss and ensure that the resulting image reflects true seabed backscatter rather than acoustic propagation effects.

Before normalization, the sonar signal has already undergone pitch and roll correction, blind zone removal, and slant range correction. The processed data now represent the measured acoustic backscatter intensity as a function of ground range for each ping. Each intensity sample is denoted as

$$I(i) = I(r_g(i))$$

where $r_g(i)$ is the horizontal ground range obtained from the geometric projection (see Equation 32). Here, $r_g(i)$ defines the physical position of the i -th corresponding seafloor point, while $I(r_g(i))$ represents

the measured acoustic intensity received from that i -th point.

For simplicity, the notation $I(i)$ is used throughout the following sections to refer to the measured backscatter intensity associated with sample i , understood as the intensity evaluated at its ground projected location $r_g(i)$. This measured intensity still contains both the true seabed reflectivity and the effects of acoustic attenuation, beam pattern, and geometric spreading.

Intensity normalization can be performed either on a complete sonar image or on individual swaths. In this work, the correction is performed on individual swaths to enable real-time operation and maintain modularity in the processing chain. The method follows the framework described in [2], where the measured sonar intensity $I(x, y)$ is modeled as a combination of two distinct components:

$$I(x, y) = R(x, y) * L(x, y)$$

Here, $R(x, y)$ represents the “*reflectance map*”, which describes the intrinsic acoustic backscatter of the seabed. It depends on physical factors such as surface roughness, material composition, and the local grazing angle. The second term, $L(x, y)$, represents the “*illumination map*”, which models large scale variations in the measured signal caused by sonar specific effects like beam pattern, transmission loss, and range dependent attenuation. The operator $*$ denotes element wise multiplication since each pixels measured intensity is affected by both the true reflectivity of the seabed and the sonars illumination pattern at that location.

The goal of intensity normalization is to recover the true seabed reflectivity $R(x, y)$ by estimating and compensating for the illumination term $L(x, y)$. Once an estimate $\hat{L}(x, y)$ of the illumination map is obtained, the estimated reflectance map can be expressed as:

$$\hat{R}(x, y) = \frac{I(x, y)}{\hat{L}(x, y)} = R(x, y) * \frac{L(x, y)}{\hat{L}(x, y)}$$

If the estimated illumination map $\hat{L}(x, y)$ closely approximates the true illumination $L(x, y)$, then $\hat{R}(x, y) \approx R(x, y)$, meaning the normalized image accurately represents the relative seabed reflectivity. This operation effectively removes large scale intensity variations caused by acoustic spreading, beam pattern effects, and medium absorption, ensuring that the remaining intensity variations primarily correspond to the physical properties of the seabed.

The illumination map $\hat{L}(x, y)$ varies slowly across the swath, representing low-frequency intensity trends rather than local seabed texture. It accounts for effects such as beam pattern variation, range dependent attenuation, and transducer gain. Several methods have been proposed to estimate this map. Filtering based approaches, such as mean or bilateral filtering, smooth the measured intensity to approximate the illumination field [2]. Physics based methods instead model the seabed as a Lambertian surface, using the relationship between grazing angle and backscatter strength to predict the illumination component [1].

In this work, $\hat{L}(x, y)$ is estimated using smoothing cubic splines, which model the large scale intensity envelope as a smooth polynomial function fitted along each individual swath. This method effectively captures gradual illumination trends without overfitting to local texture variations. The spline is obtained by minimizing the cost function:

$$p \sum_{i=1}^n |I(i) - L(i)|^2 + (1 - p) \int_1^n |D^2 L(t)|^2 dt$$

where the first term measures the deviation from the observed intensity $I(i)$, and the second penalizes excessive curvature to ensure a smooth illumination field. The parameter p controls the balance between accuracy and smoothness, while $D^2 L$ denotes the second derivative of L , enforcing continuity and smooth transitions across the swath.

After estimating $\hat{L}(i)$, the normalized reflectance profile is computed as:

$$\hat{R}(i) = \frac{I(i)}{\hat{L}(i)} \tag{33}$$

This operation removes large scale illumination effects such as acoustic spreading and transducer gain variation, producing a consistent brightness profile across range while preserving the high frequency variations related to true seabed texture.

The smoothing cubic spline approach provides a good compromise between computational efficiency and normalization accuracy. Unlike more complex illumination estimation techniques that require multi swath or physics based modeling, spline fitting can be computed quickly and independently for each swath, making it well suited for real-time embedded sonar processing. The smoothing parameter p must be determined experimentally for the specific sonar setup, but it is typically small, on the order of magnitude less than 10^{-3} . This ensures sufficient smoothing of large scale illumination variations while preserving fine scale seabed texture.

The result of the spline based normalization is a sonar image with a more uniform illumination profile, where artificial range dependent intensity variations are effectively suppressed. Figure 40 illustrates this effect, the upper image shows the raw intensity with visible horizontal artifacts, while the lower image presents the normalized result with consistent brightness and clearer seabed texture across the swath.

This normalization method offers an effective balance between computational simplicity and correction accuracy. While more advanced reflectance estimation or global filtering approaches can produce smoother illumination compensation, they are typically too computationally heavy for real-time use. In contrast, the smoothing cubic spline method achieves sufficient accuracy for onboard sonar processing and real-time SLAM integration, making it a practical and efficient choice.

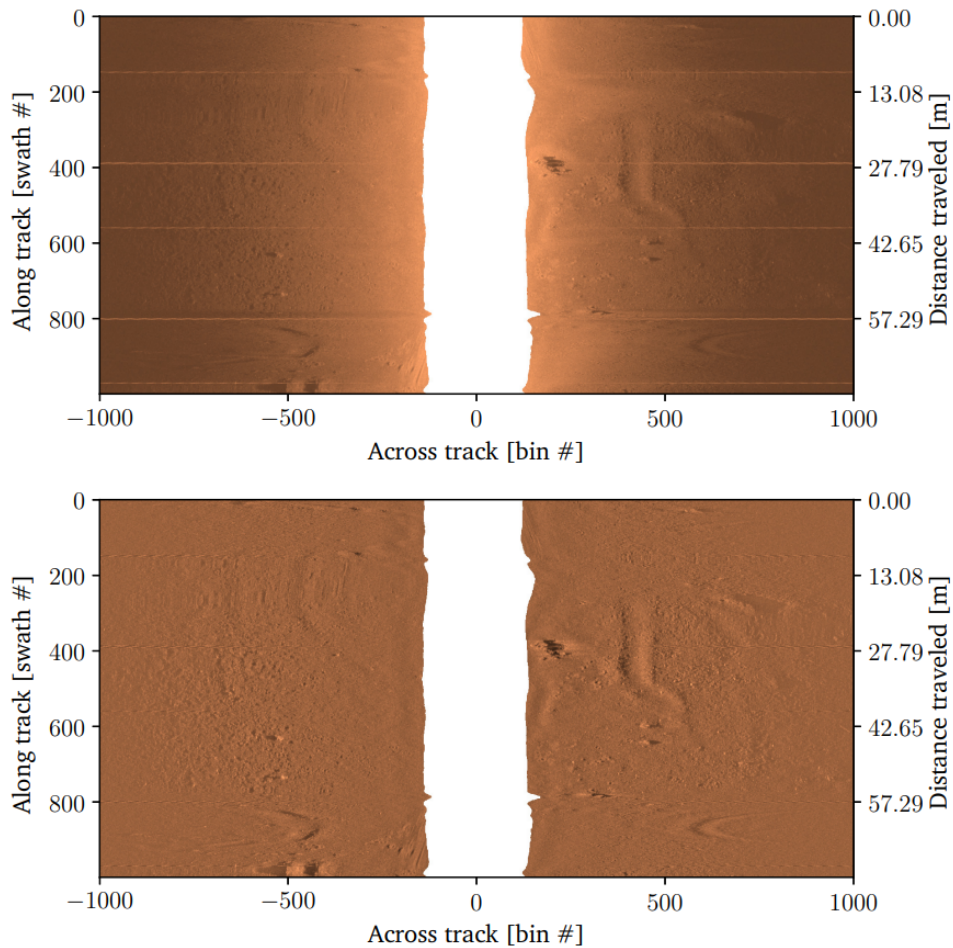


Figure 40: The top image shows the blind zone removed sonar image with raw intensity constructed from the first 1000 swaths of the dataset. The bottom image shows the same image after smoothing cubic spline normalization, where artificial horizontal lines are almost invisible and overall illumination is more uniform.^[2]

9.2.6 Stacking and Reconstruction

After each swath has been corrected for pitch, roll, slant range distortion, and intensity attenuation, the resulting data represents a consistent estimate of the seabed backscatter reflectivity in horizontal ground coordinates. To reconstruct a continuous seafloor image, the processed swaths are stacked sequentially in the temporal order of acquisition. This stacking corresponds directly to the vessels forward motion, where each new ping contributes a narrow strip of intensity data aligned along the along track axis.

During stacking, each swath is positioned according to the vessels estimated trajectory obtained from the navigation system, ensuring that positional drift and motion induced offsets are minimized. The along track resolution of the reconstructed image is determined by the ping rate and vessel speed, while the across track resolution depends on the sonars range sampling and slant to ground conversion.

The resulting 2D reconstruction forms an enhanced waterfall image where both axes correspond to physical ground distances and pixel intensity represents estimated seabed reflectivity. This corrected and normalized output marks the final stage of swath processing and serves as input for Cartesian projection and SLAM feature extraction.

9.3 Probabilistic Mapping

9.3.1 Mapping

After swath processing, the sonar data is geometrically corrected, radiometrically normalized, and expressed in ground coordinates. The next stage is to reconstruct a consistent local map that spatially represents the acoustic backscatter intensity over the seafloor. This process, referred to as “*probabilistic mapping*”, forms the core link between raw sonar measurements and the spatial representations required for SLAM based navigation and data association.

In conventional Cartesian mapping, each sonar return is assigned directly to a single grid cell using interpolation techniques such as k-Nearest Neighbor (kNN). While computationally efficient, this approach assumes that each sonar measurement corresponds to an exact reflection point on the seafloor, ignoring the finite width of the acoustic beam, the elliptical footprint, and range dependent uncertainty. As a result, such maps tend to appear artificially sharp yet physically inaccurate, particularly under conditions of noise, sparse coverage, or large angular incidence.

The probabilistic mapping framework addresses these limitations by representing each sonar beam as a spatial likelihood field rather than a deterministic ray. Instead of assigning a measurement to a single grid cell, its contribution is distributed among neighboring cells according to a physically motivated probability function derived from beam geometry, angular uncertainty, and measurement noise. This produces a continuous, uncertainty aware reflectivity map that better reflects the true acoustic interaction between the sonar and the seafloor surface.

Conceptually, the goal of the mapping stage is to transform sonar measurements into a locally consistent intensity field that preserves both geometric fidelity and radiometric realism. The resulting probabilistic map provides a robust foundation for feature extraction and SLAM, improving landmark repeatability across consecutive local maps and maintaining physical consistency even under dynamic vehicle motion and varying seabed conditions.

9.3.2 Projection from Polar to Cartesian

The projection from polar to Cartesian coordinates transforms each processed sonar swath from range bearing geometry into spatially consistent ground coordinates expressed in the NED frame. After slant range correction, every sonar sample corresponds to a point on the seafloor at a known horizontal ground range r_g and bearing relative to the transducer. Each ping line represents an along-track position determined by the vehicles estimated trajectory, while the across track direction is defined by the projected ground range r_g on either the port or starboard side.

Given the vehicle position $\mathbf{p}_{b/O}^n$ and attitude quaternion $\mathbf{q}$ from the navigation state (See Equation 7), each sonar measurement is transformed from the sonar frame to the NED frame through a complete chain of reference frames. The sonar is rigidly mounted on the vessel body with a fixed lever arm offset:

$$\mathbf{r}_{s/b}^b = \begin{bmatrix} x_{s/b}^b \\ y_{s/b}^b \\ z_{s/b}^b \end{bmatrix} \quad (34)$$

and orientation R_s^b , defining the rotation from the sonar to the body frame. This rotation depends on the sonars physical installation and differs between the port and starboard transducers. Typically, the mounting angles include a yaw offset of $\pm 90^\circ$ relative to the vessels longitudinal axis and a pitch offset corresponding to the sonar depression angle β , while roll offset is often negligible under level installation [2, 1].

After slant range correction, the ground projected location of each sonar return in the sonar frame is given by

$$\mathbf{p}_{g/s}^s = \begin{bmatrix} \pm r_g \\ 0 \\ 0 \end{bmatrix}$$

where r_g is the horizontal ground range (see Equation 32), and the sign $\pm$ indicates whether the measurement originates from the port or starboard transducer. The notation $\mathbf{p}_{g/s}^s$ denotes the position of

the seafloor point g relative to the sonar origin s , expressed in sonar coordinates.

The global position of this ground point in the NED frame is then computed through the full coordinate transformation:

$$\mathbf{p}_{g/O}^n = R_b^n(\mathbf{q}) \left(R_s^b \mathbf{p}_{g/s}^s + \mathbf{r}_{s/b}^b \right) + \mathbf{p}_{b/O}^n \quad (35)$$

where $R_b^n(\mathbf{q})$ represents the rotation matrix derived from the navigation quaternion, aligning the body frame with the NED frame.

This transformation projects each sonar return from the transducers local reference frame into world referenced Cartesian coordinates, effectively merging multiple swaths into a coherent spatial map aligned with the vehicles estimated trajectory. It forms the geometric foundation for both classical Cartesian and probabilistic mapping methods, as both approaches rely on the same coordinate transformation to establish spatial consistency between sonar samples.

9.3.3 kNN Interpolation and Grid Reconstruction

Once all sonar samples have been transformed into global coordinates, they are used to populate a local Cartesian grid representing the seafloor reflectivity. Because sonar returns are irregularly distributed along the vehicles track and sparse in some regions, many grid cells may lack direct measurements. One of the classical approaches to address this problem is the k-Nearest Neighbor (kNN) interpolation method, which generates a continuous and dense reflectivity surface from irregularly spaced sonar samples. This method follows the interpolation framework presented in [2, 38].

In this approach, a KD-tree is first constructed from all transformed sonar sample positions $\mathbf{p}_{g/O}^n$ to enable efficient nearest neighbor queries. For each grid cell centered at (x_i, y_j) , the k closest sonar samples within a maximum search radius r_{max} are identified. The Euclidean distance between a grid cell and its k -th nearest sample is given by:

$$d_k = \left\| \mathbf{p}_{g/O}^n(k) - \begin{bmatrix} x_i \\ y_j \\ 0 \end{bmatrix} \right\|$$

where $\mathbf{p}_{g/O}^n(k)$ denotes the global position of the k -th sonar sample in the NED frame.

Each neighboring sample contributes to the grid cells estimated reflectivity according to its spatial proximity, weighted by an inverse distance weighting (IDW) function:

$$w_k = \frac{1}{((d_k)^p + \epsilon)}$$

where p is the weighting exponent (typically $1 \leq p \leq 2$) controlling the influence of nearby samples, and ϵ is a small constant to prevent division by zero. A higher p value emphasizes nearby samples more strongly, producing sharper local contrast, while lower values yield smoother transitions.

The final estimated reflectivity for a grid cell is obtained as a weighted mean of the k nearest samples:

$$\hat{R}(x_i, y_j) = \frac{\sum_{k=1}^K w_k \hat{R}(k)}{\sum_{k=1}^K w_k}$$

where $\hat{R}(k)$ denotes the normalized backscatter intensity of the k -th sonar sample obtained after radiometric correction (see Equation 33).

Grid cells with no valid neighbors within the search radius r_{max} remain empty or can optionally be filled by extrapolation. Outlier suppression may also be applied by rejecting distant or inconsistent neighbors to avoid spatial artifacts.

The kNN interpolation method provides smooth and visually consistent maps while maintaining low computational cost, especially when implemented with KD-tree acceleration. It has complexity on the order of $\mathcal{O}(RC \log N)$, where R and C are grid dimensions and N is the number of sonar samples.

However, it assumes that each sonar return represents a precise reflection point, neglecting factors such as beam footprint, angular uncertainty, and range dependent spreading effects.

While effective as a geometric baseline, this deterministic interpolation approach lacks the physical realism required for uncertainty aware SLAM and feature extraction. Therefore, it is presented here primarily as a reference method, the final system instead employs a probabilistic formulation that models sonar beam uncertainty and distributes reflectivity using likelihood based mapping.

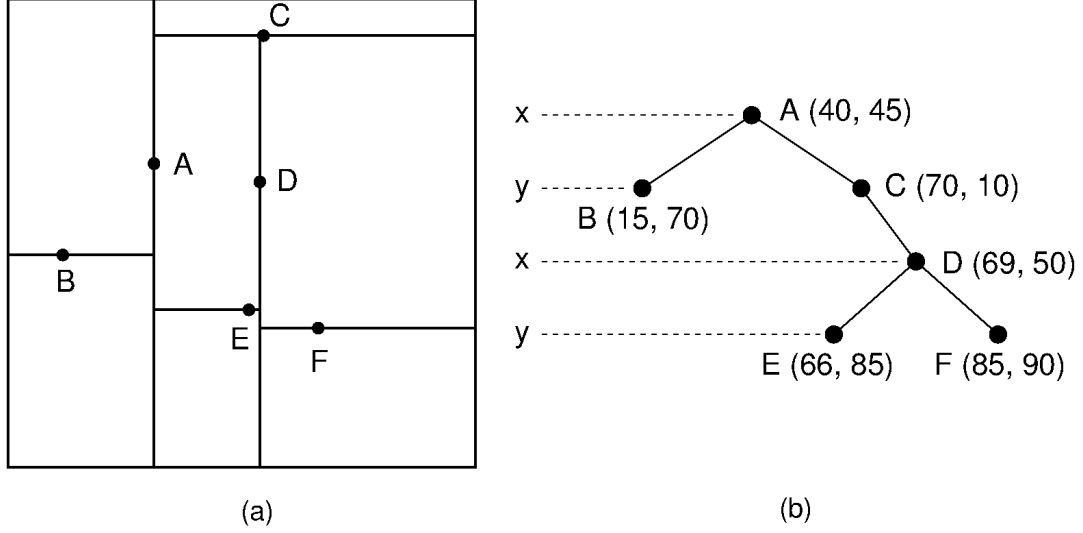


Figure 41: Illustration of a 2D KD-tree used for k-Nearest Neighbor search. (a) shows the spatial partitioning of sample points in the (x, y) plane, and (b) shows the corresponding binary tree structure where each node alternates between splitting along the x and y axes. This hierarchical structure enables efficient k -nearest neighbor queries for interpolating sonar samples in the Cartesian grid.^[38]

9.3.4 Probabilistic Interpolation and Grid Reconstruction

The probabilistic interpolation and grid reconstruction method represents the physically consistent mapping strategy used in this work. Unlike deterministic interpolation techniques such as kNN, which assume each sonar return originates from a single point, the probabilistic approach models each sonar beam as a spatial likelihood field. Each measurement contributes not only to the grid cell it directly intersects but also to its neighbors, with weights derived from the sonars beam geometry, angular uncertainty, and range dependent error characteristics.

Input

The probabilistic mapping process begins with the output of the geometric projection step (Equation 35). After this transformation, every sonar return is represented in global coordinates as $\mathbf{p}_{g/O}^n(i)$ and associated with a normalized intensity value $\hat{R}(i)$ obtained from the reflectivity normalization stage (see Equation 33).

Here, $\mathbf{p}_{g/O}^n(i)$ denotes the position of the i -th seafloor reflection point expressed in the NED frame, while $\hat{R}(i)$ represents its corresponding measured acoustic backscatter intensity. The term $\mathbf{p}$ therefore describes a “*spatial location*” in 3D space, whereas $\hat{R}$ refers to a “*measured physical quantity*” (reflectivity) at that location.

Together, they form the set of sonar observations:

$$\mathcal{S} = \left\{ \mathbf{p}_{g/O}^n(i), \hat{R}(i) \right\}_{i=1}^{N_s}$$

where N_s is the total number of processed sonar samples across all acquired swaths. Each processed swath (indexed by m) corresponds to one sonar ping sequence, and within each swath, i indexes the individual intensity samples belonging to that specific ping.

Thus, the total number of processed samples is given by:

$$N_s = \sum_{m=1}^N N_m$$

where N is the total number of processed swaths used for local map generation, and N_m is the number of valid sonar samples within the m -th swath.

In an idealized case with a constant number of valid samples per swath, this could be simplified as $N_s = N \cdot N_m$. However, in practice, each m -th processed swath or ping contains a different number of valid samples due to blind zone removal, attitude correction, and range dependent filtering. Therefore, every swath must be processed individually before aggregating all valid samples into a coherent probabilistic reflectivity map.

These samples together form the complete input dataset for the probabilistic grid reconstruction stage.

Goal and Grid Definition

The objective of the mapping step is to populate a 2D Cartesian grid representing the local seafloor reflectivity distribution. This grid is defined over world coordinates $q = (x, y)$ and stores the estimated reflectivity $\hat{R}_{prob\ map}(x, y) = \hat{R}_{prob\ map}(q) = V(q)$ for each cell.

The center of every grid cell is represented by the vector:

$$\mathbf{p}_{q/O}^n = \begin{bmatrix} x_q \\ y_q \\ 0 \end{bmatrix}$$

which defines the spatial position of grid cell q in the NED frame.

The end goal is to fill this grid with physically consistent reflectivity values that reflect how the sonar beams illuminate the seafloor over time.

In deterministic approaches such as kNN interpolation, each sonar return contributes only to the cell that lies closest to it. In contrast, the probabilistic approach considers that a sonar beam has a finite spread and that each measurement may influence several neighboring grid cells. This reduces artificial “dark gaps” between samples and produces a smoother, more realistic seafloor representation.

Conceptually, each sonar return $\mathbf{p}_{g/O}^n(k)$ contributes to the map by distributing its intensity $\hat{R}(k)$ across nearby grid cells according to the physical beam model. However, evaluating every sonar point against every grid cell is computationally expensive. Naively, this requires $\mathcal{O}(N_s N_g)$ operations, where N_s is the number of processed swath sonar samples and N_g is the number of grid cells. Later, an accelerated method proposed by Haralstad master thesis [2] will be discussed, that reduces this cost by ignoring cells too far or too oblique to be realistically illuminated.

Beam Geometry and Observation Angles

The sonar transducer emits acoustic beams characterized by a finite horizontal opening angle α_h , which defines the angular width of the acoustic footprint on the seafloor. This parameter is determined by the transducer design and is illustrated in Figures 36 and 42. To determine whether a given grid cell q could be illuminated by a specific processed swath sample m (that is, a particular sonar ping), the corresponding “observation angle” θ_{obs} is computed in the sonar frame.

Each sonar ping, indexed by m , corresponds to one transmitted acoustic pulse (swath) that produces a sequence of N_m valid intensity samples $\hat{R}(i)$ distributed along range. The beam geometry and observation angles are evaluated independently for each ping to determine which grid cells fall within the effective acoustic footprint of that single transmission.

For each grid cell position $\mathbf{p}_{q/O}^n$ and sonar ping m , the relative position of the grid cell with respect to the sonar origin is computed as:

$$\mathbf{r}_{q/s}^s = (R_s^b)^\top (R_b^n)^\top (\mathbf{p}_{q/O}^n - \mathbf{p}_{b/O}^n - R_b^n(\mathbf{q})\mathbf{r}_{s/b}^b)$$

where $\mathbf{p}_{b/O}^n$ is the vehicle position in the NED frame, obtained from the state estimator. $R_b^n(\mathbf{q})$ is the body to NED rotation matrix derived from the navigation state quaternion, also provided by the state estimator. $\mathbf{r}_{s/b}^b$ is the sonar lever arm offset relative to the body frame, a static, known constant defined by the sonars mounting position relative to the vehicles center of gravity (COG) (see Equation 34). Finally R_s^b is the fixed rotation matrix from the sonar to the body frame, determined by the sonars installation orientation (see below Equation 34).

This expression converts the position of grid cell q from world coordinates to the sonar frame at the time of ping m . The resulting vector $\mathbf{r}_{q/s}^s = [r_x^s, r_y^s, r_z^s]^\top$ represents the position of cell q relative to the sonar.

From this vector, the observation (bearing) angle is obtained by:

$$\theta_{obs} = \arctan 2(r_y^s, r_x^s) \quad (36)$$

This angle measures the azimuthal direction from the sonar to the grid cell relative to the beams acoustic axis. If $\theta_{obs} = 0$, the grid cell q lies exactly on the beam centerline. As $|\theta_{obs}|$ increases, the following cells lie further and further to the side, and if the deviation exceeds half the beamwidth, the cell is no longer illuminated:

$$|\theta_{obs}| < \frac{\alpha_h}{2}$$

Intuitively, α_h defines how wide the sonar “sees” horizontally, while θ_{obs} tells us whether a particular cell lies inside that visible region.

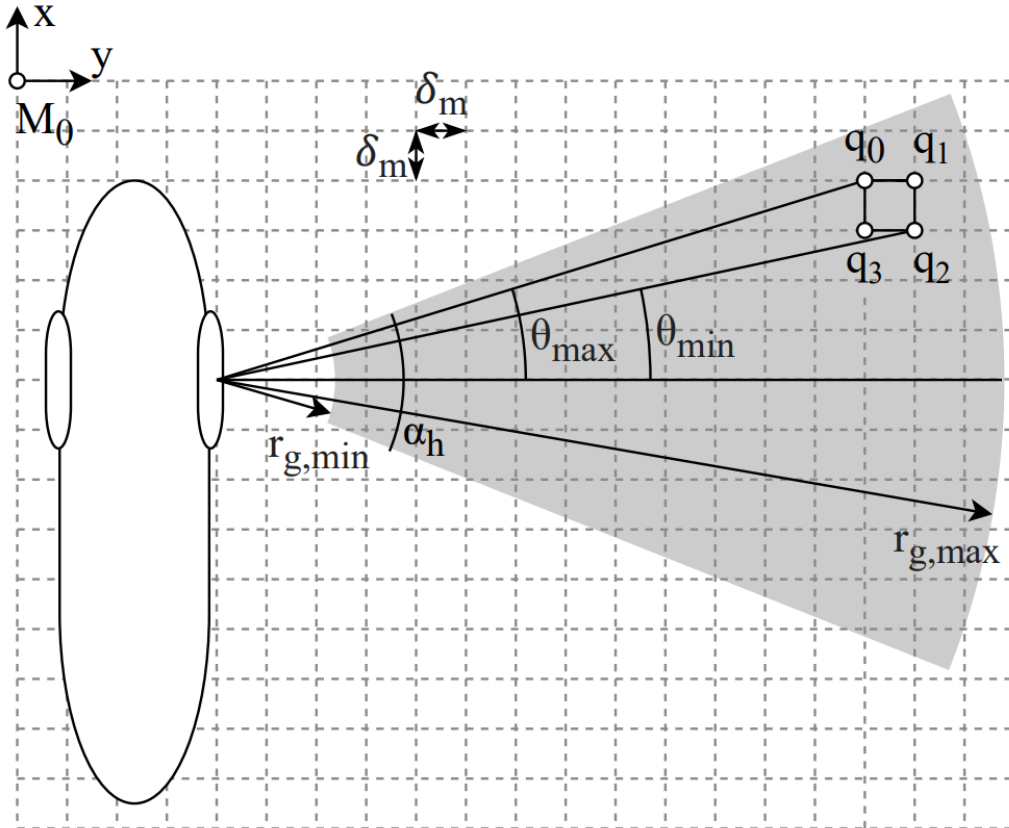


Figure 42: Illustration of the sonar beams horizontal geometry showing the acoustic axis, horizontal beamwidth α_h , and the observation angle θ_{obs} to a grid cell q . The cell lies within the illuminated footprint if $|\theta_{obs}| < \alpha_h/2$.^[2]

The way to interpret this process is as follows. At the beginning of a swath acquisition (ping sequence m), only the grid cells closest to the sonar and within its current field of view are illuminated, the remaining distant cells are outside the observation cone and can be ignored for that specific swath.

As the vehicle moves and new pings are emitted, the illuminated region gradually shifts across the grid. Cells that were visible in earlier pings may no longer fall within the beams angular range, while previously unseen cells begin to be illuminated as the sonar advances along its trajectory.

In essence, the probabilistic mapping process resembles a simplified, physically constrained form of ray tracing, where each sonar beam acts as a moving illumination cone that progressively scans the seafloor. However, unlike computer graphics lighting models, this illumination is governed by physical sonar geometry and modeled using Gaussian probability distributions to represent beam divergence and angular uncertainty, as will be detailed in the next section.

Probabilistic Beam Model

In deterministic interpolation methods such as kNN, only the nearest grid cells are updated, resulting in sharp intensity boundaries and artificial gaps. In contrast, the probabilistic beam model treats each sonar beam as a continuous angular likelihood field that spreads its energy according to a Gaussian distribution centered on the beam axis:

$$p(\theta_{obs}) = \frac{2}{\alpha_h \sqrt{2\pi}} \exp\left(-\frac{2\theta_{obs}^2}{\alpha_h^2}\right)$$

This function represents the relative contribution of a sonar beam at an angular deviation θ_{obs} from its acoustic centerline. The intensity is strongest near $\theta_{obs} = 0$ and gradually decreases toward the beam edges ($|\theta_{obs}| \approx \alpha_h/2$). (See Equation 36)

For each sonar ping m , the beam spans a finite angular region defined by its horizontal beamwidth α_h . The nominal angular limits for that ping are:

$$\theta_{obs,\min}^m = -\frac{\alpha_h}{2}, \quad \theta_{obs,\max}^m = +\frac{\alpha_h}{2}$$

However, the actual observation angle $\theta_{obs}(q)$ for each grid cell q depends on its position relative to the sonar ping m , computed earlier. (See Equation 36)

Each cell subtends a small angular width $\Delta\theta_q$ as seen from the sonar. The probability that cell q is illuminated by ping m is therefore found by integrating the beams angular intensity profile over the angular span corresponding to that specific cell:

$$P_m(q) = \int_{\theta_{obs}(q) - \frac{\Delta\theta_q}{2}}^{\theta_{obs}(q) + \frac{\Delta\theta_q}{2}} p(\theta_{obs}) d\theta_{obs}$$

Here, the integration limits differ for every grid cell because each has a unique observation angle $\theta_{obs}(q)$. This integration effectively converts the continuous beam intensity function into a discrete per cell probability, summing the energy that falls within the angular extent of that cell.

In practice, the grid cells are small enough that this integral can be accurately approximated by evaluating the Gaussian at the cell center:

$$P_m(q) \approx p(\theta_{obs}(q))$$

Before performing this computation, most grid cells can be safely excluded to reduce computational cost. For each sonar ping m , only the subset of grid cells that can physically be illuminated are considered and those within the valid sonar range window

$$|\theta_{obs}(q)| < \frac{\alpha_h}{2}$$

All other cells are ignored for that ping, as they receive negligible acoustic energy.

Algorithmically, this means that for each sonar ping m , a probability grid $P_m(q)$ is initialized

to zero and updated only for the grid cells that lie within the beams angular and range limits. Cells outside this region are skipped entirely. As the vehicle moves and subsequent pings are processed, each $P_m(q)$ contributes to a different local region of the map. Over time, these overlapping illuminated regions merge into a coherent probabilistic reflectivity map of the seafloor.

Finally, the practical formulation used to evaluate $P_m(q)$ in this work is given by:

$$P_m(q) = \begin{cases} \frac{2}{\alpha_h \sqrt{2\pi}} \exp\left(-\frac{2\theta_{obs}(q)^2}{\alpha_h^2}\right), & \text{if } |\theta_{obs}(q)| < \frac{\alpha_h}{2} \\ 0, & \text{otherwise} \end{cases} \quad (37)$$

This formulation links each cells position to its illumination probability using $\theta_{obs}(q)$ from Equation 36. Before evaluating the weight, each cell is checked for geometric validity by requiring $|\theta_{obs}(q)| < \alpha_h/2$ and a valid range, all others are assigned zero. For each ping m , the algorithm computes $\theta_{obs}(q)$ and applies the Gaussian weighting of Equation 37 to form a temporary grid $P_m(q)$, repeating this independently for all pings used in the local map.

The final probabilistic reflectivity map is then obtained by fusing all individual per ping contributions according to the complementary probability rule:

$$P(q) = \sum_{m=1}^N P_m(q) \quad (38)$$

This formulation ensures that repeated illumination of the same grid cell across multiple pings increases its overall probability of observation in a physically consistent way, without exceeding unity. Each sonar ping therefore contributes a localized Gaussian weighted illumination pattern to the grid, and as the vehicle moves, these overlapping contributions accumulate into a continuous probabilistic map of the seafloor reflectivity.

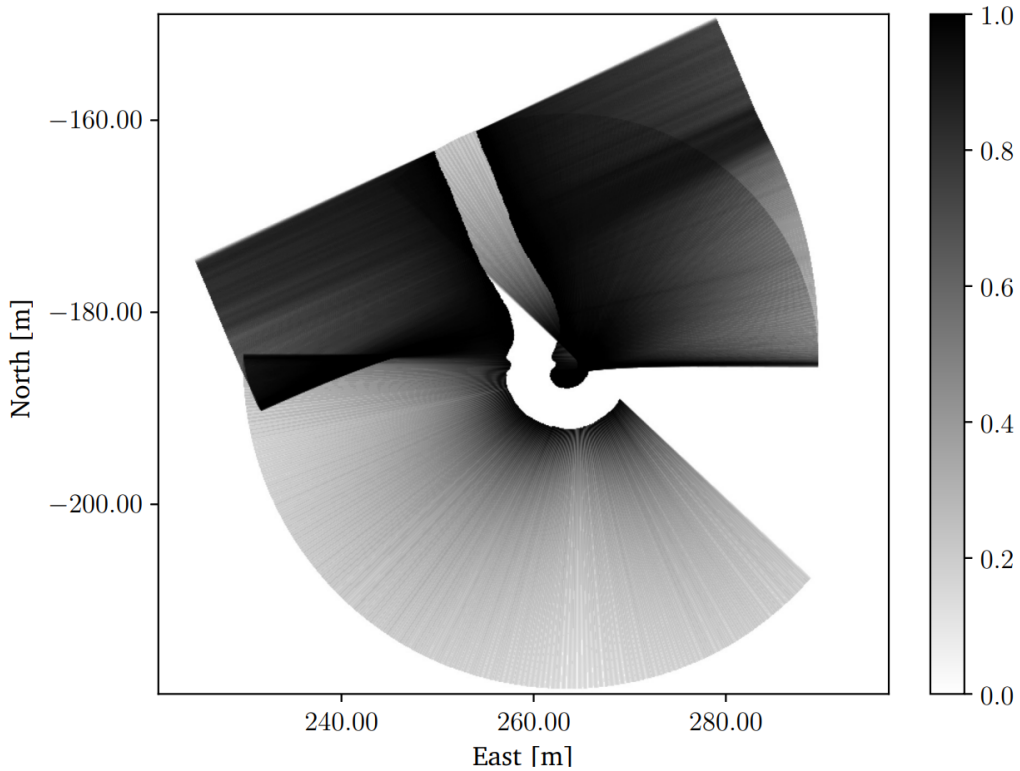


Figure 43: Visualization of the resulting probability map $P(q)$ in normalized form for better visualization. The color bar represents the illumination probability of each grid cell q , where darker color indicate higher likelihood of observation. Each visible line corresponds to the contribution from a single pings $P_m(q)$ field. When all $P_m(q)$ distributions are combined using Equation 38, the result is a complete probabilistic map showing the accumulated illumination coverage of the seafloor.^[2]

Interpolation from Waterfall to Grid Representation

After swath processing, each ping m provides a 1D sequence of normalized seabed reflectivity values $\hat{R}_m(i)$ distributed along the horizontal ground range $r_g(i)$ (see Equation 33). Together, these values form the so called “*waterfall image*”, which represents intensity as a function of range and ping index. Although this view is geometrically corrected and normalized, it still lies in the sonars acquisition geometry and not in the Cartesian grid used for mapping.

Before the probabilistic map $P(q)$ and its reflectivity field $V(q)$ can be formed, the 1D waterfall data of each swath must be projected into the 2D spatial grid shared by the map representation. This requires interpolating the discrete reflectivity samples $\hat{R}_m(i)$, defined along $r_g(i)$, into corresponding grid cell positions $q = (x_q, y_q)$ that lie within the sonar footprint of that ping.

Each grid cell q has fixed Cartesian coordinates $q = (x_q, y_q)$ in the NED frame and four corner points $\mathbf{p}_q(i)$, where $i = 0, 1, 2, 3$. The sonar position at the time of ping m is given by

$$\mathbf{p}_{s/O}^n(m) = R_b^n(\mathbf{q})\mathbf{r}_{s/b}^b + \mathbf{p}_{b/O}^n$$

where $\mathbf{p}_{b/O}^n$ and $\mathbf{q}$ is the vessel position and attitude obtained from the state estimator at the instant of ping m , and $\mathbf{r}_{s/b}^b$ is the fixed lever arm offset of the sonar relative to the vessel body frame.

Using these quantities, the slant range distance from the sonar $\mathbf{p}_{s/O}^n(m)$ at ping m to each corner of grid cell $\mathbf{p}_q(i)$ is calculated as

$$r_{s,i}^m = \left\| \mathbf{p}_q(i) - \mathbf{p}_{s/O}^n(m) \right\|$$

Each $r_{s,i}^m$ represents the acoustic distance from the sonar transducer to the i -th corner of cell q , corresponding to a position along the measured 1D intensity profile $\hat{R}_m(i)$. Later in the acceleration discussion, methods for efficiently pruning grid cells that are outside the active sonar footprint will be presented, ensuring that only a small subset of relevant cells is processed for each ping m .

Because the measured intensity samples $\hat{R}_m(i)$ are defined only at discrete range intervals of size δ_s (See Figure 42), interpolation is required to estimate the intensity value at an arbitrary slant range $r_{s,i}^m$. For this purpose, linear interpolation is applied directly between the two nearest measured samples along the 1D profile.

The interpolation weights are computed using the normalized ratio between the desired range and the sampling interval, defined as

$$w_1 = \frac{r_{s,i}^m}{\delta_s} - \left\lfloor \frac{r_{s,i}^m}{\delta_s} \right\rfloor, \quad w_2 = 1 - w_1$$

Here, the floor operator $\lfloor \cdot \rfloor$ extracts the integer part of the normalized range $\frac{r_{s,i}^m}{\delta_s}$, giving the index of the nearest lower sampled range in the processed swath of the m -th ping. The fractional part represents how far the desired range lies between two consecutive samples in this swath. If $r_{s,i}^m$ is close to the lower sample, w_1 becomes small and w_2 dominates, if it is near the upper sample, w_1 increases while w_2 decreases. This provides a smooth linear weighting between the two closest intensity samples of the processed swath. These weights can be seen as a simple yet intelligent variant of kNN interpolation, where the two nearest range samples are blended proportionally to their distance from the target point.

The corresponding interpolated reflectivity value is then given by

$$\hat{R}_m(r_{s,i}^m) = w_1 \hat{R}_m\left(\left\lfloor \frac{r_{s,i}^m}{\delta_s} \right\rfloor\right) + w_2 \hat{R}_m\left(\left\lceil \frac{r_{s,i}^m}{\delta_s} \right\rceil\right)$$

Here, both $\lfloor \cdot \rfloor$ and $\lceil \cdot \rceil$ represent discrete sample indices in the processed swath of ping m . Since the measured intensities $\hat{R}_m(i)$ are sampled at uniform range intervals δ_s , the normalized value $\frac{r_{s,i}^m}{\delta_s}$ effectively gives the “*fractional sample index*” corresponding to the slant range $r_{s,i}^m$. The floor and ceiling operators then select the two closest measured samples, the “*neighbors*” of that point, while the weights w_1 and w_2

determine how much each contributes.

These weights linearly interpolate the measured intensities between the two nearest samples in the processed swath, ensuring smooth transitions between adjacent ranges. Averaging the interpolated corner values yields the mean estimated reflectivity $V_m(q)$ for each grid cell as observed by ping m :

$$V_m(q) = \frac{1}{4} \sum_{i=0}^3 \hat{R}_m(r_{s,i}^m) = \frac{1}{4} \sum_{i=0}^3 \left[w_1 \hat{R}_m \left(\left\lfloor \frac{r_{s,i}^m}{\delta_s} \right\rfloor \right) + w_2 \hat{R}_m \left(\left\lceil \frac{r_{s,i}^m}{\delta_s} \right\rceil \right) \right] \quad (39)$$

Intuitively, this step converts the 1D processed swath, a line of measured reflectivity samples along the sonar range, into a 2D spatial representation aligned with the mapping grid. Each grid cell receives a reflectivity estimate based on how its corners relate to the measured intensity profile of the current ping, effectively “*painting*” that pings information onto the map with correct geometry and resolution.

This interpolation acts as a geometric resampling process. It transfers each pings reflectivity data from the waterfall domain into the Cartesian grid used by the probabilistic mapping framework. Once this interpolation is complete, the resulting per ping reflectivity field $V_m(q)$ can be directly fused with the corresponding probability weights $P_m(q)$ to construct the global occupancy and reflectivity map in a spatially consistent manner.

Reflectivity Fusion

After each ping m has been interpolated into the grid as the per ping reflectivity field $V_m(q)$ (see Equation 39), and weighted by its associated probability distribution $P_m(q)$ (see Equation 37), all overlapping contributions are fused together into a single reflectivity estimate for each grid cell. Producing a weighted sum of all pings contributing to the same cell:

$$V(q) = \sum_{m=1}^N P_m(q) V_m(q) \quad (40)$$

This formulation ensures that overlapping beams merge smoothly, while measurements with higher probability (ie, stronger and better aligned sonar returns) have a greater influence on the resulting grid intensity. Physically, this represents a spatial blending of all local swath reflectivity estimates based on their observation likelihoods.

Probability Map Normalization

Once the reflectivity fusion in Equation 40 has been performed, the final reflectivity map is normalized by the total accumulated observation probability $P(q)$ from Equation 38. The normalized reflectivity value for each grid cell is given by:

$$M(q) = \frac{V(q)}{P(q)} \quad (41)$$

This step compensates for varying observation density across the map, ensuring consistent intensity scaling even in regions observed by multiple overlapping pings. As a result, $M(q)$ represents the final, fully fused, and normalized local reflectivity map, a spatially consistent depiction of the seafloor surface ready for SLAM and feature extraction.

Post processing and Gap Filling

After map fusion, small unobserved gaps may remain between neighboring swaths. These are filled using a kNN extrapolation, where each empty cell is assigned the mean intensity of its k nearest observed neighbors within a defined distance threshold. To optimize computation, a flood fill method is used to identify only those cells within the active sonar footprint Q_m , avoiding unnecessary evaluation of the full grid.

Algorithm Evaluation

The probabilistic interpolation and grid reconstruction algorithm provides a physically accurate and uncertainty aware mapping framework, effectively modeling the sonar beams angular spread and measurement noise. However, the complete formulation is computationally heavy, requiring the evaluation of thousands of potential grid cells per ping and multiple Gaussian weighting operations.

As a result, the full probabilistic approach is unsuitable for real-time use. Therefore, in the next chapter, the optimized multi stage pruning and caching algorithm from Haralstad's master thesis [2] and Hoff's paper [1] is introduced. This improved implementation maintains the same probabilistic consistency while drastically reducing the computational load, enabling real-time operation on microAmpere ASV without sacrificing map quality.

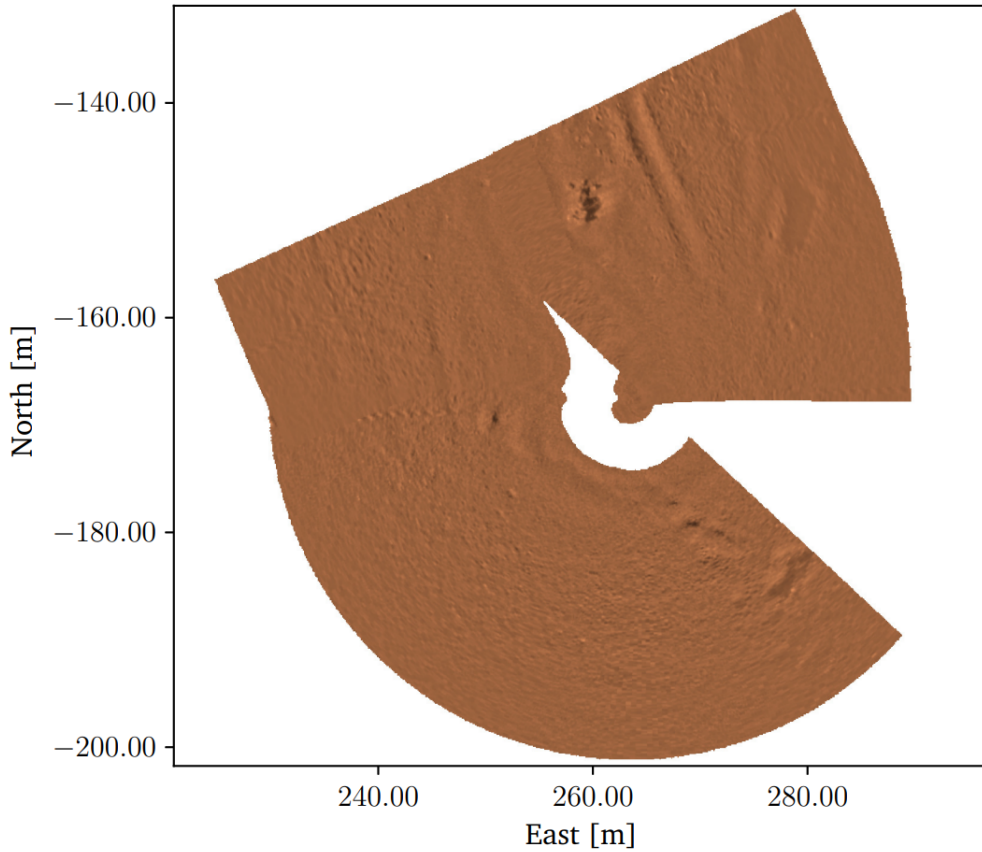


Figure 44: Final probabilistic reflectivity map after fusion and gap filling.^[2]

9.3.5 Optimized Probabilistic Interpolation and Grid Reconstruction

Computation and Acceleration

A direct implementation of the probabilistic map generation would require comparing every sonar sample with every grid cell, leading to a computational complexity of $\mathcal{O}(N_s N_g)$, where N_s is the number of sonar samples and N_g the total number of grid cells. Such a brute force approach is impractical for real-time operation. To achieve efficient execution, the algorithm applies a multi-stage pruning and caching strategy that incrementally narrows down the active grid subset for each sonar ping m before interpolation and fusion are performed.

The pruning process is organized into 3 hierarchical stages, each discarding unnecessary grid cells based on progressively more selective criteria.

Stage 1: Range Based Pruning

The first and cheapest filtering step limits grid cells based on their slant range relative to the sonar position $\mathbf{p}_{s/O}^n(m)$. Since the sonar pose and the range bounds $r_{\min}$ and $r_{\max}$ are constant for each ping m , they are computed once and reused (See Equation 32 and Figure 42). All cells are evaluated using the Euclidean distance

$$r_q = \|\mathbf{p}_q - \mathbf{p}_{s/O}^n(m)\|$$

and only those satisfying $r_{\min} < r_q < r_{\max}$ are kept. This operation quickly eliminates most of the map outside the sonar footprint, typically reducing the grid size by over an order of magnitude.

Stage 2: Angular Pruning

The remaining grid cells are then pruned by angular visibility using the observation angle $\theta_{obs}(q)$ (see Equation 36 and Figure 42). The precomputed constants from the previous stage are reused to help calculate $\theta_{obs}(q)$ efficiently. Cells outside the effective horizontal beam aperture are removed:

$$|\theta_{obs}(q)| < \frac{\alpha_h}{2}$$

This second stage eliminates cells lying outside the sonars field of view, further shrinking the active grid subset.

Stage 3: Probability Based Pruning

For the remaining candidates, the illumination probability $P_m(q)$ is evaluated using the beam model in Equation 37. The precomputed observation angle $\theta_{obs}(q)$ from the previous stage are reused to help calculate $P_m(q)$ efficiently. Cells with negligible contribution are discarded using a small predefined threshold ϵ :

$$P_m(q) \geq \epsilon$$

This final filtering step removes cells whose probability of illumination is too low to affect the map, reducing the computational domain by several additional orders of magnitude.

Combining all 3 pruning steps yields the final refined set:

$$Q_m = \left\{ q \in Q \mid r_{\min} < \|\mathbf{p}_q - \mathbf{p}_{s/O}^n(m)\| < r_{\max}, |\theta_{obs}(q)| < \frac{\alpha_h}{2}, P_m(q) \geq \epsilon \right\}$$

In practice, the number of candidate cells typically decreases from millions in the full grid, to tens of thousands after range pruning, to a few thousand after angular pruning, and finally to a few hundred high probability cells after the last stage. These cells form the final set Q_m , which is then used for interpolation and probabilistic fusion.

Optimized Equations

The modified, computationally efficient version of the complete probabilistic map generation process can then be expressed as:

$$Q_m = \left\{ q \in Q \mid r_{\min} < \|\mathbf{p}_q - \mathbf{p}_{s/O}^n(m)\| < r_{\max}, |\theta_{obs}(q)| < \frac{\alpha_h}{2}, P_m(q) \geq \epsilon \right\} \quad (42)$$

$$V_m(q) = \frac{1}{4} \sum_{i=0}^3 \left[w_1 \hat{R}_m \left(\left\lfloor \frac{r_{s,i}^m}{\delta_s} \right\rfloor \right) + w_2 \hat{R}_m \left(\left\lceil \frac{r_{s,i}^m}{\delta_s} \right\rceil \right) \right] \quad (43)$$

$$V(q) = \sum_{m \in S} \begin{cases} 0, & q \notin Q_m \\ P_m(q) V_m(q), & q \in Q_m \end{cases} \quad (44)$$

$$P(q) = \sum_{m \in S} \begin{cases} 0, & q \notin Q_m \\ P_m(q), & q \in Q_m \end{cases} \quad (45)$$

$$M(q) = \frac{V(q)}{P(q)} \quad (46)$$

$$M = \text{kNNFill}(M) \quad (47)$$

kNNFill(M)

After generating the final probabilistic reflectivity map $M(q)$, a lightweight fill in step **kNNFill**(M) is applied to ensure complete spatial coverage within the sonar footprint. This stage compensates for small unobserved gaps between adjacent swaths or at the edges of the footprint where the local probability values were too low to survive the final pruning stage.

The fill in algorithm operates on the intermediate subset $Q_{m,\text{fill}}$, obtained after the second pruning stage but before probability thresholding, defined as:

$$Q_{m,\text{fill}} = \left\{ q \in Q \mid r_{\min} < \|\mathbf{p}_q - \mathbf{p}_{s/O}^n(m)\| < r_{\max}, |\theta_{obs}(q)| < \frac{\alpha_h}{2} \right\} \quad (48)$$

This subset includes all grid cells lying inside the sonars geometric beam limits, regardless of their final illumination probability. For each unobserved cell $q \in Q_{m,\text{fill}} \setminus Q_m$, the missing reflectivity value is estimated as the mean of its K nearest valid neighbors within a predefined radius:

$$M(q) = \frac{1}{K} \sum_{k=1}^K M(q_k) \quad (49)$$

where q_k are neighboring cells already assigned valid reflectivity values from the probabilistic fusion stage.

To efficiently identify all cells belonging to $Q_{m,\text{fill}}$, a flood fill method is employed instead of scanning the entire grid. The search begins from two seed cells known to be observed, one near the left and one near the right swath boundary, and expands recursively to neighboring cells until no new valid candidates are found. This approach minimizes computation while ensuring that only spatially relevant cells within the active sonar footprint are processed.

The **kNNFill**(M) procedure produces a visually and numerically continuous map by smoothly bridging small gaps and extending valid regions, resulting in a refined reflectivity field $M(q)$ suitable for downstream SLAM, navigation, or feature extraction tasks.

Optimized Algorithm

At a high level, the complete accelerated probabilistic mapping algorithm proceeds as follows.

Inputs and outputs are summarized first, followed by the main processing steps referencing the corresponding equations for clarity.

Input:

- Processed swath sonar samples $\mathcal{S} = \{\mathbf{p}_{g/O}^n(i), \hat{R}(i)\}_{i=1}^{N_s}$ in global coordinates.
- Vehicle position $\mathbf{p}_{b/O}^n$ and attitude $\mathbf{q}$ from state estimator.
- Grid definition $Q = \{q(x, y)\}$ describing local map cells.

Output:

- Final probabilistic reflectivity map $M(q)$, optionally post processed with $\text{kNNFill}(M)$ for spatial completeness.

Algorithm Steps:

1. **Three-Stage Pruning:** Sequentially restrict computation to geometrically and probabilistically valid cells for each ping m , range filtering, angular filtering, and probability thresholding (Equation 42). Resulting subset: Q_m
2. **Interpolation:** For all $q \in Q_m$, interpolate the 1D sonar samples $\hat{R}_m(i)$ to estimate local cell intensity $V_m(q)$ using bilinear interpolation (Equation 43)
3. **Reflectivity Fusion:** Accumulate per ping reflectivity contributions weighted by illumination probability to form $V(q)$ and $P(q)$ (Equations 44 45)
4. **Normalization:** Normalize fused reflectivity by cumulative probability to obtain the map estimate $M(q)$ (Equation 46)
5. **Fill-In Post-Processing:** Apply the lightweight $\text{kNNFill}(M)$ step (Equation 47) using the subset $Q_{m,\text{fill}}$ (Equation 48) to fill gaps via K nearest neighbor averaging (Equation 49)

This optimized implementation preserves the physical consistency of the full probabilistic model while reducing complexity from $\mathcal{O}(N_s N_g)$ to approximately $\mathcal{O}(SB + RC)$, enabling real-time mapping on embedded hardware.

9.3.6 Sliding Map Window and Overlap

The local probabilistic map represents a continuously updated portion of the seafloor constructed from the most recent N processed sonar swaths. Initially, N swaths are accumulated and converted into Cartesian coordinates to form the first local map.

During operation, the map updates in a sliding window fashion: approximately one third of the previous data is retained for overlap, while the remaining two thirds are replaced by newly processed swaths. This overlap maintains spatial and temporal continuity between consecutive local maps, providing consistent feature visibility for SLAM data association and reducing discontinuities at map boundaries.

The choice of N determines the trade off between temporal resolution and mapping stability. A larger N yields smoother, more stable maps but introduces higher latency and lower update rate, while a smaller N increases temporal responsiveness at the cost of reduced spatial context and fewer observable landmarks. In practice, N is tuned empirically based on the vehicles speed, ping rate, and sonar range, typically corresponding to a window of several hundred to a few thousand swaths.

For real-time execution, the sliding map buffer is implemented as a “ring buffer”, where new swaths overwrite the oldest entries once the capacity N is reached. This structure maintains a constant memory footprint and allows efficient data replacement in $\mathcal{O}(1)$ time, ensuring deterministic performance even during long-duration missions.

The sliding window mechanism ensures that the local probabilistic map always reflects the most recent portion of the seafloor while maintaining a controlled overlap with previous maps. This enables smooth temporal transitions, stable landmark tracking, and spatial continuity required for reliable SLAM operation.

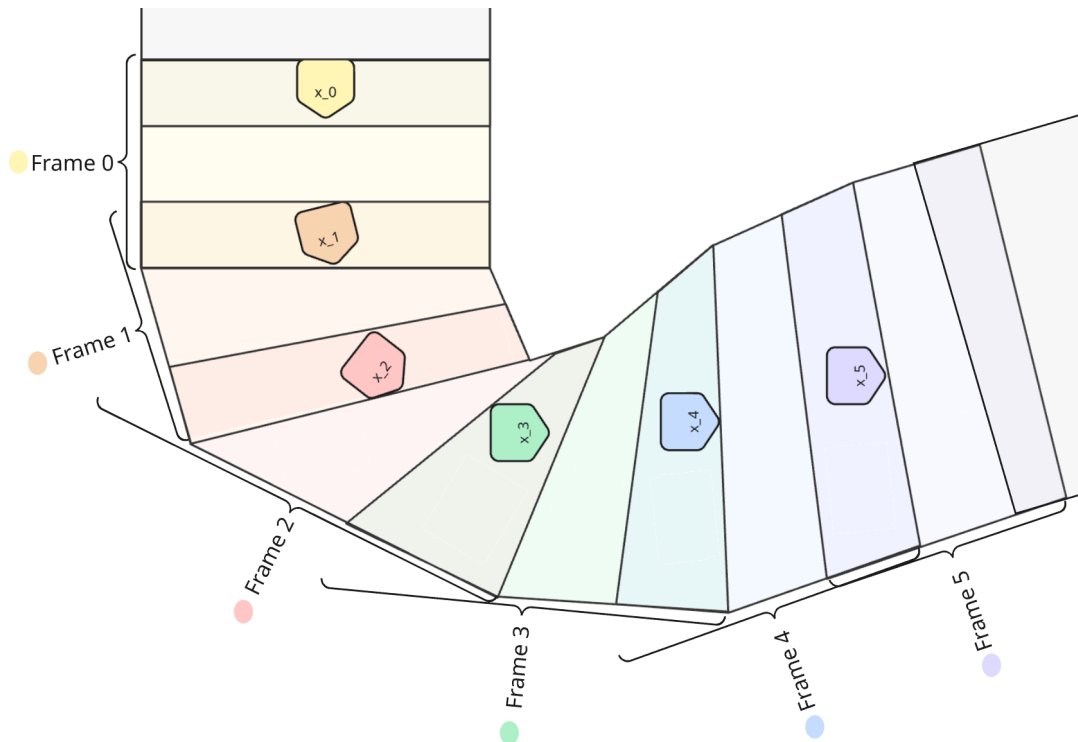


Figure 45: Sliding window mechanism for local map updates. Each frame represents a local probabilistic map composed of N swaths. As new data arrives, the window advances while retaining one third of the previous samples for overlap, ensuring continuity and real-time operation through a ring buffer structure.

9.4 Feature Extraction

9.4.1 Landmarks

Once the local reflectivity map M has been generated, the next step is to identify distinct and stable seafloor features that can serve as landmarks for SLAM. These landmarks correspond to repeatable acoustic structures, such as bright echo highlights or dark shadow regions that persist across consecutive local maps and provide reliable reference points for localization and data association.

The feature extraction process operates directly on the normalized reflectivity map M , treating it as an intensity field analogous to a grayscale image. The objective is to detect, describe, and represent acoustically distinctive regions in a compact form that can later be matched across time. This involves applying image processing methods such as edge preserving filtering, adaptive thresholding, and connected component analysis to isolate coherent acoustic responses from background texture and noise.

Each detected landmark is defined by its spatial centroid $\mathbf{p}_L = [x_L, y_L]^\top$ in the local Cartesian frame, and characterized through a descriptor summarizing its local reflectivity and texture pattern. To interface with the SLAM framework, every landmark is expressed as a measurement in polar form:

$$\mathbf{z}^i = [z_r^i, z_\theta^i]^\top$$

where z_r^i and z_θ^i represent the range and bearing of the landmark relative to the vehicle, assumed to be positioned at the origin of the local map with its heading aligned to the x axis. This provides a compact, self contained observation consistent with the range bearing model $h(\mathbf{x}, \mathbf{m}^i)$ used in SLAM. (see Equation 12)

The goal of this stage is not only to detect features but also to define a standardized data interface for the SLAM system. The expected input is the processed reflectivity map M , while the outputs are a set of landmark measurements $\mathbf{z}^i$ with associated uncertainty and descriptors.

Subsequent sections describe the detailed image processing pipeline used for landmark detection, the computation of measurement and uncertainty, and the design of compact descriptors that enable reliable feature matching across local maps.

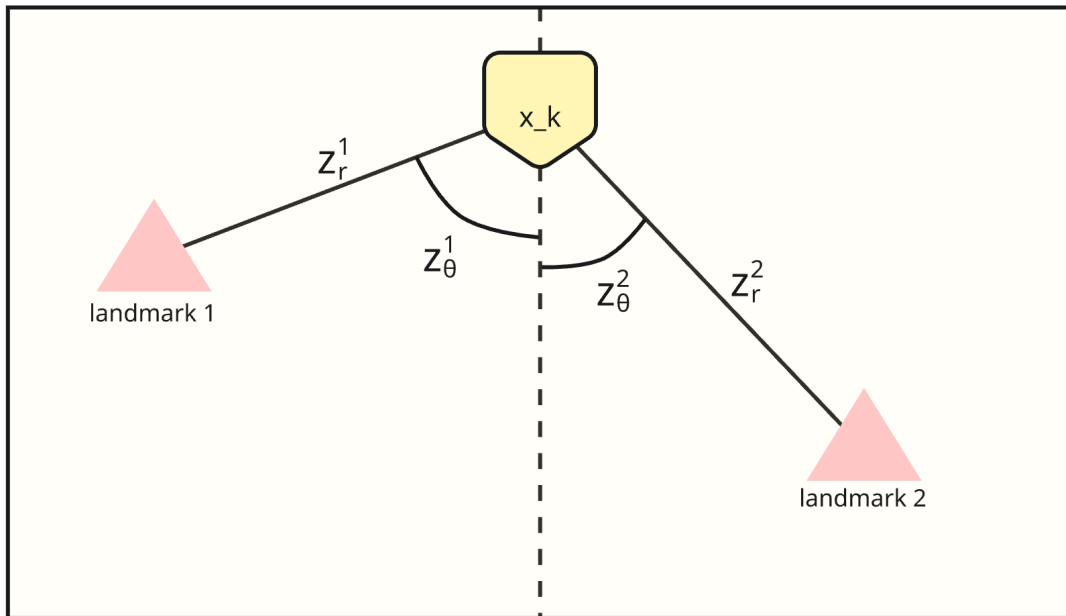


Figure 46: Landmark detection and conversion to range bearing formulation relative to the vehicle position at the local map origin.

9.4.2 Landmark Detection

Image processing and detection

The detection of acoustic landmarks in the reflectivity map $M(x, y)$ follows the same principles as classical image feature extraction. The map can be viewed as a grayscale image, where each pixel intensity represents the local acoustic reflectivity of the seafloor. Hence, identifying landmarks becomes a problem of detecting distinctive regions that stand out in brightness or darkness relative to their surroundings.

The challenge arises from the presence of speckle noise and uneven illumination across the map, both of which can obscure genuine features. To address this, the reflectivity map is first preprocessed using an edge preserving filter to suppress high frequency noise while maintaining the important structural gradients that define echoes and shadows.

Subsequent steps operate within a local kernel (neighborhood) around each pixel, where local intensity statistics are computed and compared against adaptive thresholds. This adaptive approach makes the detector robust to varying background intensity and contrast, allowing consistent landmark detection under different seafloor textures or sonar angles.

Finally, standard image processing tools, such as morphological filtering and connected component analysis, are used to remove spurious responses and cluster neighboring detections into coherent regions. The centroids of these regions define the landmark coordinates in the local map frame, serving as reliable acoustic features for subsequent SLAM processing.

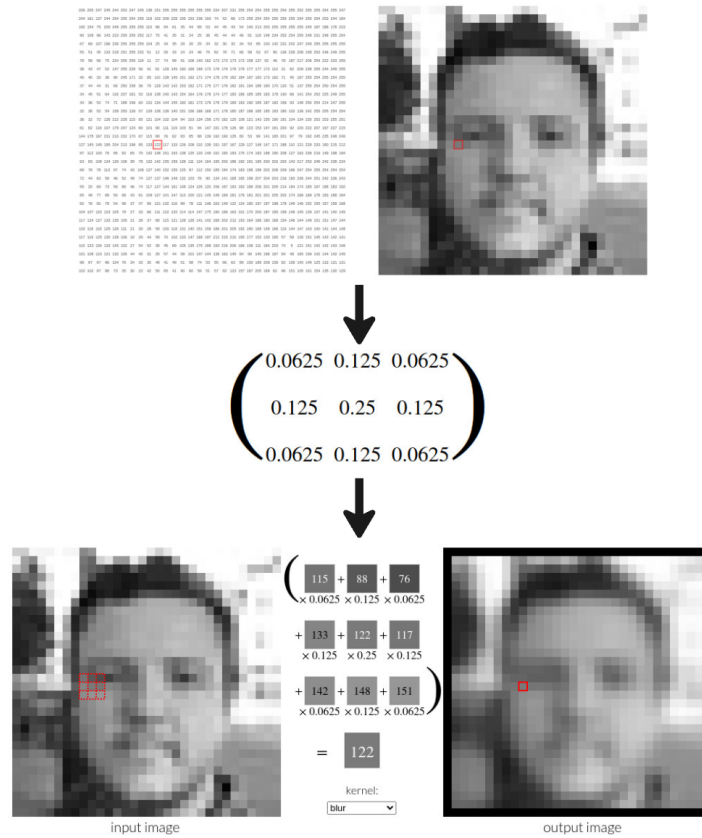


Figure 47: Illustration of a kernel based filtering operation. Each output pixel is computed as the weighted sum of neighboring pixels within the kernel window. Shown here is a 3×3 Gaussian filter with $\sigma = 1$, demonstrating the basic principle of convolution based image processing. The concept of applying a local kernel window is fundamental in image processing and is used extensively throughout this chapter for filtering, statistical analysis, and feature detection.^[39]

Edge preserving filtering

To suppress high frequency speckle noise while maintaining the sharp acoustic transitions that define echoes and shadows, the reflectivity map $M(x, y)$ is preprocessed using an “*edge preserving bilateral filter*”. Unlike traditional Gaussian smoothing, which performs averaging purely based on spatial distance, the bilateral filter combines two weighting functions, a “*spatial kernel*” that smooths nearby pixels and a “*range kernel*” that preserves edges by considering intensity similarity.

The filtered map $M_s(x, y)$ is computed as:

$$M_s(x, y) = (G_s * G_r * M_s)(x, y)$$

$$M_s(x, y) = \frac{1}{W(x, y)} \sum_{i=-n}^n \sum_{j=-n}^n G_s(i, j) G_r(M(x+i, y+j) - M(x, y)) M(x+i, y+j), \quad n = \frac{N-1}{2}$$

where G_s is the “*spatial Gaussian kernel*” that decays with pixel distance, and G_r is the “*range Gaussian kernel*” that decays with intensity difference. It is important to note that this operation is applied locally within the $N \times N$ neighborhood, centered at (x, y) , not over the entire image.

For each output pixel, a new adaptive kernel is computed based on the local spatial and intensity structure, allowing the filter to smooth homogeneous regions while preserving sharp edges within that local window.

$$G_s(i, j) = \exp\left(-\frac{i^2 + j^2}{2\sigma_s^2}\right), \quad G_r(\Delta M) = \exp\left(-\frac{(\Delta M)^2}{2\sigma_r^2}\right)$$

The normalization factor $W(x, y)$ ensures proper weighting:

$$W(x, y) = \sum_{i=-n}^n \sum_{j=-n}^n G_s(i, j) G_r(M(x+i, y+j) - M(x, y)), \quad n = \frac{N-1}{2}$$

Here, σ_s controls the degree of spatial smoothing, determining how far neighboring pixels contribute based on their spatial distance from the center. Increasing σ_s causes the filter to average over a larger neighborhood, producing smoother results but potentially blurring small features.

In contrast, σ_r governs the sensitivity to intensity differences between pixels. A small σ_r strongly penalizes averaging across large intensity changes, preserving sharp edges such as echo shadow boundaries. Increasing σ_r makes the filter less edge aware, gradually approaching standard Gaussian smoothing where intensity differences are ignored.

This combined spatial range filtering effectively denoises homogeneous seafloor regions while maintaining crisp echo shadow edges, which is crucial for reliable landmark detection. Typical configurations use $N \in [5, 11]$, $\sigma_s \in [1.0, 3.0]$, and $\sigma_r \in [0.5, 2.0]$, depending on sonar resolution and speckle intensity.

It is important to note that the spatial and range kernels do not necessarily have to be Gaussian. While Gaussian kernels are the most common due to their smooth weighting and numerical stability, other kernel formulations can be used to tailor the filtering behavior to specific sonar characteristics.

Alternative options include “*box filters*” for uniform averaging, “*exponential kernels*” for stronger edge emphasis, or “*Lorentzian kernels*” for robust outlier suppression. More advanced variants such as the “*trilateral filter*” or “*guided filter*” can also be explored to improve edge preservation and contrast consistency.

The optimal kernel choice depends on the dataset and acoustic conditions. Gaussian filtering provides a solid baseline with balanced noise suppression and edge retention, but experimenting with different kernel combinations can yield better edge definition and more stable feature extraction performance in specific environments.

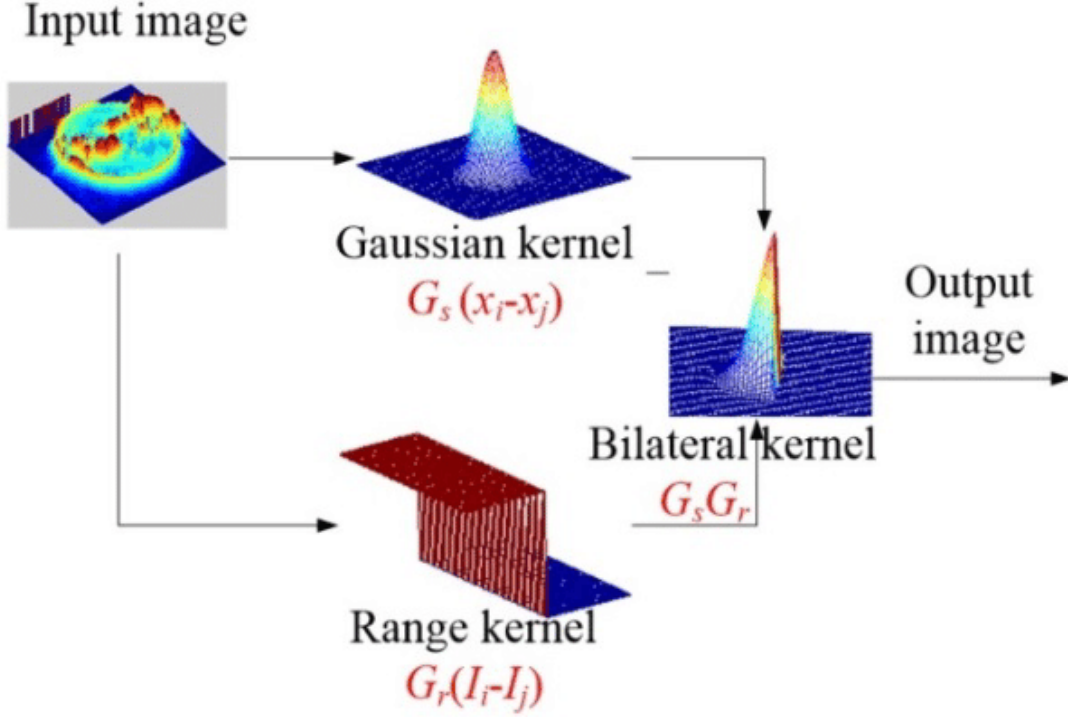


Figure 48: Example of bilateral edge-preserving filtering applied to a grayscale reflectivity map. The filter combines a spatial Gaussian kernel G_s with an intensity-based range kernel G_r to reduce speckle noise while maintaining sharp echo and shadow boundaries.^[40]

Local statistical analysis

After bilateral filtering, each pixel in the filtered map $M_s(x, y)$ is analyzed within a local neighborhood window $\mathcal{N}(x, y)$ of size $N \times N$. This window $N \times N$ is typically larger than the kernel used in the bilateral filter, since it aims to capture broader local intensity variations rather than fine scale noise. While the bilateral filter preserves edges and reduces speckle noise on a small spatial scale, the larger statistical window provides contextual information about the surrounding seafloor texture, enabling adaptive feature detection under varying background and illumination conditions.

For each pixel, a square window $\mathcal{N}(x, y)$ of size $N \times N$ is centered at (x, y) and moved (slid) across the entire map, similar to the convolution process used during filtering. Within this window, the local portion of the filtered map $M_s(x, y)$ is analyzed using a Gaussian-weighted mean and standard deviation:

$$\mu_{\mathcal{N}}(x, y) = \frac{\sum_{(i,j) \in \mathcal{N}(x,y)} G_{\sigma_{\mu}}(i, j) M_s(i, j)}{\sum_{(i,j) \in \mathcal{N}(x,y)} G_{\sigma_{\mu}}(i, j)}$$

$$\sigma_{\mathcal{N}}(x, y) = \sqrt{\frac{\sum_{(i,j) \in \mathcal{N}(x,y)} G_{\sigma_{\mu}}(i, j) (M_s(i, j) - \mu_{\mathcal{N}}(x, y))^2}{\sum_{(i,j) \in \mathcal{N}(x,y)} G_{\sigma_{\mu}}(i, j)}}$$

where $i, j \in [-n, n]$, $n = \frac{N-1}{2}$

Here, N defines the total window size (eks, 15×15 pixels), and n represents the half width of that window, determining how far the kernel extends from the central pixel in each direction.

It is important to note that this computation is performed locally within each kernel window centered on (x, y) , not across the entire image.

The Gaussian weights $G_{\sigma_\mu}(i, j)$ are defined only over this local neighborhood $\mathcal{N}(x, y)$ as:

$$G_{\sigma_\mu}(i, j) = \exp\left(-\frac{i^2 + j^2}{2\sigma_\mu^2}\right)$$

Here, σ_μ controls how strongly neighboring pixels influence the local statistics. Pixels closer to the center of the kernel contribute more to the mean and variance, while distant pixels have exponentially less weight.

This Gaussian weighted formulation makes the adaptive thresholding process more robust to sharp transitions and boundary effects, since it smooths local estimates of $\mu_{\mathcal{N}}$ and $\sigma_{\mathcal{N}}$ without blurring the underlying feature edges. Typical configurations use $\sigma_\mu \in [3, 7]$, depending on the desired level of spatial smoothing relative to the window size N .

These weighted statistics effectively describe the acoustic texture of the surrounding seafloor. Regions with low $\sigma_{\mathcal{N}}$ are homogeneous and unlikely to contain distinctive features, whereas regions with high $\sigma_{\mathcal{N}}$ typically correspond to structural transitions such as object edges, echoes, or shadow boundaries. The computed $\mu_{\mathcal{N}}$ and $\sigma_{\mathcal{N}}$ fields are then used in the adaptive thresholding stage to identify locally dominant features.

The neighborhood size $N \times N$ and weighting parameter σ_μ must be chosen carefully, too small a window makes the detector sensitive to noise, while too large a window may over-smooth important local contrast. In practice, window sizes between 15×15 and 31×31 pixels with $\sigma_\mu \in [3, 7]$ work well for typical sonar resolutions and feature scales.

Adaptive thresholding

Once the local mean $\mu_{\mathcal{N}}(x, y)$ and standard deviation $\sigma_{\mathcal{N}}(x, y)$ have been computed, each pixel is classified based on how much its intensity deviates from the local background. A pixel is marked as a potential feature if its absolute deviation exceeds an adaptive, locally scaled threshold:

$$|M_s(x, y) - \mu_{\mathcal{N}}(x, y)| > k \sigma_{\mathcal{N}}(x, y)$$

where k is a sensitivity factor that determines how strong a deviation must be to qualify as a landmark candidate.

This condition is applied independently to every pixel, producing a binary mask of the same size as the reflectivity map. Pixels that satisfy the inequality are marked as “*TRUE (1)*”, and all others as “*FALSE (0)*”. This effectively highlights areas whose intensity differs significantly from their local background, adapting dynamically to local contrast and texture variations.

Positive deviations ($M_s(x, y) > \mu_{\mathcal{N}}$) indicate bright *echo landmarks*, typically caused by strong reflections or elevated objects.

Negative deviations ($M_s(x, y) < \mu_{\mathcal{N}}$) reveal dark *shadow landmarks*, which correspond to acoustic occlusions or surface depressions.

This adaptive thresholding approach ensures that the detector remains robust under varying illumination, noise, and seafloor reflectivity. Unlike global methods such as “*Otsus method*”, which compute a single global threshold for the entire image, this local approach adapts to spatially varying conditions, which is crucial for sonar imagery, where intensity levels can change drastically between regions due to grazing angle, texture, and attenuation effects.

Intuitively, if a region has a high local mean $\mu_{\mathcal{N}}$ (bright background), only much brighter or darker spots are selected, preventing false detections in already bright areas. Conversely, in darker or smoother regions, even small local deviations may pass the threshold, allowing subtle yet meaningful features to be detected.

The constant k is tuned empirically according to sonar noise characteristics, typically $k \in [1.5, 3.0]$.

Smaller k values increase sensitivity, detecting weaker features at the cost of more false positives, while larger k values yield cleaner detections but may miss faint landmarks.

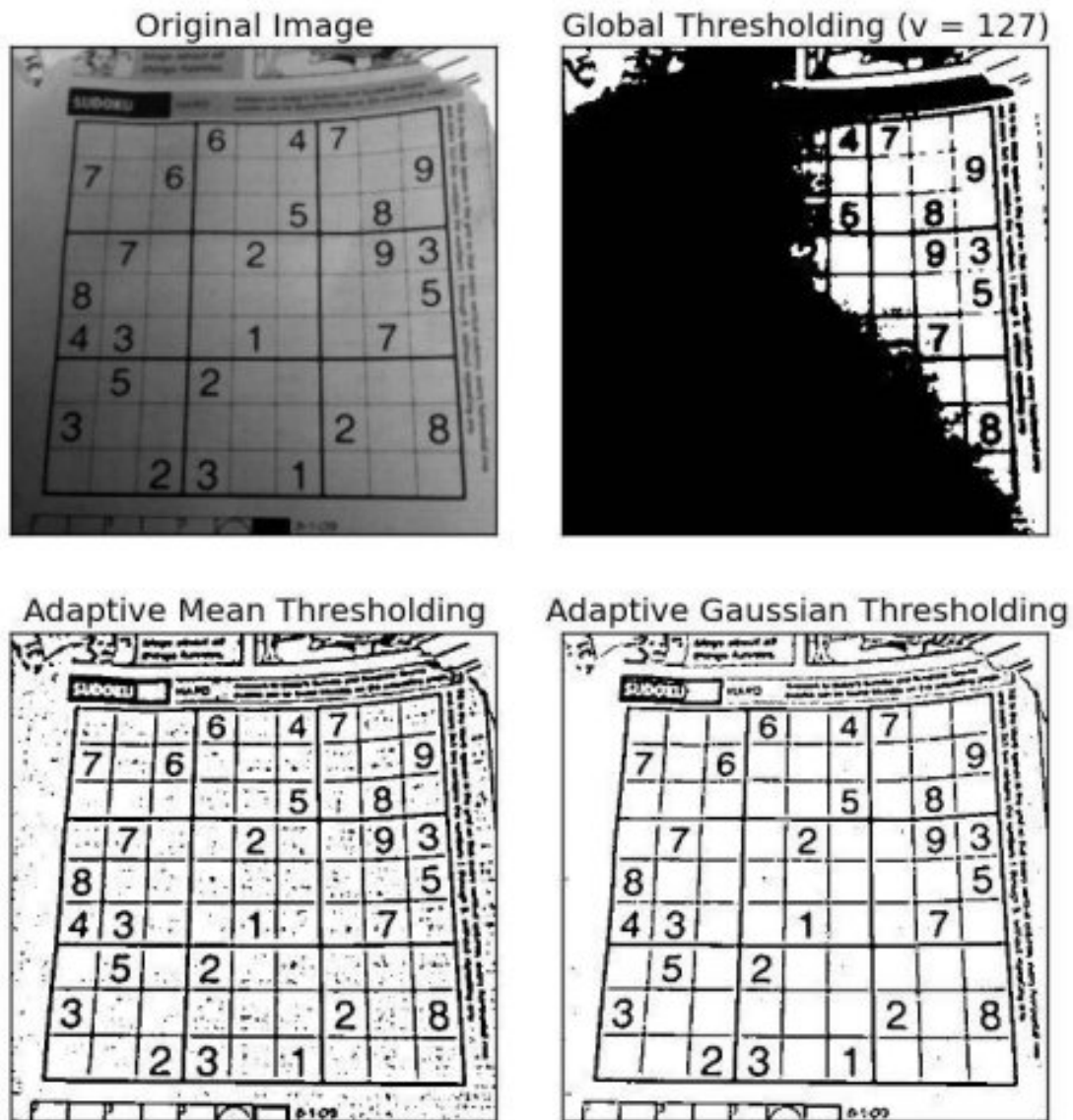


Figure 49: Comparison between global and adaptive thresholding methods. The global approach applies a single threshold across the entire image, causing poor segmentation under uneven illumination. In contrast, adaptive methods compute thresholds locally within small neighborhoods. The adaptive mean thresholding preserves most structures but introduces minor speckle artifacts, while the adaptive Gaussian thresholding provides a more robust and balanced result, maintaining clear edges and overall feature integrity.^[41]

Morphological filtering

After adaptive thresholding, the resulting binary mask of candidate pixels typically contains isolated noisy responses and small fragmented blobs caused by residual speckle or local contrast fluctuations. To refine the detection and enforce spatial consistency, the mask is processed using morphological operations that act directly on the pixel structure of the image.

⊖ **Erosion:** Erosion shrinks bright (foreground) regions by eliminating boundary pixels that do not have full neighbourhood support. It effectively removes small isolated pixels, thin artefacts, and detaches weakly connected components.

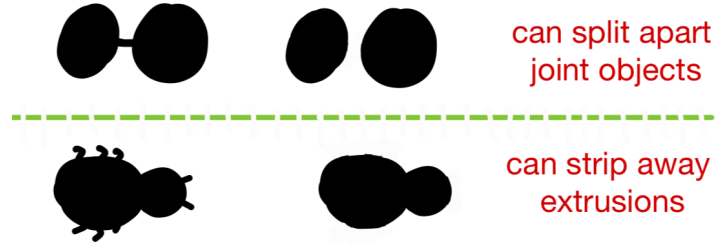


Figure 50: Illustration of the erosion operation: the structuring element slides across the binary mask and removes pixels on the boundaries of foreground regions when the neighbourhood condition is not met.^[42]

⊕ **Dilation:** Dilation expands bright regions by adding pixels on their boundaries if at least one pixel in the structuring element neighbourhood was already “on”. It fills small gaps and reconnects nearby fragmented responses, enlarging coherent regions.

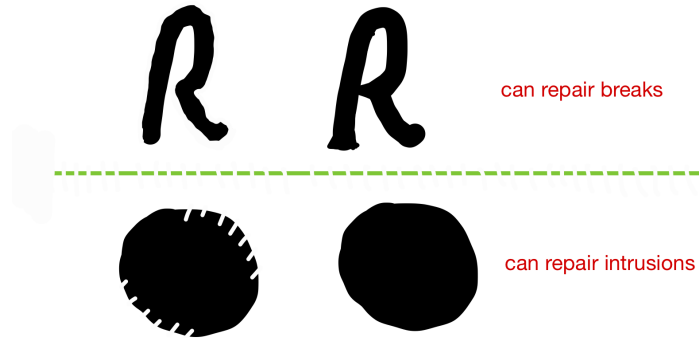


Figure 51: Illustration of the dilation operation: the structuring element slides across the binary mask and a pixel is set to foreground if any pixel in the structuring neighbourhood is foreground, thereby growing the region.^[42]

Compound operations (opening and closing): By executing erosion and dilation in sequence, we can tailor the morphological effect:

$$M_{\text{open}} = (M_{\text{bin}} \ominus B) \oplus B, \quad M_{\text{close}} = (M_{\text{bin}} \oplus B) \ominus B$$

Opening (erosion → dilation) removes small foreground speckles while preserving the shape of larger objects. Closing (dilation → erosion) fills small holes and reconnects background gaps inside foreground components.

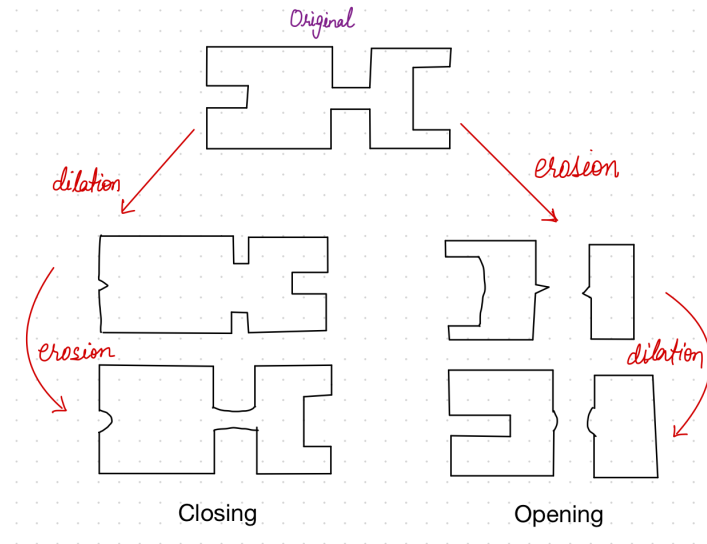


Figure 52: Visual comparison of compound morphological operations: opening vs closing and their effect on the binary mask.^[42]

Recommended sequence for landmark extraction: For the seafloor landmark extraction pipeline, the following morphological sequence is proven effective:

1. **Opening:** with a small structuring element, removes isolated noise pixels and tiny speckle blobs.
2. **Closing:** with a slightly larger structuring element, fills small holes within candidate regions and reconnects weak fragments.

This sequence ensures that only coherently sized, well shaped regions remain, effectively filtering noise while preserving potential echo and shadow regions for subsequent centroid extraction. By applying “*opening first*” (to suppress noise) and “*closing second*” (to fill and reconnect), region integrity is maximized while avoiding false merging of unrelated features. Reversing this order (closing before opening) often leads to premature merging of small noise blobs or incomplete hole removal.

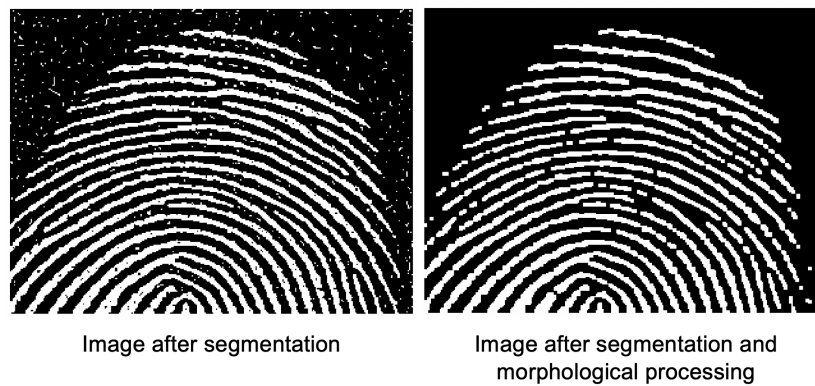


Figure 53: Example of Morphological Processing.^[42]

Labelling

Once the binary mask has been morphologically filtered, the next step is to identify and separate individual feature blobs that correspond to potential landmarks. This is achieved using a “*connected component labeling*” process, which assigns a unique integer ID to each spatially connected region of active pixels in the binary map.

Each pixel in the binary image is examined and grouped with its neighbors based on a connectivity rule, typically:

- **4-connectivity:** pixels are connected if they share an edge (top, bottom, left, or right),
- **8-connectivity:** pixels are connected if they share either an edge or a corner (includes diagonals).

Using this rule, the algorithm scans through the binary image and assigns a label value L_i to every pixel that belongs to the same connected region:

$$M_{\text{label}}(x, y) \in \{0, 1, 2, \dots, L\}$$

Here, $M_{\text{label}}(x, y)$ is the “*labeled map*”, meaning every pixel in the original binary mask now stores an integer label instead of just 0 or 1. L is the total number of detected blobs (connected components) in the image. $\mathcal{R}_L$ denotes the “*set of all pixels*” (x, y) that share the same label L , ie, all pixels that belong to that particular region or “*blob*”.

In other words, each unique label L identifies one continuous feature region, and $\mathcal{R}_L$ simply collects all pixels that make up that region.

This turns the binary mask into a labeled feature map, where each blob such as an echo or shadow is now clearly separated and can be analyzed individually. After labeling, the connected regions $\mathcal{R}_L$ are used to compute geometric and intensity based properties like centroid position, area, or local reflectivity patterns for further landmark characterization.

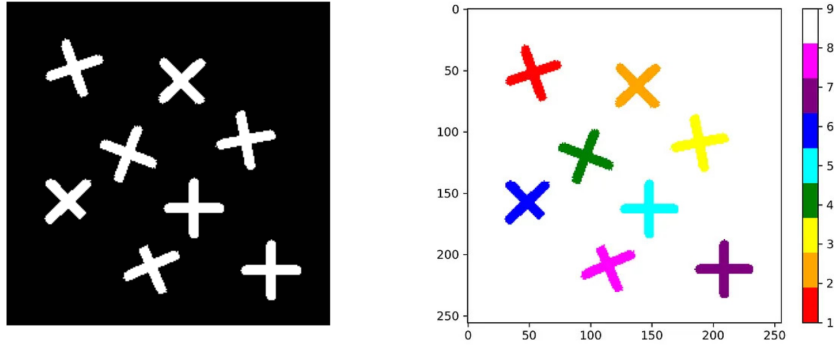


Figure 54: Example of binary image transformation before and after connected component labeling. The left image shows the original binary mask, while the right image displays the labeled result, where each distinct color represents a unique label L . All pixels sharing the same label form the region set $\mathcal{R}_L$, corresponding to one detected connected component. These labeled regions can then be used for computing centroid, area, and other geometric or texture based descriptors for feature extraction.^[43]

Centroid extraction

For each connected region $\mathcal{R}_L$, the landmark position is computed as the intensity independent geometric centroid:

$$\mathbf{p}_L = \begin{bmatrix} x_L \\ y_L \end{bmatrix} = \frac{1}{N_L} \sum_{(x,y) \in \mathcal{R}_L} \begin{bmatrix} x \\ y \end{bmatrix} \quad (50)$$

Here, N_L denotes the total number of pixels within the connected region $\mathcal{R}_L$, which contains all pixels belonging to the same labeled feature (echo or shadow).

Intuitively, this operation computes the center of mass of the blob by averaging the x and y coordinates of all pixels that form it. The resulting centroid $\mathbf{p}_L = [x_L, y_L]^\top$ represents the spatial position of that landmark in the local map frame.

9.4.3 Landmark Measurement

After extracting each landmark centroid $\mathbf{p}_L = [x_L, y_L]^\top$ (Equation 50), it is converted into the range-bearing measurement used by the SLAM observation model $h(\mathbf{x}, \mathbf{m}^i)$ (Equation 11). With the vehicle placed at the origin and aligned with the x -axis, the measurement becomes

$$\mathbf{z}^i = \begin{bmatrix} z_r^i \\ z_\theta^i \end{bmatrix} = \begin{bmatrix} \sqrt{x_L^2 + y_L^2} \\ \text{atan2}(y_L, x_L) \end{bmatrix} \quad (51)$$

where z_r^i and z_θ^i are the landmark range and bearing. This direct polar form matches the SLAM sensor model and allows immediate use in the update step without additional transformations.

9.4.4 Landmark Measurement Uncertainty

Every measurement $\mathbf{z}^i$ obtained from Equation 51 is subject to uncertainty arising from sensor noise, acoustic propagation effects, and image processing errors.

To properly integrate these measurements into the SLAM optimization, each observation is assigned a covariance matrix $\mathbf{R}_{z^i}$ that quantifies the expected error in range and bearing. This matrix defines the confidence associated with the measurement constraint in the graph, weighting its influence during the nonlinear least squares optimization.

Following the range dependent noise formulation in Equation 14, the covariance for each landmark measurement $\mathbf{z}^i = [z_r^i, z_\theta^i]^\top$ is modeled as:

$$\mathbf{R}_{z^i} = \begin{bmatrix} \sigma_r^2(1 + \alpha_r z_r^{i2}) & 0 \\ 0 & \sigma_\theta^2(1 + \alpha_\theta z_r^{i2}) \end{bmatrix} \quad (52)$$

Here, σ_r and σ_θ denote the base noise levels in range and bearing, while α_r and α_θ represent range dependent scaling factors.

Intuitively, as the landmark distance z_r^i increases, the signal to noise ratio degrades due to beam spreading, attenuation, and reduced pixel resolution. This leads to larger uncertainty in both range and bearing, which is reflected by the quadratic scaling terms in the covariance. At short ranges, the measurements are more reliable and the uncertainty approaches the base values σ_r^2 and σ_θ^2 .

Within the graph based SLAM framework, this covariance directly defines the information matrix $\mathbf{\Omega}_{z^i} = \mathbf{R}_{z^i}^{-1}$ used to weight the measurement residuals during optimization, ensuring that reliable, close landmarks contribute more strongly than distant or uncertain ones.

9.4.5 Descriptor Assignment

Descriptor purpose

Each detected landmark must be uniquely characterized to enable consistent recognition across multiple local maps. Descriptors encode distinctive information about the landmarks local appearance and spatial structure, allowing the SLAM system to match observations taken from different viewpoints or times.

Following the formulation in “*Robust Imaging Sonar-based Place Recognition and Localization in Underwater Environments*” [44], the proposed approach employs two complementary descriptor sets. A “*strong descriptor*” capturing local reflectivity and texture statistics, and a “*weak descriptor*” encoding coarse geometric context. This separation claims to improve robustness. Intensity based features remain stable across illumination and angle variations, while geometric terms enhance distinctiveness when consistent viewing conditions are available.

Strong descriptors

The strong descriptor represents the intrinsic appearance of the landmark region $\mathcal{R}_L$ using local intensity statistics and texture measures:

$$\mathbf{d}_L^{\text{strong}} = [\mu_I, \sigma_I, \text{contrast}, \text{entropy}] \quad (53)$$

Each component in the strong descriptor captures a distinct visual property of the acoustic landmark region, enhancing robustness against local illumination changes, speckle noise, and varying sonar viewing angles. The mean intensity μ_I (Equation 54) represents the overall brightness of the region, while the standard deviation σ_I (Equation 55) captures internal reflectivity variation. The contrast term (Equation 56) measures how strongly the region stands out from its surroundings, and entropy (Equation 58) quantifies the diversity and complexity of its texture pattern through the local intensity distribution.

Together, these four quantities form a compact yet expressive representation of each landmarks reflectivity and texture characteristics. They are largely invariant to rotation, scale, and moderate illumination changes, providing stable and repeatable appearance based matching across different viewpoints and sonar passes.

μ_I : Mean intensity

The mean reflectivity μ_I represents the average brightness of all pixels within the region $\mathcal{R}_L$:

$$\mu_I = \frac{1}{N_L} \sum_{(x,y) \in \mathcal{R}_L} M_s(x,y) \quad (54)$$

where $M_s(x,y)$ is the filtered reflectivity map and $N_L = |\mathcal{R}_L|$ is the number of pixels in the region. Intuitively, μ_I indicates how reflective or shadowed the region is on average, bright regions correspond to echoes (high return), while dark regions correspond to shadows (low return).

σ_I : Intensity variance

The intensity standard deviation σ_I measures local variation in reflectivity:

$$\sigma_I = \sqrt{\frac{1}{N_L} \sum_{(x,y) \in \mathcal{R}_L} (M_s(x,y) - \mu_I)^2} \quad (55)$$

$N_L = |\mathcal{R}_L|$ is the number of pixels in the region. A high σ_I implies strong internal texture or gradients (typical for rocky or structured seafloor), while low σ_I indicates uniform regions such as flat sediment or shadowed zones.

contrast: Local contrast

Contrast quantifies the normalized difference between maximum and minimum intensities within the region:

$$\text{contrast} = \frac{I_{\max} - I_{\min}}{I_{\max} + I_{\min} + \epsilon} \quad (56)$$

where $I_{\max}$ and $I_{\min}$ are the maximum and minimum pixel values in $\mathcal{R}_L$, and ϵ is a small constant to avoid division by zero.

Contrast provides a scale independent measure of how distinctly the feature stands out against its background, higher contrast implies stronger detectability.

entropy: Local entropy

Entropy quantifies the information content or complexity of intensity variations within the region $\mathcal{R}_L$. It is computed from the local intensity histogram of the region, which captures how frequently different brightness values occur.

To obtain entropy, the pixel intensities inside the landmark region $\mathcal{R}_L$ are first analyzed statistically through their local histogram. This histogram represents how frequently each intensity value occurs within the region and provides a compact description of its brightness distribution.

The full dynamic range of the reflectivity map $M(x, y)$ is divided into K discrete bins or categories (for example, $K = 32$ or $K = 64$). Each bin groups together pixels with similar intensity levels, capturing the distribution of brightness values in that region.

Let h_i denote the number of pixels in $\mathcal{R}_L$ whose intensity falls into the i -th bin, and $N_L = |\mathcal{R}_L|$ the total number of pixels in the region. The normalized probability of occurrence for each intensity bin is then computed as:

$$p_i = \frac{h_i}{N_L}, \quad i \in [1, K] \quad (57)$$

This probability distribution $\{p_i\}$ reflects how pixel intensities are spread across the available range: a few dominant bins indicate a uniform texture (low variation), while many bins with roughly equal weights indicate a complex, highly variable texture. This property is then defined as entropy:

$$\text{entropy} = \left(- \sum_{i=1}^K p_i \log_2(p_i + \epsilon) \right) \quad (58)$$

where ϵ is a small constant (eks, 10^{-12}) to prevent undefined logarithms for empty bins.

Intuitively, entropy measures how unpredictable or “*rich*” the texture is.

Low entropy $\rightarrow$ most pixels have similar intensities (smooth or uniform region, eks., shadow or uniform texture).

High entropy $\rightarrow$ pixel intensities vary widely (complex texture, eks, rocky or reflective area).

This metric complements the contrast and variance features by encoding local structural diversity rather than simple brightness variation, improving landmark distinctiveness across sonar passes.

Weak descriptors

The weak descriptor provides complementary geometric cues that describe the landmarks spatial configuration relative to the sonar frame:

$$\mathbf{d}_L^{\text{weak}} = [A_L, \rho_L, \theta_L, \Delta_r] \quad (59)$$

Each component in the weak descriptor captures a geometric or spatial characteristic of the landmark relative to the sonar frame, complementing the appearance based information encoded in the strong descriptor. The landmark area A_L is computed directly from the number of pixels in the region as $A_L = N_L = |\mathcal{R}_L|$, providing a simple measure of the features spatial extent in the local map.

The polar coordinates (ρ_L, θ_L) correspond to the range and bearing of the landmark (Equation 51), linking its geometric position to the vehicle centered reference frame.

To compute the radial intensity gradient Δ_r , the region $\mathcal{R}_L$ is divided into two radial subsets relative to the sonar origin, ie map origin $M(0, 0)$. This will create an “inner” half $\mathcal{R}_L^{\text{in}}$ and an “outer” half $\mathcal{R}_L^{\text{out}}$. This division is based on the average radial distance of each pixel, where $r = \sqrt{x^2 + y^2}$. Pixels with smaller r radius (closer to the sonar) belong to the inner set $\mathcal{R}_L^{\text{in}}$, while those with larger r radius belong to the outer set $\mathcal{R}_L^{\text{out}}$:

$$\mathcal{R}_L^{\text{in}} = \{(x, y) \in \mathcal{R}_L \mid r < r_{\text{median}}\}, \quad \mathcal{R}_L^{\text{out}} = \{(x, y) \in \mathcal{R}_L \mid r \geq r_{\text{median}}\}$$

For each subset, the mean reflectivity is computed as:

$$\mu_I^{\text{in}} = \frac{1}{N_{\text{in}}} \sum_{(x, y) \in \mathcal{R}_L^{\text{in}}} M(x, y), \quad \mu_I^{\text{out}} = \frac{1}{N_{\text{out}}} \sum_{(x, y) \in \mathcal{R}_L^{\text{out}}} M(x, y)$$

and the corresponding mean radius:

$$r_{\text{in}} = \frac{1}{N_{\text{in}}} \sum_{(x, y) \in \mathcal{R}_L^{\text{in}}} r, \quad r_{\text{out}} = \frac{1}{N_{\text{out}}} \sum_{(x, y) \in \mathcal{R}_L^{\text{out}}} r$$

The radial intensity gradient is then approximated by a simple finite difference:

$$\Delta_r = \frac{\mu_I^{\text{out}} - \mu_I^{\text{in}}}{r_{\text{out}} - r_{\text{in}}}$$

This quantity expresses how the mean reflectivity of the landmark changes with increasing range. A negative Δ_r indicates that intensity decreases with distance (typical for shadow regions), a positive Δ_r indicates increasing brightness (as in echo highlights), and values near zero correspond to uniform regions with little depth dependent variation.

Note that computing the radial intensity gradient is relatively expensive compared to its descriptive value one gets out of it. If real-time performance is a concern, Δ_r can be omitted from the weak descriptor set.

Full descriptor

Combining the appearance based descriptor in Equation 53 and the geometric descriptor in Equation 59, the final landmark descriptor is constructed as a single concatenated feature vector:

$$\mathbf{d}^i = [\mathbf{d}_L^{\text{strong}}, \mathbf{d}_L^{\text{weak}}] \quad (60)$$

This unified representation captures both the intrinsic reflectivity pattern of a landmark i and its spatial configuration relative to the sonar frame, balancing appearance robustness and geometric distinctiveness.

During feature association, the strong descriptor primarily drives the similarity metric, ensuring reliable appearance based matching, while the weak descriptor provides an additional geometric validation layer that helps reject ambiguous or spatially inconsistent correspondences.

For practical implementation, the descriptors are compared using nearest neighbor search methods such as FLANN or kNN-based matching, allowing efficient correspondence evaluation between consecutive local maps in graph based SLAM optimization.

This combined descriptor structure ensures that landmarks are discriminative, repeatable, and computationally manageable, providing a compact yet expressive interface between feature extraction and SLAM data association.

9.4.6 Feature Extraction Output

The final output of the feature extraction module consists of validated landmarks, each described by:

$$\mathcal{L}_i = \{\mathbf{z}^i, \mathbf{R}_{z^i}, \mathbf{d}^i\}$$

This minimal representation provides all essential information for the SLAM data association and optimization steps comprising the landmark measurement, its uncertainty, and a discriminative descriptor for efficient matching.

Each detected landmark is thus represented compactly by its measurement $\mathbf{z}^i$ (Equation 51), its associated uncertainty $\mathbf{R}_{z^i}$ (Equation 52), and its appearance geometry descriptor $\mathbf{d}_L^i$ (Equation 60).

Together, these three components form the minimal yet complete information set required for graph based SLAM, defining what was observed, how reliable that observation is, and how it can be uniquely recognized across consecutive local maps.

10 Data Association

10.1 Introduction

Data association (DA) determines whether each new landmark observation corresponds to an existing map feature or should initialize a new one, making it one of the most critical components of SLAM. Its role is to ensure that the map remains internally consistent, every accepted match becomes a fixed constraint in the estimator, directly influencing all future pose and landmark updates [45]. Because a single incorrect association can corrupt the entire map or cause unbounded drift, SLAM relies on DA to filter out impossible or unlikely matches before they enter the optimization pipeline. This is achieved through strict statistical gating based on the Mahalanobis distance, which defines a confidence region in measurement space and limits the set of landmarks that must be considered for matching.

Most SLAM systems assume static or near static environments because reliably handling moving objects remains an open research challenge. Several approaches exist, robust estimators, non Gaussian noise models, motion removal, or explicit tracking, but none are universally reliable. For this thesis the seabed is treated as near static over short and medium time spans, which suits underwater mapping where sediment and structures change slowly. This simplifies DA to a geometric and probabilistic matching problem built on the extended range bearing model used throughout the thesis.

Every association that passes gating becomes a factor linking the current pose to a landmark, meaning the backend optimizer fully trusts the decision. To avoid catastrophic drift, the system relies solely on JCBB, using it in two modes depending on the situation. In ordinary local operation, tight gating leaves only 1 or very few candidates, causing JCBB to collapse naturally into a simple nearest neighbour outcome without performing full joint checks. In ambiguous cases or during loop closure attempts, JCBB switches to its full mode and evaluates joint compatibility using the full SLAM covariance, ensuring that only geometrically consistent multi landmark associations are accepted. Descriptor based loop closure triggering decides when this expensive global JCBB evaluation is necessary and when the lightweight local mode is sufficient.

Although many alternative DA frameworks exist, probabilistic methods like PDA/JIPDA, multi-hypothesis tracking, and mixture model formulations, they are too slow or fragile for real-time Side Scan Sonar SLAM (SSS SLAM) and are therefore only explored but not used in this thesis. The adopted pipeline instead couples gating, JCBB, and descriptor based loop closure detection into a unified scheme that in theory remains computationally efficient while still robust against false matches. This chapter outlines this DA strategy and motivates why these choices are best suited for reliable underwater SLAM.

10.2 Gating

Gating is the first filtering step in data association and removes landmark candidates that are statistically incompatible with a new measurement. For a predicted measurement $\hat{z} = h(\hat{x})$ with innovation covariance

$$S = HPH^\top + R \quad (61)$$

The idea is the same as in the “*State Estimation*” chapter, the comparison between z and $\hat{z}$ must be done in measurement space, so the state covariance P is pushed through the measurement Jacobian H to form the innovation covariance S , which describes how uncertainty looks after the nonlinear transformation $h(x)$ (see Equation (11)-(14)). The sensor noise R is added according to the noise model in (14), since the predicted measurement itself carries uncertainty. For the deterministic range bearing model (11) the Jacobian can be derived analytically, but the extended model (13) makes H too messy to write in closed form, so it is computed numerically by perturbing each state component slightly, recomputing $h(x)$, and using the resulting finite differences to assemble the Jacobian. This gives a clean and robust H even when the measurement model is complex.

a sensor reading z is only considered if it falls inside this uncertainty region. The check is done using the Mahalanobis distance

$$d^2 = \|z - \hat{z}\|_S^2 = (z - \hat{z})^\top S^{-1} (z - \hat{z}) \quad (62)$$

which measures the innovation in the units of its predicted noise rather than raw Euclidean space.

To understand why Mahalanobis distance is used, Figure 55 shows the key intuition. The left plot (Euclidean) treats all directions equally, creating a circular inlier region. The right plot (Mahalanobis) adapts to the covariance, high uncertainty directions are stretched, low uncertainty directions are tight, and correlations tilt the ellipse. This ellipse is exactly the gate used in DA and SLAM problem. A measurement must fall inside this ellipse to be considered a candidate, if it lies outside, it is rejected immediately.

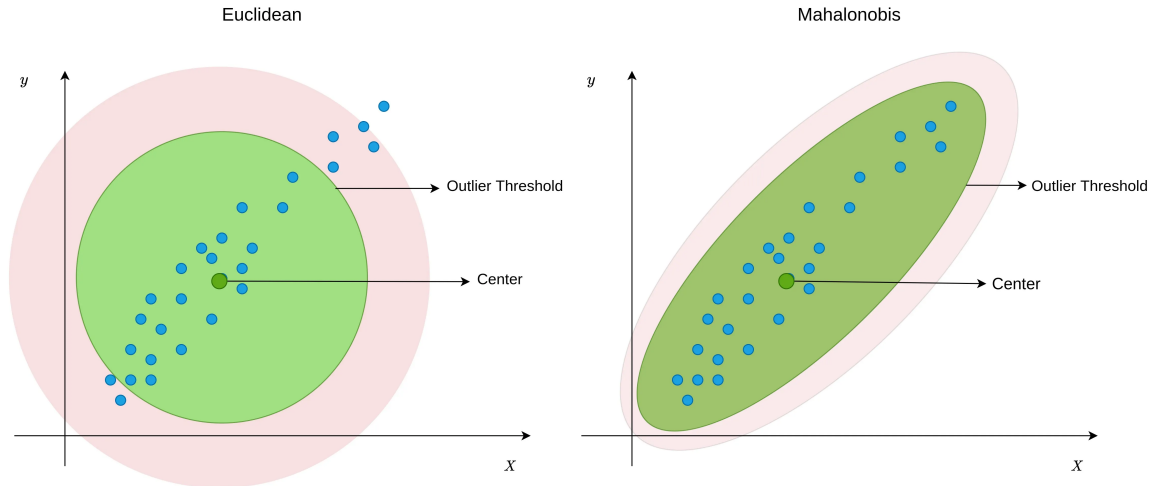


Figure 55: Euclidean vs Mahalanobis residual contours. Euclidean distance yields circular gates. Mahalanobis distance uses the covariance to stretch and tilt the inlier region into an ellipse aligned with the uncertainty.^[46]

The Mahalanobis distance d^2 provides a single scalar test of how well a measurement matches its prediction in the 2D range-bearing space. Because it directly incorporates the geometry of the covariance S , it measures the error in “units of expected noise” rather than raw Euclidean distance. Gating then becomes a simple rule, a candidate landmark is accepted only if $d^2 < \gamma$, where γ sets the size of the allowed uncertainty ellipse around the predicted measurement. A small γ means only very close (statistically strong) matches are accepted, a larger γ allows more uncertain ones.

Since this thesis assumes Gaussian measurement noise throughout the estimation pipeline, a correct innovation behaves like a normalised 2D Gaussian vector. Squaring each Gaussian component and summing them produces a random variable that follows a χ^2 distribution with 2 DOF. This is why the squared Mahalanobis distance naturally links to the χ^2 distribution, under the Gaussian assumption, d^2 directly expresses how likely the innovation is. The threshold is therefore chosen as $\gamma = \chi^2_2(p)$, where p is the desired confidence level taken from the Gaussian CDF (Cumulative Distribution Function). According to calculations in paper on multinormal distributions and chi squared [47] choosing $p = 0.95$ for 2 DOF yields $\gamma \approx 5.99$, meaning 95% of all correct 2D innovations should satisfy $d^2 < 5.99$, choosing $p = 0.99$ gives $\gamma \approx 9.21$. These values simply correspond to the radius of a 2D Gaussian confidence region expressed in Mahalanobis units. If $d^2 < \gamma$ the measurement is statistically consistent, and if $d^2 > \gamma$ it is rejected.

This thesis uses 2 separate gating modes depending on the situation. A wide “*global gate*” with a large γ is used when loop closure candidates are possible, allowing uncertain but potentially important matches to pass. In ordinary short range operation a much tighter “*local gate*” is used instead, aggressively pruning nearby candidates for computational efficiency. These 2 modes are applied independently depending on context, and the choice of which gate to use is handled later in the data association pipeline.

10.3 Nearest Neighbor

Once gating removes all statistically invalid candidates, the remaining step is simply to choose which of the surviving landmarks the measurement should be associated with. The Nearest Neighbor (NN) rule selects the candidate with the smallest Mahalanobis distance d^2 inside the gate. Formally, for the set of gated landmarks $\mathcal{L}$, NN chooses

$$j^* = \arg \min_{j \in \mathcal{L}} d_j^2$$

where d_j^2 is the Mahalanobis distance between the measurement and the predicted observation of landmark j .

This makes NN extremely fast and easy to implement, since it only requires evaluating the distance for each surviving landmark and selecting the smallest value. It commits to exactly one landmark, avoiding branching or combinatorial growth.

The drawback is that NN is brittle in cluttered environments, because it always picks the single closest candidate even if several lie at similar distances or if a spurious detection happens to be slightly closer. However, underwater seabed landmarks are sparse, well separated, and predominantly static, making NN a good starting algorithm for this thesis.

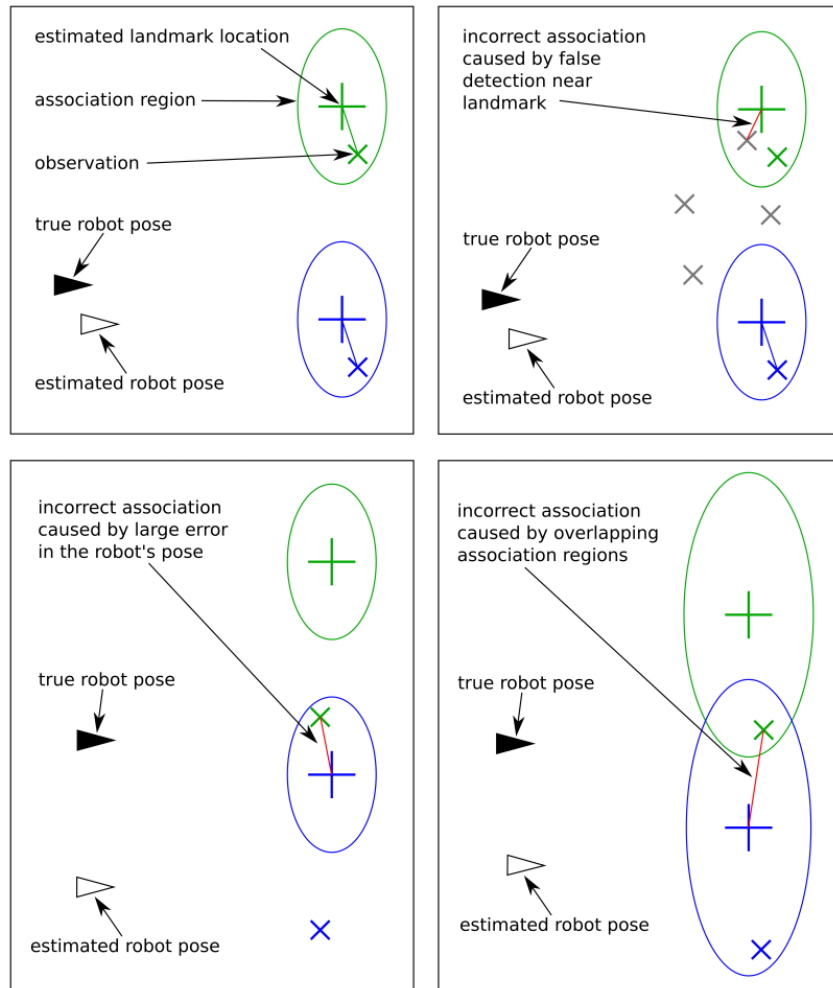


Figure 56: Examples of individual nearest-neighbor association. Each landmark (+) has an association region (ellipse), based on the uncertainty in that landmarks location and the noise in the observation model. Each observation (X) is associated to its nearest landmark, if the observation falls in that landmarks association region. If there are multiple observations falling in the same landmarks association region, the nearest is chosen. This strategy may choose incorrect associations if there are many false alarms (top right), if there is large pose uncertainty (bottom left), or if an observation falls in the association region of multiple landmarks (bottom right).^[48]

10.4 Probabilistic Methods

Probabilistic data association methods attempt to handle ambiguity explicitly by reasoning over multiple possible landmark matches rather than committing to a single nearest neighbour. They represent association uncertainty with probability distributions and update the SLAM system using weighted or multi hypothesis formulations. While powerful in theory, these methods become computationally heavy and often numerically fragile when applied online in larger SLAM problems with many landmarks. [14]

PDA (Probabilistic Data Association) distributes probability mass across all gated candidates instead of selecting a single landmark, updating the filter with a weighted combination of them. This soft assignment can handle mild ambiguity but becomes unreliable once many landmarks fall inside the gate, and its computational cost scales poorly without offering clear benefits for sparse seabed features.

JIPDA (Joint Integrated Probabilistic Data Association) combines the ideas from JPDA and IPDA, extending PDA by evaluating all measurement to landmark assignment combinations jointly rather than treating matches independently. This reduces the greediness of standard PDA, but it also requires a highly accurate probabilistic model of landmark geometry and measurement behaviour. For sonar based features, landmark shapes are often ambiguous, weakly defined, and far from Gaussian, making these likelihood models hard to specify correctly and easy to mis tune making DA to overconfident. In practice this leads to inconsistent or low quality associations, while the added mathematical and computational complexity offers little benefit for the SSS SLAM setting. These methods can work in controlled environments, but for seabed mapping they are simply not a suitable fit.

MHT (Multi Hypothesis Tracking) keeps several data association hypotheses alive in parallel and prunes weaker ones over time. It is considered the gold standard for ambiguous tracking problems, but its hypothesis tree grows exponentially with ambiguity, making it usable only offline or with extreme pruning. For SLAM it is typically far too slow for real time use.

Particle filter approaches such as FastSLAM embed data association within each particle so different hypotheses can coexist naturally. While attractive conceptually, SLAM particle filters are known to degrade due to sample depletion, degeneracy, and reliance on accurate proposal distributions. These weaknesses make them fragile for sonar SLAM where measurement noise is high and data arrival is dense.

Overall, although probabilistic methods offer richer representations of association uncertainty, they do not scale well and perform poorly in real time for the sparse but high throughput setting of SSS SLAM. For these reasons they are not adopted in this thesis, where gating combined with NN DA algorithm provides a simpler and more computationally efficient solution.

10.5 Mixture Models

Mixture model approaches handle data association uncertainty by allowing several competing hypotheses to coexist rather than forcing a single discrete match. Instead of selecting one landmark, the front-end of SLAM creates a mixture factor that encodes multiple possible associations, and each hypothesis is carried into the back-end of SLAM as part of a unified factor. The SLAM optimizer then resolves the ambiguity by selecting the hypothesis that forms the most globally consistent solution with the rest of the graph. This avoids premature front-end decisions and provides a theoretically elegant way to handle ambiguous or noisy measurements. [49]

A variety of methods exist within this family, including Max Mixture models and MMSAM (Mixture Model Smoothing and Mapping). These frameworks extend classical smoothing by embedding multiple measurement models inside each factor, allowing the optimizer to choose between them. Conceptually this is appealing, no association is thrown away early, ambiguity is preserved, and the final solution is determined by full nonlinear optimization rather than by a brittle nearest neighbour rule. For inherently uncertain sensing domains, such as sonar or radar, mixture models represent robust formulations of DA.

However, according to Haralstad master thesis [2], mixture models introduce a severe practical limitation, mixture factors destroy the sparsity structure required for efficient incremental solvers that modern optimizers in SLAM back-end use. Because each mixture factor couples several hypotheses, the factor graph becomes substantially denser, and incremental smoothing (iSAM type methods) can no longer operate efficiently. MMSAM and related mixture smoothing techniques therefore fall back to full batch smoothing methods (SAM), which must repeatedly rebuild and solve large linear systems. Even though each sonar 2D local map frame contains only a handful of landmarks, the repeated construction and elimination of dense mixture factors cause computational costs to grow rapidly, making online operation infeasible.

For these reasons, mixture model DA is not used in this thesis. While conceptually powerful and well suited for offline or post processing scenarios, mixture smoothing does not meet the real-time constraints of SSS SLAM.

10.6 RANSAC

RANSAC (Random Sample Consensus) is a robust model fitting algorithm designed to identify inliers within noisy or partially corrupted data. The method repeatedly samples small random subsets of measurements, fits a simple geometric model to each subset, and evaluates how many remaining points agree with that model within a tolerance. After many iterations, the model with the largest consensus set is selected, and all outliers are discarded. Because RANSAC optimizes agreement rather than likelihood, it remains reliable even when a large portion of the measurements are outliers.

In classical vision and LiDAR SLAM, RANSAC is widely used to filter correspondence pairs before estimating geometric transformations between frames. Examples include fitting a homography between two camera images or estimating a rigid body transformation between two point clouds. The algorithm ensures that only geometrically consistent matches influence the pose estimate, while spurious or mismatched features are rejected automatically.

For landmark based SLAM with sparse sonar detections, RANSAC is far less suitable. Side scan sonar landmarks are highly viewpoint dependent, affected by shadows and intensity distortions, and lack the stable geometric structure (corners, edges, planes) that camera or LiDAR features exhibit. This makes it difficult to define a reliable geometric model that RANSAC can fit. As a result, applying RANSAC at the landmark association level offers little robustness and often produces unstable results unless heavy preprocessing is used.

RANSAC remains a powerful tool for rigid geometric fitting, but due to the weak and ambiguous geometric structure of sonar landmarks, it is not used for data association in this thesis.

10.7 JCBB

10.7.1 Overview

Joint Compatibility Branch and Bound (JCBB) is a robust data association method designed for feature based SLAM systems, where a single wrong association can corrupt the entire map and cause catastrophic estimation failure. Unlike nearest neighbour approaches that evaluate each measurement to landmark match independently, JCBB evaluates entire sets of associations and accepts only those that are statistically compatible when considered jointly. This is essential in SLAM, because robot pose uncertainty induces strong correlations between all landmark predictions, even if each individual pairing lies inside its own Mahalanobis gate, the combination may still be geometrically inconsistent once correlations are accounted for. JCBB directly addresses this by enforcing that both individual compatibility and joint compatibility must hold before accepting any association set.

The core philosophy behind JCBB is as follows, a measurement sequence should only be associated with landmarks if the entire set of correspondences could realistically have been produced by the same robot pose under the SLAM covariance. This makes JCBB highly robust in ambiguous situations, especially when multiple landmarks fall inside overlapping covariance ellipses by rejecting any combination of matches that is inconsistent with the systems uncertainty geometry. JCBB uses a branch and bound search to efficiently explore the combinatorial hypothesis space, pruning branches as soon as joint or individual compatibility fails. Although the worst case complexity is exponential, branch and bound pruning and covariance structure make the algorithm practical for modest numbers of measurements, and therefore well suited for local SLAM updates in this thesis. [50]

10.7.2 Mathematical Preparation for JCBB

JCBB builds directly on the prediction and gating machinery already defined in the previous section on Gating (See equation (62)). For each landmark j , the predicted measurement

$$\hat{z}_j = h(\hat{x})$$

and the innovation

$$\nu_j = z - \hat{z}_j$$

are identical to those used for Mahalanobis gating. Likewise, the linearised measurement Jacobian H_j and the innovation covariance

$$S_j = H_j P H_j^\top + R$$

are exactly the same quantities introduced in the Gating subsection and need not be recomputed differently for JCBB (See Equation (61)). Individual compatibility is therefore unchanged:

$$d_j^2 = \nu_j^\top S_j^{-1} \nu_j < \gamma$$

where γ is the χ^2 threshold for a 2D measurement (See Equation (62)).

In addition, JCBB extends the gating test from individual compatibility tests, to entire association hypotheses. Here assume a batch of m measurements $z_1, \dots, z_m$, and that a hypothesis

$$a = \{(z_1 \leftrightarrow j_1), (z_2 \leftrightarrow j_2), \dots, (z_m \leftrightarrow j_m)\}$$

assigns measured landmark z_k to landmark j_m .

Then, the stacked innovation vector is formed simply by concatenating the individual innovations already defined in the Gating section (See Equation (62)):

$$\nu_a = \begin{bmatrix} \nu_{j_1} \\ \nu_{j_2} \\ \vdots \\ \nu_{j_m} \end{bmatrix} \quad (63)$$

Similarly, the stacked Jacobian is

$$J_a = \begin{bmatrix} H_{j_1} \\ H_{j_2} \\ \vdots \\ H_{j_m} \end{bmatrix} \quad (64)$$

where each H_{j_k} is the same per landmark Jacobian used in the standard gating test.

The block diagonal measurement noise is

$$R_a = \text{diag}(R, \dots, R)$$

again using the same noise model R defined earlier.

To compute joint compatibility we also require the full SLAM covariance P , which contains all pose and landmark uncertainties and their cross correlations. For the SLAM state

$$X = \begin{bmatrix} x \\ m_1 \\ \vdots \\ m_N \end{bmatrix} \quad (65)$$

where x is robot pose and m_N are landmarks poses, the covariance has the block structure

$$P = \begin{bmatrix} P_{xx} & P_{xL} \\ P_{Lx} & P_{LL} \end{bmatrix} \quad (66)$$

where P_{xx} is the robot pose covariance, P_{xL} are pose to landmark cross covariances, and P_{LL} contains all landmark to landmark covariances.

This is the same covariance used in the gating expression $S_j = H_j P H_j^\top + R$, but here it must be applied to the stacked hypothesis.

With these ingredients, the joint innovation covariance becomes

$$S_a = J_a P J_a^\top + R_a,$$

which is a direct extension of the gating covariance S_j to the stacked hypothesis.

In the end JCBB declares a hypothesis a statistically valid if gating conditions are met (See equation (62))

$$d_a^2 = \nu_a^\top S_a^{-1} \nu_a < \gamma \quad (67)$$

This ensures that all proposed associations in the hypothesis are mutually consistent under the full SLAM covariance P , not just individually inside their own gates.

10.7.3 Covariance Structure and Practical Extraction

JCBB relies on the full SLAM covariance P introduced earlier in (66), because the joint compatibility test must account for how pose uncertainty and landmark correlations interact. The pose block P_{xx} describes local pose uncertainty, the cross terms P_{xL} determine how pose errors map into each landmark prediction, and the landmark to landmark block P_{LL} captures how all landmarks are correlated through past robot motion. In gating, this covariance appears inside the per landmark expression $S_j = H_j P H_j^\top + R$. JCBB uses the same idea but applies it to the entire stacked hypothesis, requiring all blocks of P rather than only P_{xx} .

In classical EKF-SLAM the full covariance P is explicitly maintained, so accessing any sub-block is immediate. Modern graph based optimizers used in this project, however, do not store P directly. Instead, they keep uncertainty implicitly in an internal factorization. As a result the cost of extracting covariance blocks differs greatly, the pose covariance P_{xx} is cheap to obtain, but retrieving P_{xL} requires more computation, and obtaining P_{LL} is significantly slower. The details behind this behaviour are explained fully in the “*Optimizer*” chapter, but the key point for DA is simple, different blocks of P have very different extraction/computational costs.

10.7.4 Local vs Global JCBB Usage

Because of the different costs involved in retrieving the various covariance blocks, JCBB is applied with different levels of covariance information depending on the DA scenario. In short range, frame to frame operation the uncertainty is dominated by the robot pose, meaning that P_{xx} alone is sufficient for filtering and compatibility checks. Using only P_{xx} keeps the computation fast and lightweight, and in practice collapses the decision to a NN style algorithm outcome whenever only a single candidate survives gating. When several landmarks fall inside the gate, JCBB cannot evaluate mutual consistency without P_{xL} and P_{LL} and therefore reduces to selecting the candidate with the smallest individual Mahalanobis distance, again similar behaviors to NN algorithm. This behavior is acceptable locally because 2D local map sonar landmarks are sparse, well separated, and ambiguous local configurations are rare.

Global DA is different. When the global gate suggests that a loop closure may be present, the correlations between multiple landmarks become important, and JCBB must rely on the full covariance structure. In this regime the blocks P_{xL} and P_{LL} are required in addition to P_{xx} to evaluate true joint compatibility across all candidate associations. Extracting these blocks is significantly more expensive, so full JCBB is used only during loop closure attempts and never during routine short range mapping. The loop closure pipeline and its use of full covariance JCBB are described in detail in the subsequent chapter. The important point is that almost all DA steps run in the fast local mode, while full joint checks are required only during the relatively rare loop closure events, meaning the overall computational cost remains well balanced in theory.

10.7.5 JCBB Algorithm

With all compatibility machinery defined in the previous subsections, the JCBB algorithm itself is quite intuitive. The goal is simply to search over all feasible associations for a batch of measurements, but to prune impossible or inconsistent branches as early as possible using the individual and joint tests already introduced.

Assume m measurements $\{z_1, \dots, z_m\}$ are obtained at the same robot pose, and from the Gating stage each measurement landmark z_k has a (usually small) gated candidate set $\mathcal{C}_k = \{j_1, \dots, j_{n_k}\}$. JCBB processes the measurements sequentially from 1 to m and incrementally builds a hypothesis

$$a = \{(z_1 \leftrightarrow j_1), (z_2 \leftrightarrow j_2), \dots\}$$

At recursion depth k , JCBB tries assigning measurement z_k to:

- one of its gated landmarks $j \in \mathcal{C}_k$, or
- a null association (meaning z_k does not match any landmark)

For each attempted assignment, JCBB performs the following steps:

1. **Individual compatibility:** The assignment $(z_k \leftrightarrow j)$ must satisfy the same Mahalanobis gate $d_j^2 < \gamma$ defined earlier (see (62)). If it fails this test, the branch is discarded immediately.
2. **Partial joint assembly:** JCBB forms the partial stacked innovation vector ν_a (See Equation (63)) and partial stacked Jacobian J_a (See Equation (64))
3. **Joint compatibility:** Using the covariance blocks of P appropriate for the current DA mode (local or global), JCBB evaluates partial joint Mahalanobis consistency (See Equation (67)) If the partial hypothesis is already inconsistent, the entire branch is pruned.
4. **Branch bound pruning:** JCBB computes a simple bound on how many additional associations the remaining measurements could contribute. If even the best case completion cannot beat the current best hypothesis size, the branch is pruned without further exploration.

JCBB continues descending the tree, exploring only branches that remain individually and jointly compatible and that still have the potential to outperform the best hypothesis found so far. This pruning is extremely powerful, in practice the vast majority of branches are rejected early because joint compatibility is a strong global constraint, and even small inconsistencies in geometry or covariance structure immediately invalidate the hypothesis.

Once all measurements have been processed, JCBB returns the hypothesis containing the largest number of jointly compatible associations. If several hypotheses have the same size, JCBB selects the first consistent one encountered. The key result is that JCBB will always return the maximal jointly compatible set, if a consistent configuration exists under the SLAM covariance P , JCBB is guaranteed to find it.

This behaviour makes JCBB far more reliable than NN or gating alone, especially when multiple landmarks lie inside overlapping uncertainty ellipses. While NN commits immediately to the closest candidate and can be misled by local ambiguity, JCBB evaluates full association sets under the joint uncertainty model, providing a principled and highly robust DA mechanism suitable for both local updates and loop closure events in this thesis.

10.7.6 FastJCBB

Several accelerated variants of JCBB exist, most notably FastJCBB [51], which aim to reduce the computational cost of the joint compatibility checks. Classical JCBB must repeatedly assemble and invert the joint covariance S_a as a hypothesis grows, giving an $O(m^2)$ update cost for m measurements. FastJCBB exploits the conditional independence structure of measurements taken at the same pose to update the joint test incrementally, avoiding full recomputation and greatly reducing per step cost. Importantly, FastJCBB produces the exact same associations as classical JCBB, the improvement is purely computational, not algorithmic.

However, the exponential nature of the underlying branch and bound search remains. These faster variants perform well only when each measurement has a modest number of gated candidates. In the context of SSS SLAM this condition is naturally satisfied, local 2D sonar map frames typically contain only 1-5 usable landmarks per measurement, as reported in Haraldstads master thesis [2]. This makes JCBB and its fast variants practical and reliable for the small, sparse landmark sets encountered in underwater mapping.

10.7.7 Practical Use and Limitations

JCBB is most effective in exactly the situations where NN algorithm becomes unreliable, when pose uncertainty causes several landmark ellipses to overlap and multiple candidates pass the Mahalanobis gate. By evaluating full association sets under the SLAM covariance, JCBB is able to reject geometrically inconsistent combinations that NN would accept, providing strong protection against false matches that could corrupt the map. This robustness makes JCBB particularly valuable for the DA tasks in this thesis, where even a single incorrect association may undermine both local mapping and loop closure verification.

JCBB, however, is not a general purpose solution. Its search complexity grows exponentially with the number of measurements, and the covariance information it requires can be expensive to extract from modern incremental optimizers. It also inherits the limitations of linearised compatibility tests, meaning that large pose errors or badly predicted landmark geometry may cause all hypotheses to be rejected. For these reasons, JCBB is applied with different covariance detail rather than being turned on or off. In this thesis JCBB is always used, but in two modes, a fast local mode that uses only P_{xx} and therefore effectively collapses to a NN style outcome when several landmarks lie inside the gate, and a full global mode that incorporates P_{xL} and P_{LL} during loop closure checks. In ordinary short range mapping this makes JCBB lightweight and efficient, while during loop closure it provides the full robustness of joint compatibility when it matters most. This in theory keeps the front end fast in routine operation while still ensuring strong protection against false associations in the rare but critical global events.

10.8 Loop Closure and Matching

10.8.1 The Problem of Loop Closure

Running full global JCBB every frame is infeasible because its complexity and the required covariance blocks P_{xx}, P_{xL}, P_{LL} would overwhelm the SLAM budget. The system therefore separates “local” DA (frame to frame) from “global” DA (loop closure). Both begin with gating, local gating is tight and yields only a few candidates, while global gating is wider and produces many, requiring full joint compatibility. JCBB then runs either in lightweight form using only P_{xx} for local DA or in full form using P_{xx}, P_{xL}, P_{LL} during loop closure. Since loop closures are rare, global JCBB is triggered only occasionally and does not break real-time performance.

The remaining question is simply how the system knows when a loop closure might be happening. Global JCBB cannot be used to detect this, because that would defeat the whole purpose of avoiding its heavy cost. Instead the system needs a fast and approximate hint that the robot might be revisiting an old area, here a lightweight descriptor based matching algorithm is chosen as each landmark has its own descriptor already attached.

10.8.2 Descriptor Based Loop Closure Matching

Before any global DA can be attempted, the system compares the newly observed sonar landmarks against stored descriptors from older parts of the map. Each 2D sonar landmark is assigned both weak and strong descriptors. Weak descriptors are used first with very relaxed thresholds, pruning only the largest outliers. Strong descriptors are used next to filter the result down to a small number of plausible candidates. It is important to note that this step does not determine the data association, it merely determines whether a loop closure might exist. If no descriptors match at all, then the landmarks are judged unique enough that only local DA is required. If descriptor similarity rises above a threshold, then a loop closure is possible, and global DA is triggered.

Descriptor similarity is computed using nearest neighbour search in descriptor space (not the geometric NN used earlier for DA). This search is implemented with a KD-tree backend, with FLANN providing fast comparison across the stored landmark descriptors. Matching returns a similarity score, if one match stands out clearly above the rest or exceeds a fixed threshold, the system flags a loop closure candidate. A periodic fallback trigger (eks every 50 frames) is also used as a safety net to avoid missing loop closures that generate weak descriptor responses.

10.8.3 Local vs Global Loop Closure Decisions

To avoid meaningless self matching, the system maintains a short sliding window of recent submaps (eks the last 5 frames). These are excluded from global matching, since they belong to the current local area and should be handled only by local DA. The loop closure search is therefore always performed against map regions older than this window. With this rule, the loop closure decision becomes:

1. Compute descriptor similarity between the new sonar frame and all older submaps beyond the local window
2. If similarity exceeds the threshold, trigger global gating followed by full JCBB
3. Otherwise, perform local gating and run JCBB in local mode with only P_{xx}

This design ensures that global DA runs only when necessary, keeping the system real time while still catching true loop closures reliably.

10.8.4 Landmark Revalidation

When an old landmark is successfully reobserved and verified by global JCBB, it is revalidated and treated as a fresh observation. Its “age” in the map is reset so that it again participates in local DA, ensuring that long term drift does not push repeated landmarks out of the local mapping window.

This combination, fast descriptor matching, selective global DA, and revalidation, provides in theory a more practical and efficient loop closure mechanism while maintaining the robustness guarantees of full JCBB exactly where it matters.

10.9 Integrated Data Association Framework for SLAM

The complete DA pipeline combines local JCBB, global JCBB, descriptor based loop closure triggering, and landmark aging into a single unified framework. Each new scan follows the same sequence of operations, switching between local and global modes depending on whether a loop closure is likely.

10.9.1 Processing Each New Scan

For every incoming scan, the system first determines whether loop closure is plausible. This decision is made before any geometric DA step:

- Perform descriptor similarity checks using a FLANN Matcher on all submaps older than the local horizon N . Young submaps are ignored to avoid trivially matching against very recent poses
- A periodic fallback trigger (eks every 50 scans) is evaluated to guard against missed opportunities

If any descriptor match exceeds a predefined threshold or stands out strongly from all others, the system marks the scan as a loop closure candidate.

10.9.2 Local vs Global DA Branch

Once the trigger phase is complete, the DA mode is selected:

- **If loop closure is suspected:** The system switches into global mode. A wide “*global gating*” step is applied to the older map region, yielding a larger and more ambiguous candidate set. JCBB is then run using the full covariance structure P_{xx}, P_{xL}, P_{LL} to verify the loop closure. This step is expensive, but it is executed rarely
- **Otherwise (default case):** Local DA is performed. A tight “*local gate*” is applied to the most recent N submaps, producing only a few spatially separated candidates. JCBB then runs using only P_{xx} , which is cheap to extract and behaves similarly to an NN method when only a single candidate survives. This mode dominates runtime and ensures real-time performance

10.9.3 Landmark Aging

Landmark management is handled via a simple aging mechanism:

- Newly observed landmarks begin as “young” and remain inside the local window for N scans
- Each scan increments the age of all young landmarks
- Once a landmark exceeds age N , it becomes an “old” landmark and joins the pool considered for loop closure triggering
- Old landmarks do not need further aging, they remain in the global set indefinitely

10.9.4 Landmark Revalidation

If an old landmark is successfully matched and passes JCBB during loop closure verification, it is revalidated:

- Its age is reset to zero
- It re-enters the local map as a young landmark

This prevents the system from repeatedly treating the same valid landmark as an old loop closure candidate and improves long term consistency.

10.9.5 Summary of the Full Pipeline

Putting everything together, each scan proceeds as follows:

1. **Loop closure trigger:** Evaluate descriptor similarity (FLANN) + periodic check on all submaps older than N
2. **If triggered:** Run global gating and full JCBB using P_{xx}, P_{xL}, P_{LL} to verify loop closure
3. **If not triggered:** Run local gating and lightweight JCBB using only P_{xx} on recent submaps
4. **Landmark aging:** Age all young landmarks, those exceeding N join the loop closure candidate pool
5. **Landmark revalidation:** Any old landmark confirmed by JCBB becomes young again

10.10 Factor Construction

Whenever data association confirms that a landmark has been observed, the front end updates the factor graph. IMU preintegration is paused, a new pose node is created, and the accumulated IMU factor is added between the new and previous poses. If the landmark is new, the system initializes a fresh landmark node using the current measurement as its prior estimate, which makes the initial factor residual effectively zero at insertion. If the landmark already exists, a new measurement factor is added between the current pose and that landmark, tightening its position estimate. Loop closures follow the same mechanism but link the current pose to older landmarks, enforcing global consistency and correcting long term drift.

All landmark updates, new landmarks, re-observations, and loop closures, use the same extended range bearing residual model. The measurement noise is assumed zero mean Gaussian,

$$\mathbf{n}^i \sim \mathcal{N}(\mathbf{0}, \mathbf{R}_{z^i})$$

with $\mathbf{R}_{z^i}$ given by the range dependent covariance in (14). Under this model, the factor graph residual becomes

$$r^i = h_{\text{corr}}(\mathbf{x}, m^i) - z_k$$

where $h_{\text{corr}}(\mathbf{x}, m^i)$ is the extended range bearing measurement function from (13), incorporating incidence angle corrections and distance dependent noise characteristics, and z_k is the observed landmark measurement. This extended model is used throughout the thesis, while the simpler deterministic form in (11) may be used for debugging and testing. These noisy residuals are inserted directly into backend optimizer of SLAM as standard nonlinear factors, enabling consistent incremental optimization as new scans arrive.

11 Optimizers

11.1 Introduction

Optimizers are the engine behind the SLAM update step. Sensors add constraints, but the optimizer decides how the state moves to satisfy them. In practice, the problem is posed as a Maximum A Posteriori (MAP) problem, seeking the most likely trajectory and landmarks given all measurements and priors. With standard assumptions such as Gaussian noise and first order linearization, the MAP problem becomes a nonlinear least squares problem solved iteratively. At each iteration, the residuals are linearized around the current estimate and an increment that improves the state is computed. This “correction” is what lets SLAM reduce drift, enforce loop closures, and keep the map consistent. [52]

There are two dominant families for doing state estimation in SLAM, filtering and smoothing. Filtering methods, like EKF-SLAM (Extended Kalman Filter) and particle-filter SLAM (FastSLAM), maintain a rolling belief over the current state only (or a short window). They update online/live as measurements arrive by propagating the state and compressing all past information into the filters covariance or a set of weighted particles. This is simple and has low memory, but it throws away structure in old constraints and it can become inconsistent after many linearizations, especially when revisiting places (loop closures) or when correlations span long time intervals. Particle filters can represent multi modal beliefs but scale poorly with dimension and often need heavy resampling and clever proposal distributions to avoid degeneracy, something that is difficult to achieve in practice. [53][54]

Smoothing methods keep a dense record of the variables we care about (eks, the whole robot trajectory and, if needed, landmarks) and all the measurement factors that tie them together. Instead of only “where am I now?”, smoothing asks “what is the entire trajectory and map that best fits everything we have ever seen?”. This global view tends to produce better accuracy and consistency, especially when closing loops or fusing many asynchronous sensors. Computationally, smoothing exposes sparse structure, meaning each measurement only touches a few variables, so the global normal equations are large but very sparse. Modern linear algebra plus careful data structures exploit that sparsity and outperform classical filters on realistic SLAM workloads. That is why smoothing has become the predominant approach in modern SLAM systems.

An estimate of the unknowns is maintained, denoted θ (robot poses, and landmarks). Each measurement yields a residual that indicates how far the current estimate is from the expected sensor reading. Close to the current estimate, a small nudge in θ gives an approximate change in the residuals. This is a first order (linear) approximation. Stacking all residuals together, that approximation looks like this [55]:

$$r(\theta + \Delta\theta) \approx A\Delta\theta - b$$

Here A is the Jacobian, it describes how each residual changes with each variable. The vector b is the residual at the current estimate (with a sign convention so the equation above points toward reducing error). The small vector $\Delta\theta$ is the “correction” to be computed. [55]

The update step chooses $\Delta\theta$ that reduces all residuals as much as possible. The best practice here is to use MAP approach. In this section, measurement noise is assumed Gaussian (this is an approximation, not always true, and non Gaussian cases are discussed later). After linearizing the residuals around the current estimate, the MAP problem becomes a least-squares fit [55]:

$$\Delta\theta^* = \arg \min_{\Delta\theta} \|A\Delta\theta - b\|^2 \quad (68)$$

Optimize this estimate so that change in $\Delta\theta^*$ is equal to 0, ie linearize. When linearized, equation (68) can be simplified to a so called normal equation [55]:

$$A^T A \Delta\theta = A^T b$$

This equation system can be then be solved by Cholesky decomposition of $A^T A$ or by optimization algorithms that will be discussing down below. Solve this linear system for $\Delta\theta$, then update/correct the estimate [55]:

$$\theta = \theta + \Delta\theta$$

For stability on harder problems Levenberg-Marquardt damping can be added, however the core idea stays the same across these optimizer algorithms. [35]

Classical batch smoothing forms the full information matrix, eliminates variables in a chosen order, and solves for all states together. That is accurate but not ideal for online use. Every new measurement would, in principle, require rebuilding and refactoring a large system, with cost growing with mission length. Real robots need real-time behavior, so iterative methods are preferred, incremental smoothing that reuses previous computation. The idea is to keep the factorization of the linearized problem in a data structure that can be updated locally when new factors arrive, only touching the parts of the graph that actually change.

This is where Iterative Smoothing and Mapping (iSAM and iSAM2) methods come in. They exploit that SLAM data are very sparse and mostly locally connected, a new odometry or measurement links a pose to a neighbor pose or a nearby landmark, not to everything. iSAM maintains a square root factor (via QR) and updates it incrementally using Givens rotations, with occasional reordering to control fill in. It keeps uncertainty queries fast and avoids full resolves, except when needed. iSAM2 goes further by expointing factor graphs and organizing the factorization into a Bayes tree data type (a directed tree of cliques). On new measurements, only the impacted cliques are relinearized and refactored, and variables are reordered incrementally. As a result, work scales with the local update rather than the entire graph, this makes update step “fluid”.

In modern SLAM the hard part isn’t “doing SLAM”, it’s solving the SLAM optimization fast as data grows. Most methods use the same MAP correction loop. Linearize, solve for $\Delta\theta^*$, update θ . The real difference lies in how the problem is represented and updated. Smart data structures and good variable ordering keep data structures sparse and decoupled, and solves quick. Meanwhile bad data structure representation of data causes slowdowns.

This is exactly why, for SLAM on marine vessels, especially ASVs with tight space, power, and compute budgets but strict real-time needs, iterative smoothing methods like iSAM2 are a strong fit. They reuse prior factorizations, add new measurements as local factors, relinearize and refactor only the affected cliques, and reorder variables incrementally. In practice that means low latency, bounded memory and CPU load, and accuracy close to batch solutions, even on long missions.

11.2 iSAM

11.2.1 Getting to SLAM update step

Before computing a good estimate, defining simple models for robot motion and sensor observations is crucial. The motion model describes state evolution, and the measurement model describes sensor readings. States are x_i for robot poses, controls are u_i , and measurements are z_k for landmarks. Stack all unknowns into θ , poses and landmarks.

Motion (process) model:

$$\begin{aligned} x_i &= f_i(x_{i-1}, u_i) + w_i \\ w_i &\sim N(0, Q_i) \end{aligned}$$

Given the previous state x_{i-1} and control u_i , the next state x_i comes from a model f plus uncertainty noise in the model itself w_i . This uncertainty captures things like currents, slip, and actuator errors. f_i can be a discrete time dynamics update or use plain odometry. Assuming Gaussian w_i is a handy start so MAP becomes least squares. Later this uncertainty model can be switched to robust or heavy tailed noise model if needed.

Measurement model:

$$\begin{aligned} z_k &= h_k(x_{i_k}, l_{j_k}) + v_k \\ v_k &\sim N(0, R_k) \end{aligned}$$

Each measurement z_k depends on state x_{i_k} and landmarks l_{j_k} transformed using measurement transform function $h_k(\cdot)$, this allows state estimate to become estimated measurement position. In addition this measurement has noise v_k which is modeled as Gaussian noise for simplifications later on when calculating.

Prior:

$$x_0 \sim N(\mu_0, \Sigma_0)$$

A prior anchors the graph (otherwise the problem is underdetermined up to a global transform). It can encode GPS at the start, a known dock pose, or simply a weak “zero” prior to fix gauge.

Predictions should match measurements. In a perfect world, every residual (prediction minus measurement) would be zero. In practice, model errors and sensor noise make the residuals nonzero. Estimation is about choosing the state update that makes all residuals as small and as statistically consistent as possible.

This is where MAP algorithm comes in. MAP (Maximum A Posteriori) is the principled way to fuse everything we know. A prior on the state, the motion model, and all measurements. It combines them through probability, weighting each residual by its uncertainty. With Gaussian noise, the negative log posterior becomes a sum of squared (weighted) residuals. That gives us a single objective to minimize, where more reliable terms (small covariance) count more. This is better than ad hoc weighting and naturally handles many sensors.

Motion and measurement functions are nonlinear (angles, rotations, ranges). Minimizing the nonlinear MAP cost directly is hard. Linearization lets us solve it iteratively. At the current estimate approximate the nonlinear functions by their first order Taylor expansion, solve a linear least squares problem for a small increment, update the estimate, and repeat. This is all shown in the iSAM paper [55] where linearized forms of the system becomes:

$$\begin{aligned} f_i(x_{i-1}, u_i) - x_i &\approx (F_i^{i-1} \Delta x_{i-1} - \Delta x_i) - a_i \\ F_i^{i-1} &:= \left. \frac{\partial f_i(x_{i-1}, u_i)}{\partial x_{i-1}} \right|_{x_{i-1}^0} \\ a_i &= x_{i-1}^0 - f_i(x_{i-1}^0, u_i) \end{aligned} \tag{69}$$

$$\begin{aligned}
h_k(x_{i-1}, u_i) - z_k &\approx (H_k^{i_k} \Delta x_{i_k} - J_k^{j_k} \Delta l_{j_k}) - c_k \\
H_k^{i_k} &:= \left. \frac{\partial h_k(x_{i_k}, l_{j_k})}{\partial x_{i_k}} \right|_{(x_{i_k}^0, l_{j_k}^0)} \\
J_k^{j_k} &:= \left. \frac{\partial h_k(x_{i_k}, l_{j_k})}{\partial l_{j_k}} \right|_{(x_{i_k}^0, l_{j_k}^0)} \\
c_k &= z_k - h_k(x_{i_k}^0, l_{j_k}^0)
\end{aligned} \tag{70}$$

Plug the linearized odometry (69) and measurement (70) models into a single objective over the stacked increment vector $\Delta\theta$ (all pose and landmark updates). The goal is to pick the small change $\Delta\theta^*$ that jointly reduces all linearized residuals. Each factor becomes a linear row in the relevant increments.

For an odometry factor i , the linearized residual is

$$r_i^{\text{odo}} = F_i^{i-1} \Delta x_{i-1} + G_i^i \Delta x_i - a_i,$$

where F and G are the odometry Jacobians. Because odometry constrains the relative change between x_{i-1} and x_i , the block on Δx_i is $-I$ ($G_i^i = -I$). Here a_i is the current odometry prediction error.

For a measurement factor k connecting pose x_{i_k} to landmark l_{j_k} , the residual is

$$r_k^{\text{meas}} = H_k^{i_k} \Delta x_{i_k} + J_k^{j_k} \Delta l_{j_k} - c_k,$$

with H and J the measurement Jacobians with respect to the involved pose and landmark, and c_k the corresponding prediction error.

Each residual is measured with a Mahalanobis norm $\|r\|_\Sigma^2 := r^\top \Sigma^{-1} r$, using its own covariance, Λ_i for odometry and Γ_k for measurements. This matters because Mahalanobis distance “bakes in” uncertainty. Directions the sensor is confident about are penalized more. Noisy or correlated directions are penalized less, and the metric tilts along correlated axes. As a result, the errors are not judged in plain Euclidean meters/radians but in “standard-deviation units” tailored to each factor. Intuitively, this turns “Euclidean space + covariance” into Mahalanobis space, where the residual ellipses already encode the right weighting. That is why the covariance symbols appear inside the cost, uncertainty is not ignored, it’s embedded in how distance is measured (see Figure 55 and Equation (62) in the Data Association chapter for intuitive geometric explanation on Mahalanobis distance). With this, the whole objective of equation (71) is just “add up all these linearized residuals, each judged fairly in its own noise units, and pick the $\Delta\theta^*$ that makes the total smallest”. Intuitively, factor can be visualized as spring pulling on the variables. Mahalanobis scaling makes the springs stiff along low noise directions and soft along high noise ones, so the solution balances all pulls by their reliability.

Collecting all linearized factors with their covariances, the MAP update $\Delta\theta^*$ is obtained by minimizing the following Mahalanobis-weighted least-squares objective:

$$\Delta\theta^* = \arg \min_{\Delta\theta} \left\{ \sum_{i=1}^M \|F_i^{i-1} \Delta x_{i-1} + G_i^i \Delta x_i - a_i\|_{\Lambda_i}^2 + \sum_{k=1}^K \|H_k^{i_k} \Delta x_{i_k} + J_k^{j_k} \Delta l_{j_k} - c_k\|_{\Gamma_k}^2 \right\} \tag{71}$$

Here the Mahalanobis term is only used as a weighting rule inside the optimizer, every residual is scaled by its own covariance, nothing more. In the iSAM cost it simply acts as a noise aware weight that tells the optimizer how strongly each factor should pull on the solution. (see Figure 55 and Equation (62) in the Data Association chapter for intuitive geometric explanation on Mahalanobis distance)

Equation (71) is a sum of Mahalanobis residuals (process terms use Λ_i , measurement terms use Γ_k). To turn that into one clean least squares system, first step is to “whiten” each residual so its noise is unit, for scalars divide by the standard deviation, for vectors apply the covariance’s square root inverse to the residual and its Jacobians Σ^{-1} . After whitening, all errors are ordinary Euclidean ones, so the covariance symbols can be dropped, stack the Jacobians into one big sparse matrix A , stack the prediction errors into b , and solve the standard least squares problem (72).

$$\Delta\theta^* = \arg \min_{\Delta\theta} \|A\Delta\theta - b\|^2 \tag{72}$$

Here, θ stacks all unknowns (robot poses x and landmarks l), A is the single large, sparse (whitened) measurement Jacobian formed by stacking the block Jacobians F, G, H , and J from the linearized motion and measurement models, and b is the stacked prediction error vector that collects the current odometry errors a and measurement errors c with a consistent sign convention. Intuitively, A describes how residuals change for small state perturbations, b encodes the present mismatch between predictions and measurements, and solving equation (72) yields the best local correction $\Delta\theta^*$ used to update the estimate.

In the linearized setting, the optimal increment $\Delta\theta^*$ is found by setting the gradient of the least squares objective to zero. This yields the normal equations according to iSAM paper [55]:

$$A^T A \Delta\theta = A^T b$$

Solving this system is typically performed using a numerically stable square root method (QR/Cholesky) rather than forming an explicit inverse. This gives the optimal correction $\Delta\theta^*$. The state estimate is then updated as follows:

$$\theta \leftarrow \theta + \Delta\theta^*$$

11.2.2 Incremental QR for fast updates (iSAM)

The linearized SLAM subproblem is solved by least squares. Solving the normal equations $(A^T A)\Delta\theta = A^T b$ with Cholesky can be fast but very unstable and ill conditioned as the problem grows (it squares the condition number and increases fill in). iSAM avoids this by working directly with the whitened Jacobian A using QR factorization, and by updating that factorization incrementally when new factors arrive.

Batch square root form (QR on the Jacobian) can be shown in iSAM paper [55] to be of form:

$$A = Q \begin{bmatrix} R \\ 0 \end{bmatrix}, \quad Q^T Q = I, \quad R: \text{upper triangular}$$

$$\begin{bmatrix} d \\ e \end{bmatrix} = Q^T b$$

$$\|A\Delta\theta - b\|^2 = \|R\Delta\theta - d\|^2 + \|e\|^2$$

The iSAM paper [55] shows that after QR the equation is:

$$A\Delta\theta - b = \begin{bmatrix} R \\ 0 \end{bmatrix} \Delta\theta - \begin{bmatrix} d \\ e \end{bmatrix}, \quad \Rightarrow \quad \|A\Delta\theta - b\|^2 = \|R\Delta\theta - d\|^2 + \|e\|^2.$$

Put simply, once QR factorization is performed, the error splits into two parts. To make the total error as small as possible, set the first term to zero and solve:

$$R\Delta\theta^* = d \tag{73}$$

leaving $\|e\|^2$ as the (minimal) residual norm. If R has full rank, this linearized system has one singular unique solution $\Delta\theta^*$.

In iSAM the matrix R is upper triangular, so equation (73) is solved by back substitution (no matrix inverse). This gives a fast, numerically stable way to compute the correction and update the state $\theta \leftarrow \theta + \Delta\theta^*$ without heavy compute.

11.2.3 What is R? The square root information matrix

At the end of QR, the triangular factor R satisfies the following form:

$$R^T R = A^T A.$$

This means $A^T A$ (the information matrix obtained by linearization) is represented by the “square root” R . Working with R keeps all the curvature of the problem but in a form that is easier to use and numerically safer because R is upper triangular, so computations reduce to cheap substitution methods instead of expensive matrix inverses. Uncertainty can also be extracted directly from R . The state covariance is given by:

$$\Sigma = (A^T A)^{-1} = (R^T R)^{-1},$$

A dense inverse is never built. In reality, when entries of the uncertainty Σ are needed, solve small triangular systems with $R^\top$ and R , and read only the pose blocks and the pose to landmark blocks of interest. Since R is sparse and triangular, this is fast and stable, and it avoids forming $A^\top A$. (See 11.2.8 Data Association from R)

11.2.4 Matrix Factorization for building QR (Givens rotations)

Here *Givens rotations* is used to build an upper triangular factor R from the (whitened) Jacobian A by zeroing entries below the diagonal, one at a time. This yields a QR factorization without forming $A^\top A$ and without explicitly storing Q .

A Givens rotation is a 2×2 orthogonal transform applied to two rows (or two columns) to annihilate one chosen entry. Givens rotation matrix is defined as:

$$G(\varphi) = \begin{bmatrix} \cos \varphi & \sin \varphi \\ -\sin \varphi & \cos \varphi \end{bmatrix} \quad (74)$$

Start at the leftmost non zero column of the A matrix and sweep to the right, one column at a time. In each column, pick two rows, “ k ” (the current pivot row) and “ i ” (a row below it), and apply the small “rotate and combine” equation (74) so the entry under the diagonal in that column becomes zero. Only those two rows are mixed, the new row “ k ” becomes a bit of the old row “ k ” plus a bit of row “ i ”, and the new row “ i ” becomes a bit of the old row “ i ” minus a bit of row “ k ”. Repeat down the column until all subdiagonal entries are gone, then move to the next column on the right. (see Figure 57 down bellow for a visual of one Givens step)

As the algorithm sweeps the columns of A , the matrix is transformed into the upper triangular form, this is R , and the full Q doesn’t need to be formed to get to the result. Apply the same row rotations to b as you eliminate entries so the right hand side stays consistent. After the initial factorization of A matrix, new measurements don’t require rebuilding A . Later it is shown that new whitened rows can be appended beneath the current R , then a short sequence of row rotations re-triangularizes R . In other words, updates operate directly on R and b , and A is bypassed for incremental steps.

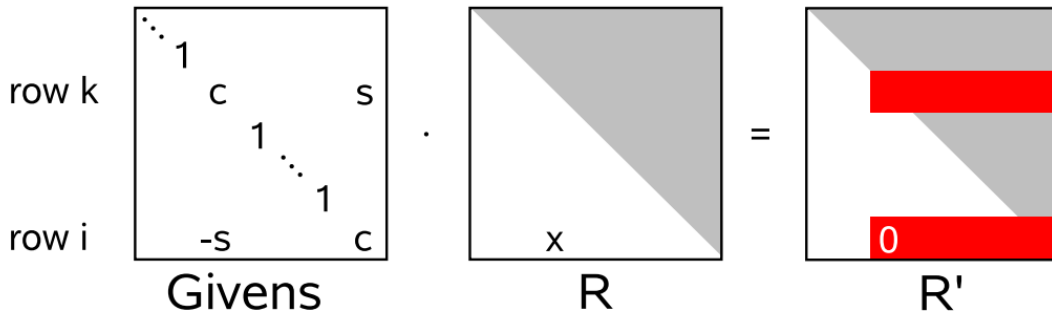


Figure 57: One Givens step in QR. The entry marked “ x ” is eliminated by rotating two rows, only the entries shown in red are modified, and the exact pattern depends on sparsity. Repeating this column wise (left to right) turns the matrix into an upper triangular R . Apply the same rotation to the b vector to keep the least squares system consistent.^[55]

In order to make R upper triangular, φ value must be chosen precisely to zero out a single sub diagonal entry in preliminary matrix, either be it A matrix on batch step or R matrix on iterative steps. The rotation angle φ is computed from the two entries in the current column, the pivot $x = a_{kk}$ and the subdiagonal $y = a_{ik}$.

$$\begin{aligned} r &= \sqrt{x^2 + y^2} = \sqrt{a_{kk}^2 + a_{ik}^2} \\ c &= \cos \varphi = \frac{x}{r} = \frac{a_{kk}}{r} \\ s &= \sin \varphi = \frac{y}{r} = \frac{a_{ik}}{r} \end{aligned}$$

Solving for φ gives the following answer, where $\alpha = x = a_{kk}$ and $\beta = y = a_{ik}$:

$$(\cos \varphi, \sin \varphi) = \begin{cases} (1, 0), & \text{if } \beta = 0, \\ \left(-\frac{\alpha}{\beta} \frac{1}{\sqrt{1 + (\alpha/\beta)^2}}, \frac{1}{\sqrt{1 + (\alpha/\beta)^2}} \right), & \text{if } |\beta| > |\alpha|, \\ \left(\frac{1}{\sqrt{1 + (\beta/\alpha)^2}}, -\frac{\beta}{\alpha} \frac{1}{\sqrt{1 + (\beta/\alpha)^2}} \right), & \text{otherwise.} \end{cases} \quad \text{with } \alpha := a_{kk}, \beta := a_{ik}. \quad (75)$$

These coefficients in equation (75) give the same rotation as (74).

Givens rotations guarantee that the (i, k) entry of the working matrix becomes zero, and they preserve lengths. First the two numbers $[x, y]^\top$ are rotated. When embedded in the full matrix, the same rotation is applied to the affected parts of the two rows and to the matching entries of b . In practice, embed $G_{(i,k)}(\varphi)$ so it acts only on rows k and i , and apply the same rotation to b to keep the least squares system consistent.

11.2.5 Incremental Updating

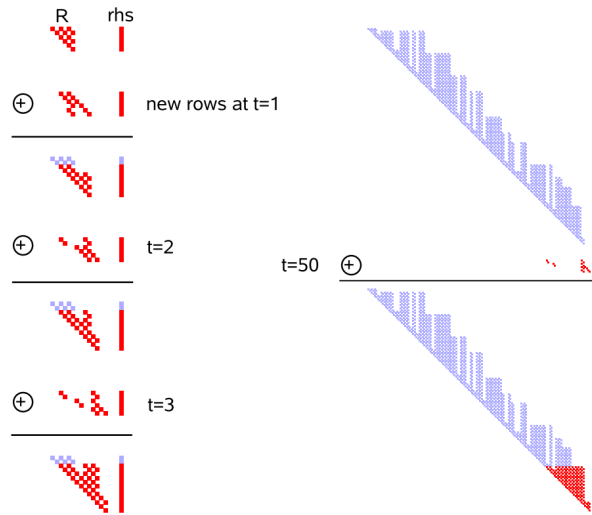


Figure 58: Incremental update of the factored system. A new whitened row $w^\top$ and RHS (Right Hand Side) entry γ are appended beneath the current R and d . A short sequence of Givens rotations restores the upper triangular form, yielding updated R' and d' . Unchanged entries are shown in light color, only a small stencil is touched each step, so update cost stays bounded.^[55]

After the initial QR factorization, maintain the solution in “square root” form, an upper triangular matrix R and a transformed right hand side d . Here, R is the triangular factor that satisfies $R^\top R = A^\top A$ (the Gauss Newton information), and d is the top part of $Q^\top b$. When a new measurement arrives, first whiten it (divide by its standard deviation or apply the square root information of its covariance) so it has unit variance. The whitened measurement contributes a new row $w^\top$ to the Jacobian and a new scalar γ to the RHS (Right Hand Side). Notice that A is NOT rebuild. Instead, append $w^\top$ under the current R , and γ under the current d , which produces a system that is “almost” triangular but has one non triangular row at the bottom.

$$R' = \begin{bmatrix} R \\ w^\top \end{bmatrix}, \quad d' = \begin{bmatrix} d \\ \gamma \end{bmatrix}$$

Next, re-triangularize locally with Givens rotations (74). Only touching the columns where the new whitened Jacobian row $w^\top$ has nonzeros (i.e., the variables that this new factor actually connects to, such as a pose x_i or a landmark l_j). Starting from the leftmost such column, each rotation mixes the current pivot row with the new bottom row to kill one sub diagonal entry. Repeat the process until the entire bottom row is zero and the matrix is upper triangular again. The equation would look something like this:

$$\begin{bmatrix} R \\ w^\top \end{bmatrix} \xrightarrow{\text{Givens rotation on affected columns}} \begin{bmatrix} R' \\ 0 \end{bmatrix}$$

While the matrix is rotates, apply the same rotations to the right hand side so that the least squares system stays consistent. Here d is the transformed RHS (Right Hand Side) before the update and γ is the new whitened RHS entry that pairs with $w^\top$. After the rotations, the top block becomes the updated RHS d' used for solving, and the final bottom entry becomes a small leftover error e_{new} that adds to the total residual.

$$\begin{bmatrix} d \\ \gamma \end{bmatrix} \xrightarrow{\text{same rotations}} \begin{bmatrix} d' \\ e_{\text{new}} \end{bmatrix}$$

Intuitively, the new row $w^\top$ is “folded up” into the triangular structure by a short chain of 2x2 rotations that only touch the connected variables, everything else is left alone. Then get the correction by a fast back substitution on the updated matrix R' and vector d' :

$$R' \Delta \theta^* = d'$$

11.2.6 Loop Closure

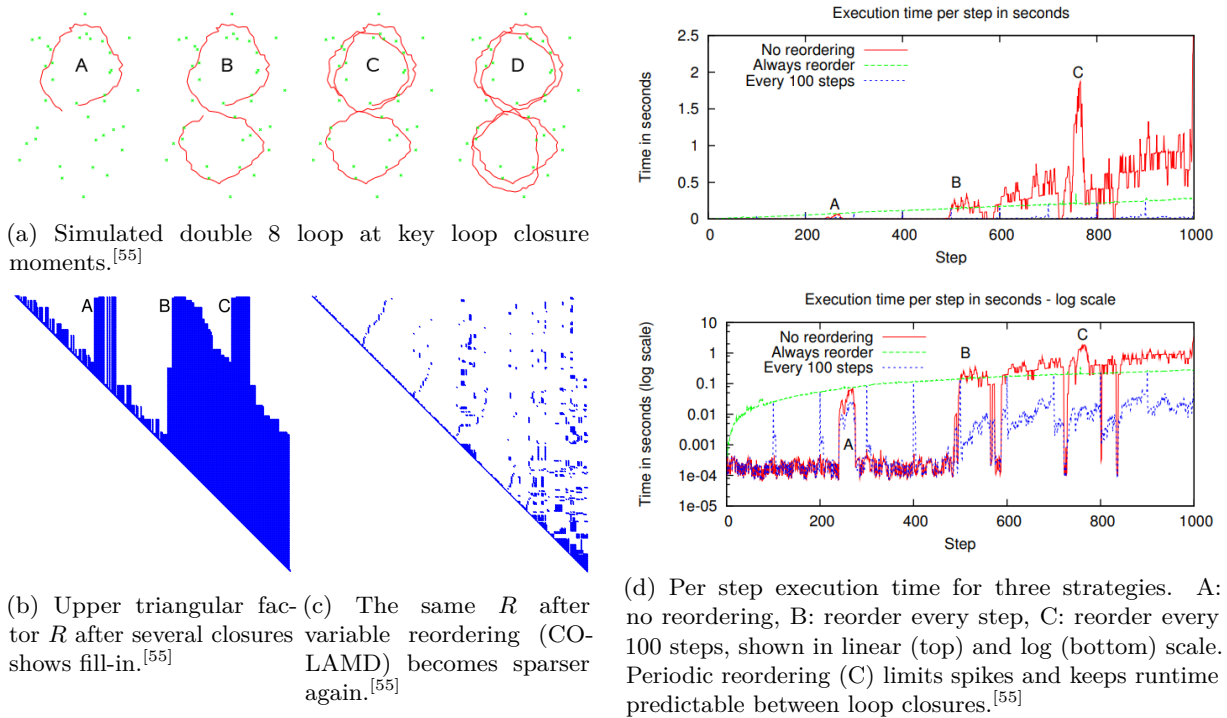


Figure 59: Effect of loop closures and variable reordering (iSAM).^[55]

Loop closures tie together far-apart parts of the trajectory (and landmarks), which makes previously separate columns interact. In QR terms this creates fill-in, R gets extra non zeros, so updates, back substitution, and selected covariance queries get slower and memory grows. This can be fixed by variable reordering. The goal is to pick a new elimination order that preserves sparsity. In practice run a heuristic like COLAMD (Column Approximate Minimum Degree) (often the block version for pose/landmark blocks) on the Jacobian’s sparsity matrix A , then do batch factorization on the whole A matrix using rotations (74) with that order [55]. Reordering costs time because the factor must be rebuilt with a new permutation, but it pays back by making subsequent updates cheap again.

Because reordering is expensive, it is not done at every step. Instead, reordering is performed periodically every N steps (eks, 50 - 200) so the cost stays predictable. In marine AUV and ASV runs this keeps compute bounded. Between reorders incremental updates are fast and local. After a loop closure, one spike occurs (reorder + refactor), then the system returns to low latency. Practical tips when doing this step is to keep poses as blocks (block COLAMD) to reduce fill-in, align reordering with planned relinearization passes, and monitor simple stats (nonzeros in R , update time) to decide when N is too small (wasting time reordering) or too large (letting fill in snowball).

11.2.7 Re-Linearization

Re-linearization keeps the local model valid. The QR and update methods assume the system is locally linear around the current estimate, but with angles, three dimensional motion, and nonlinear sensor data, that approximation drifts as the robot moves. If it is not refreshed, increments grow, the optimizer biases the map, and loop closures can fail. The remedy is to re-linearize and recompute Jacobians at the current state for the factors that matter. Doing this for every factor at every step is too expensive, so in practice new factors are always linearized, and older ones are refreshed only when needed.

In iSAM the practical schedule is to run incremental updates between maintenance cycles, then perform a full re-linearization every N steps. Between cycles, old factors are not re-linearized, new factors are whitened and inserted, and R is updated incrementally. At the cycle boundary after N steps, the entire problem is re-linearized at the current estimate, the full Jacobian A is rebuilt conceptually, variables are reordered with COLAMD to restore sparsity, and the system is refactored using equation (74) to obtain a fresh triangular R . For this reason variable reordering is usually grouped with batch re-linearization at the same N step.

N must be chosen carefully, by balancing freshness vs compute. If N is too large, the linearization point drifts far from reality, Jacobians no longer match the true geometry, corrections become biased, loop closures pull hard, and the map can warp (a classic “stale linearization” issue). If N is too small, the system stops often to re-linearize and refactor, wasting CPU and power, and reducing real-time throughput.

11.2.8 Data Association from R

For data association, Mahalanobis based approach is often used instead of plain nearest neighbour Approach. Nearest neighbour measures raw Euclidean distance and ignores sensor noise and correlations. Mahalanobis measures the innovation in the units of its uncertainty, so noisy directions count less, precise directions count more, and correlated components are handled correctly (see Figure 55 and Equation (62) in the Data Association chapter for intuitive geometric explanation on Mahalanobis distance). For a candidate match between current pose x_i and landmark l_j , form the innovation ν_k and score:

$$d_k^2 = \nu_k^\top \Xi_k^{-1} \nu_k, \quad \Xi_k = J_k \Sigma J_k^\top + \Gamma_k$$

where $J_k = [H^{x_i} \ H^{l_j}]$ is the linearized measurement Jacobian, Γ_k is the sensor noise, and Σ is the state covariance.

This type of data association often uses gating with a chi-square test. The test keeps matches that are within $d_k^2 \leq \chi_{m,\alpha}^2$ (right dimension m , chosen confidence α). From the survivors, pick the “minimum cost” one (or solve a global assignment using d_{ij}^2 as the cost matrix if several features compete). One thing to note about this approach is that the full dense $\Sigma = (R^\top R)^{-1}$ is not required for data association, only the local covariances influenced by the measurement. These covariances can be extracted directly from the square root information matrix R . Pose blocks and pose to landmark blocks are available online as well. Landmark terms on the other hand are either a approximate fast conservative estimate or computed exactly on demand. This keeps the association real-time for the most part. Here are the partitioned forms. Full state covariance (poses vs. landmarks):

$$\Sigma = \begin{bmatrix} \Sigma_{xx} & \Sigma_{xL} \\ \Sigma_{Lx} & \Sigma_{LL} \end{bmatrix}$$

where Σ_{xx} is pose to pose, Σ_{LL} is landmark to landmark, and $\Sigma_{xL} = \Sigma_{Lx}^\top$ is pose to landmark.

The 2×2 submatrix needed for a single candidate (x_i, l_j) :

$$\Sigma_{\{x_i, l_j\}} = \begin{bmatrix} \Sigma_{x_i x_i} & \Sigma_{x_i l_j} \\ \Sigma_{l_j x_i} & \Sigma_{l_j l_j} \end{bmatrix}$$

Fast marginals from the square root factor (online/live)

In iSAM paper [55], they propose keeping the current pose last in the ordering. Then the covariances

needed for association, the pose variance $\Sigma_{x_i x_i}$ and the pose to landmark cross terms $\Sigma_{x_i l_j}$ come straight from the square root factor R with two small triangular solves:

$$R^\top Y = B, \quad RX = Y$$

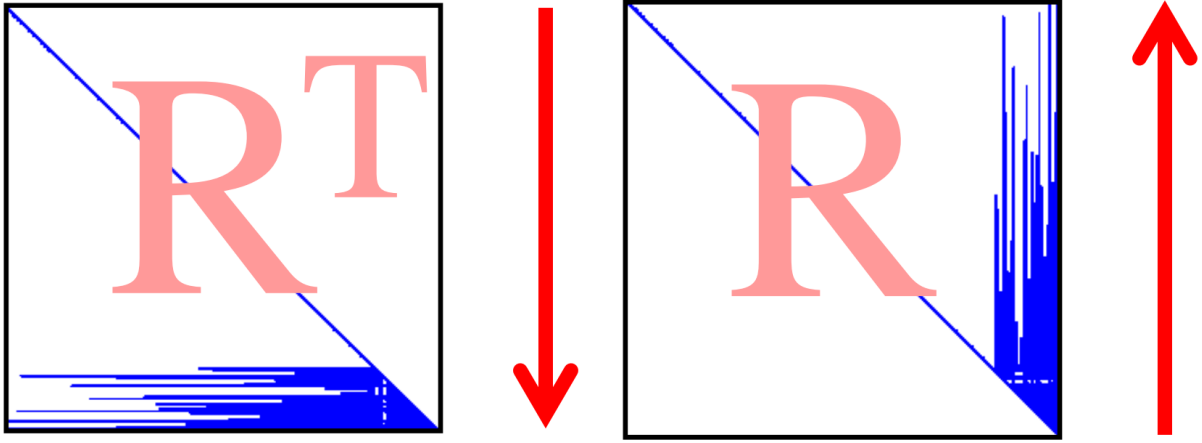
where $B = \begin{bmatrix} 0 \\ I_{d_x} \end{bmatrix}$ simply selects the last pose block of size d_x . Because R is upper triangular and B is zero above the last block, the forward solve gives:

$$Y = [0, \dots, 0, R_{ii}^{-1}]^\top$$

This Y preliminary matrix is the clue, where only d_x back substitutions are needed to get full X vector. Reading the result X yields, in one pass:

$$\Sigma_{x_i x_i} \text{ (bottom-right block of } \Sigma) \quad \Sigma_{l_j x_i} = \Sigma_{x_i l_j}^\top \text{ for connected } l_j$$

Intuitively “solve up” then “solve down” on R for the last pose, and get exactly the columns of $\Sigma = (R^\top R)^{-1}$ that matter for gating in data association. This can be done every step without forming any dense inverses, only using R square root information matrix iteratively. (See Figure 60)



(a) $R^\top Y = B$ (forward substitution). Sweep “down” the matrix to form Y from a selector B that picks the last pose block.

(b) $RX = Y$ (back substitution). Sweep “up” the matrix to obtain the desired columns $X = \Sigma_{:, x_i}$ (pose and pose to landmark).

Figure 60: Grab the needed covariances in two quick steps: first solve “down” with $R^\top$, then solve “up” with R . Only touch entries near the last pose block, so each update stays fast and cheap.^[55]

Conservative landmark covariances (online/live)

Exact landmark landmark blocks Σ_{jj} (or old pose to landmark $\Sigma_{(i-n)j}$) are expensive to extract at every step. Therefore in iSAM paper [55] they propose using a safe, conservative bound built from the current pose covariance and the measurement noise via the linearized back projection:

$$\tilde{\Sigma}_{jj} = \bar{J} \begin{bmatrix} \Sigma_{ii} & 0 \\ 0 & \Gamma \end{bmatrix} \bar{J}^\top$$

where $\bar{J}$ is the Jacobian of the local inverse measurement model, Σ_{ii} is the current pose covariance, and Γ is the measurement noise. This upper bounds landmark uncertainty (never over confident), is fast, and works well for online Mahalanobis gating. As more measurements of a landmark arrive, this bound typically tightens.

An important caveat to mention is that the true Γ (measurement noise) is usually unknown. The iSAM recipe is to choose Γ conservatively so the algorithms doesn’t get overly confident. That keeps things safe but can make the gate too tight, causing the data association to reject more matches than it should. Later, a more reliable way to approximate Γ from measurement characteristics is described. It avoids overconfidence and uses cues such as sonar range and resolution and landmark confidence, so the

gate is realistic and not risky.

Exact landmark covariances (on demand)

When accuracy is needed (eks: on risky loop closure, conflicting hypotheses), Data Association can recover exact Σ_{jj} and $\Sigma_{(i-n)j}$ without forming the full dense inverse information matrix $\Sigma = (A^T A)^{-1} = (R^T R)^{-1}$. Because the covariance is the inverse of the information matrix:

$$\Sigma = (R^T R)^{-1}, \quad R^T R \Sigma = I$$

Needed entries can be extracted without forming the full inverse by solving for two triangular matrixes:

$$R^T Y = I, \quad R \Sigma = Y$$

The solve uses only the “nonzero” entries of R , so computation touches only the parts that matter, not the whole matrix. iSAM walks backwards along those non zero links and gives us exactly the covariance numbers σ_{ij} Data Association ask for. If R is mostly banded, this is near linear time. This exact method should only be used when its really needed (eks: a few Σ_{jj} blocks to check on a loop closure). It’s slower than the conservative shortcut, but still much faster than inverting the whole matrix.

Alternative way to finding Γ

Instead of relying on an overly conservative fixed measurement noise Γ (as done in the original iSAM formulation [55]), it can be derived directly from the physical sensor model. Γ represents the measurement noise covariance, the statistical uncertainty of the sensor reading itself, independent of any landmark or map uncertainty.

For sonar based mapping, this measurement noise can be obtained directly from the range dependent model defined in Equation (14). Here

$$\Gamma_i = \Sigma_{\text{sonar}}^i = \mathbf{R}_{\text{sonar}}(r_i)$$

where the uncertainty increases quadratically with range due to beam spreading, attenuation, and multipath effects. This model already captures the per axis range and bearing variances $\sigma_r^2(1 + \alpha_r r_i^2)$ and $\sigma_\theta^2(1 + \alpha_\theta r_i^2)$, providing a realistic, range aware characterization of sonar noise around the corrected measurement prediction $h_{\text{corr}}(\mathbf{x}, m^i)$ (see Equation 13).

Using a physically grounded Γ_i instead of a constant conservative value improves realism and stability in both optimization and data association. However, great care must be taken not to make Γ_i too small (overconfident), which would make the Mahalanobis gate overly strict and reject valid matches, making the optimizer unstable and crumbling the map, ie divergence from true map and trajectory. In practice, the model should be tuned from representative sonar datasets and can be adaptively scaled with signal to noise ratio, reflection strength, or detection confidence to maintain a good balance between robustness and accuracy.

11.2.9 Algorithm

At each time step absorb new information, linearize around the current estimate, solve a small least squares, and update. Periodically refresh linearization and variable order. Concretely:

1. **Add factors (whiten first):** take the new odometry/measurements and add them to the graph. “Whiten” them so every residual has unit noise (as described before (72)).
2. **Linearize at the current guess θ :** turn the nonlinear motion/measurement models into local linear ones using the Jacobians in (69) and (70). This gives the summed Mahalanobis cost (71), which after whitening becomes one least squares problem (72).
3. **Keep a triangular system up to date (QR):** append the new (whitened) rows and apply a few Givens rotations (74) so the matrix stays upper triangular R . Update the right hand side b vector the same way (see “Incremental Updating”).
4. **Solve for the small change:** because R is triangular, solve (73) by back substitution (fast) to get the correction .

5. **Update the estimate:** replace the old state with the improved one, $\theta \leftarrow \theta + \Delta\theta^*$.
6. **Every N steps (maintenance):** refresh accuracy by re-linearizing all factors at the new θ , reorder variables (eks: COLAMD algorithm) to keep things sparse, and refactor with Givens (74) to get a clean R .
7. **Data Association update:** read the needed covariances from R (pose and pose to landmark live, landmark blocks conservative or exact on demand) and run Mahalanobis gating in Data Association for next matches.

11.2.10 Limitations

iSAM is fast between updates but has practical downsides. They come from the “data structure”, not the underlying SLAM algorithm. iSAM keeps a single, global square root information matrix R (from $A^\top A$) and does periodic maintenance (reordering + re-linearization). This makes updates simple, but couples cost to global structure and variable ordering instead of just local changes.

- **Latency spikes at maintenance:** Periodic global variable reordering and re-linearization trigger stalls (especially after loop closures), since R must be refactored end to end.
- **Fill in growth between reorders:** Incremental QR on a fixed order accumulates fill in in R , touching more entries per update and increasing time/memory step by step.
- **All or nothing re-linearization:** iSAM typically refreshes many factors at maintenance even if most variables barely moved, wasting Jacobian recomputations.
- **Global refactor on ordering changes:** Any change to elimination order implies a large refactor of the global R , regardless of how small the new information is.
- **Broad marginal queries are costly:** Last pose and nearby cross terms can be pulled quickly from R , but wide Σ blocks (eks: many landmarks or older poses) require multiple triangular solves and can be costly.
- **Schedule sensitivity:** Choosing “every N steps” for reorder/re-linearize is heuristic, too small wastes time, too large lets fill in and linearization error grow, causing jitter and warp in the map.
- **Numerical robustness vs simplicity:** Working with A and R avoids explicit $A^\top A$, but long incremental runs plus fill in can still hurt conditioning and stability if ordering lags.

These limitations motivated **iSAM2**, which replaces the single monolithic R that is very static, with a Bayes tree representation that is dynamic and updates only the affected parts. We address iSAM2 and how it mitigates the issues above in the next chapter.

11.3 iSAM2

11.3.1 Introduction and Motivation

iSAM2 was developed to overcome the practical limitations of iSAM. The core optimization method in iSAM (nonlinear least squares solved through incremental QR factorization) is sound and provides accurate solutions. The bottlenecks arise not from the underlying algorithms, but from the static data structure used to maintain the problem. In iSAM the system is stored as a single global square root information matrix R . While this representation is compact and efficient for batch updates, it leads to several issues during incremental operation.

First, the R matrix is “global”. Adding new factors or loop closures often changes many rows and columns, which requires expensive refactorization. This causes latency spikes, especially when loop closures occur and large parts of the trajectory suddenly become coupled. Second, relinearization in iSAM is also global. To maintain accuracy, the system periodically refreshes Jacobians for all factors, which forces full reconstruction of the R matrix. Third, variable reordering to reduce fill in and keep R sparse is again an all or nothing operation, with cost proportional to the entire problem size. Together, these properties mean that iSAM, while efficient between updates, still suffers from periodic heavy computation that disrupts real time performance.

iSAM2 addresses these limitations by introducing a new data structure, the “Bayes tree”. Instead of representing the system as a static R matrix, iSAM2 leverages the factor graph formulation of SLAM, applies variable elimination, and interprets the resulting structure as a chordal Bayes net. This Bayes net can then be compactly represented as a Bayes tree, which retains all probabilistic information while enabling local updates. With the Bayes tree, adding new measurements or relinearizing states only modifies the affected cliques in the tree, leaving the rest untouched. This local property eliminates the global refactorization bottlenecks of iSAM, smooths out computation over time, and makes the algorithm scalable to large and long term mapping problems. [35, 56]

11.3.2 Factor Graphs

A factor graph is a bipartite graph that connects variable nodes (poses and landmarks) to factor nodes (priors, motion, and measurements). It encodes the same estimation problem as in iSAM, but makes sparsity and locality explicit because each factor touches only a few variables.

Let the variables be robot poses $x_1, \dots, x_M$ and landmarks $l_1, \dots, l_N$, and let $\Theta = \{x_1, \dots, x_M, l_1, \dots, l_N\}$. According to the iSAM2 papers [35, 56] the posterior factorizes as

$$f(\Theta) = \prod_i f_i(\Theta_i),$$

Here, each factor f_i depends only on its adjacent variables Θ_i . How each factor is modeled depends on the situation. Because odometry and measurements are uncertain in the real world, factors should be represented with a probabilistic model. One of the easiest probabilistic models is Gaussian, which is easy to model and will work well later when working in Mahalanobis form to optimize the problem and solve it in simple manner (See Figure 55 and Equation (62) for Mahalanobis form). Factor functions can therefore be modeled as follows:

$$\text{Prior: } f_p(x_0) \propto \exp\left(-\frac{1}{2}\|\mu_0 - x_0\|_{P_0}^2\right),$$

$$\text{Odometry: } f_i(x_{i-1}, x_i) \propto \exp\left(-\frac{1}{2}\|f_i(x_{i-1}, u_i) - x_i\|_{Q_i}^2\right),$$

$$\text{Measurement: } f_k(x_{i_k}, l_{j_k}) \propto \exp\left(-\frac{1}{2}\|h_k(x_{i_k}, l_{j_k}) - z_k\|_{R_k}^2\right).$$

This is in a way very similar to how iSAM models the system, however here in iSAM2, the system is represented as factor graphs. This graph view exposes conditional independence directly in the topology. Each factor only connects nearby variables in time or space, which later yields a sparse linear system.

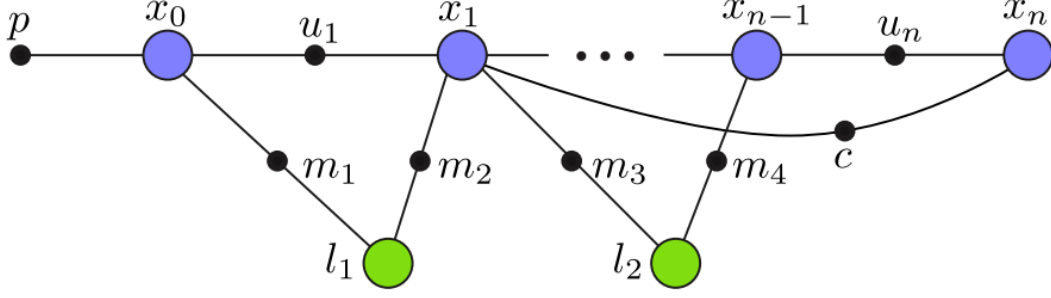


Figure 61: Picture from iSAM2 paper [35] describes factor graph formulation of the SLAM problem. Variable nodes (large circles) are poses $x_0, \dots, x_n$ and landmarks l_1, l_2 . Factor nodes (small solid circles) represent a prior p , odometry u_i , landmark measurements m_i , and a loop closure constraint c . This approach can represent any cost function, including factors that connect more than two variables.

This example shows how local connectivity induces sparsity. This will be useful when solving the optimization problem later down below. Each factor touches only its adjacent variables, so after linearization (to compute a local optimum) the Jacobian rows have nonzeros only in those columns. The resulting matrix is sparse, which helps the optimizer.

11.3.3 From Factor Graphs to the SLAM Optimization Problem

Here SLAM is represented with a factor graph $G = (\mathcal{F}, \Theta, \mathcal{E})$. There are two node types, factor nodes $f_i \in \mathcal{F}$ that encode pieces of information (prior, odometry, measurements, loop closures), and variable nodes $\theta_j \in \Theta$ that hold unknowns (poses, landmarks, calibration). An edge $e_{ij} \in \mathcal{E}$ is drawn only when factor f_i depends on variable θ_j . This wiring is important, missing edges mean independence, and the pattern of edges controls which variables interact in the estimation problem.

The graph specifies how a global objective splits into simple parts:

$$f(\Theta) = \prod_i f_i(\Theta_i),$$

Here Θ_i collects only the variables that touch factor f_i . In the SLAM setting, a prior p anchors the first pose, odometry factors u relate consecutive poses, landmark factors m couple a pose with a landmark, and loop closure factors c link poses that see the same place again (see Picture 61). The same framework can also handle factors that involve three or more variables, for example a factor that ties a pose to a landmark and a camera intrinsics block (calibration), or “separator” variables shared in cooperative mapping. The key point is that the graph can host any cost term as long as it states which variables it touches.

Under Gaussian measurement models as defined in the previous subsection, each factor has the form:

$$f_i(\Theta_i) \propto \exp\left(-\frac{1}{2} \|h_i(\Theta_i) - z_i\|_{\Sigma_i}^2\right),$$

This Gaussian form will simplify calculations as Gaussian is nice to work with. Here $h_i(\cdot)$ predicts what the sensor should see from the current variables Θ_i , z_i is the actual measurement, and Σ_i is the measurement covariance. The notation $\|e\|_{\Sigma}^2 \triangleq e^\top \Sigma^{-1} e$ is the squared Mahalanobis distance, which measures error in “units of its noise” (directions with low variance are penalized more). (See Figure 55 and Equation (62))

To combine all information, simply multiply the factor likelihoods. Products are awkward to optimize, instead take a negative logarithm to turn the product into a sum. Terms that do not depend on Θ drop out, and each Gaussian factor becomes a squared Mahalanobis residual weighted by its covariance. The result is one scalar objective that collects the prior, all odometry factors, all landmark measurements, and any loop closures. Small covariances make a factor count more, large covariances count less. In short, multiply the factors and take the negative log to get a single sum of squared errors. This can be write in compact form as:

$$\Theta^* = \arg \min_{\Theta} \frac{1}{2} \sum_i \|h_i(\Theta_i) - z_i\|_{\Sigma_i}^2$$

Here Θ collects all unknowns, such as poses, landmarks, and other parameters. Each factor h_i depends only on a small subset Θ_i , so each residual couples only those variables. After linearization this gives a sparse system, because most variables do not appear together in any single residual.

This is our MAP function in nonlinear form.

This nonlinear cost is not solved in one shot because the measurement transform $h_i(\cdot)$ is nonlinear (because of angles, ranges, bearing-only sensors, etc...). As in the iSAM system, linearize around the current estimate and solve iteratively. Choose a current estimate Θ^0 and look for a small update $\Delta\theta$ that improves the fit. For each factor take a 1st-order Taylor expansion around Θ^0 , which turns that factor into a simple linear residual in the increment $\Delta\theta$ that touches only its adjacent variables. After whitening by $\Sigma_i^{-1/2}$, stack all linearized factors into one sparse least-squares problem:

$$\Delta\Theta^* = \arg \min_{\Delta\Theta} (-\log(f(\Delta\Theta))) = \Delta\theta^* = \arg \min_{\Delta\theta} \|A \Delta\theta - b\|^2 \quad (76)$$

Here $A \in \mathbb{R}^{m \times n}$ is the measurement Jacobian (one row block per factor), b stacks the whitened prediction errors, and $\Delta\theta$ is the n -dimensional increment. The sparsity pattern of A is dictated by the factor graph, a row has nonzeros only in the columns of the variables that appear in that factor.

Here the linear least squares problem is solved in a numerically stable manner. One route is the normal equations $A^\top A \Delta\theta = A^\top b$ and a Cholesky factorization $A^\top A = R^\top R$ followed by forward and back substitution to recover $\Delta\theta$. Another route is QR factorization on A , which yields an upper triangular system $R \Delta\theta = d$ that can be solved by back substitution. Both routes are standard, in practice QR factorization methods are preferred for stability.

After solving for $\Delta\theta$, update the state $\theta \leftarrow \theta + \Delta\theta$ and repeat, linearize, solve, update. This is Gauss-Newton. If the problem is difficult (poor linearization, strong nonlinearity), add Levenberg-Marquardt damping and instead solve $(A^\top A + \lambda I) \Delta\theta = A^\top b$, which blends Gauss-Newton with a trust-region step to keep updates safe. Stop when $\Delta\theta$ is small or the cost no longer decreases.

This is the same least squares core as in iSAM. The difference is the data structure. iSAM works with the Jacobian A and its triangular factor R . iSAM2 keeps the factor graph as the main object, which carries the same information as A from iSAM but in a graph form. Eliminate variables to get a Bayes net and store it as a Bayes tree. This lets iSAM2 update only the cliques touched by new factors instead of refactoring everything. The next subsection shows how this is done.

11.3.4 From Factor Graphs to Bayes Networks

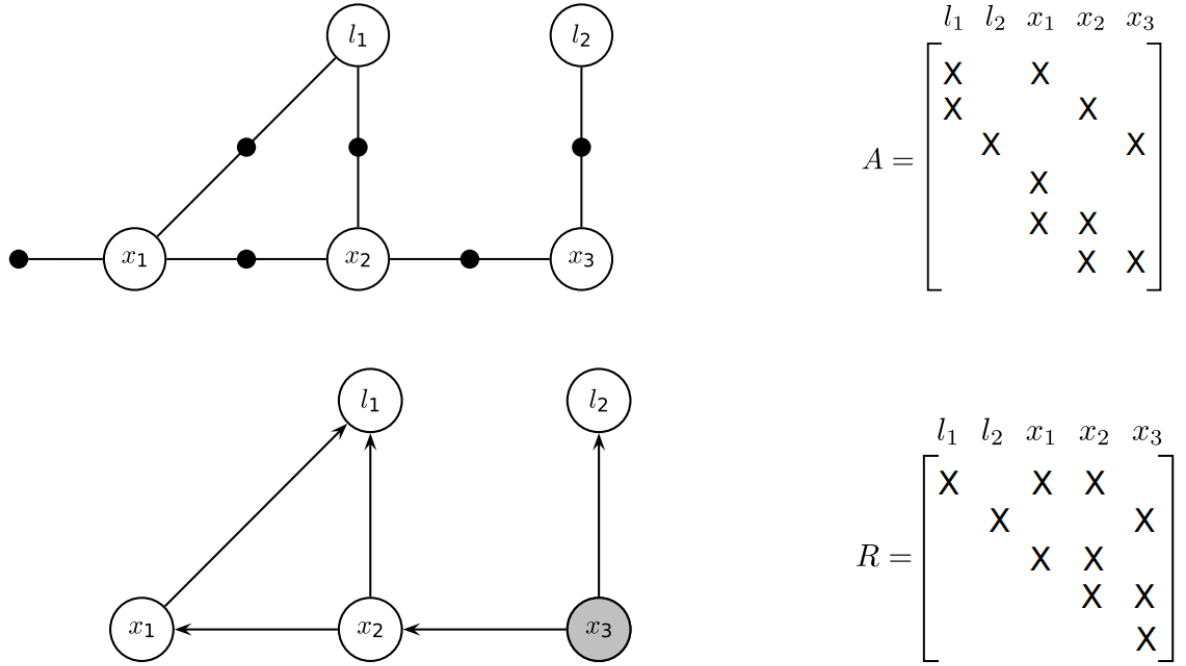


Figure 62: Picture from iSAM2 paper [35]. (Top) factor graph and associated Jacobian A for a small SLAM example with poses x_1, x_2, x_3 , landmarks l_1, l_2 , and a prior on x_1 (Bottom) the chordal Bayes net and the square root factor R obtained by eliminating in the order l_1, l_2, x_1, x_2, x_3 . The last eliminated variable (the root) is shaded. [35]

Starting from the least squares form in (76), the whitened measurement matrix A is factored as in iSAM. An elimination order is chosen, (for the example in Picture 62 it is l_1, l_2, x_1, x_2, x_3), and sparse QR with Givens rotations (74) is applied. This produces an orthogonal Q and an upper triangular R . Because R is triangular, solving by back substitution proceeds variable by variable in the chosen order. That solve can be read as a chain of simple Gaussian conditionals, one per variable, where the off diagonal entries to the right of each pivot indicate which previously eliminated variables that conditional depends on. Drawing arrows from the parent variables to the current variable turns the same structure into a directed graphical model. In short, for a chosen ordering, sparse QR on the factor graph yields an R that encodes a Bayes network, this is what the bottom panel of Picture 62 shows.

Instead of running sparse QR on the whitened Jacobian, the same result can be obtained by eliminating variables directly on the factor graph using bipartite elimination game methods [57]. Starting from (76), choose an order, then for each variable collect its adjacent factors, combine them, marginalize that variable out, and attach the resulting factor to the remaining neighbors. Repeat until all variables are removed. For any fixed ordering, the purely graphical elimination produces the same dependency pattern and the same square root information factor as numeric QR factorization. This procedure forms a Bayes network with chordal properties, and in the matrix view yields an upper triangular R with $R^\top R = A^\top A$ (see Picture 62). The advantage is that A does not need to be assembled, and Givens rotations and explicit QR factorization are avoided. The computation stays on the graph, the ordering controls fill, and the desired chordal structure is obtained. In practice this is the same QR algebra, just done in a graph aware way that avoids unnecessary intermediate fill and work according to Good Column Orderings for Sparse QR Factorization paper [57].

This observation is the bridge to iSAM2. A chordal Bayes network groups naturally into cliques, this can be represented as a Bayes tree. The next subsections use this tree, rather than a single global R , to support local updates and avoid global re-factorizations.

11.3.5 R as a Bayes tree data structure

A key outcome of variable elimination on the SLAM factor graph is a chordal Bayes network. Chordal means that in the moralized (undirected) view every long cycle has a shortcut edge, which keeps parents of a variable grouped in small cliques and prevents excessive fill in during elimination. This property is what makes the square root factor R sparse and tractable, and it also sets up a clean bridge to a tree representation of the same information. [35, 56]

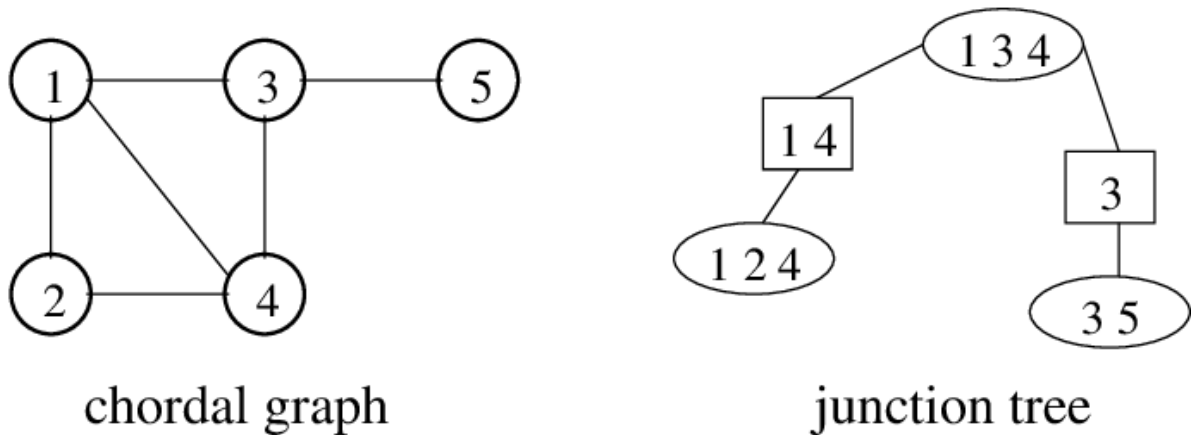


Figure 63: Picture from Robert Castelo paper [58] shows an example of a simple Chordal graph (Left side) and corresponding small cliques/junction tree (Right side). Eliminating in a good order produces a chordal Bayes net whose moralized graph can be grouped into cliques/junction trees. This is the structure that keeps R matrix sparse.

When linearizing and eliminating in some order, the resulting triangular system R still satisfies $R^\top R = A^\top A$, but each row block of R matrix now carries a specific meaning. Each row block of R belongs to the variable that was just eliminated. That row says, in Gaussian form, how this variable depends on a few variables that were eliminated earlier. Think of those earlier variables as its parents. When solving $R\Delta\theta = d$ by back substitution, start at the last row and move upward. That is the same as computing each variable from its parents in turn. So R is not just numbers in a triangle. It is the same set of conditional relationships as the Bayes network, written in matrix form.

Because the Bayes net is chordal, its conditionals naturally group into cliques. Consecutive row blocks of R that share the same separator form a clique. Cliques that share a separator are connected to form a Bayes tree. Each clique stores frontal conditionals, child given the parent separator, and each edge carries only the shared separator variables. The tree encodes the same numeric information as R , but it is organized by locality rather than by a single global ordering.

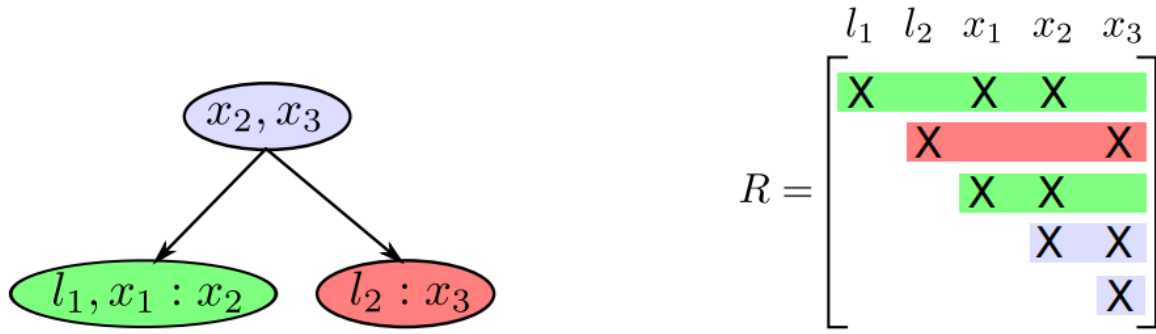


Figure 64: Picture from Bayes tree paper [56] shows Bayes tree (left) and its matching rows in the square root factor R (right). Colors indicate which contiguous row blocks of R belong to each clique.

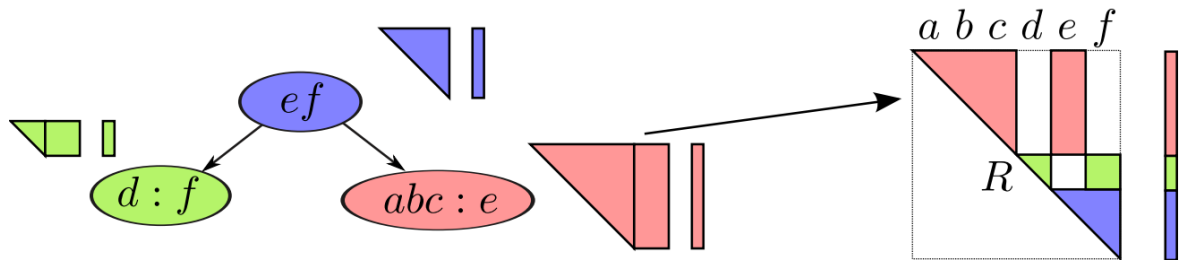


Figure 65: Picture from iSAM2 paper [35] shows each clique contains the conditional of its frontal variables given the separator. The same entries appear in the corresponding rows/columns of R . Back substitution on R mirrors evaluating the tree from leaves to root, while marginal queries follow the few cliques that touch the queried variables.

The takeaway is simple. Eliminating a factor graph gives a chordal Bayes net. Grouping its conditionals yields a Bayes tree. The square root factor R contains the very same conditionals in matrix form. Thus R can be viewed as a Bayes tree data structure in matrix form. iSAM2 stores and updates this Bayes tree directly instead of one global R , which preserves the numerical benefits of the square root form while enabling strictly local updates. This means when new measurements arrive or when some variables must be re-linearized, only the cliques on a small subtree are touched and the rest of the structure stays unchanged, making computational complexity manageable and the data set grows.

11.3.6 Incremental Updates directly on Bayes tree

iSAM2 never rebuilds the whole system when new data arrives. A new odometry or landmark factor only touches a few variables in the factor graph, so only the matching subtree in the Bayes tree is modified. All other branches stay exactly the same. This is the key difference from iSAM, where adding a factor could force a global re-factorization of the single R matrix.

Two simple rules explain why updates stay local. First is that information flows upward in the tree during elimination, so changes propagate only from the touched variables toward the root. Second, a factor becomes active when the first variable in its local elimination order is eliminated, so only the paths from those variables up to the root can be affected, while unrelated subtrees remain untouched.

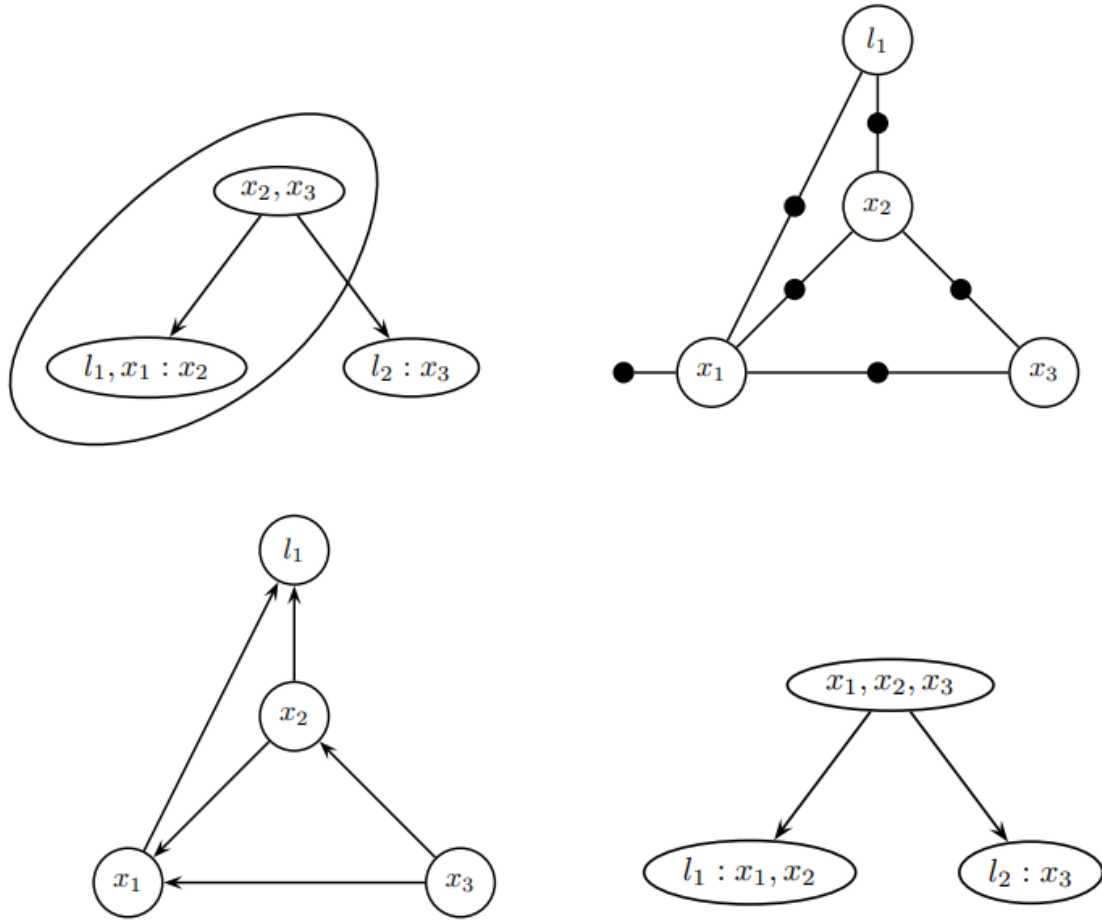


Figure 66: Picture taken from Bayes tree paper [56] shows how to update a Bayes tree with a new factor. (Top left) Affected cliques when adding a factor between x_1 and x_3 , the right branch is unaffected. (Top right) Local factor graph rebuilt from those cliques plus the new factor. (Bottom left) Local chordal Bayes net after elimination. (Bottom right) New Bayes subtree with the untouched “orphan” subtree reattached.

One update works as follows. first, locate the cliques that contain the variables touched by the new factor and follow their ancestors up toward the root, this marks the only region that needs work, while the rest of the tree becomes “orphans” that stay valid and untouched. Next, convert the conditionals stored in those marked cliques back into a small local factor graph and insert the new factor. Then re-eliminate just this local graph (using the same graph aware elimination as discussed previously) to produce a new chordal Bayes net and its updated Bayes subtree. Finally, reattach the orphan subtrees at the proper separators. Only this small subtree changes, everything else is reused. (See Picture 66)

The result is incremental and predictable computation. iSAM2 edits only the cliques touched by the new information, runs a small local elimination, and solves by back substitution along that subtree. There are no global re-factorization spikes, and accuracy is maintained by re-linearizing only the variables in the affected region when needed.

11.3.7 Loop Closure and Incremental Reordering

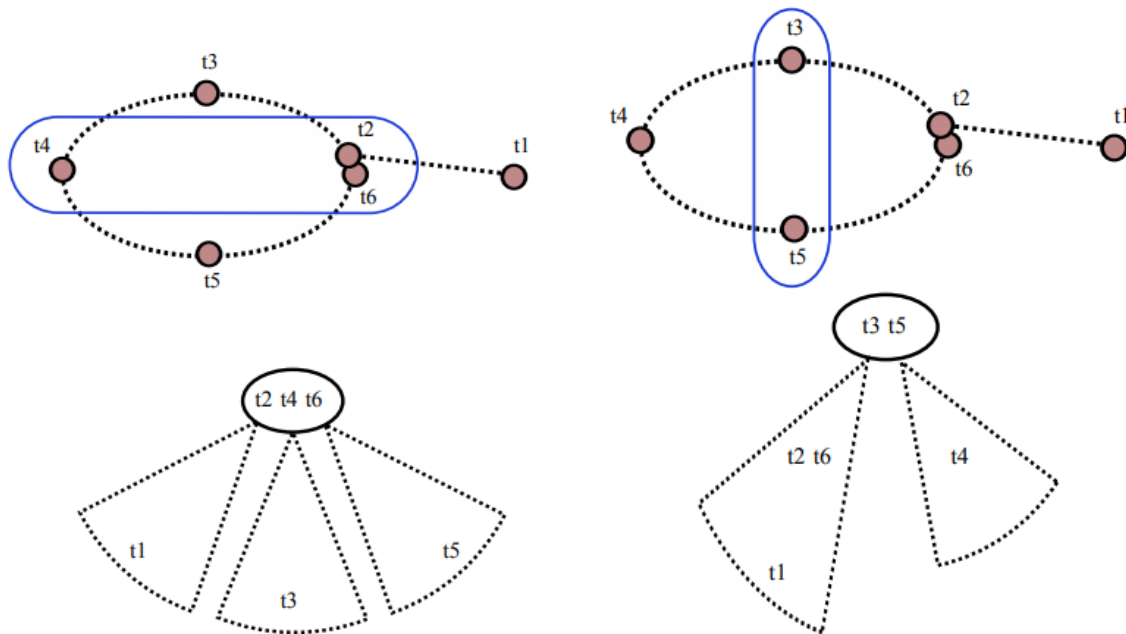


Figure 67: Picture taken from Bayes tree paper [56] shows loop closure with incremental reordering. Two batch optimal orderings (top) yield different Bayes trees (bottom). For online operation, the ordering should keep the newest variables near the root, so future updates affect only a small subtree.

Loop closures add a factor between two far apart poses. In a Bayes tree this never forces a global re-factorization. Only the cliques on the (unique) paths from those two pose cliques up to the root are affected, all other subtrees are untouched “orphans” and are reused as is. The affected top region is converted back to a local factor graph, the loop closure factor is added, and the region is re-eliminated to produce a new local chordal Bayes net and Bayes subtree. The unchanged subtrees are then reattached at the matching separators. This keeps updates local and predictable, unlike iSAMs global R re-factorization after loop closure. [35, 56]

A good variable ordering is still crucial because it controls fill in (clique sizes) during elimination. iSAM2 performs incremental reordering only over the affected variables, rather than periodic global reorderings. A simple and effective rule is to force the most recent variables (the ones new factors usually touch) to be eliminated last (i.e. near the root). Practically, this is implemented with a constrained COLAMD heuristic. Here the newest pose blocks are kept at the end of the order while letting COLAMD algorithm find a sparse order for the rest. This produces small, stable updates at each step, even when loops close. [56]

Batch orderings found by nested dissection (or similar heuristics) can look equally good for a one time solve because they produce comparable sparsity. For online SLAM the next update matters. The preferred ordering therefore leaves the newest poses at or near the Bayes tree root, so the next odometry or loop closure factor changes only a small subtree. If the newest pose lies deep in the tree, the same update must rewrite many cliques. For this reason iSAM2 uses constrained reorderings, keeping recent variables last in the order near the root and letting the heuristic arrange the rest. This keeps updates local and cheap. [56]

This constrained COLAMD heuristic will not yield a globally optimal ordering, however it reliably reduces fill in and the amount of work near the update, keeps recent variables near the root, and avoids large latency spikes. In practice it delivers close to batch sparsity while saving compute time, and when needed it is reapplied only to the affected subtree, so the cost stays proportional to that small region rather than the whole tree.

11.3.8 Fluid Re-Linearization

iSAM2 stops doing periodic global “re-linearize everything”. Instead iSAM2 only refresh (re-linearize) the parts of the problem that truly need it, right when they need it. The Bayes tree makes this easy, new measurements change only a small subtree, so iSAM2 recomputes just that piece and leave the rest of the tree alone. This keeps the math accurate without the big stalls that happened in iSAM after loop closures or long runs. [35, 56]

How it works in practice is simple. Always keep a running correction vector Δ from the latest linear solve. If a variable’s change is tiny, treat its current linearization point as “good enough” and do not touch it. If a variable’s change is larger than a small threshold (call it β), mark that variable for re-linearization. Then mark the cliques that contain those variables and their ancestors in the Bayes tree. Only those cliques are rebuilt, go back to the original nonlinear factors for those cliques, recompute their Jacobians at the new linearization point, add cached “marginal” factors from untouched children, and eliminate again to update just the top of the tree. Everything else remains as it was. Note that this re-linearization threshold can be set per state. Positions (x, y) can use a looser (higher) threshold so they are refreshed less often, while the heading angle, being more nonlinear should use a tighter (lower) threshold. [56]

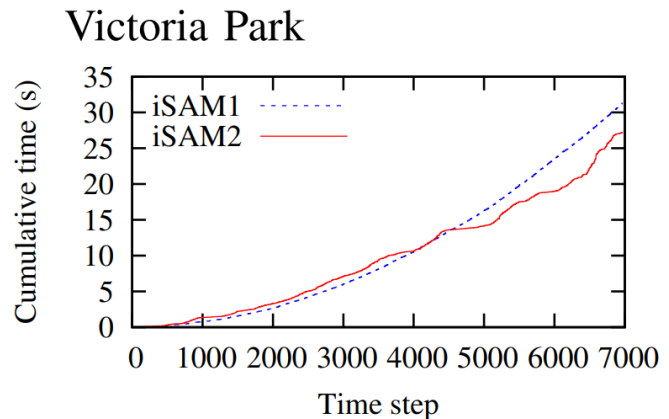
Solving for $\Delta\theta$ is also done only where needed. First solve on the modified top of the tree. Then walk into children only if any parent update exceeded a small propagation threshold (call it α). This “only follow when it matters” rule avoid needless work in distant parts of the map while still keeping accuracy where the robot is and where measurements arrived. Together, the two thresholds (β to decide who to re-linearize, and α to decide where to propagate the solve) give iSAM2 its fluid, real-time/online behavior. Accuracy when and where it matters, speed everywhere else.

This local, threshold strategy removes the need for heavy batch steps and keeps runtime smooth over long missions just like incremental reordering does the same. In essence what iSAM2 does is it exploits Factor graphs representations of the map and Bayes tree data structure to do dynamic variable reordering and re-linearization, no need for periodic batch calculations like in iSAM. On standard datasets (Victoria Park) the cumulative computation of iSAM2 grows much more slowly than iSAM, because re-linearization and re-elimination are confined to small subtrees rather than the full graph. And over time the discrepancy will only grow where iSAM will slow down whilst iSAM2 will hold its compute much more efficient. (See Picture 68)

Victoria Park



Victoria Park SLAM map used in the original iSAM benchmarks.



Cumulative time vs. step for iSAM (periodic batch) and iSAM2 (fluid). iSAM2 stays faster as the dataset grows.

Figure 68: Picture from the Bayes tree paper [56]. In iSAM2, relinearization is fluid, only the affected cliques are recomputed and the rest of the tree is reused.

11.3.9 Sparse Factor Graphs

Over long missions the factor graph can accumulate many near duplicate constraints (revisits of the same place, repeated landmark sightings from similar viewpoints). This “Eiffel Tower effect” slowly densifies the graph and enlarges cliques in the Bayes tree, which increases update cost. The fix is sparsification. Here keep the informative constraints and summarize or drop the redundant data points while preserving the important information flow. In practice this is done locally where data arrive. Retain the freshest odometry and a few diverse loop closures, and replace discarded constraints by a light summary on the small separator set where they would have entered the tree. Intuitively, keep the “shape” of the information at the boundary and forget interior details that are now redundant. Simple policies such as keyframing (keeping only selected poses as variables), pruning highly correlated measurements, and limiting per landmark observation count work well. The result is a graph that stays sparse, cliques that stay small, and a Bayes tree that remains cheap to update even in very long runs.

11.3.10 Beyond Gaussian Assumptions (Robust Estimators)

Pure Gaussian residuals are fragile in the face of outliers (bad data association, spurious loop closures, moving objects, changing environment). Robust estimators fix this by replacing the quadratic loss with a robust loss that grows slower than a square. Common choices include Huber (quadratic near zero, linear in the tails), Cauchy, or Tukey. In iSAM2 this is a drop in change at the factor level. Each robust loss yields a weight for its residual, updated as the estimate improves (iteratively re-weighted least squares). Factors that fit well keep high weight, inconsistent ones are down weighted, so they no longer dominate the solution. This makes incremental updates and fluid re-linearization safer because a single wrong constraint will not trigger large edits high up in the tree. For hard loop closures, one can also use switchable or graduated penalties that let the optimizer “turn off” a suspect factor until there is enough supporting evidence. The Bayes tree concept stays the same, only local factor weights and linearization adapt, so robustness comes with little extra complexity.

11.3.11 Data Association from the Bayes Tree

The same as in iSAM, iSAM2 scores candidate matches with a Mahalanobis distance, which measures the innovation in units of its uncertainty. However in iSAM2 the needed covariances are read directly from the Bayes tree without forming a dense matrix. Pose and nearby pose to landmark covariances are obtained efficiently by following only the few cliques that contain those variables and their separators. This keeps online/real-time gating fast for data association. Queries involving far away landmarks or many old poses may touch a larger portion of the tree and therefore cost more, but they remain practical on demand. In practice this combines a cheap, conservative bound (for routine gating) with exact small block queries when decisions are critical (e.g. verifying a loop closure). The result is reliable association with predictable compute cost during operation.

11.3.12 Algorithm

At each step iSAM2 absorbs new measurements, touch only the relevant cliques in the Bayes tree, solve for a small increment, and update the state. iSAM2 combines the linear update with fluid re-linearization, so work stays local and predictable. Concretely, one iteration looks like this (matching the structure summarized in the iSAM2 and Bayes tree papers [35, 56])

1. **Add new data:** Insert the new factors (odometry, measurements, loop closures) into the factor graph. If new states appear, add them to the estimate Θ .
2. **Mark what to refresh (fluid re-linearization):** Keep the last increment $\Delta\theta$. If a state moved more than a small threshold β , mark it for re-linearization. Only pass this mark to neighbors if the parent changed more than a smaller threshold α . This finds the small set that really needs work now.
3. **Build a small local problem:** Take just the cliques in the Bayes tree that touch the marked states (and their ancestors up to the root) and turn them back into a tiny factor graph, everything else becomes reusable “orphans”.
4. **Order and eliminate locally:** Find a sparse order for this small graph using a constrained COLAMD (keep the newest states last, near the root), then eliminate to make a new local Bayes subtree.

5. **Reattach orphans:** Connect the untouched subtrees back at the correct separators. Only the edited subtree changed, the rest is reused.
6. **Solve where needed:** Back solve on the updated top of the tree to get a new increment $\Delta\theta$. Propagate the solve into children only when the parent’s change is big enough (same α rule).
7. **Update the estimate:** Apply the increment on the whole factor graph, $\Theta \leftarrow \Theta \oplus \Delta\Theta$ (where $\theta = \Theta$ and $\Delta\theta = \Delta\Theta$). Keep $\Delta\Theta$ for the next steps re-linearization test.
8. **Keep variable ordering healthy (incremental):** When a loop closure grows cliques near the root, run constrained COLAMD again, but only on that small region to reduce fill and keep the newest states near the root.
9. **Data association update:** Fetch the needed local covariances from the Bayes tree, compute far away covariances only on demand, and compare predicted and observed features using chosen Data Association method.

11.3.13 Limitations

While iSAM2 removes the big computation spikes seen in iSAM, some practical and theoretical limits remain.

- **Ordering is heuristic, not optimal:** Choosing a variable order that minimizes fill in is NP-hard, so iSAM2 relies on constrained COLAMD and related heuristics. These give good, stable performance online but cannot guarantee the globally best sparsity.
- **Clique growth in dense areas.** Heavy revisiting of the same places (“Eiffel Tower effect”) or many near duplicate constraints can enlarge cliques near the root. Updates remain local, but the cost of each local elimination grows with clique size. In long runs, sparsification/keyframing is often needed to keep the graph light.
- **Threshold tuning for fluid re-linearization:** The accuracy/speed trade off depends on two small thresholds (who to re-linearize and how far to propagate the solve). These must be tuned for the sensor and motion model. Too loose can delay accuracy, too tight does extra work.
- **Faraway covariances can be expensive:** Exact marginal/covariance queries are done by recursive message passing on the tree (dynamic programming style). Queries that span long paths or large separators touch more cliques and therefore cost more.
- **Gaussian least-squares core:** Outliers and non Gaussian effects are not handled by iSAM2 alone. Robust losses or switchable constraints must be added at the factor level to down weight bad data, otherwise accuracy can degrade during long missions.
- **Nonlinearity still matters:** Poor initial guesses or highly nonlinear measurements can require multiple Gauss-Newton/Levenberg-Marquardt steps. iSAM2 just makes each step local.
- **Engineering complexity and memory:** Compared to a single global R in iSAM, the Bayes tree in iSAM2 adds much more moving parts (cliques, separators, cached/orphan subtrees, incremental reordering). Correct, efficient implementations are more involved, and memory still grows with map size unless one prunes or summarizes.

In practice, most of these limits can be mitigated with careful design. Using keyframing/sparsification to cap clique size, robust losses or switchable constraints to handle outliers, and constrained incremental reordering to keep updates local. The main hurdle is engineering complexity. This is where the open source GTSAM library (from the Georgia Tech team behind iSAM/iSAM2) is invaluable. It ships a production quality Bayes tree/iSAM2 implementation, clean factor graph APIs, robust noise models, and utilities for ordering and re-linearization, making it a practical starting point for both research and deployment. GTSAM is a widely used research framework for factor graph based sensor fusion and SLAM, and will be discussed in more detail in later chapters.

12 Global Map and Trajectory

12.1 Introduction

The global map and trajectory view serves as the final visualization layer of the Side Scan SONar SLAM (SSS SLAM) pipeline. As the optimizer produces updated pose estimates, the system incrementally renders an expanding global seafloor map in semi real time. This provides an intuitive and practical way to inspect mapping quality, observe drift behaviour, validate loop closures, and debug the entire pipeline as the mission unfolds. Beyond debugging, the global map becomes a valuable product for post processing, mission analysis, and integration into higher level autonomy modules such as collision avoidance, risk assessment, and future path planning. Since all computationally expensive steps, normalization, projection, probabilistic interpolation, and stitching, have already been handled in the local map generation stage, the global mapping layer is designed to remain lightweight and efficient. Its purpose is not to reinterpret or re-estimate sonar data, but simply to accumulate the already processed local Cartesian maps into a coherent global frame and display them in a stable, continuous, easy to examine form.

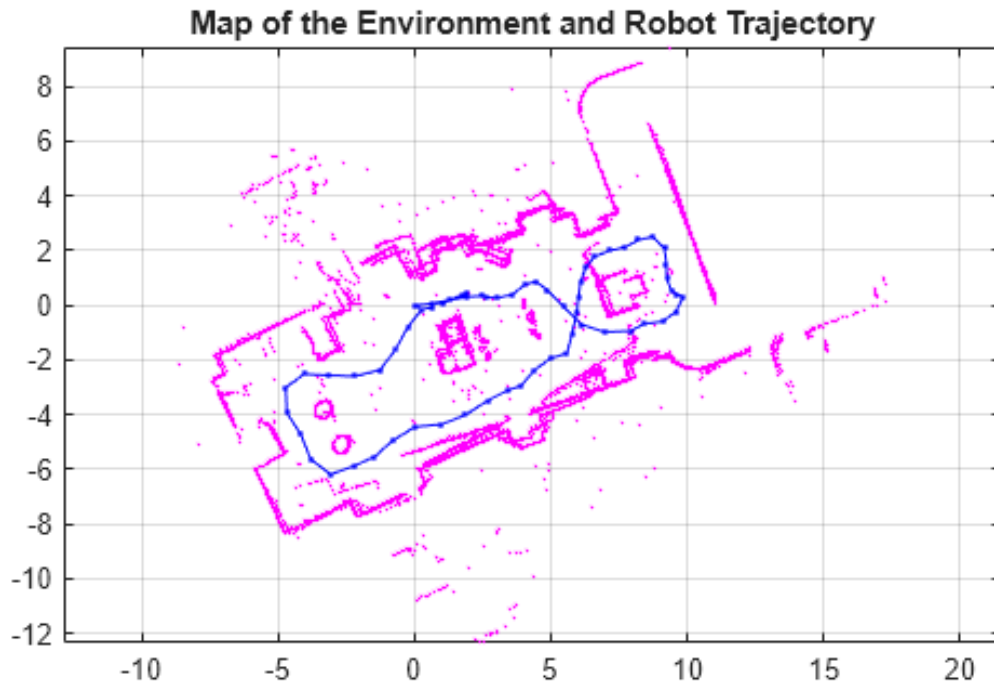


Figure 69: Example of a 2D global map reconstructed from LiDAR scans, with the robots estimated trajectory overlaid.^[59]

12.2 Global Trajectory and Global Map Generation

The global trajectory is obtained directly from the SLAM back end. The optimizer outputs each estimated vehicle pose in the Lie group $SE(3)$, which already encodes the full 3D rotation and translation of the platform. Generating the trajectory therefore becomes trivial, see Equation (4). Each optimized pose is appended to a trajectory buffer and plotted in the global frame without additional computation. No filtering or smoothing is required at this stage, since all corrections (including loop closures and re-linearization updates) are already reflected in the optimizers output.

The global map generation builds on the same principle. Each local Cartesian map tile originates from the local map pipeline, where slant range correction, pitch and roll compensation, probabilistic interpolation, and normalization have already been applied. Because these tiles are internally consistent and represented in a local body centered frame, the global layer only needs to apply the corresponding $SE(3)$ pose to transform every pixel into the global coordinate system, see Equation (5). Conceptually, this is identical to standard computer graphics rendering, take an image patch, apply a rigid body transform, and place it in a world canvas.

As the vessel moves, thousands of local maps accumulate. To manage this efficiently, a KD tree or similar kNN based spatial structure can be used to index pixel locations and enable fast insertion, lookup, and updates as the map grows. This becomes especially important when evaluating overlapping regions between newer and older map tiles. Due to the sliding window behaviour described in the “*Local Map Generation*” chapter, where each new tile shares approximately 1/3rd overlap with the previous tile, the global map frequently receives multiple pixel estimates for the same global location. Since the local maps are already intensity normalized and internally smoothed, the global fusion strategy can remain simple, whenever two or more pixels fall into the same global cell, their values are averaged. This produces smooth transitions between overlapping tiles without introducing additional bias or artifacts.

Because all heavy preprocessing has already been performed in local map generation, the global map does not require probabilistic fusion, filtering, or advanced occupancy grid models. Its role is strictly compositional, transform, place, and blend. The final structure can also be stored in a lightweight database or grid based memory layout to support efficient post processing or rapid access during mission playback. This allows the global map to act not only as a visual tool but also as a reusable data product ready for integration into autonomy frameworks such as obstacle avoidance, seafloor classification, semantic mapping, and long term planning tasks.

Overall, the global trajectory and map module is intentionally simple. The optimizer provides accurate $SE(3)$ poses, the local map generation pipeline provides consistent Cartesian tiles, and the global layer merely stitches them together into a coherent and continuously growing world representation. This results in a robust and computationally efficient final stage of the SSS SLAM system.

13 Implementation Frameworks And Tools

13.1 Introduction

Developing a complete SLAM and state estimation system entirely from scratch is both time consuming and technically challenging. Each subsystem, from sensor fusion and motion estimation to optimization and data association requires extensive mathematical formulation, careful numerical implementation, and detailed validation. Reimplementing these complex components not only slows progress but also increases the risk of introducing subtle numerical errors and inconsistencies between modules.

To mitigate these challenges and focus on higher level research goals, this work builds upon established open source frameworks that provide reliable, modular, and well tested implementations of key algorithms. These frameworks form the foundation of modern robotics software stacks, allowing efficient integration of sensors, state estimators, optimizers, and visualization tools.

ROS2 is used as the central middleware for data handling, real-time communication, and debugging through tools such as RViz2 and rqt. The CAOR-MINES ParisTech UKF-M library provides a robust foundation for nonlinear state estimation and multi sensor fusion. OpenCV is utilized for image processing, feature extraction, and local map generation, forming the core of the visual and probabilistic perception pipeline. GTSAM handles factor graph construction, preintegration, and optimization, while MRPT supports data association and perception modules for the SLAM front-end.

All selected frameworks are open source and distributed under permissive research oriented licenses, ensuring that they can be freely used, modified, and integrated into this work while maintaining transparency and reproducibility.

13.2 ROS2 Middleware Framework

The Robot Operating System 2 (ROS2) serves as the core communication and integration framework in this work. It provides a distributed, real-time capable middleware that enables modular data flow between sensors, estimators, and control systems. ROS2 is built on the Data Distribution Service (DDS) standard, which allows decentralized peer-to-peer communication without relying on a central master node. This ensures robust and scalable operation across multiple computing units, a key requirement for autonomous platforms like the microAmpere ASV.

ROS2 organizes functionality in layered components, as illustrated in Figure 70. The operating system layer abstracts the underlying hardware and supports cross platform execution on Linux, Windows, and macOS. The ROS Middleware (RMW) layer implements DDS, providing reliable message delivery and configurable Quality of Service (QoS) policies for different data types. The Client Library (RCL) defines node behavior through APIs such as `rclcpp` (C++) and `rclpy` (Python), while the user code layer contains the application specific nodes developed for sensing, estimation, and control tasks.

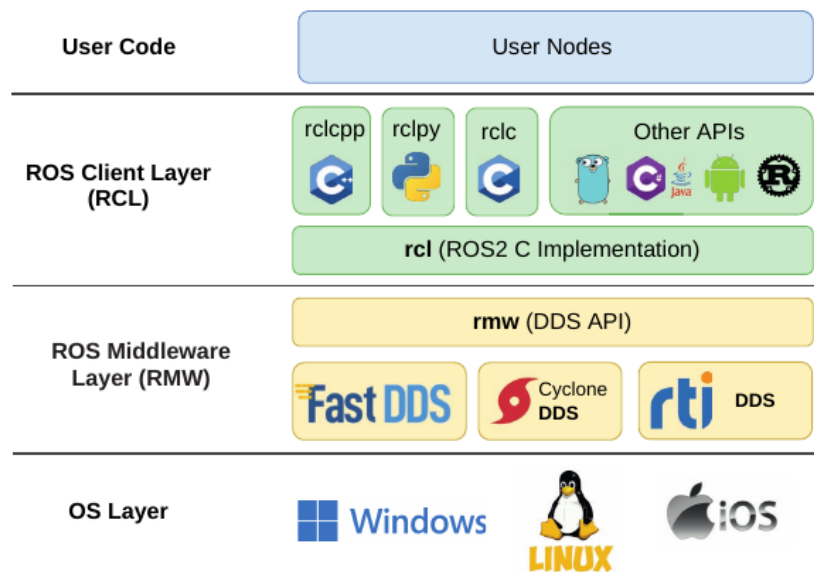


Figure 70: Layered architecture of ROS2 showing the relation between the OS, middleware, client library, and user application layers.^[60]

Data exchange in ROS2 follows the publish-subscribe paradigm. Each node can publish messages to a topic or subscribe to existing ones, allowing asynchronous and decoupled communication. For request-response operations, ROS2 uses services, while longer, goal driven interactions use actions. These mechanisms enable efficient communication between high rate data streams (eks, IMU, GNSS) and low frequency updates (eks, mission control). The middleware handles discovery, serialization, and message transport automatically, while QoS settings allow fine-grained control of reliability, latency, and timing behavior.

ROS2 also provides integrated tools for debugging, visualization, and development. Logging utilities, introspection tools, and visualization frameworks such as RViz2 and rqt enable real-time monitoring of system states, sensor topics, and algorithmic outputs. These features are essential for testing, validation, and troubleshooting during SLAM and navigation experiments.

ROS2 modular design allows independent packaging and deployment of nodes, simplifying updates and debugging. Python based launch files handle configuration and startup, making ROS2 an efficient backbone for distributed estimation and control.

ROS2 is released under the Apache 2.0 license, which permits free use, modification, and redistribution with proper attribution. Some components migrated from ROS1 retain the 3-Clause BSD license, both of which allow unrestricted research and development use. The ROS2 framework and source code are available at <https://www.ros.org/> [61] and on its official GitHub repository <https://github.com/ros2> [62].

13.3 CAOR-MINES ParisTech UKF-M Library

The UKF-M (Unscented Kalman Filter on Manifolds) library, developed by CAOR-MINES ParisTech, provides a practical framework for nonlinear state estimation on manifolds such as $SO(3)$ and $SE(3)$ [30][24]. It extends the standard UKF [23] to handle rotations and translations directly on their respective manifolds, removing the need for Jacobian derivation and linearization.

In this work, UKF-M is used as the main state estimation backend for integrating IMU, GNSS, and SLAM data into a unified navigation solution. Each measurement source is represented by a nonlinear measurement function $h(\mathbf{x})$ with an associated covariance that reflects its uncertainty. The filter propagates the mean and covariance directly on the manifold, maintaining proper handling of rotational and translational geometry. This allows consistent fusion of measurements expressed in different coordinate frames and Manifold formulations.

The implementation used here follows the official open source version maintained by CAOR-MINES ParisTech, available at <https://github.com/CAOR-MINES-ParisTech/ukfm> [63]. The repository provides a structured implementation of the UKF-M formulation along with reference examples for sensor fusion and motion estimation. The library has been verified in several research projects involving inertial and visual odometry [30][24].

The UKF-M library is released under the BSD-3-Clause license. This license permits modification, redistribution, and academic use, as long as copyright and license notices are preserved. It ensures that derived work remains properly attributed to the original developers while allowing open research collaboration and reproducibility.

13.4 OpenCV Computer Vision Library

The Open Source Computer Vision Library (OpenCV) is a widely used open source framework for computer vision, image processing, and feature extraction. Originally developed by Intel in 1999 and now maintained by an active open source global community, it provides a comprehensive collection of algorithms for image analysis, geometric transformations, and visual perception tasks [64][65].

OpenCV is implemented in C++ and designed for high performance visual computation, offering efficient data structures and hardware accelerated routines for real-time image processing. In this work, the C++ API is used primarily for the local map generation subsystem, where OpenCV supports both low-level image operations and high-level feature extraction. It is employed for filtering, thresholding, contour detection, and morphological transformations in probability maps, as well as for detecting and describing key features used in local mapping and spatial data association.

The main motivation for using OpenCV lies in its mature, optimized implementation and extensive algorithmic coverage. Its core data types such as `cv::Mat`, `Point`, and `Rect` simplify the manipulation of image like probabilistic grids and map structures. These properties make it particularly suitable for handling local occupancy or likelihood maps, where visual operations directly translate to probabilistic updates. Moreover, its modular design and cross library interoperability allow seamless integration with other systems, such as ROS2 via `cv_bridge`, and downstream state estimation frameworks like GTSAM and UKF-M for sensor fusion and visual SLAM.

The official OpenCV website and source code repository are available at <https://opencv.org/> [64] and <https://github.com/opencv> [65]. The C++ library for OpenCV is distributed under the Apache-2.0 license, permitting unrestricted academic and commercial use, modification, and redistribution with appropriate attribution.

In summary, OpenCV offers a robust, computationally efficient, and versatile foundation for image and map processing. Its optimized C++ implementation and comprehensive functionality make it an essential component for the perception and local mapping pipeline developed in this work.

13.5 MRPT Data Association and Perception Tools

The Mobile Robot Programming Toolkit (MRPT) is a comprehensive open source C++ framework that provides essential tools for perception, mapping, localization, and robotic data processing. It is designed to support rapid development and testing of SLAM and navigation algorithms by offering a large collection of reusable software modules for sensing, motion modeling, and data fusion. MRPT is cross platform and integrates well with other robotics ecosystems such as ROS2, making it widely used in both academic and industrial research.

In this work, MRPT is used primarily for data association and landmark management within the SLAM front-end. It includes advanced probabilistic data association algorithms such as the Joint Compatibility Branch and Bound (JCBB) method, which tests multiple measurement to landmark hypotheses simultaneously to ensure consistency across the entire observation set. This greatly reduces the risk of incorrect correspondences in cluttered or repetitive environments. MRPT also supports alternative association strategies, such as Nearest Neighbor (NN) and Mahalanobis distance based matching, allowing flexible adaptation to different sensor configurations and noise conditions.

Beyond data association, MRPT offers a wide range of perception and robotics utilities. It includes libraries for point cloud processing, image handling, 3D visualization, occupancy grid mapping, particle filters, extended Kalman filters, and rawlog recording for dataset playback. These tools simplify tasks such as sensor calibration, dataset generation, and visualization of SLAM outputs. The framework also provides simulation environments, motion models for wheeled and differential robots, and extensive support for range and vision based sensors.

Together, these features make MRPT a powerful framework for developing and validating SLAM front-ends. Its modular architecture allows selective integration of only the required components, reducing implementation time while maintaining flexibility and reliability. The official repository and related packages are available at <https://github.com/mrpt> [66].

MRPT is distributed under the BSD-3-Clause license, permitting modification, redistribution, and research use while requiring acknowledgment of the original authors and inclusion of the license text in derived work.

13.6 GTSAM Factor Graph Framework

The Georgia Tech Smoothing and Mapping (GTSAM) library is an open-source C++ framework for representing and solving estimation problems using factor graphs. Developed by Frank Dellaert and the BORG Lab at the Georgia Institute of Technology, GTSAM provides a flexible and efficient foundation for robotics and computer vision tasks involving probabilistic inference, including SLAM, structure from motion, and multi sensor fusion [67][68][69].

At its core, GTSAM models estimation problems as a product of small, local probability functions known as “*factors*”, each depending on a subset of variables (eks, robot poses, velocities, landmarks, biases). These factors form a sparse graphical model, allowing efficient computation of the Maximum a Posteriori (MAP) estimate through variable elimination and nonlinear least squares optimization. This formulation provides a clear, modular representation of the SLAM problem while naturally capturing uncertainty and correlations between states.

GTSAM performs optimization directly on manifolds using Lie group representations ($SE(2)$, $SE(3)$) for rotation and translation. This avoids the singularities of Euler angles or quaternions by updating states through tangent space retraction, ensuring consistent and stable 3D motion estimation and sensor fusion [67].

GTSAM features an advanced IMU preintegration module based on the work of Forster, Carlone, and Dellaert [31] and the earlier visual inertial framework by Lupton and Sukkarieh [70]. It compresses high rate inertial data into a single relative motion factor, reducing computation while preserving accuracy. Using on manifold integration with bias correction and covariance propagation, it enables tight multi sensor fusion within the factor graph.

For real-time applications, GTSAM implements the iSAM2 algorithm [35]. iSAM2 maintains a Bayes tree data structure that enables local re-linearization and partial updates when new factors or variables are added. Instead of re-optimizing the entire graph, only the affected subtrees are updated, allowing smooth incremental estimation without large computational spikes. This makes iSAM2 ideal for online SLAM systems, where continuous sensor updates must be processed with bounded latency. The solver uses variable reordering (eks, constrained COLAMD) to minimize matrix fill in and preserve sparsity, ensuring scalability even for large scale mapping problems.

In practice, GTSAM separates the model structure from the state representation. A `NonlinearFactorGraph` stores all factors, while a `Values` container holds the current estimates of unknown variables. New sensor measurements are added as typed factors, such as `BetweenFactor`, `PriorFactor`, or `ImuFactor`, each with its associated noise model. GTSAM then performs nonlinear optimization using solvers like Gauss-Newton or Levenberg-Marquardt. For online updates, the iSAM2 interface provides continuous refinement of the solution, while preserving consistency between past and current estimates.

In this work, GTSAM forms the core of the SLAM backend, handling factor graph construction, IMU preintegration, and real-time optimization through iSAM2. IMU, GNSS, and SLAM measurements are fused as factors that continuously refine the estimated trajectory and map. The modular design also allows straightforward extension with additional sensor types, such as visual or sonar and acoustic inputs.

GTSAM is built on a strong academic foundation, combining key advances in probabilistic robotics and factor graph optimization [56] [55] [35]. Its modular design enables rapid prototyping of new factor types, testing of optimization methods, and integration with real world datasets. The framework has become a standard in research for its clarity, scalability, and rigorous mathematical grounding in SLAM and estimation.

GTSAM is released under the BSD license, allowing modification and redistribution for academic and commercial use with proper attribution. The library and source code are publicly available at <https://gtsam.org/> [68] and <https://github.com/borglab> [69].

In summary, GTSAM provides a robust foundation for real-time SLAM, combining factor graphs, manifold optimization, preintegration, and incremental solvers for efficient multi sensor fusion.

References

- [1] Hoff Simon, Andreas Hagen, Haraldstad Vegard, Reitan Hogstad Bjoornar, and Varagnolo Damiano. “Side-scan sonar based landmark detection for underwater vehicles”. In: (Oct. 2024). URL: <https://ntnuopen.ntnu.no/ntnu-xmlui/handle/11250/3172808> (visited on 09/13/2025).
- [2] Haraldstad Vegard. “A side-scan sonar based simultaneous localization and mapping pipeline for underwater vehicles”. Master’s thesis. Norwegian University of Science and Technology (NTNU), 2023. URL: <https://ntnuopen.ntnu.no/ntnu-xmlui/handle/11250/3086270> (visited on 09/13/2025).
- [3] Bjørnar Reitan Hogstad. “Side-Scan Sonar Imaging and Error-State Kalman Filter Aiding Unmanned Underwater Vehicle (UUV) to Autonomy”. Master’s thesis. Norwegian University of Science and Technology (NTNU), Feb. 2022. URL: <https://ntnuopen.ntnu.no/ntnu-xmlui/handle/11250/3032962> (visited on 10/11/2025).
- [4] United Nations. “THE 17 GOALS”. In: (2025). URL: <https://sdgs.un.org/goals> (visited on 12/09/2025).
- [5] Flyguys. “LiDAR vs. Sonar: Whats the Difference?” In: (Apr. 12, 2022). URL: <https://flyguys.com/lidar-vs-sonar-whats-the-difference/> (visited on 12/04/2025).
- [6] Flyguys. “Longitudinal and transverse waves”. In: (Sept. 13, 2019). URL: <https://www.sciencelearn.org.nz/resources/2816-sound-wave-interference> (visited on 12/04/2025).
- [7] Zhao Zhiwei, Wu Jinqui, Qi Xiaofei, Qiao Gang, Zhang Wenbo, Zhang Chaofan, and Guo Kang. “Design of a Broadband Cavity Baffle Bender Transducer”. In: (May 16, 2022). URL: <https://www.mdpi.com/2077-1312/10/5/680> (visited on 12/04/2025).
- [8] Wikipedia. “Side-scan sonar”. In: (Sept. 16, 2025). URL: https://en.wikipedia.org/wiki/Side-scan_sonar (visited on 12/04/2025).
- [9] Cao Vo Tran Luan. “Design and Implementation of microAmpere: An Advanced USV for Autonomous Docking Research”. In: (Jan. 2025).
- [10] Reimers Henrik. “NMPC and blueboat hardware”. In: (Jan. 2025).
- [11] Safran Navigation & Timing. “STIM300 Datasheets”. In: (May 2025). URL: <https://safran-navigation-timing.com/document/stim300-datasheets/> (visited on 10/11/2025).
- [12] u-blox. “ZED-F9P Module”. In: (May 9, 2025). URL: <https://www.u-blox.com/en/product/zed-f9p-module> (visited on 10/11/2025).
- [13] Thor Inge Fossen. “Fossen’s Marine Craft Model”. In: (). URL: <https://fossen.biz/html/marineCraftModel.html> (visited on 10/12/2025).
- [14] Brekke Edmund. *Fundamentals of Sensor Fusion: Target Tracking, Navigation and SLAM*. NTNU, 2020.
- [15] Zeithöfler Julian. “Nominal and observation-based attitude realization for precise orbit determination of the Jason satellites”. In: (June 2019). URL: https://www.researchgate.net/figure/illustrates-the-principle-of-gimbal-lock-The-outer-blue-frame-represents-the-x-axis-the_fig4_338835648 (visited on 10/12/2025).
- [16] David. “How Quaternions Produce 3D Rotation”. In: (June 2019). URL: <https://penguinmaths.blogspot.com/2019/06/how-quaternions-produce-3d-rotation.html> (visited on 10/12/2025).
- [17] Atanasov Nikolay. “ECE276A: Sensing & Estimation in Robotics: Lecture 12: SO(3) and SE(3) Geometry and Kinematics”. In: (). URL: https://natanaso.github.io/ece276a2020/ref/ECE276A_12_SO3_SE3.pdf (visited on 10/12/2025).
- [18] Wagner Tobias. “Position information and their coordinate systems WGS84 and ETRS”. In: (Mar. 25, 2024). URL: <https://genesys-offenburg.de/support/application-aids/gnss-basics/position-information/> (visited on 10/12/2025).
- [19] SBG Systems. “NED (North-East-Down) Frame”. In: (). URL: <https://www.sbg-systems.com/glossary/ned-north-east-down/> (visited on 10/12/2025).
- [20] Xin Hu. “Robust nonlinear control design for dynamic positioning of marine vessels with thruster system dynamics”. In: (Oct. 2018). URL: https://www.researchgate.net/figure/North-east-down-frame-and-vessel-fixed-frame-26_fig1_325388618 (visited on 10/12/2025).

- [21] Kanevsky Alex, Huitt Carpenter Mark, Gottlieb David, and Jan S. Hesthaven. “Robust nonlinear control design for dynamic positioning of marine vessels with thruster system dynamics”. In: (Aug. 2007). URL: https://www.researchgate.net/publication/222525996_Application_of_implicit-explicit_high_order_Runge-Kutta_methods_to_discontinuous-Galerkin_schemes (visited on 10/16/2025).
- [22] Kaess Michael and Dellaert Frank. “Factor Graphs for Robot Perception”. In: (2017). URL: <https://www.cs.cmu.edu/~kaess/pub/Dellaert17fnt.pdf> (visited on 10/29/2025).
- [23] Wan Eric and Van Der Merwe R. “The Unscented Kalman Filter for Nonlinear Estimation”. In: (Oct. 2000). URL: <https://ieeexplore.ieee.org/document/882463/authors#authors> (visited on 10/17/2025).
- [24] Brossard Martin, Barrau Axel, and Bonabel Silvere. “A Code for Unscented Kalman Filtering on Manifolds (UKF-M)”. In: (Mar. 11, 2020). URL: <https://arxiv.org/pdf/2002.00878> (visited on 11/05/2025).
- [25] Lei Mao. “Introduction to Bayesian Filter”. In: (May 21, 2019). URL: <https://leimao.github.io/article/Introduction-to-Bayesian-Filter/> (visited on 10/17/2025).
- [26] MATLAB. “An Optimal State Estimator | Understanding Kalman Filters, Part 3”. In: (Mar. 27, 2017). URL: <https://se.mathworks.com/videos/understanding-kalman-filters-part-3-optimal-state-estimator--1490710645421.html> (visited on 10/17/2025).
- [27] Alex Becker. “Kalman Filter: from the Ground Up”. In: (2023). URL: <https://kalmanfilter.net/ekf.html> (visited on 10/17/2025).
- [28] Liu Wei, Song Dan, Wang Zhipeng, and Fang Kun. “Comparative Analysis between Error-State and Full-State Error Estimation for KF-Based IMU/GNSS Integration against IMU Faults”. In: (Nov. 11, 2019). URL: <https://www.mdpi.com/1424-8220/19/22/4912> (visited on 10/17/2025).
- [29] Hertzberg Christoph, Wagner René, Frese Udo, and Lutz Schröder. “Integrating Generic Sensor Fusion Algorithms with Sound State Representations through Encapsulation of Manifolds”. In: (July 6, 2011). URL: <https://arxiv.org/abs/1107.1119> (visited on 10/17/2025).
- [30] Brossard Martin, Barrau Axel, and Bonabel Silvere. “Unscented Kalman Filtering on Lie Groups”. In: (Sept. 2017). URL: <https://hal.science/hal-01489204v3/document> (visited on 11/05/2025).
- [31] Forster Christian, Carlone Luca, Dellaert Frank, and Scaramuzza Davide. “On-Manifold Preintegration for Real-Time Visual-Inertial Odometry”. In: (Oct. 30, 2016). URL: <https://arxiv.org/abs/1512.02363> (visited on 10/30/2025).
- [32] Hatleskog Johan, Nissov Morten, and Alexis Kostas. “IMU-Preintegrated Radar Factors for Asynchronous Radar-LiDAR-Inertial SLAM”. In: (Sept. 26, 2025). URL: <https://arxiv.org/abs/2509.22288> (visited on 10/30/2025).
- [33] J.H Seyed and E.B Castelan. “Evaluation of Monocular Visual-Inertial SLAM: Benchmark and Experiment”. In: (Nov. 2019). URL: https://www.researchgate.net/publication/340811939_Evaluation_of_Monocular_Visual-Inertial_SLAM_Benchmark_and_Experiment (visited on 10/30/2025).
- [34] Koller Friedman. “Factor Graphs and GTSAM”. In: (May 22, 2019). URL: <https://gtsam.org/tutorials/intro.html> (visited on 11/08/2025).
- [35] Kaess Michael, Johannsson Hordur, Roberts Richard, Ila Viorela, Leonard John, and Dellaert Frank. “iSAM2: Incremental smoothing and mapping with fluid relinearization and incremental variable reordering”. In: (Aug. 18, 2011). URL: <https://ieeexplore.ieee.org/document/5979641> (visited on 09/14/2025).
- [36] King Peter, Anstey Benjamin, and Vardy Andrew. “Sonar image registration for localization of an underwater vehicle”. In: (Oct. 2017). URL: https://www.researchgate.net/publication/324314762_Sonar_image_registration_for_localization_of_an_underwater_vehicle (visited on 11/08/2025).
- [37] Vlad Lazar Alexandru, Delia Bugheanu Alexandra, Viorel Ungureanu Gheorghe, and Dan Ionescu Alexandru. “Side-scan sonar mapping of anthropic influenced seafloor: a case study of Mangalia Harbour”. In: (Jan. 2013). URL: https://www.researchgate.net/publication/275637568_Side-scan_sonar_mapping_of_anthropic_influenced_seafloor_a_case_study_of_Mangalia_Harbour (visited on 11/08/2025).

- [38] OpenDSA. “KD Trees”. In: (Oct. 15, 2025). URL: <https://opendsa-server.cs.vt.edu/ODSA/Books/Everything/html/KDtree.html> (visited on 11/08/2025).
- [39] Powell Victor. “Image Kernels: Explained Visually”. In: (). URL: <https://setosa.io/ev/image-kernels/> (visited on 11/10/2025).
- [40] xu Long and Yan Yihua. “Guided filtering for solar image/video processing”. In: (Mar. 2017). URL: https://www.researchgate.net/publication/318968983_Guided_filtering_for_solar_image_video_processing (visited on 11/10/2025).
- [41] OpenCV. “Image Thresholding”. In: (Nov. 9, 2025). URL: https://docs.opencv.org/4.x/d7/d4d/tutorial_py_thresholding.html (visited on 11/10/2025).
- [42] Prateek Chhikara. “Understanding Morphological Image Processing and Its Operations”. In: (Mar. 30, 2022). URL: <https://towardsdatascience.com/understanding-morphological-image-processing-and-its-operations-7bcf1ed11756/> (visited on 11/10/2025).
- [43] Shaig Hamzaliyev. “Implementing Connected Component Analysis for Image Processing from Scratch with Python”. In: (Apr. 1, 2024). URL: <https://medium.com/@h.shaig93/connected-component-analysis-implementing-from-scratch-with-python-c5931785493b> (visited on 11/10/2025).
- [44] Kim Hogyun, Kang Gilhwan, Jeong Seokhwan, Ma Seungjun, and Younggun Cho. “Robust Imaging Sonar-based Place Recognition and Localization in Underwater Environments”. In: (May 24, 2023). URL: <https://arxiv.org/pdf/2305.14773> (visited on 11/10/2025).
- [45] Zhou Wu, E Shiju, Cao Zhenxin, and Dong Ying. “Review of SLAM Data Association Study”. In: (2016). URL: <https://www.atlantis-press.com/article/25859609.pdf> (visited on 12/02/2025).
- [46] Cansiz Sergen. “Multivariate Outlier Detection in Python: Multivariate Outliers and Mahalanobis Distance in Python”. In: (Mar. 20, 2021). URL: <https://medium.com/data-science/multivariate-outlier-detection-in-python-e946cfc843b3> (visited on 09/15/2025).
- [47] G.Brereton Richard. “The chi squared and multinormal distributions”. In: (Oct. 17, 2014). URL: <https://analyticalsciencejournals.onlinelibrary.wiley.com/doi/pdf/10.1002/cem.2680> (visited on 12/03/2025).
- [48] Berzan Constantin and J.Russell Stuart. “Monte Carlo Methods for SLAM with Data Association Uncertainty”. In: (Aug. 13, 2015). URL: <https://www2.eecs.berkeley.edu/Pubs/TechRpts/2015/EECS-2015-191.pdf> (visited on 12/03/2025).
- [49] Doherty Kevin, P.Baxter David, Schneeweiss Edward, and J.Leonard John. “Probabilistic Data Association via Mixture Models for Robust Semantic SLAM”. In: (Sept. 29, 2019). URL: <https://arxiv.org/abs/1909.11213> (visited on 12/03/2025).
- [50] Neira Jose and D.Tardos Juan. “Data Association in Stochastic Mapping using the Joint Compatibility Test”. In: (Dec. 31, 2001). URL: <https://ieeexplore.ieee.org/document/976019> (visited on 12/03/2025).
- [51] Shen Xiaotong, Frazzoli Emilio, Rus Daniela, and H.Ang Jr Marcelo. “Fast Joint Compatibility Branch and Bound for Feature Cloud Matching”. In: (Dec. 1, 2016). URL: <https://ieeexplore.ieee.org/document/7759281> (visited on 12/03/2025).
- [52] Cadena Cesar, Carlone Luca, Carrillo Henry, Latif Yasir, Scaramuzza Davide, and Neira José. “Past, Present, and Future of Simultaneous Localization and Mapping: Toward the Robust-Perception Age”. In: (Dec. 31, 2016). URL: <https://ieeexplore.ieee.org/document/7747236> (visited on 09/14/2025).
- [53] Durrant-Whyte Hugh and Bailey Tim. “Simultaneous localization and mapping: part I”. In: (June 30, 2006). URL: <https://ieeexplore.ieee.org/document/1638022> (visited on 09/14/2025).
- [54] Durrant-Whyte Hugh and Bailey Tim. “Simultaneous localization and mapping: part II”. In: (Sept. 30, 2006). URL: <https://ieeexplore.ieee.org/document/1678144> (visited on 09/14/2025).
- [55] Kaess Michael, Ranganathan Ananth, and Dellaert Frank. “iSAM: Incremental Smoothing and Mapping”. In: (Dec. 31, 2008). URL: <https://ieeexplore.ieee.org/document/4682731> (visited on 09/14/2025).
- [56] Kaess Michael, Ila Viorela, Roberts Richard, and Dellaert Frank. “The Bayes Tree: An Algorithmic Foundation for Probabilistic Robot Mapping”. In: (Jan. 2010). URL: <https://www.cs.cmu.edu/~kaess/pub/Kaess10wafr.pdf> (visited on 09/14/2025).

- [57] Heggernes Pinar and Matstoms Pontus. “Finding Good Column Orderings for Sparse QR Factorization”. In: (Sept. 2000). URL: https://www.researchgate.net/publication/2498292_Finding_Good_Column_Orderings_for_Sparse_QR_Factorization (visited on 09/25/2025).
- [58] Castelo Robert. “The Discrete Acyclic Digraph Markov Model in Data Mining”. In: (Jan. 2002). URL: https://www.researchgate.net/publication/46624302_The_Discrete_Acyclic_Digraph_Markov_Model_in_Data_Mining (visited on 09/25/2025).
- [59] Matlab. “Build Map from 2-D Lidar Scans Using SLAM”. In: (2025). URL: <https://se.mathworks.com/help/lidar/ug/build-map-from-2d-lidar-scans-using-slam.html> (visited on 12/04/2025).
- [60] Rosell Jan, Palomo Leopold, and Avellaneda. “Appendix: ROS 2 - Introduction to ROS”. In: (2023). URL: <https://sir.upc.edu/projects/rostutorials/appendixRos2/index.html> (visited on 10/15/2025).
- [61] Open Robotics. “ROS - Robot Operating System”. In: (2025). URL: <https://www.ros.org/> (visited on 11/07/2025).
- [62] ROS2. “ROS2”. In: (2025). URL: <https://github.com/ros2> (visited on 11/07/2025).
- [63] CAOR-MINES ParisTech. “UKF-M”. In: (Oct. 29, 2021). URL: <https://github.com/CAOR-MINES-ParisTech/ukfm> (visited on 11/07/2025).
- [64] OpenCV. “OpenCV”. In: (). URL: <https://opencv.org/> (visited on 11/11/2025).
- [65] OpenCV. “OpenCV GitHub”. In: (). URL: <https://github.com/opencv> (visited on 11/11/2025).
- [66] MRPT. “MRPT”. In: (Nov. 7, 2025). URL: <https://github.com/mrpt> (visited on 11/07/2025).
- [67] Dellaert Frank. “Factor Graphs and GTSAM: A Hands-on Introduction”. In: (Sept. 2012). URL: <https://repository.gatech.edu/server/api/core/bitstreams/b3606eb4-ce55-4c16-8495-767bd46f0351/content> (visited on 09/25/2025).
- [68] GTSAM. “GTSAM: Factor graphs for Sensor Fusion in Robotics”. In: (). URL: <https://gtsam.org/> (visited on 11/07/2025).
- [69] BORG Lab. “BORG Lab”. In: (). URL: <https://github.com/borglab> (visited on 11/07/2025).
- [70] Lupton Todd and Sukkarieh Salah. “Visual-Inertial-Aided Navigation for High-Dynamic Motion in Built Environments Without Initial Conditions”. In: (Nov. 29, 2011). URL: <https://ieeexplore.ieee.org/document/6092505> (visited on 11/07/2025).